

INDICPARAM: Benchmark to evaluate LLMs on low-resource Indic Languages

Ayush Maheshwari, Kaushal Sharma[♣], Vivek Patel[♣], Aditya Maheshwari[♣]

[♣]Indian Institute of Management Indore, India ; [♣]BHARATGEN

ayush.hakmn@gmail.com, {kaushals,vivekp,adityam}@iimidr.ac.in

Abstract

While large language models excel on high-resource multilingual tasks, low- and extremely low-resource Indic languages remain severely under-evaluated. We present INDICPARAM, a human-curated benchmark of over 13,000 multiple-choice questions covering 11 such languages (Nepali, Gujarati, Marathi, Odia as low-resource; Dogri, Maithili, Rajasthani, Sanskrit, Bodo, Santali, Konkani as extremely low-resource). We evaluated 20 LLMs, both proprietary and open-weights, which reveals that even the top-performing Gemini-2.5 reaches 58% average accuracy, followed by GPT-5 (45) and DeepSeek-3.2 (43.1). We additionally label each question as knowledge-oriented or purely linguistic to discriminate factual recall from grammatical proficiency. Further, we assess the ability of LLMs to handle diverse question formats-such as list-based matching, assertion-reason pairs, and sequence ordering-alongside conventional multiple-choice questions. INDICPARAM provides insights into limitations of cross-lingual transfer and establishes a challenging benchmark for Indic languages. The dataset is available at <https://huggingface.co/datasets/bharatgenai/IndicParam>¹.

1 Introduction

Large language models (LLMs) have delivered state-of-the-art results across a range of multilingual tasks, particularly in high- and medium-resource settings such as translation, named entity recognition, and question answering (Liu et al., 2024). However, systematic evaluation for low and extremely-low-resource Indic languages remains limited. Existing benchmarks for Indic languages have significant coverage for major languages (e.g. Hindi, Tamil, Telugu) (Singh et al., 2024; Kakwani

¹Scripts to run benchmark are present at <https://github.com/ayushbits/IndicParam>

Language	#Words (in M)	#Speakers (in M)
Nepali	1642.9	19.4
Marathi	1541.2	99.1
Gujarati	934.1	60.3
Odia	333.8	42.6
Sanskrit	16.9	3.1
Maithili	14.0	14.3
Konkani	2.8	2.6
Santali	0.8	7.7
Bodo	0.8	1.5
Dogri	0.6	2.8
Rajasthani	-	25.8
Total	4480	279.1

Table 1: Number of words (in millions) present in FineWeb2 corpus and number of speakers (in millions) for each language considered in INDICPARAM.

et al., 2020), yet they provide only partial or no support for several understudied languages. Additionally, they rarely include coverage for code-mixed usage and evaluation.

In Table 1, we present word counts for the INDICPARAM languages within FineWeb2 (Penedo et al., 2025). FineWeb2 is a cleaned pretraining corpus comprising of approximately 3 trillion words from around 1800 languages (excluding English) gathered from more than 100 Common Crawl snapshots. For the 11 languages analyzed in this benchmark, the FineWeb2 corpus contains 4.49 billion words. Four languages-Nepali, Marathi, Gujarati, and Odia-account for 4.45 billion of these words, leaving only 36 million words for the remaining six languages. Collectively, these 11 languages represent about 280 million speakers across the Indian subcontinent, yet their aggregate web presence is disproportionately small relative to their speaker base. Based on the availability of web corpus, we classify Marathi, Gujarati, Nepali and Odia as low-resource languages while remaining seven languages as extremely low-resource languages.

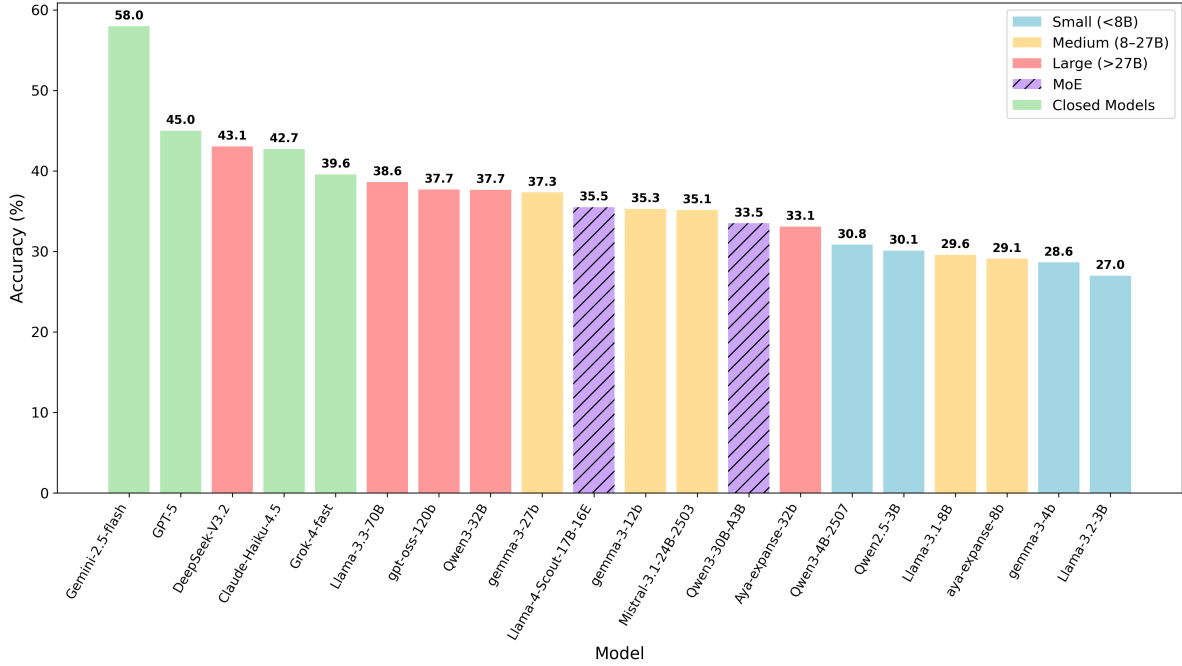


Figure 1: Average performance for all languages on INDICPARAM. Except DeepSeek-V3.2, all closed models outperform best performing open-weight models.

Low resource (4): Nepali, Gujarati, Marathi, Odia

Extremely-low resource (7): Dogri, Maithili, Rajasthani, Sanskrit, Bodo, Santali, Konkani

India, with a population of approximately 1.4 billion, is home to over 120 languages, at least 30 of which have one million or more speakers ([Office of the Registrar General & Census Commissioner, India, 2018](#)). These languages span multiple families, for instance, Marathi and Hindi belong to the Indo-European branch; Kannada and Telugu are Dravidian; Santali is Austro-Asiatic; and Bodo is Sino-Tibetan. The Indian Constitution recognizes 22 languages in the Eighth Schedule, yet most existing benchmarks concentrate on 11 major languages that together account for more than 93% of speakers, largely reflecting the limited availability of web-scale data for others ([IAMAI, 2024](#)). With recent advances in multilingual LLMs, language understanding has improved for many of these widely used languages ([Hu et al., 2020](#); [Qin et al., 2025](#)). However, despite a few benchmarks covering the majority set of 11 languages, there exists almost no evaluation resources for other Indic languages ([Kumar et al., 2022](#); [Madasu et al., 2023](#)).

To address these limitations, this paper intro-

duces INDICPARAM, a human-supervised evaluation set comprising more than 13,000 questions for 11 low- and extremely low-resource Indic languages. We also include a code-mixed Sanskrit-English language set evaluation. We hypothesize that despite limited pretraining data in several of these languages, LLMs can exploit cross-lingual transfer among typologically and script-wise related languages to improve performance on these target languages. To separate language understanding from factual recall, each question is labeled as either linguistic (*e.g.*, morphology, syntax, semantics, discourse) or knowledge-based (world facts and entities).

We evaluate 20 LLMs spanning open- and closed-source systems and a range of parameter scales and architectures on INDICPARAM. The evaluated models includes closed models such as GPT-5, Gemini-2.5, Claude-Haiku-4.5, and open weight models such as Llama4-Scout, DeepSeek-V3.2 among others. In Figure 1, we present the average performance of all models. Larger models with substantial exposure to multilingual Indic corpora tend to outperform smaller models. However, no model exceeds a 58% average score on INDICPARAM. Gemini-2.5 achieves the highest overall accuracy at 58, followed by GPT-5 at 45, Claude-Haiku at 42.7 and Grok-4 at 39.6, indicating that model robustness on these languages remains lim-

ited even among state-of-the-art proprietary models. Open-weight models narrow the gap with DeepSeek-V3.2 (DeepSeek-AI, 2025a), which is a 685B parameter model (43.1%) while Llama 3.3-70B attains average accuracy of 39.6. Overall, results indicate that multilingual performance for Indic languages remains challenging across both closed and open models, with substantial room for improvement in cross-lingual generalization. Our contributions can be summarised as follows:

1. We present INDICPARAM, a benchmark consisting of 13K+ questions on 11 low- and extremely-low resource Indic languages, as well as a code-mixed English-Sanskrit variant.
2. We provide granular annotations for each instance in the benchmark, including question-type and question-category labels, to enable systematic analysis of LLM capabilities.
3. We conduct an extensive evaluation encompassing 20 LLMs, including both proprietary closed-source and open-weight models.

2 Related Work

LLMs have demonstrated substantial improvements in language understanding performance for several Indic languages (SarvamAI; Pundalik et al., 2025). Multiple Indic multilingual benchmarks evaluate such progress, primarily targeting major mid- and high-resource languages (Singh et al., 2025, 2024; Maheshwari et al., 2024). IndicGenBench (Singh et al., 2024) proposes a benchmark on 29 Indic languages, including those examined in this study. Its evaluation suite includes tasks such as reading comprehension and translation, which are primarily adapted from English benchmark datasets like XQUAD (Artetxe et al., 2020) and FLORES (Goyal et al., 2022). IndicGenBench QA tasks restrict answers to spans supported by the given passage. In contrast, INDICPARAM extends this evaluation paradigm by assessing both language understanding and domain-specific general knowledge of LLMs using questions drawn from graduate-level examinations.

IndicGLUE (Kakwani et al., 2020) is a suite of supervised NLP tasks for prominent Indic languages, with a particular emphasis on Hindi, including classification tasks such as news categorization and headline prediction, as well as natural language inference, among others. IndicXTREME (Doddapaneni et al., 2023) broadens task coverage to named entity recognition, question answer-

ing, and paraphrase detection, and expands language coverage by translating and curating benchmarks in 11 Indic languages; however, many long-tail languages remain outside its scope. Recent Indic QA benchmarks further diversify evaluation: ParamBench (Maheshwari et al., 2025) focuses on graduate-level questionnaires with varied question types in Hindi, while MILU (Verma et al., 2025) contains 79K questions across 11 Indian languages, including Odia, Gujarati, and Marathi, which are also covered in INDICPARAM. Sanskriti (Maji et al., 2025a) and Pariksha (Watts et al., 2024) are English-language benchmarks designed to assess the socio-cultural alignment of LLMs. Drishtikon (Maji et al., 2025b) further extends this line of work to multimodal cultural understanding across 15 Indic languages.

Despite this progress, most benchmarks concentrate on the majority of 11 widely used Indian languages, with very limited coverage for extremely low-resource languages. In this paper, we introduce graduate-level language understanding questionnaires in 4 low-resource and 7 extremely low-resource languages, enabling targeted assessment of both language competence and general knowledge for underrepresented Indic languages.

3 INDICPARAM

INDICPARAM is a high-quality QA benchmark consisting of 13,207 questions that evaluates language understanding and general knowledge of multilingual LLMs in 11 Indic languages. These languages are spoken by approximately 280 million people across the Indian subcontinent, of whom 10 languages are recognized as scheduled languages in the Indian Constitution. Rajasthani, though not a scheduled language, is spoken by roughly 25 million people. We also include a QA set of code-mixed version of Sanskrit where questions include a mix of English and Sanskrit.

The language-wise distribution of questions is shown in Table 2. The questions are collected from UGC-NET², which is a nationwide examination administered by a government agency to determine eligibility for PhD admission and appointment to teaching positions in Indian universities and colleges. The exam is offered in around 85 subjects and is conducted twice annually. Each test consists of two papers composed of multiple-choice questions (MCQs). We developed the dataset by down-

²<https://ugcnet.nta.ac.in>

Language	#Ques	LU(%)	Script	Code
Nepali	1038	18.8	Deva	npi
Marathi	1245	4.7	Deva	mar
Gujarati	1044	0.6	Gujarati	guj
Odia	577	20.3	Orya	ory
Maithili	1286	10.1	Deva	mai
Konkani	1328	2.5	Deva	gom
Santali	873	11.3	Olck	sat
Bodo	1313	-	Deva	brx
Dogri	1027	18.3	Deva	doi
Rajasthani	1190	27.8	Deva	-
Sanskrit	1315	20.7	Deva	san
Sans-Eng	971	11.5	-	-
Total	13207			

Table 2: Distribution of question-answer pairs for different languages in INDICPARAM. ‘Sans-Eng’ denotes a separate set of Sanskrit-English code-mixed question-answer pairs. LU refers to % of manually classified language-understanding questions; rest are categorized as knowledge-related (*c.f.* Section 3.1).

loading question papers and answer keys from the official examination website for the period between 2012-2018.

Each language comprises multiple PDF question papers, many machine-readable and a subset with non-selectable text. Document layouts vary, with some single-column and most two-column. To ensure uniform accessibility, all PDFs were processed with a proprietary OCR system, and the extracted text was used for downstream curation and annotation. To the best of our knowledge, this corpus has not appeared in prior LLM benchmarking studies and constitutes a newly curated, human-authored dataset explicitly designed for graduate-level evaluation in Indic contexts. We describe annotation setup and team structure in Section 8 in Appendix.

3.1 Question Classification

In addition to annotating question-answer pairs in INDICPARAM, each question is assigned to one of two categories: (a) **language understanding** (LU) that includes questions related to linguistics and grammar, and (b) **general knowledge** (GK) targeting fact-based queries. Given the low- and extremely low-resource nature of the studied languages, this evaluation investigates model performance under data scarcity conditions and ex-

amines the extent to which cross-lingual transfer mechanisms can compensate for limited in-language training data.

The distribution of questions across these categories for each language is presented in Table 2. The LU column reports, for each language, the proportion of questions that test language understanding while the remaining questions belonging to the GK category and focusing on factual knowledge. The share of LU questions varies substantially, from under 1% in Gujarati to nearly 28% in Rajasthani, with several other languages such as Odia and Sanskrit also exhibiting over 20% LU coverage.

3.2 Question Type Classification

Following ParamBench (Maheshwari et al., 2025), we annotate each question-answer pair with its corresponding question type. We classify them into six primary categories that capture the diversity of assessment formats (see Table 3): multiple-choice questions (MCQ), assertion and reasoning (A&R), list matching, blank filling, identify incorrect statement (IS), and ordering. The majority (73%) of the questions are MCQs followed by list matching (9%), ordering (6.5%), A&R (6.1%), IS (4.1%) and blank filling (1.1%). The language wise breakdown of all languages remains the same except for Bodo, Gujarati, and Dogri, where the proportion of MCQs remain 36.6%, 33% and 21.7%, respectively, which are significantly lower than other languages in the benchmark. The detailed statistics of language-wise splits of question types is present in Table 6. In Table 11, we present example questions from different languages and their respective question class and question type.

3.3 Quality control and Verification

We used a multistep verification pipeline for linguistic precision and integrity of data. Primary data was taken from high-resolution, native PDFs from the official UGC NET portal to minimize extraction noise. Text extraction was script-tailored: LLMWhisperer³ for Devanagari scripts-Sanskrit, Maithili, Marathi-to preserve complex conjuncts, and Surya OCR (Paruchuri and Team, 2025) for Gujarati and Odia, followed by intensive manual correction by experts.

A post-verification audit on a random 5% sample (10,427 words and 61,547 characters) reported

³<https://unstract.com/llmwhisperer/>

Question Type	#Questions
Multiple-choice	9653
Assertion & Reason	811
List Matching	1185
Fill in the blanks	157
Identify incorrect statement	545
Ordering	856
Total	13207

Table 3: Distribution of questions across all languages in INDICPARAM.

a Word Error Rate (WER) of 0.68% and a Character Error Rate (CER) of 0.12%, confirming high reliability of the processed text. Please refer to the Appendix section 8.1 for more details.

3.4 Setup

We evaluated 20 state-of-the-art models spanning both open-weight and proprietary LLMs. The open-weight models range from 3 billion to over 685 billion parameters, including both dense and mixture-of-experts (MoE) architectures. These models collectively represent the state of the art in multilingual reasoning, code understanding, and multimodal generation. All models were used in their instruction-tuned configurations and evaluated under a zero-shot prompting setup. The prompt used during evaluation is present in Table 8. We apply a single, standardized prompt template across all languages and models. We also experimented with alternative prompt formulations, including prompts written in different languages; however, these variations did not meaningfully affect the results.

Open-weights models are retrieved from publicly available checkpoints at HuggingFace and use transformers library for inference. We load parameter weights in bf16 precision for all open-weight models except Llama4-Scout which was loaded with 8-bit quantization. We use greedy decoding while generating predictions by setting max-tokens as 50, temperature as 0, do_sample flag to false and batch size is set to 16. We disable thinking mode for Qwen3-MoE and keep reasoning levels at low for GPT-OSS-120B. For Gemma and Mistral, we follow their respective strategies from response generation instead of transformers pipeline function. We use OpenRouter (<https://openrouter.ai>) to generate predictions for closed models. We next describe the models included in our evaluation.

[//openrouter.ai](https://openrouter.ai)) to generate predictions for closed models. We next describe the models included in our evaluation.

1. **Qwen Series (Team-Qwen3, 2025):** We evaluated multiple models of the Qwen family, including Qwen2.5-3B, Qwen3-4B-2507, Qwen3-30B-A3B, and Qwen3-32B. The Qwen 3 series integrates substantial architectural and pre-training improvements over the 2.5 generation, trained on roughly 36 trillion multilingual tokens spanning 119 languages. The Qwen3-30B-A3B variant adopts a mixture-of-experts (MoE) design with 3 billion active parameters per forward pass, providing strong efficiency while maintaining performance parity with larger dense models.

2. **Llama Series (Team, 2024):** We evaluated Metas Llama-3.2-3B, Llama-3.1-8B, and Llama-3.3-70B. These models are trained on a mixture of high-quality multilingual and code data totaling approximately 15 trillion tokens. We also evaluated on Llama-4-Scout-17B-16E which is an MoE model having a 17 billion active parameters with 16 experts with a total of 109 billion model parameters. This model is trained on 40T tokens specifically designed to improve multilingual text and images.

3. **Gemma Series (Team, 2025):** We evaluated Gemma-3 models at 4B, 12B, and 27B parameter scales. Developed by Google DeepMind, the Gemma 3 family employs hybrid instruction-fine-tuning strategies and trained on diverse multilingual datasets. These models have demonstrated strong alignment and competitive performance across multilingual and reasoning benchmarks.

4. **Aya Expanse (Aryabumi et al., 2024):** The Aya-Expanse-8B and Aya-Expanse-32B models are instruction-tuned multilingual LLMs. These models prioritize cultural and linguistic diversity and are trained on extensive parallel datasets that include over 100 different languages.

5. **Mistral 3.1 (MistralAI):** We evaluated the Mistral-Small-3.1-24B-2503 which is a 24 billion parameter multi-lingual model achieving competitive performance while maintaining fast inference throughput.

6. **DeepSeek V3.2 (DeepSeek-AI, 2025b):** We evaluated DeepSeek-V3.2-Experimental model consisting of 685B parameters. It employs specialized routing and sparsity mechanisms to reduce inference costs while preserving high-quality reasoning, code synthesis, and mathematical rea-

soning performance.

7. **GPT-OSS-120B (OpenAI et al., 2025):** We examined the GPT-OSS-120B model, an open-weight MoE model by OpenAI, trained on trillions of tokens with a focus on STEM, coding, and general knowledge. It has a total of 116.8B parameters and 5.1B active parameters per token per forward pass.

8. **Frontier Proprietary Models:** For completeness, we also benchmarked several leading closed-source systems accessible through public APIs: GPT-5, Claude-Haiku-4.5, and Grok-4-Fast.

- *GPT-5* (OpenAI) represents the current flagship frontier model, incorporating multimodal reasoning and tool-use capabilities.

- *Claude-Haiku-4.5* (Anthropic) is a lightweight yet high-accuracy variant of the Claude 4.5 family designed for efficiency-critical environments.

- *Grok-4-Fast* (xAI) is optimized for rapid inference and contextual awareness within streaming conversational environments.

- *Gemini-2.5-Flash* (Comanici et al., 2025) (Google) is a sparse MoE model and a distilled version of a larger Gemini-2.5 model.

4 Results and discussion

4.1 Overall Model Performance

The evaluation results presented in Figure 1 demonstrate that average performance on INDIC-PARAM for 20 evaluated models remains moderate, though several interesting patterns emerge when grouped by model scale. Among the small-capacity models (<8B parameters), the Qwen family performs best: Qwen3-4B and Qwen2.5-3B achieves the accuracy of 30.8 and 30.1 respectively. Other models in this category, including Gemma-3-4B, and Llama-3.2-3B, remain around the 27-30% range.

In the medium-size category (8B-27B), performance improves considerably, with Gemma-3-27B leading at 37.3%, followed closely by Gemma-3-12B (35.3) and Mistral-Small-3.1-24B (35.1). Other models like Aya-Expansive-8B and Llama-3.1-8B score less than 30%. These findings underscore that model scale beyond 8B yields measurable gains, but pre-training coverage and multilingual representation continue to play a decisive role in performance.

Among the larger and frontier models ($\geq 27B$), overall accuracy rises substantially. The best performance is achieved by Gemini-2.5, which

attains an average of 58%, followed by GPT-5 (45), DeepSeek-V3.2 (43.1), Claude-Haiku-4.5 (42.7), and Grok-4-Fast (39.6). Open-weight models such as Llama-3.3-70B and GPT-OSS-120B achieve 38.6 and 37.7, respectively, while the Llama-4-Scout-17B-16E models remain slightly lower at 35.5. These results indicate that while data and parameter size enhance overall accuracy, the highest-performing models also benefit from refined fine-tuned on multilingual or Indic-rich corpora.

Overall, the evaluation reveals a clear upward trajectory in accuracy with increasing model scale, yet even the most capable frontier systems such as Gemini-2.5, GPT-5 and DeepSeek-V3.2 demonstrate that substantial headroom remains for further enhancement in Indic language understanding. The results affirm that cross-lingual generalization, fine-tuning quality, and the underlying token distribution remain central determinants of downstream accuracy.

4.2 Performance on Low- and Extremely Low-Resource Indic Languages

Table 4 and Table 5 compare contemporary LLMs on eight extremely low-resource Indic languages (Dogri, Maithili, Rajasthani, Sanskrit, Sanskrit-English code-mixed, Bodo, Santali, and Konkani) and four higher (yet still low-resource) Indic languages (Nepali, Gujarati, Marathi, and Odia) respectively. On extremely low-resource languages, Gemini-2.5 achieves the highest average of 59.2%, followed by GPT-5 (45.1), open-weight DeepSeek-3.2 at 43.7. No model exceeds 53 on the most underrepresented languages (Bodo and Santali), with most models remaining below 36%. Gemini-2.5 shows strong performance on Sanskrit (72.4) and Sanskrit-English code-mixed text (80.3). Despite Gemini-2.5 performing remarkably better than other LLMs, the remaining models exhibit substantially lower performance, suggesting that strong Indic-language capability is not yet widespread across current LLMs.

With the exception of Rajasthani and Sanskrit, all models including Gemini scores less than 54% for all other languages. Notably, even very large parameter Deepseek model scores under 44%, aside from the Sanskrit variants. The relatively stronger results for Rajasthani may stem from its linguistic proximity to Hindi, which is typically well represented in model pretraining corpora. Despite limited web-scale data availability, Sanskrit

Size	Model	Dogri	Maithili	Rajasthani	Sanskrit	Sans-Eng	Bodo	Santali	Konkani	Average
Small	Qwen2.5-3B	32.1	28.8	28.6	31.9	<u>34.8</u>	<u>28.3</u>	33.1	<u>31.4</u>	30.9
	Llama-3.2-3B	27.9	21.9	26.3	28	<u>27.9</u>	<u>27.8</u>	27.4	28.8	26.9
	Gemma-3-4b	30.5	24.5	27.6	29.1	32.2	25.8	31.7	29.1	28.5
	Qwen3-4B	<u>34.6</u>	<u>31.6</u>	<u>30</u>	<u>33.8</u>	33.6	25.1	<u>33.8</u>	29.6	<u>31.2</u>
Medium	Aya-8b	31.5	29.9	30.8	29.4	29.9	28.1	30.8	26.9	29.5
	Llama-3.1-8B	26.4	29	31.8	32.2	34	26.8	30.6	29.5	30.0
	Gemma-3-12b	37.2	31.6	35	37.1	41.8	<u>30.6</u>	33.7	35.7	35.1
	Mistral3.1-24b	<u>38</u>	<u>37.4</u>	34.8	<u>40</u>	44.1	<u>30.1</u>	35.7	32	36.2
	Gemma-3-27b	37.6	34.4	<u>39.8</u>	38.6	<u>49.1</u>	28.9	<u>36.7</u>	<u>36.5</u>	<u>37.3</u>
Large	Aya-32b	35.2	33.2	34.6	35.2	39	27.2	35.6	33.1	33.7
	Qwen3-32B	<u>43.4</u>	34	33.8	39.2	48.6	<u>31.8</u>	39.7	34.5	33.9
	Llama-3.3-70B	37.3	<u>37.5</u>	<u>38.2</u>	41.7	51.6	<u>28.6</u>	40.1	37.3	37.6
	DeepSeek-3.2	43.3	<u>41.4</u>	<u>40.1</u>	<u>51.3</u>	<u>61.4</u>	36.4	41.5	<u>37.6</u>	<u>43.7</u>
MoE	Qwen3-A3B	35.7	30.9	31.6	36.7	44.2	<u>29.9</u>	35.9	28.3	<u>38.6</u>
	Llama-4-Scout	30.5	31.3	34.8	<u>44</u>	<u>48.7</u>	<u>25.9</u>	<u>36.3</u>	34.6	35.4
	gpt-oss-120b	<u>39.7</u>	<u>35.6</u>	<u>39.4</u>	40.2	48.1	28.3	35.6	<u>35.4</u>	37.4
Closed	GPT-5	<u>45.7</u>	<u>43.4</u>	<u>44.6</u>	<u>54.8</u>	<u>64.6</u>	<u>31.6</u>	40.4	39	<u>45.1</u>
	Grok-4-fast	40.2	38.1	39.1	42.9	54.2	30.6	38.8	37.4	39.7
	Claude-4.5	44.8	42.4	44.1	47.8	58.1	28.1	40.4	<u>42.1</u>	43.0
	Gemini-2.5	54.2	54	66.6	72.4	80.3	48.1	52.9	48.4	59.2

Table 4: Performance on extremely-low resource languages on INDICPARAM. We **bold** and *italicize* the best overall performance, underline and *italicize* the best performance in each model-size category and underline the second best overall performance.

Size	Model	Nepali	Gujarati	Marathi	Odia	Average
Small	Qwen2.5-3B	27.1	<u>29.5</u>	27.9	28.8	28.2
	Llama3.2-3B	26.2	26.1	28.6	27	27.1
	Gemma-3-4b	<u>29</u>	26.2	29.9	31.5	28.9
	Qwen3-4B	28.7	27.4	<u>30.5</u>	<u>34.7</u>	<u>29.8</u>
Medium	Aya-8b	28.2	27.7	28.3	28.1	28.1
	Llama3.1-8B	25.5	27.2	32.2	29.3	28.7
	Gemma-3-12b	34.8	<u>33.8</u>	38.4	34.5	35.6
	Mistral3.1-24b	34.9	32.6	32.9	27.6	32.6
	Gemma3-27b	<u>39.3</u>	31.1	<u>39.9</u>	<u>39.9</u>	<u>37.4</u>
Large	Aya-32b	31.6	30.7	35.5	34.5	33.0
	Qwen3-32B	33	28.4	33.2	28.9	31.2
	Llama3.3-70B	35	35.9	40.8	40.7	37.9
	DeepSeek-3.2	<u>39.7</u>	<u>39.5</u>	<u>44.6</u>	<u>42.1</u>	<u>41.6</u>
MoE	Qwen3-A3B	<u>37.2</u>	33.5	43.5	<u>40.4</u>	<u>38.7</u>
	Llama-4-Scout	<u>34.6</u>	27.3	<u>43.5</u>	<u>35</u>	35.5
	gpt-oss-120b	37.1	<u>37</u>	39.7	39.9	38.3
Closed	GPT-5	43.4	<u>40</u>	<u>48.4</u>	<u>48.7</u>	<u>44.9</u>
	Grok-4-fast	40.9	35.9	39.8	40.7	39.2
	Claude-4.5	<u>43.8</u>	37	43.9	44	42.0
	Gemini-2.5	52	51.7	55.7	64.5	54.9

Table 5: Performance on low resource languages on INDICPARAM. We **bold** and *italicize* the best overall performance, underline and *italicize* the best performance in each model-size category and underline the second best overall performance.

is a well-studied language in the NLP literature, which may partially explain its comparatively bet-

ter performance.

Among smaller models, the Qwen family out-

performs Gemma and Llama models. In contrast, Gemma3-27B surpasses Qwen3-32B, which is counter-intuitive to the assumption that Qwen3’s larger pretraining corpus necessarily yields consistent downstream gains. Overall, within the open-weight models, DeepSeek performs best, suggesting benefits from its architectural choices and larger parameter scale.

In the low-resource group, Gemini-2.5 again leads with 54.9%, ahead of GPT-5 (44.9), Claude-4.5 (42.0) and DeepSeek-3.2 (41.6). Among other open models, Gemma-3-27b (37.4) and the MoE Qwen3-A3B (38.7) perform best in their respective size categories. Notably, Gemini-2.5 achieves the highest performance in Odia (64.5), followed by Marathi (55.7) having a 8.8 points drop. In contrast, other models exhibit a much smaller performance gap between these two languages.

Except for Gemini, all other models score below 50% across each of the four languages considered. Small-size models achieve their highest score in Odia (34.7), and medium-size models likewise obtain their best result in Odia (39.9). Among larger open-weight models, DeepSeek attains the strongest performance, scoring 44.6 on Marathi. Overall, these findings suggest that improving performance on low-resource Indic languages likely requires both stronger general pretraining data and targeted, language-specific resources.

4.3 Performance on General Knowledge and Language Understanding Tasks

We distinguish between two complementary capabilities: (i) *General Knowledge* questions that require factual reasoning and culturally grounded world knowledge (Table 9), and (ii) *Language-Specific Understanding* questions that probe grammatical, lexical, morphosyntactic, and discourse-level proficiency within each Indic language (Table 10). LU questions forms a very small percentage of overall questions, particularly for Gujarati (0.6%).

4.3.1 General Knowledge

Frontier models exhibit clear separation from the rest of the models. Gemini-2.5 achieves the highest average (58), followed by GPT-5 (54.8), Claude-4.5 (43.5) and DeepSeek-3.2 (43.3). Among open-weight systems, Llama-3.3-70B (39.3) and gpt-oss-120b (38) perform competitively, while Grok-4-fast attains 40.3. Large

gains are observed with scale: medium-sized models (8B–27B) top out at 37.9 (Gemma-3-27b), and small models (<8B) remain below 32 (Qwen3-4B: 31.3). As shown in Table 9, Gemini dominates Sanskrit (74.4) and Sanskrit–English code-mixed (80.3) contexts, reflecting superior classical-language representation.

4.3.2 Language-Specific Understanding

Performance on pure linguistic tasks is higher than GK questions on all languages except Sanskrit. Except Gemini, none of the models rarely exceed 67% on any single language (see Table 10). Gemini-2.5 again leads (69.8 average), followed by GPT-5 (52.5) Claude-4.5 (49.8) and DeepSeek-3.2 (46.3). Several medium-scale models punch above their parameter count: Llama-4-Scout (41.7, especially strong on Marathi) and Gemma-3-27b (42.2) occasionally outperform larger dense models on morphologically complex languages. Smaller models struggle severely, with many scores in the 20-35% range; as evident in Table 10. Sanskrit, Sanskrit+Eng, Dogri, Nepali and Santali prove particularly challenging across the board.

A comparative analysis of GK and LU reveals distinct scaling behaviours. General knowledge accuracy correlates strongly with model scale (Pearson $r = 0.91$ across the 20 evaluated models), whereas language-specific understanding shows a weaker but still substantial correlation ($r = 0.73$). The correlation between the two dimensions is moderately high ($r = 0.79$), indicating that top-tier factual reasoning usually co-occurs with stronger linguistic proficiency, yet meaningful divergences remain.

Several models achieve notably balanced performance. Gemma-3-27b (GK: 37.9, LU: 40.9) and Mistral3.1-24b (GK: 35.3, LU: 42.7) outperform many larger dense models on linguistic tasks, demonstrating that continued pre-training on diverse Indic corpora and instruction tuning can compensate for parameter count in morphosyntactically demanding settings. Similarly, MoE-based Llama-4-Scout reaches 41.8 on language understanding while scoring only 35.6 on general knowledge, highlighting the value of sparse architectures for grammar-heavy tasks.

These results underscore that future progress on low- and extremely low-resource Indic languages will require not only continued scaling but also (i) higher-quality and more diverse Indic pre-

training data, (ii) dedicated cross-lingual continuation training, and (iii) architectural innovations that prioritise fine-grained morphological and syntactic representation.

5 Conclusion

We present INDICPARAM consisting of >13k questions for 11 low and extremely-low resource Indic languages. Our benchmark reveals that even the strongest contemporary LLMs, including frontier proprietary systems, remain far from reliable on graduate-level questions in low- and extremely low-resource Indic languages, with no model surpassing 45% average accuracy except Gemini-2.5. While larger closed models such as Gemini-2.5, GPT-5, Claude-4.5 and open-weight model, DeepSeek-3.2 consistently outperform open-weight alternatives. These findings indicate INDICPARAM as a challenging, human-curated evaluation suite that fills a key gap in existing Indic benchmarks and underline the need for more balanced pretraining and adaptation for under-represented Indic languages.

6 Limitations

While INDICPARAM substantially broadens evaluation for low- and extremely low-resource Indic languages, the benchmark is derived exclusively from UGC-NET language papers in linguistics and literature, which biases the content toward academic exam styles and may not fully represent everyday or domain-general language use. Further, our evaluation is conducted using a log-likelihood-based multiple-choice setup in a zero-shot configuration, which may yield different results from alternative evaluation schemes such few-shot prompting, or chain-of-thought prompting, and therefore does not capture the full range of behaviors that these models might exhibit under richer prompting strategies.

7 Ethics

All data described in this work was collected from publicly available resources. We sought permission of UGC-NTA for releasing the benchmark. INDICPARAM is released for non-commercial research use; code and annotation scripts are licensed under MIT. All annotation was conducted by consenting native speakers compensated at local rates, and no personally identifiable exam-candidate information appears in the benchmark.

Users interested in commercial use should contact UGC-NTA for relevant permissions.

References

- Mikel Artetxe, Sebastian Ruder, and Dani Yogatama. 2020. On the cross-lingual transferability of monolingual representations. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 4623–4637.
- Viraat Aryabumi, John Dang, Dwarak Talupuru, Saurabh Dash, David Cairuz, Hangyu Lin, Bharat Venkitesh, Madeline Smith, Jon Ander Campos, Yi Chern Tan, Kelly Marchisio, Max Bartolo, Sebastian Ruder, Acyr Locatelli, Julia Kreutzer, Nick Frosst, Aidan Gomez, Phil Blunsom, Marzieh Fadaee, Ahmet Üstün, and Sara Hooker. 2024. [Aya 23: Open weight releases to further multilingual progress](#).
- Gheorghe Comanici, Eric Bieber, Mike Schaeckermann, Ice Pasupat, Noveen Sachdeva, Inderjit Dhillon, Marcel Blistein, Ori Ram, Dan Zhang, Evan Rosen, et al. 2025. Gemini 2.5: Pushing the frontier with advanced reasoning, multimodality, long context, and next generation agentic capabilities. *arXiv preprint arXiv:2507.06261*.
- DeepSeek-AI. 2025a. Deepseek-v3.2-exp: Boosting long-context efficiency with deepseek sparse attention.
- DeepSeek-AI. 2025b. Deepseek-v3.2-exp: Boosting long-context efficiency with deepseek sparse attention.
- Sumanth Doddapaneni, Rahul Aralikkatte, Gowtham Ramesh, Shreya Goyal, Mitesh M Khapra, Anoop Kunchukuttan, and Pratyush Kumar. 2023. Towards leaving no indic language behind: Building monolingual corpora, benchmark and models for indic languages. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 12402–12426.
- Naman Goyal, Cynthia Gao, Vishrav Chaudhary, Peng-Jen Chen, Guillaume Wenzek, Da Ju, Sanjana Krishnan, MarcAurelio Ranzato, Francisco Guzmán, and Angela Fan. 2022. The flores-101 evaluation benchmark for low-resource and multilingual machine translation. *Transactions of the Association for Computational Linguistics*, 10:522–538.
- Junjie Hu, Sebastian Ruder, Aditya Siddhant, Graham Neubig, Orhan Firat, and Melvin Johnson. 2020. Xtreme: A massively multilingual multi-task benchmark for evaluating cross-lingual generalisation. In *International conference on machine learning*, pages 4411–4421. PMLR.
- IAMAI. 2024. [Internet in india 2024](#). Research report, Internet and Mobile Association of India (IAMAI). Prepared in collaboration with Kantar.

- Divyanshu Kakwani, Anoop Kunchukuttan, Satish Golla, Gokul NC, Avik Bhattacharyya, Mitesh M Khapra, and Pratyush Kumar. 2020. IndicNLPsuite: Monolingual corpora, evaluation benchmarks and pre-trained multilingual language models for indian languages. In *Findings of the association for computational linguistics: EMNLP 2020*, pages 4948–4961.
- Aman Kumar, Himani Shrotriya, Prachi Sahu, Amogh Mishra, Raj Dabre, Ratish Puduppully, Anoop Kunchukuttan, Mitesh M Khapra, and Pratyush Kumar. 2022. IndicNlg benchmark: Multilingual datasets for diverse nlg tasks in indic languages. In *Proceedings of the 2022 conference on empirical methods in natural language processing*, pages 5363–5394.
- Yang Liu, Meng Xu, Shuo Wang, Liner Yang, Haoyu Wang, Zhenghao Liu, Cunliang Kong, Yun Chen, Maosong Sun, and Erhong Yang. 2024. Omgeval: An open multilingual generative evaluation benchmark for large language models. *arXiv preprint arXiv:2402.13524*.
- Lokesh Madasu, Gopichand Kanumolu, Nir-mal Surange, and Manish Shrivastava. 2023. Mukhyansh: A headline generation dataset for indic languages. In *Proceedings of the 37th Pacific Asia Conference on Language, Information and Computation*, pages 620–634.
- Ayush Maheshwari, Ashim Gupta, Amrith Krishna, Atul Kumar Singh, Ganesh Ramakrishnan, Anil Kumar Gourishetty, and Jitin Singla. 2024. [Samayik: A benchmark and dataset for English-Sanskrit translation](#).
- Ayush Maheshwari, Kaushal Sharma, Vivek Patel, and Aditya Maheshwari. 2025. Parambench: A graduate-level benchmark for evaluating llm understanding on indic subjects. *arXiv preprint arXiv:2508.16185*.
- Arijit Maji, Raghvendra Kumar, Akash Ghosh, Sri-parna Saha, et al. 2025a. Sanskriti: A comprehensive benchmark for evaluating language models’ knowledge of indian culture. *arXiv preprint arXiv:2506.15355*.
- Arijit Maji, Raghvendra Kumar, Akash Ghosh, Nemil Shah, Abhilekh Borah, Vanshika Shah, Nishant Mishra, Sriparna Saha, et al. 2025b. Drishtikon: A multimodal multilingual benchmark for testing language models’ understanding on indian culture. *arXiv preprint arXiv:2509.19274*.
- Team MistralAI. [Mistralai/mistral-small-3.1-24b-instruct-2503](#) ð hugging face.
- Office of the Registrar General & Census Commissioner, India. 2018. [Census of india 2011: Paper 1 of 2018 – language, india, states and union territories \(table c-16\)](#). Census of India 2011, Paper 1 of 2018, Language Data (Table C-16).
- OpenAI, :, Sandhini Agarwal, Lama Ahmad, Jason Ai, Sam Altman, Andy Applebaum, Edwin Arbus, and Rahul K. 2025. [gpt-oss-120b & gpt-oss-20b model card](#).
- Vikas Paruchuri and Datalab Team. 2025. Surya: A lightweight document ocr and analysis toolkit. <https://github.com/VikParuchuri/surya>. GitHub repository.
- Guilherme Penedo, Hynek Kydlíek, Vinko Sabolec, Bettina Messmer, Negar Foroutan, Amir Hossein Kargaran, Colin Raffel, Martin Jaggi, Leandro Von Werra, and Thomas Wolf. 2025. [Fineweb2: One pipeline to scale them all – adapting pre-training data processing to every language](#).
- Kundeshwar Pundalik, Piyush Sawarkar, Nihar Sahoo, Abhishek Shinde, Prateek Chanda, Vedant Goswami, Ajay Nagpal, Atul Singh, Viraj Thakur, Vijay Dewane, Aamod Thakur, Bhargav Patel, Smita Gautam, Bhagwan Panditi, Shyam Pawar, Madhav Kotcha, Suraj Racha, Saral Sureka, Pankaj Singh, Rishi Bal, Rohit Saluja, and Ganesh Ramakrishnan. 2025. [Param-1 BharatGen 2.9b model](#).
- Libo Qin, Qiguang Chen, Yuhang Zhou, Zhi Chen, Yinghui Li, Lizi Liao, Min Li, Wanxiang Che, and Philip S. Yu. 2025. [A survey of multilingual large language models](#). *Patterns*, 6(1).
- SarvamAI. Sarvam AI | Sovereign Indian AI Ecosystem for LLMs, Agents, and AI Assistants — sarvam.ai. <https://www.sarvam.ai/>. [Accessed 20-08-2025].
- Abhishek Kumar Singh, Vishwajeet Kumar, Rudra Murthy, Jaydeep Sen, Ashish Mittal, and Ganesh Ramakrishnan. 2025. Indic qa benchmark: A multilingual benchmark to evaluate question answering capability of llms for indic languages. In *Findings of the Association for Computational Linguistics: NAACL 2025*, pages 2607–2626.
- Harman Singh, Nitish Gupta, Shikhar Bharadwaj, Dinesh Tewari, and Partha Talukdar. 2024. Indicgenbench: A multilingual benchmark to evaluate generation capabilities of llms on indic languages. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 11047–11073.
- Gemma 3 Team. 2025. [Gemma 3 technical report](#).
- Llama 3 Team. 2024. [The llama 3 herd of models](#).
- Team-Qwen3. 2025. [Qwen3 technical report](#).
- Sshubam Verma, Mohammed Safi Ur Rahman, Vishwajeet Kumar, Rudra Murthy Venkataramana, and Jaydeep Sen. 2025. Milu: A multi-task indic language understanding benchmark. In *Annual Conference of the North American Chapter of the Association for Computational Linguistics*.

Ishaan Watts, Varun Gumma, Aditya Yadavalli, Vivek Seshadri, Manohar Swaminathan, and Sunayana Sitaram. 2024. Pariksha: A large-scale investigation of human-llm evaluator agreement on multilingual and multi-cultural data. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 7900–7932.

Appendix

8 Annotation Setup

For each language, we first collected official UGC-NET language papers and corresponding answer keys from the official website⁴ and converted all files into a uniform machine-readable format. Question papers appeared in double-column layouts, and many PDFs contained non-selectable text. Hence, all documents were processed through a proprietary OCR pipeline to extract text consistently across heterogeneous layouts and scan qualities. During this stage, each question was tagged with its language, subject, exam session, and year, and answer keys were aligned to their corresponding items.

Following OCR, human annotators manually corrected recognition errors, restored missing characters and diacritics, and normalised punctuation and spacing so that questions and options follow a pre-specified parsing format. Beyond surface correction, annotators standardised structural formattings such as option markers, numbering schemes, and line breaks. Each final entry in INDICPARAM is stored with explicit fields for question text, question type, four answer options, correct option, language, subject, year, and unique question identifier.

Annotation was carried out by native speakers having expertise in the target language and having prior experience reading graduate-level exam material. Since many papers mix language-focused and knowledge-focused content, annotators were required to demonstrate strong proficiency in the relevant Indic script. All annotators received detailed written guidelines describing how to handle incomplete questions, ambiguous answer keys, and exam-specific conventions, as well as how to assign labels for language understanding (LU) versus knowledge-related (KR) items and for the six question types used in the benchmark. Prior to full-scale annotation, annotators were given a sample dataset along with worked examples. They were then assigned trial files, which were reviewed and corrected by the reviewers. Feedback was provided iteratively, and only after demonstrating consistent accuracy were annotators assigned larger batches of data. For question-answer pairs, annotation quality followed a focused two-step review pipeline. In the first step, annotators corrected OCR errors in the question and options, aligned each item with the official answer key, and ensured that exactly one option was marked as correct. In the second step, a senior reviewer re-checked these corrected pairs against the original exam papers, verifying that the question text and the designated correct option was faithful to the source. Annotators were compensated at a rate of \$1 per 10 questions.

8.1 Quality Control and Verification

A multi-stage quality control and verification pipeline was implemented to ensure that the dataset maintained integrity and linguistic precision. This first involved acquiring the primary source data directly from the official website of UGC NET, in high quality, natively generated PDFs, thereby ensuring high initial resolution and low extraction noise.

For the text extraction task, a approach was used based on script requirements. For languages using the Devanagari script, such as Sanskrit, Maithili, and Marathi, LLMWhisperer⁵ was used because it has been optimized for retaining complex Indic conjuncts. In contrast, non-Devanagari scripts, such as Gujarati and Odia, had significant extraction limits on multiple platforms. After comparing several options, Surya OCR (Paruchuri and Team, 2025) was implemented, but it had more script-specific mistakes that required intensive manual correction by subject area experts in Gujarati and Odia. All the Devanagari extractions then underwent parallel manual verification, wherein human annotators followed strict linguistic rules to correct ocr-induced errors and ensure structural consistency.

Quantification of final data quality was done via a post-verification audit on a randomized 5% sample of the total file corpus. A sample of this size consists of 10,427 words and 61,547 characters, with 71 errors, which is a negligibly small WER of 0.68% and CER of 0.12%.

⁴<https://ugcnet.nta.nic.in/>

⁵<https://unstract.com/llmwhisperer/>

9 Ethical considerations

This work introduces a new benchmark dataset derived from publicly available exam questions. As the dataset is built exclusively from existing public materials and involves no new data collection from human participants, no formal review by an Institutional Review Board (IRB) or equivalent ethics committee was required or conducted. Instead, the authors performed an internal assessment of data provenance, potential societal impacts, licensing considerations, and responsible use, in line with community standards for responsible machine learning research (e.g., NeurIPS Ethics Guidelines).

9.1 Intended use and beneficiaries

The benchmark is designed to support research in evaluating LLM understanding on Indic subjects. It aims to provide a challenging, realistic evaluation resource for the academic and AI research communities, as well as developers working on automated educational tools or knowledge-intensive systems. By releasing this dataset, we hope to foster progress in fair, transparent, and reproducible benchmarking of models at the graduate level, ultimately benefiting educators, students, and researchers seeking improved AI capabilities in knowledge assessment.

9.2 Licensing and copyright considerations

The original UGC-NET exam questions are publicly released by the National Testing Agency (NTA), Government of India, primarily for candidate preparation and result-related purposes (available on ugc-net.nta.ac.in). No explicit redistribution license (e.g., Creative Commons) or statement prohibiting non-commercial research use is provided on the official website.

On November 21, 2025, we proactively contacted the NTA/UGC-NET team via email to seek formal permission for including these publicly available questions in ParamBench. Our request emphasized that the benchmark is strictly for non-commercial academic research under the DST-funded BharatGen project (now part of the national IndiaAI Mission), with full attribution to "UGC-NET, National Testing Agency, Government of India" in the paper and dataset. We attached the draft paper for their review. As of the latest revision of this manuscript, we await their response and will update the dataset release accordingly should any conditions be specified.

Pending explicit permission, we consider our transformative use (curated selection, cleaning, and re-formatting into a machine-readable benchmark for 13,207 questions focused on Indian languages) permissible under fair dealing provisions of the Indian Copyright Act, 1957, for non-commercial research and educational purposes. We do not redistribute raw original question papers/PDFs; only processed question-answer pairs with metadata are released. The derived IndicParam dataset will be made available under the Creative Commons Attribution 4.0 International (CC-BY-4.0) license, requiring attribution to both this work and the original UGC-NET source.

10 Question Types-wise results

Table 6 presents the distribution of question types across subjects, and Table 7 provides the corresponding performance results for these question types.

10.1 Question Types

Table 6 provides a detailed breakdown of the types of questions present in each language. MCQs dominate the distribution, but a substantial number of list matching, AssertionReasoning(A&R), identification of incorrect statements (IS), ordering and blank filling questions contribute to different question types.

10.2 Question type-wise results

Table 7 shows the performance of each model on INDICPARAM by question type. In the smaller-sized models, Qwen3-4B demonstrates better performance in a majority formats of questions, which includes MCQ, IS, fill in the blanks and Ordering. This model outperforms alternative 34B models, including Llama3.2-3B and Gemma3-4B. In the medium-size model category, Gemma3-27B shows the best overall performance, with considerable improvements in MCQ, A&R, and fill in the blanks. At the same time,

Language	MCQ	A&R	List Matching	Blank Filling	IS	Ordering	Total
Nepali	957	1	38	5	25	12	1038
Marathi	933	83	139	1	54	35	1245
Gujarati	345	172	208	0	128	191	1044
Odia	379	40	70	1	22	65	577
Maithili	1265	0	0	21	0	0	1286
Konkani	1069	30	134	19	45	31	1328
Santali	772	36	46	11	2	6	873
Bodo	481	288	270	17	0	257	1313
Dogri	223	153	217	1	231	202	1027
Rajasthani	1114	4	11	14	13	34	1190
Sanskrit	1210	1	39	34	11	20	1315
Sans-Eng	905	3	13	33	14	3	971
Total	9653	811	1185	157	545	856	13207

Table 6: Different question types across all languages in INDICPARAM. A&R refers to Assertion and Reason type question, IS refers to identification of incorrect statement. ‘Sans-Eng’ denotes a separate set of Sanskrit-English code-mixed question-answer pairs which forms a separate set.

Gemma3-12B and Mistral3.1-24B also show competitive results in List Matching and Ordering, respectively. Large models show a considerable improvement in their ability to handle different question types. Llama3.3-70B model performs best on most question types, especially MCQ, list matching, A&R and IS. DeepSeek-3.2 performs best on MCQ, list matching, A&R shows good results compared to other large-sized models in the category. Within MoE models, Qwen3-A3B performs best on MCQ, IS and fill in the blanks type questions. On the other hand, gpt-oss-120b shows the best results on list matching, A&R, and ordering related questions. Closed-source models consistently outperform their open-source counterparts. GPT-5, achieves best scores across almost all question categories, including MCQ, fill in the blanks, list matching, A&R and ordering. In contrast, Claude-4.5 demonstrates best performance for IS questions. Grok4-fast performance is consistently lower than that of GPT-5 and Claude-4.5.

11 Performance on KR and LU questions

The trends observed in Table 9 and Table 10 show that accuracy improves consistently with increasing model scale, with frontier models achieving the strongest results. As discussed in Section 4.3.1, general knowledge tasks display steady but moderate gains, whereas Section 4.3.2 highlights far greater variability in language-specific tasks, reflecting the difficulty of deep multilingual grounding. Architectural improvements, including MoE routing, enable mid-sized models narrow the performance gap with larger systems.

12 Zero shot prompting template

In Table 8, we provide the zero-shot prompt used while evaluating all the models.

Size	Model	MCQ	List Matching	A&R	Blanks	IS	Ordering
Small-size	Qwen2.5-3B	30.3	32.6	25.6	26.1	27.7	30.5
	Llama3.2-3B	26.9	25.2	29.3	28.7	22.4	31.2
	Gemma3-4b	28.7	27.6	<u>30.8</u>	28	27.5	28.7
	Qwen3-4B	<u>31.5</u>	<u>27.8</u>	27.1	<u>31.8</u>	<u>28.4</u>	<u>32.6</u>
Medium-size	Aya-8b	29.1	28.9	32.3	34.4	25.5	27.9
	Llama3.1-8B	30.5	23.2	32.7	35	24.4	27.6
	Gemma3-12b	35.1	<u>35.9</u>	36.7	33.1	<u>37.1</u>	34.8
	Mistral3.1-24b	35.7	34.3	27.1	33.8	35.8	<u>37</u>
	Gemma3-27b	<u>38.1</u>	32.8	<u>37.2</u>	<u>38.9</u>	36.3	35.7
Large-size	Aya-32b	33.8	36.4	31.4	<u>36.9</u>	27.2	31.3
	Qwen3-32B	34.6	24.6	32.8	29.9	31.9	29.8
	Llama3.3-70B	<u>37</u>	<u>39.7</u>	<u>41.6</u>	34.4	<u>40.7</u>	<u>38.1</u>
	DeepSeek-3.2	43.2	47.2	44.5	<u>45.9</u>	38	<u>37.1</u>
MoE	Qwen3-A3B	<u>39.9</u>	33.1	37.9	<u>40.1</u>	<u>35.6</u>	34
	Llama-4-Scout	37.5	28.3	34.6	35	30.8	26.4
	gpt-oss-120b	37.9	<u>38</u>	<u>38.7</u>	38.2	32.5	<u>37.4</u>
Closed	GPT-5	<u>46.1</u>	41.3	41.4	<u>47.8</u>	43.3	<u>41.6</u>
	Grok4-fast	40.4	39.2	33.7	36.9	38.5	36.9
	Claude-4.5	43.9	39	40	37.6	<u>45.7</u>	36.6
	Gemini-2.5	57.2	65.6	55	59.5	58.5	51.4

Table 7: Question-type wise performance of all models on INDICPARAM.

Zero-Shot Prompting
Prompt = f"""Question: {'question_text'} Options: A) {'option_a'} B) {'option_b'} C) {'option_c'} D) {'option_d'} The above question is written in {language} language. Please analyze the question and options carefully, and select the correct answer. Respond ONLY with one letter (A, B, C, or D) corresponding to the correct option. Do not provide any explanation or additional text.”

Table 8: Zero-Shot prompt applied across all models for evaluation

Model	Gujarati	Konkani	Maithili	Marathi	Oriya	Rajasthani	Sanskrit	Sans-Eng	Dogri	Nepali	Santali
Qwen2.5-3B	<u>29.3</u>	<u>31.5</u>	29.2	28.2	29.1	27.1	31.7	<u>35.5</u>	32.2	27.3	33.5
Llama3.2-3B	26.2	28.2	21.1	28.3	27.4	26.3	27.8	27.9	28	25.1	27.3
Gemma-3-4b	26.3	29.1	23.8	29.2	30.4	25.3	29.5	32.3	30.4	28.2	31.5
Qwen3-4B	27.3	29.3	<u>31.5</u>	<u>29.7</u>	<u>32.6</u>	<u>27.4</u>	<u>34.5</u>	33.6	<u>35.6</u>	<u>28.9</u>	<u>33.9</u>
Aya-8b	27.6	26.6	29	28.3	27.8	31.5	29.7	30.7	31.2	27.2	30.1
Llama-3.1-8B	27.3	29.1	27.9	31.8	28.9	29.3	32.4	34.2	25.9	24.6	30.4
Gemma-3-12b	<u>33.9</u>	35.5	29.7	37.5	33	32.6	39	43.1	36.7	34.8	33.5
Mistral3.1-24b	<u>32.3</u>	31.7	<u>36.3</u>	32.1	28	32.3	<u>41.3</u>	45.2	38	34.8	35.8
Gemma-3-27b	31.3	<u>36</u>	33.2	<u>38.9</u>	<u>37.8</u>	<u>36.9</u>	39.9	<u>49.4</u>	<u>37.8</u>	<u>39.4</u>	<u>36.7</u>
Aya-32b	28.4	32.5	31.7	32.5	28.3	33	36.3	40.6	35.3	32.5	35.5
Qwen3-32B	35.9	34.2	31.5	40	38.5	29.5	40.1	49.5	43.1	33.6	39.7
Llama-3.3-70B	33.6	37.2	35.2	42.7	<u>39.3</u>	36.6	42.8	52.1	37.5	35.8	39
DeepSeek-3.2	<u>39.6</u>	<u>37.3</u>	<u>39.7</u>	<u>43.9</u>	<u>38.7</u>	<u>37.4</u>	<u>53.5</u>	<u>62.9</u>	<u>42.8</u>	<u>39.5</u>	<u>40.7</u>
Qwen3-A3B	30.5	28.3	29.6	34.9	33.9	29.1	38.2	44.7	35.8	30.8	35.3
Llama-4-Scout	27.4	34.4	30	<u>42.1</u>	32.2	32.2	<u>44.3</u>	49.4	30	34	35.7
gpt-oss-120b	<u>37</u>	<u>34.9</u>	<u>33.9</u>	38.7	<u>36.3</u>	<u>35.5</u>	41.5	<u>50.2</u>	<u>38.7</u>	<u>36.1</u>	<u>35.7</u>
GPT-5	<u>40.1</u>	38.5	41.4	<u>47.7</u>	44.1	40.8	<u>57.8</u>	<u>66.3</u>	45.1	41.5	40.1
Grok-4-fast	35.9	37.1	36.5	38.8	38.3	37.1	45.2	55.9	39.9	40.6	38.1
Claude-4.5	37	<u>41.9</u>	40.3	42.9	39.8	39.7	48.8	59.5	44.9	<u>43.4</u>	40.3
Gemini-2.5	51.8	47.6	50.4	54.3	60.9	64	74.4	80.3	52.4	50.4	52.3

Table 9: Performance of models on knowledge related question category.

Model	Gujarati	Konkani	Maithili	Marathi	Oriya	Rajasthani	Sanskrit	Sans-Eng	Dogri	Nepali	Santali
Qwen2.5-3B	<u>66.7</u>	27.3	25.4	20.7	27.4	32.4	<u>32.6</u>	<u>29.7</u>	<u>31.9</u>	26.2	30.3
Llama3.2-3B	16.7	<u>51.5</u>	28.5	34.5	25.6	26.4	28.8	27.9	27.7	30.8	28.3
Gemma-3-4b	16.7	30.3	30.8	43.1	35.9	33.6	27.7	31.5	30.9	<u>32.3</u>	33.3
Qwen3-4B	50	39.4	<u>33.1</u>	<u>46.6</u>	<u>42.7</u>	<u>36.7</u>	30.7	33.3	29.8	27.7	<u>33.3</u>
Aya-8b	50	39.4	38.5	27.6	29.1	29.1	28.4	23.4	33	32.8	<u>36.4</u>
Llama-3.1-8B	16.7	45.5	39.2	39.7	30.8	38.2	31.4	32.4	28.7	29.7	32.3
Gemma-3-12b	16.7	42.4	<u>49.2</u>	56.9	40.2	41.5	29.5	31.5	<u>39.4</u>	34.9	35.4
Mistral3.1-24b	83.3	45.5	46.9	48.3	25.6	41.2	<u>34.8</u>	35.1	<u>37.8</u>	35.4	35.4
Gemma-3-27b	0	<u>57.6</u>	44.6	<u>60.3</u>	<u>47.9</u>	<u>47.6</u>	33.3	<u>46.8</u>	36.7	<u>39</u>	36.4
Aya-32b	<u>33.3</u>	<u>57.6</u>	46.2	46.6	31.6	38.8	31.1	27	34.6	35.4	36.4
Qwen3-32B	33.3	45.5	56.2	56.9	49.6	44.8	36	41.4	44.7	41	40.4
Llama-3.3-70B	16.7	42.4	<u>57.7</u>	58.6	44.4	42.4	37.5	47.7	36.2	<u>43.1</u>	48.5
DeepSeek-3.2	<u>16.7</u>	<u>48.5</u>	56.9	58.6	<u>55.6</u>	<u>47</u>	<u>42.8</u>	<u>49.5</u>	<u>45.7</u>	40.5	<u>47.5</u>
Qwen3-A3B	<u>50</u>	30.3	43.1	48.3	36.8	38.2	30.7	40.5	35.6	34.9	40.4
Llama-4-Scout	16.7	45.5	42.3	70.7	46.2	41.5	<u>42.8</u>	<u>43.2</u>	32.4	36.9	<u>41.4</u>
gpt-oss-120b	33.3	<u>54.5</u>	<u>50.8</u>	60.3	<u>53.8</u>	<u>49.7</u>	34.8	31.5	<u>44.1</u>	<u>41.5</u>	35.4
GPT-5	<u>33.3</u>	<u>60.6</u>	<u>60.8</u>	<u>63.8</u>	<u>66.7</u>	<u>54.5</u>	<u>43.2</u>	<u>51.4</u>	<u>48.4</u>	<u>51.8</u>	43.4
Grok-4-fast	33.3	51.5	52.3	60.3	50.4	44.2	33.7	40.5	41.5	42.6	<u>44.4</u>
Claude-4.5	33.3	51.5	60.8	63.8	<u>60.7</u>	<u>55.8</u>	<u>43.6</u>	46.8	44.1	<u>45.6</u>	41.4
Gemini-2.5	33.3	78.8	85.4	84.5	78.6	73.3	64.4	80.2	62.2	59	57.6

Table 10: Performance of models on language understanding (LU) question category.

13 Examples questions in *INDICPARAM*

Table 11 presents examples of six distinct types of questions for each language used in *INDICPARAM*.

Question	option(a)	option(b)	option(c)	option(d)	Ans	Type	Class	Lang
1. बालिवधस्य वर्णनमस्ति रामायणस्य	सुन्दरकाण्डे	किष्किन्धा काण्डे	अरण्यकाण्डे	बालकाण्डे	b	MCQ	G	san
2. 'मन्त्रिपरिषदं द्वादशमात्यान्कुर्वीत' इति कस्य मान्यता ?	बार्हस्पत्यानाम्	कौटिल्यस्य	औशनसाम्	मानवानाम्	d	MCQ	G	san
3. अधोऽडिकतानां समीचीनमुत्तरं चिनुत - (a) सरमा-पणि 1. बृहदारण्यकोप-सम्वादः निषत् (b) स्वाध्यायान्मा 2. ऋग्वेदस्य प्रमदः दशममण्डले (c) कल्पः 3. तैत्तिरीयोपनिषत् (d) आत्मनस्तु 4. हस्तः कामाय सर्वं प्रियं भवति (a) (b) (c) (d)	1 3 2 4	4 2 3 1	2 3 4 1	3 2 1 4	c	Order	G	san
4. भाशा दी सभर्ने थमां लौहकी इकाई ऐ :	ध्वनि	ध्वनिग्राम	रूपग्राम	वाक्य	a	MCQ	L	doi
5. पैहली चंदी च दिती गे दियें प्रविष्टियों दा दूई चंदी दियें प्रविष्टियें कन्नै स्हेई मिलान करो : चंदी-1 चंदी-2 (अ) डोगरी काव्य-चर्चा (i) प्रो. रामनाथ शास्त्री (ब) परख-पड़ताल (ii) शिवनाथ (स) डोगरी साहित्य दा इतिहास (iii) प्रो. चम्पा शर्मा (द) बाबा जित्तो (iv) ओम गोस्वामी कोड : (अ) (ब) (स) (द)	(ii) (i) (iv) (ii) (iii) (iv) (i) (iii) (iv) (ii)	(iii) (iv) (i) (iii) (ii) (i) (iv) (ii)	(i) (iii) (iv) (ii)	(i) (iii) (iv) (ii)	c	LM	G	doi
6. 'इक लड़ाई होर' ते 'बंजर' दे रचेता न :	मदन मोहन शर्मा ते मोहन सिंह ।	मोहन सिंह ते नरसिंह देव जम्वाल ।	मोहन सिंह ते मदन मोहन शर्मा ।	जितेन्द्र शर्मा ते दीनू भाई पंत ।	c	Order	G	doi

7. બન્ને યાદીની વિગતો સર-ખાવી સાચાં જોડકાં બનાવો : (a) શેષાદ્રિ (i) વિજયરાય વૈદ્ય (b) રામ વૃંદાવની (ii) ત્રિભુ-વનદાસ લુહાર (c) ત્રિશુળ (iii) રાજેન્દ્ર શાહ (d) મયૂરાનંદ (iv) ખબરદાર (a) (b) (c) (d)	(i) (iv) (iii) (ii)	(iv) (iii) (ii) (i)	(ii) (iv) (iii) (i)	(iii) (ii) (i) (iv)	b	LM	G	guj
8. નીચેનાં વિધાનોને કાર્યકાર-ણસંબંધે તપાસો : (A) 'વદતોવ્યાઘાત' મધ્યમપદલોપી સમાસ છે. (R) મધ્યમપદલોપી સમાસ સર્વપદપ્રધાન છે.	(A) અને (R) બન્ને સાચાં છે.	(A) અને (R) બન્ને ખોટાં છે.	(A) સાચું છે અને (R) ખોટું છે.	(A) ખોટું છે અને (R) સાચું છે.	b	A&R	G	guj
9. નીચેનામાંથી સુસંગત વિગત-જૂથ જણાવો :	વીસમી સદી, વૈશ્વાનર, નટમંડળ	ગુજરાત વિદ્યા- સભા, ગુજરાતી સાહિત્ય પરિષદ, ગુજરાત સાહિત્ય અકાદમી	ઊડણ ચરકલડી, સોયનું નાકું, મનીષા	ખીચડી, અંતઃસ્ત્રોતા, આવતી- કાલનો સૂરજ	b	MCQ	G	guj
10. ફાવો તો પર્યાય વૈંચૂન કાઢૂન વાક્ય પૂર્ણ કરાત. સોळाव्या शेंकड्यातल्या रामायणाच्या हातबरपांत	ठांयीं ठांयीं पुर्तुगेज उतरां मेळटात.	कांयच पुर्तुगेज उतरां मेळनात.	संस्कृत उतरां भरसून पुर्तुगेजीक कोंकणीची सया मारता.	देशी आनी अपभ्रंशी भारतीय भासो मेळटात.	b	Blank fill- ing	G	gom
11. भाशीक नादांचो भाशेचे बांदावळीचे नदरेंतल्यान अभ्यास करपी व्याकरणाक किर्ते म्हणतात ?	अर्थविज्ञान	नादविज्ञान	वाक्य- विचार	स्वनीम- विचार	d	MCQ	G	gom
12. 'काळोकिट' हो समास हांतलो खंयचो ?	उपमावाचक कर्मधारय समास	विशेषण उभयपद समास	द्वंद्व समास	बहुव्रीहि समास	b	MCQ	L	gom
13. हुनकासँ भेंट भेल छल' लिखने छथि	मणिपद्म	अमर	सुमन	मधुप	a	MCQ	G	mai
14. "शिव की विष पचाय यदि लितथि नहि माथे ।"	देखितथि	बुझितथि	करितथि	सकितथि	d	Blank fill- ing	G	mai
15. 'एकावली परिणय' में अंगीरस अछि	शान्त	अद्भुत	हास्य	श्रृंगार	d	MCQ	L	mai

16. पुढीलपैकी कोणते नाटक विजय तेंडुलकर यांनी लिहिले नाही ?	गृहस्थ	सूर्यास्त	बेबी	श्रीमंत	b	MCQ	G	mar
17. पुढीलपैकी कोणती कादंबरी ऐतिहासिक नाही ?	वज्राघात	कालिकामूर्ति	सावळ्या तांडेल	दुर्देवी रंगू	b	IS	G	mar
18. पुढील साहित्यकृतींचा कालानुसार क्रम लावा.	धग, किडलेली माणसे, औदुंबर, स्मृतिचित्रे,	किडलेली माणसे, औदुंबर, स्मृतिचित्रे, धग,	धग, औदुंबर, स्मृतिचित्रे, किडलेली माणसे,	औदुंबर, स्मृतिचित्रे, किडलेली माणसे, धग,	d	Order	G	mar
19. शास्त्रीय मार्क्सवादी दृष्टिमा आइडियोलजी भन्नाले के बुझिन्छ ?	भ्रमपूर्ण चेतना ।	सैद्धान्तिक आस्था	प्रचलित विचारधारा ।	पुस्तकहरू पढेर पाइने ज्ञान ।	a	MCQ	G	nep
20. वाक्यमा शब्दहरू माझ रहने सम्बन्धलाई के भनिन्छ ?	विन्यासक्रमी सम्बन्ध ।	सहचारक्रमी सम्बन्ध ।	व्यतिरेकी सम्बन्ध ।	परस्परव्यापि सम्बन्ध ।	a	MCQ	L	nep
21. नेपाली भाषामा प्रयोग हुने 'चोलो', 'पटुका', 'मुन्धुम', 'बाउसे' शब्दहरू कुन भाषा परिवारबाट आएका हुन् ?	भारोपेली ।	भोट-बर्मेली ।	आग्नेय ।	द्रविड ।	b	MCQ	L	nep
22. "राजस्थानी भाषा रौ विगसाव नागर अपभ्रंश सूं हुयौ है।" ओ कथन है -	डॉ. एल.पी. टेस्सिटोरी	डॉ. ग्रियर्सन	डॉ. सुनीतिकुमार चटर्जी	डॉ. सुकुमार सेन	b	MCQ	G	Raj
23. राजस्थानी अनुवाद री दीठ सूं किसौ अनुवाद रौ जोड़ौ गलत है ?	गांधीजी री आत्मकथा - आईदानसिंह भाटी	भरथरी शतक - मनोहर शर्मा	मेघदूत - नारायणसिंह भाटी	रचाव - चेतन स्वामी	d	IS	G	Raj
24. बुगचौ' सबद रौ सही अर्थ है :	लुगाइयां रै कपड़ा राखण रौ कपड़े रौ थेलौ	लुगाइयां री लकड़ी री पेटी	लुगाइयां री कपड़ा राखण री लोह री पेटी	मड़दां रै कपड़ा राखण री लकड़ी री पेटी	a	MCQ	L	Raj
25. स हि कदाचिद् वाच्ये विधिरूपे प्रतिषेधरूपः' अत्र स इत्यनेन कोऽभिप्रेतः ?	वाच्यार्थः	लक्ष्यार्थः	व्यङ्ग्यार्थः	तात्पर्यार्थः	c	MCQ	L	sans-mix
26. कौषीतकि उपनिषद् कस्य वेदस्य अस्ति ? कौषीतकि उपनिषद् किस वेद की है ? Of which Veda is कौषीतकि उपनिषद् ?	अथर्ववेदस्य	सामवेदस्य	ऋग्वेदस्य	कृष्णयजुर्वेदस्य	c	MCQ	G	sans-mix

27. ଅଧୋଲିଖିତେଷୁ କେନ ସହ କସ୍ୟ ସମ୍ବନ୍ଧ: ? ତାଲିକା ଚିନୁତ - (a) ସତ୍କାର୍ଯ୍ୟବାଦ: (i) ନ୍ୟାୟବୈଶେଷିକାଣାମ୍ (b) ପରମାଣୁବାଦ: (ii) ବୌଦ୍ଧାନାମ୍ (c) ବିବର୍ତ୍ତବାଦ: (iii) ସଂଖ୍ୟାନାମ୍ (d) ବିଜ୍ଞାନବାଦ: (iv) ଅଦ୍ୱୈତବେଦାନ୍ତିନାମ୍ (a) (b) (c) (d)	(iii) (ii) (i) (iv)	(iii) (iv) (i) (ii)	(iii) (i) (iv) (ii)	(iii) (i) (ii) (iv)	c	LM	G	sans- mix
28. ସେଦାୟ ହାପଡ଼ାମକୋ କାଥା ଲେକାତେ ହୋଝ ହୋପୋନକୋବାକ୍ ପାରିସ ଦୋ ଓକୋ ଜାୟଗା ରେ କୋ ହାଟିଜ ଲେଦା ?	ଚାମ୍ପା	ଚାୟ	ସାସାଡବେଡା	ଜାରପୀ	c	MCQ	G	sat
29. ଲାତାର ରେ ଓଲ ଆକାନ ଥାପିତ କାଥା (A) ଆର ଓଜେ (R) ରେନାକ୍ ଓକା ଗାଦେଲ ବାଝୁତୀ ସୁହିୟା ? ଥାପିତ (A) : ଭାରତୀୟ ନୋଜୋର ତେ ଗାୟାନ ସାଁବହେତୁ ଦୋ ସୁକ ରେ ମୁଘାତ ଲାକତୀୟାନା । ଓଜେ (R) : ଚେଦାକ୍ ଜେ ନୋବା କାଥା ହୋଁ ପଞ୍ଚିମ ଦିସୋମ ରେନ ସାଁପହେତୁହିୟା ନୋଜୋର ତେ ହୋଁ ନାପାୟା ।	(A) ଆର (R) ବାଝୁସୁହିୟା	(A) ସୁହିୟା, (R) ବାଝୁସୁହିୟା	(A) ବାଝୁସୁହିୟା, (R) ସୁହିୟା	(A) ଆର (R) ବାନାର ସୁହିୟା	d	A&R	G	sat
30. ଓକା ପାରସୀ ଘାରୋଚୁ ରେନାକ୍ ଓକା ପାରସୀ କାନା ? ଓନା ସୁହି ମିଲାବ ମେଁ । ସୂଚୀ - I ସୂଚୀ - II a.ଆର୍ଯ୍ୟ ଭାଷା ପରିବାର 1. ତିବ୍ବତୀ b. ଅଗ୍ନେୟ ଭାଷା ପରିବାର 2. ମାଲ୍ତୋ c. ଦ୍ରାବିଡ଼ ଭାଷା ପରିବାର 3. ମୁଣ୍ଡା d. ଚୀନୀ ଭାଷା ପରିବାର 4. ବଂଗଳା କୂଟ : a b c d	1 2 3 4	4 3 2 1	3 1 4 2	2 4 3 1	b	LM	G	sat
31. ମୁକୁ ଅକ୍ଷର ଶେଷରେ କଣ ଥାଏ ?	ଅକ୍ଷର	ସୂରଧୂନି	ବ୍ୟଞ୍ଜନଧୂନି	ସୂର୍ଜଧୂନି	b	MCQ	L	ory
32. ପ୍ରକାଶକାଳ ଅନୁଯାୟୀ ଉପଯୁକ୍ତ କ୍ରମ ନିର୍ଦ୍ଧାରଣ କର ।	ଉଷା, ଚିଲିକା, ପାର୍ବତୀ, ମହାଯାତ୍ରା	ଚିଲିକା, ଉଷା, ପାର୍ବତୀ, ମହାଯାତ୍ରା	ପାର୍ବତୀ, ଚିଲିକା, ଉଷା, ମହାଯାତ୍ରା	ଉଷା, ପାର୍ବତୀ, ଚିଲିକା, ମହାଯାତ୍ରା	d	Order	G	ory
33. କେଉଁଟି ବ୍ୟୋମକେଶ ତ୍ରିପାଠୀଙ୍କ ନାଟକ ନୁହେଁ ?	ସିଂହଦ୍ୱାର	ଜାଗରଣ	ମୁଠାଏମାଟି	ସୁନାଫରୁଆ	c	IS	G	ory

34. बाहायनाने गाहायाव होनाय मेथाइफोरनि बाहायजानाय सम फारि साजायनायखौ सायख - I. बैसागो मेथाइ II. दोहोरोम सिबिनाय मेथाइ III. बेलाड IV. मैगं खानायाव खननाय मेथाइ	II III IV I	III III IV	IV III I II	I III IV III	b	Order	G	brx
35. अलंबार' लाइसिखौ सोर सुजुर्दोमोन	मदाराम ब्रह्म	सुकुमार बसुमतारी	सतिस चन्द्र बसुमतारी	प्रम'द चन्द्र ब्रह्म	d	MCQ	G	brx
36. गाहायनि बुंफोरनाय (A) आरो जाहोन (R) खौ गेबें ना गोरान्थि बाहायनानै सायख : बुंफोरनाय (A) : आथिखालाव मोननाय बासिराम जोहोलाव बेलाडखौ थारै आबुं बेलाड एबा natural ballad बुंथावा । जाहोन (R) : मानोना बे बेलाडा लिरनाय महराव फोसावजानाय ।	(A)-आरों (R) मोननैबो गेबें	(A)-आ गेबें आरो (R)-आ गोरान्थि	(A) आरो (R) मोननैबो गोरान्थि	(A)-आ गोरान्थि आरो (R) आ गेबें	b	A&R	G	brx

Table 11: Sample questions from all languages in INDICPARAM along with their options, answers, question types, and question classes, with languages indicated using ISO-639 codes. We use *Order* to denote ordering-type questions, and the abbreviations *G* and *L* refers to General Knowledge (GK) and Language Understanding (LU) questions respectively.

TransLIST: A Transformer-Based Linguistically Informed Sanskrit Tokenizer

Jivnesh Sandhan¹, Rathin Singha², Narein Rao¹, Suvendu Samanta¹,
Laxmidhar Behera^{1,4} and Pawan Goyal³

¹IIT Kanpur, ²UCLA, ³IIT Kharagpur, ⁴IIT Mandi

jivnesh@iitk.ac.in, rsingha108@g.ucla.edu,
nrao20@iitk.ac.in, pawang@cse.iitkgp.ac.in

Abstract

Sanskrit Word Segmentation (SWS) is essential in making digitized texts available and in deploying downstream tasks. It is, however, non-trivial because of the *sandhi* phenomenon that modifies the characters at the word boundaries, and needs special treatment. Existing *lexicon driven* approaches for SWS make use of Sanskrit Heritage Reader, a lexicon-driven shallow parser, to generate the complete candidate solution space, over which various methods are applied to produce the most valid solution. However, these approaches fail while encountering out-of-vocabulary tokens. On the other hand, *purely engineering* methods for SWS have made use of recent advances in deep learning, but cannot make use of the latent word information on availability.

To mitigate the shortcomings of both families of approaches, we propose **Transformer based Linguistically Informed Sanskrit Tokenizer** (TransLIST) consisting of (1) a module that encodes the character input along with latent-word information, which takes into account the *sandhi* phenomenon specific to SWS and is apt to work with partial or no candidate solutions, (2) a novel soft-masked attention to prioritize potential candidate words and (3) a novel path ranking algorithm to rectify the corrupted predictions. Experiments on the benchmark datasets for SWS show that TransLIST outperforms the current state-of-the-art system by an average 7.2 points absolute gain in terms of perfect match (PM) metric.¹

1 Introduction

Sanskrit is considered as a cultural heritage and knowledge preserving language of ancient India. The momentous development in digitization efforts has made ancient manuscripts in Sanskrit readily available for the public domain. However, the usability of these digitized manuscripts is limited

due to linguistic challenges posed by the language. SWS conventionally serves the most fundamental prerequisite for text processing step to make these digitized manuscripts accessible and to deploy many downstream tasks such as text classification (Sandhan et al., 2019; Krishna et al., 2016b), morphological tagging (Gupta et al., 2020; Krishna et al., 2018), dependency parsing (Sandhan et al., 2021; Krishna et al., 2020a), automatic speech recognition (Kumar et al., 2022) etc. SWS is not straightforward due to the phenomenon of *sandhi*, which creates phonetic transformations at word boundaries. This not only obscures the word boundaries but also modifies the characters at juncture point by deletion, insertion and substitution operation. Figure 1 illustrates some of the syntactically possible splits due to the language-specific *sandhi* phenomenon for Sanskrit. This demonstrates the challenges involved in identifying the location of the split and the kind of transformation performed at word boundaries.

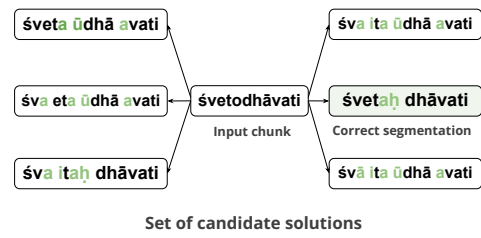


Figure 1: An example to illustrate challenges posed by *sandhi* phenomenon for SWS task.

The recent surge in SWS datasets (Krishna et al., 2017; Krishnan et al., 2020) has led to various methodologies to handle SWS. Existing *lexicon-driven* approaches rely on a *lexicon driven* shallow parser, popularly known as Sanskrit Heritage Reader (SHR) (Goyal and Huet, 2016a).² This line of approaches (Krishna et al., 2016a, 2018, 2020b)

¹The codebase and datasets are publicly available at: <https://github.com/rsingha108/TransLIST>

²<https://sanskrit.inria.fr/DICO/reader.fr.html>

formulate the task as finding the most accurate semantically and syntactically valid solution from the candidate solutions generated by SHR. With the help of the significantly reduced exponential search space provided by SHR and linguistically involved feature engineering, these *lexicon driven* systems (Krishna et al., 2020b, 2018) report close to state-of-the-art performance for the SWS task. However, these approaches rely on the completeness assumption of SHR, which is optimistic given that SHR does not use domain specific lexicons. These models are handicapped by the failure of this preliminary step. On the other hand, *purely engineering* based knowledge-lean data-centric approaches (Hellwig and Nehrdich, 2018; Reddy et al., 2018; Aralikatte et al., 2018) perform surprisingly well without any explicit hand-crafted features and external linguistic resources. These *purely engineering* based approaches are known for their ease of scalability and deployment for training/inference. However, a drawback of these approaches is that they are blind to latent word information available through external resources.

There are also lattice-structured approaches (Zhang and Yang, 2018; Gui et al., 2019; Li et al., 2020) (originally proposed for Chinese Named Entity Recognition (NER), which incorporate lexical information in character-level sequence labelling architecture). However, these approaches cannot be directly applied for SWS; since acquiring word-level information is not trivial due to *sandhi* phenomenon. To overcome these shortcomings, we propose **Transformer-based Linguistically Informed Tokenizer (TransLIST)**. TransLIST is a perfect blend of *purely engineering* and *lexicon driven* approaches for the SWS task and provides the following advantages: (1) Similar to *purely engineering* approaches, it facilitates ease of scalability and deployment during training/inference. (2) Similar to *lexicon driven* approaches, it is capable of utilizing the candidate solutions generated by SHR, which further improves the performance. (3) Contrary to *lexicon driven* approaches, TransLIST is robust and can function even when candidate solution space is partly available or unavailable.

Our key contributions are as follows: (a) We propose the linguistically informed tokenization module (§ 2.1) which accommodates language-specific *sandhi* phenomenon and adds inductive bias for the SWS task. (b) We propose a novel soft-masked

attention (§ 2.2) that helps to add inductive bias for prioritizing potential candidates keeping mutual interactions between all candidates intact. (c) We propose a novel path ranking algorithm (§ 2.3) to rectify the corrupted predictions. (d) We report an average 7.2 points perfect match absolute gain (§ 3) over the current state-of-the-art system (Hellwig and Nehrdich, 2018).

We elucidate our findings by first describing TransLIST and its key components (§ 2), followed by the evaluation of TransLIST against strong baselines on a test-bed of 2 benchmark datasets for the SWS task (§ 3). Finally, we investigate and delve deeper into the capabilities of the proposed components and its corresponding modules (§ 4).

2 Methodology

In this section, we will examine the key components of TransLIST which includes a linguistically informed tokenization module that encodes character input with latent-word information while accounting for SWS-specific *sandhi* phenomena (§ 2.1), a novel soft-masked attention to prioritise potential candidate words (§ 2.2) and a novel path ranking algorithm to correct mispredictions (§ 2.3).

2.1 Linguistically Informed Sanskrit Tokenizer (LIST)

Lexicon driven approaches for SWS are brittle in realistic scenarios and *purely engineering* based approaches do not consider the potentially useful latent word information. We propose a win-win/robust solution by formulating SWS as a character-level sequence labelling integrated with latent word information from the SHR as and when available. TransLIST is illustrated with an example *śvetodhāvati* in Figure 2. SHR employs a Finite State Transducer (FST) in the form of a lexical juncture system to obtain a compact representation of candidate solution space aligned with the input sequence. As shown in Figure 2(a), we receive the candidate solution space from the SHR engine. Here, *śvetah dhāvati* and *śveta ūdhā avati* are two syntactically possible splits.³ It does not suggest the final segmentation. The candidate space includes words such as *śva*, *śvetah* and *etaḥ* whose boundaries are modified with respect to the input sequence due to *sandhi* phenomenon. SHR gives us mapping (head and tail position) of all

³Only some of the solutions are shown for visualization.

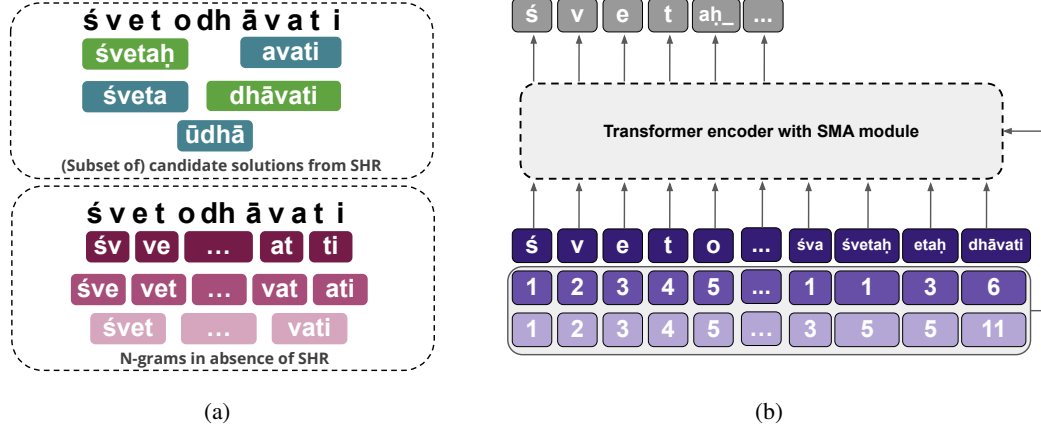


Figure 2: Illustration of TransLIST with a toy example “śvetodhāvati”. Translation: “The white (horse) runs.” (a) LIST module: We use the candidate solutions (two possible candidate solutions are highlighted with ■, ■ colors where the latter is the gold standard) from SHR if available; in the absence of SHR, we resort to using n-grams ($n \leq 4$). (b) TransLIST architecture: In span encoding, each node is represented by head and tail position index of its character in the input sequence. ■, ■, ■ denote tokens, heads and tails, respectively. The SHR helps to include words such as śva, śvetah and etaḥ whose boundaries are modified with respect to input sequence due to sandhi phenomenon. Finally, on the top of the Transformer encoder, classification head learns to predict gold standard output shown by ■ for the corresponding input character nodes only.

the candidate nodes with the input sequence. In case such mapping is incorrect for some cases, we rectify it with the help of deterministic algorithm by matching candidate nodes with the input sentence and finding the closest match. In the absence of SHR, we propose to use all possible n-grams ($n \leq 4$)⁴ which helps to add inductive bias about neighboring candidates in the window size of 4.⁵ We feed the candidate words/n-grams to the Transformer encoder and the classification head learns to predict gold standard output for the corresponding input character nodes only. The output vocabulary consists of unigram characters (e.g., ś, v), bigrams and tri-grams (e.g., aḥ_). The output vocabulary contains ‘_’ to represent spacing between words. Consequently, TransLIST is capable of using both character-level modelling as well as latent word information as and when available. On the other hand, *purely engineering* approaches rely only on character-level modelling and *Lexicon driven* approaches rely only on word-level information from SHR to handle *sandhi*.

2.2 Soft Masked Attention (SMA)

Transformers (Vaswani et al., 2017) have been proven to be effective for capturing long-distance dependencies in a sequence. The self-attention property of a Transformer facilitates effective interaction between character and available latent word information. There are two preliminary prerequisites for effective modelling of inductive bias for tokenization: (1) Allow interactions between the candidate words/characters within and amongst chunks. (2) Prioritize candidate words containing the input character for which a prediction is being made (e.g., in Figure 2(b), śva and śvetah are prioritized amongst the candidate words when predicting for the character ś).⁶ The vanilla self-attention (Vaswani et al., 2017) can address both the requirements; however, it has to self-learn the inductive bias associated with prioritisation. It may not be an effective solution in low-resourced settings. On the other hand, if we use hard-masked attention to address the second prerequisite, we lose mutual interactions between the candidates. Hence, we propose a novel soft-masked attention which helps to address both the requirements effectively. To the best of our knowledge, there is no existing soft-masked attention similar to ours. We formally discuss this below.

⁶We find that failing to meet any one of the prerequisites leads to drop in performance (§ 4).

⁴We do not observe significant improvements for $n > 4$.

⁵Our probing analysis (Figure 4) suggests that char-char attention mostly focuses on immediate neighbors. Refer to § 4 for detailed ablations on LIST variants.

Self-attention maps a query and a set of key-value pairs to an output as discussed in Vaswani et al. (2017). For an input $x = (x_1, \dots, x_n)$ where $x_i \in R^{d_x}$, self-attention gives an output $z = (z_1, \dots, z_n)$ where $z_i \in R^{d_z}$. We presume the standard formulation of vanilla self-attention (Vaswani et al., 2017) where d_x is the dimension of input word representation and d_z is the projection dimension. Here, $W^Q, W^K, W^V \in R^{d_x \times d_z}$ are parameter matrices. For simplicity, we ignore multi-head attention in equations 1, 2 and 3.

$$z_i = \sum_{j=1}^n \alpha_{ij} (x_j W^V) \quad (1)$$

$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{k=1}^n \exp(e_{ik})} \quad (2)$$

$$e_{ij} = \frac{(x_i W^Q)(x_j W^K)^T}{\sqrt{d_z}} \quad (3)$$

In **soft-masked attention**, we provide a prior about interactions between candidate words and the input characters using a span encoding ($s_{ij} \in R^{d_z}$) (Li et al., 2020). Intuitively, it helps inject inductive bias associated with prioritisation whilst maintaining mutual interactions between the candidates.

Formally, we modify Equation 2 to define soft masked attention as:

$$\alpha_{ij}^{SM} = \frac{M_{ij} \exp(e_{ij})}{\sum_{k=1}^n M_{ik} \exp(e_{ik})} \quad (4)$$

where $M \in R^{n \times n}$, $M_{ij} \in [0, 1]$. M_{ij} is defined as:

$$M_{ij} = \frac{(x_i W^Q)(s_{ij} W^R)^T}{\sqrt{d_z}} \quad (5)$$

$W^R \in R^{d_z \times d_z}$ is a learnable parameter which projects s_{ij} into a location-based key vector space. Summarily, the proposed SMA module helps to prioritize potential candidate words with the help of separation, inclusion and intersection information between nodes. Finally, we calculate the output z with the help of the proposed SMA as follows:

$$z_i = \sum_{j=1}^n \alpha_{ij}^{SM} (x_j W^V) \quad (6)$$

Next, we discuss the span position encoding.

Span position encoding is one of the backbones of the proposed soft-masked module. It is utilized to capture the interactions between the candidate words and the sequence of input characters. Each

span/node (which is a character/word and its corresponding position in the input sentence) is represented by the head and tail which denote the position index of the initial and final characters of the token in the input sequence, as shown in Figure 2(b). The span of character is characterized by the same head and tail position index. For example, $head[i]$ and $tail[i]$ represent the head and tail index of span x_i , respectively. The separation, inclusion and intersection information between nodes x_i and x_j can be captured by the four distance equations 7-10.

$$d_{ij}^{(hh)} = head[i] - head[j] \quad (7)$$

$$d_{ij}^{(ht)} = head[i] - tail[j] \quad (8)$$

$$d_{ij}^{(th)} = tail[i] - head[j] \quad (9)$$

$$d_{ij}^{(tt)} = tail[i] - tail[j] \quad (10)$$

The final span encoding is a non-linear transformation of these 4 distances:

$$s_{ij} = \text{ReLU}(w_s(p_{d_{ij}^{(hh)}} \oplus p_{d_{ij}^{(ht)}} \oplus p_{d_{ij}^{(th)}} \oplus p_{d_{ij}^{(tt)}})) \quad (11)$$

where $w_s \in R$ is a learnable parameter, $\oplus$ is a concatenation operation and $p_d \in R^{\frac{d_z}{4}}$ is a sinusoidal position encoding similar to Vaswani et al. (2017).

2.3 Path Ranking for Corrupted Predictions (PRCP)

Our error analysis (§ 4) suggests that sometimes the proposed system predicts words that are not part of the candidate solution space. These mistakes can be rectified with the help of SHR’s candidate solutions by appropriately substituting suitable candidates. We refer to the prediction corresponding to a chunk that does not fall in the candidate solution space, as a *corrupted prediction* and define a *path* as the sequence of characters in a candidate solution for a given input. We enumerate all the possible directed paths (In Figure 2(a), two possible candidate solutions are highlighted with  and  colors) corresponding to the input (with a corrupted prediction) and formulate the task as a *path ranking problem*. While designing the path scoring function (S), we consider the following criteria: (1) Select a path consisting of semantically coherent candidate words. We use an integrated judgment from two sources. First, we prefer a path having a high log-likelihood (LL) score as per TransLIST

to choose a semantically coherent path in line with the contextual information of TransLIST. Second, we reinforce the scoring function (S) by considering the perplexity score (ρ) for the path from the character-level language model. (2) To avoid paths consisting of over-generated segmentation provided by SHR, we use a penalty proportional to the number of words ($|W|$) present in the path to prefer paths with less number of words. This gives us the following path scoring function (S):

$$S = \frac{LL_{TransLIST}}{\rho_{CharLM} \times |W|}$$

where

$LL_{TransLIST}$ = log-likelihood by TransLIST

ρ_{CharLM} = Perplexity score by CharLM

$|W|$ = Number of words present in path

3 Experiments

Data and Metrics: Currently, Digital Corpus of Sanskrit (Hellwig, 2010, DCS) has more than 600,000 morphologically tagged text lines. It consists of digitized constructions composed in prose or poetry over a wide span of 3000 years. Summarily, DCS is a perfect representation of various writing styles depending on time and domains. We use two available benchmark datasets (Krishna et al., 2017, SIGHUM)⁷ and (Krishnan et al., 2020, Hackathon) for SWS. Both datasets are subset of DCS (Hellwig, 2010). These datasets also come with candidate solution space generated by SHR for SWS. We prefer Krishna et al. (2017, SIGHUM) over a relatively larger dataset (Hellwig and Nehrdich, 2018) to obviate the time and efforts required for obtaining candidate solution space. We obtain the ground truth segmentation solutions from DCS. We could not use DCS10k (Krishna et al., 2020b) due to partly missing gold standard segmentation (inflections) for almost 50% data points. SIGHUM consists of 97,000, 3,000 and 4,200 sentences as train, dev, test set, respectively. Similarly, Hackathon consists of 90,000, 10,332 and 9,963 sentences as train, dev and test set, respectively. We use the following word-level evaluation metrics: macro-averaged Precision (P), Recall (R), F1-score (F) and the percentage of sentences with perfect matching (PM).

⁷<https://zenodo.org/record/803508#.YRdZ43UzaXJ>

Hyper-parameter settings: For the implementation of TransLIST, we build on top of codebase by Li et al. (2020). We use the following hyper-parameters for the best configuration of TransLIST: number of epochs as 50 and a dropout rate of 0.3 with a learning rate of 0.001. We release our codebase and datasets publicly under the Apache license 2.0. All the artifacts used in this work are publicly available for the research purpose. For all the systems, we do not use any pretraining. All the input representations are randomly initialized. We use GeForce RTX 2080, 11 GB GPU memory computing infrastructure for our experiments.

Baselines: We consider two *lexicon-driven* approaches where Krishna et al. (2016a, SupPCRW) formulate SWS as an iterative query expansion problem and Krishna et al. (2018, Cliq-EBM) deploy a structured prediction framework. Next, we evaluate four *purely-engineering* based approaches, namely, Encoder-Decoder framework (Reddy et al., 2018, Seq2Seq), character-level sequence labelling system with combination of recurrent and convolution element (Hellwig and Nehrdich, 2018, rcNN-SS), vanilla **Transformer** (Vaswani et al., 2017) and character-level Transformer with relative position encoding (Yan et al., 2019, TENER). Finally, we consider lattice-structured approaches originally proposed for Chinese NER which incorporate lexical information in character-level sequence labelling architecture. These approaches consist of lattice-structured LSTM (Zhang and Yang, 2018, **Lattice-LSTM**), graph neural network (GNN) based architecture (Gui et al., 2019, **Lattice-GNN**) and Transformer based architecture (Li et al., 2020, **FLAT-Lattice**). **TransLIST:** As per § 2.1, we report two variants: (a) TransLIST_{ngrams} which makes use of only n-grams, and (b) TransLIST which makes use of SHR candidate space.

Results: Table 1 reports the results for the best performing configurations of all the baselines on the test set of benchmark datasets for the SWS task.⁸ Except *purely engineering* based systems (Seq2seq, TENER, Transformer and rcNN-SS), all systems leverage linguistically refined candidate solution space generated by SHR. Among the lattice-structured systems, FLAT-Lattice demonstrates competing performance against rcNN-SS.

⁸We do not compare with recently proposed variant of Clique-EBM (Krishna et al., 2020b) and seq2seq baseline (Aralikatte et al., 2018) due to unavailability of codebase. Also, they do not report performance on these two datasets.

	SIGHUM				Hackathon			
Model	P	R	F	PM	P	R	F	PM
Seq2seq	73.44	73.04	73.24	29.20	72.31	72.15	72.23	20.21
SupPCRW	76.30	79.47	77.85	38.64	-	-	-	-
TENER	90.03	89.20	89.61	61.24	89.38	87.33	88.35	49.92
Lattice-LSTM	94.36	93.83	94.09	76.99	91.47	89.19	90.31	65.76
Lattice-GNN	95.76	95.24	95.50	81.58	92.89	94.31	93.59	70.31
Transformer	96.52	96.21	96.36	83.88	95.79	95.23	95.51	77.70
FLAT-Lattice	96.75	96.70	96.72	85.65	<u>96.44</u>	<u>95.43</u>	<u>95.93</u>	<u>77.94</u>
Cliq-EBM	96.18	<u>97.67</u>	<u>96.92</u>	78.83	-	-	-	-
rcNN-SS	<u>96.86</u>	96.83	96.84	<u>87.08</u>	96.40	95.15	95.77	77.62
TransLIST _{ngrams}	96.97	96.77	96.87	86.52	96.68	95.74	96.21	79.28
TransLIST	98.80	98.93	98.86	93.97	97.78	97.44	97.61	85.47

Table 1: Performance evaluation between baselines in terms of P, R, F and PM metrics. The significance test between the best baselines, rcNN-ss, FLAT-lattice and TransLIST in terms of recall/perfect-match metrics: $p < 0.05$ (as per t-test, for both the datasets). We do not report the performance of SupPCRW and Cliq-EBM on Hackathon dataset due to unavailability of codebase. On SIGHUM, we report numbers from their papers. The best baseline’s results for the corresponding datasets are underlined. The overall best results per column are highlighted in bold.

We find that rcNN-SS and FLAT-Lattice perform the best among all the baselines on SIGHUM and Hackathon datasets, respectively.

Both the TransLIST variants outperforms all the baselines in terms of all the evaluation metrics with TransLIST providing an average 1.8 points (F) and 7.2 points (PM) absolute gain with respect to the best baseline systems, rcNN-SS (on SIGHUM) and FLAT-Lattice (on Hackathon). Even when the SHR candidate space is not available, the proposed system can use TransLIST_{ngrams}, which provides an average 0.11 points (F) and 0.39 points (PM) absolute gain over the best baselines. TransLIST_{ngrams} gives comparable performance to rcNN-SS on SIGHUM dataset, while on the Hackathon dataset, it performs significantly better than FLAT-Lattice ($p < 0.05$ as per t-test). The wide performance gap between TransLIST and TransLIST_{ngrams} demonstrates the effectiveness of using SHR candidate space, when available. Summarily, we establish a new state-of-the-art results with the help of meticulously stitched LIST, SMA and PRCP modules. The knowledge of the candidate space by SHR gives an extra advantage to TransLIST. Otherwise, natural choice is the proposed purely engineering variant TransLIST_{ngrams} when that is not available.

4 Analysis

In this section, we investigate various questions to dive deeper into the proposed components and

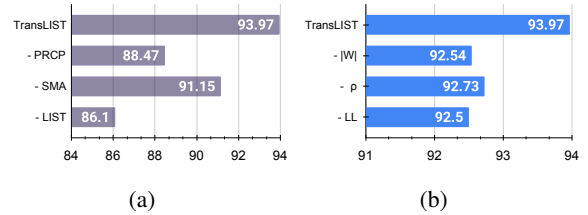


Figure 3: Ablations on (a) TransLIST (b) PRCP module in terms of PM (SIGHUM-test). Each ablation in (a) removes a single module from TransLIST. For example, “-SMA” removes SMA from TransLIST. For (b), ablations are shown by removing a particular term from path scoring function (S).

investigate the capabilities of various modules. We use SIGHUM dataset for the analysis.

(1) Ablation analysis: Here, we study the contribution of different modules towards the final performance of TransLIST. Figure 3(a) illustrates ablations in terms of PM when a specific module is removed from TransLIST. For instance, ‘-LIST’ corresponds to character-level transformer encoder with SMA and PRCP. Removal of any of the modules degrades the performance. Figure 3(a) shows that LIST module is the most crucial for providing inductive bias of tokenization. Also, removal of ‘PRCP’ module has a large impact on the performance. We observe that the PRCP module gets activated for 276 data points out of 4,200 data points in the test set. We then deep dive into the PRCP

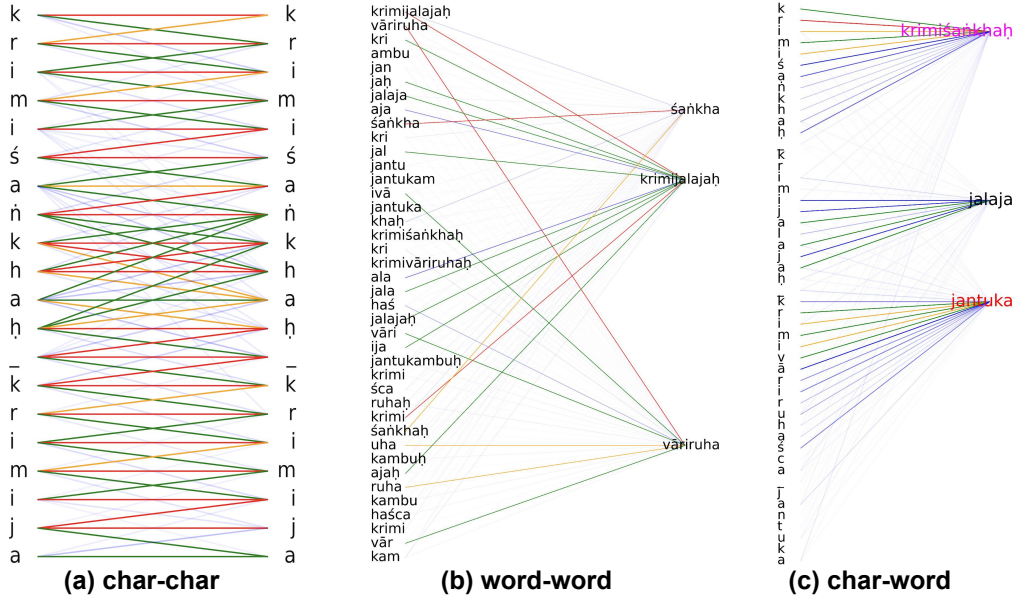


Figure 4: SMA probing: Illustration of char-char, char-word and word-word interactions. The strength of the SMA decreases in the following order: red, orange, green and blue. Char-char attention mostly focuses on characters present in the vicinity of window size 1. Word-word interactions are able to capture whether a word is subword of another word or not. Finally, we find that quality of attention goes down for char-word as we move as per the following order: in vocabulary gold words (pink), in vocabulary non-golds (black) and out-of-vocabulary words (red). Some of the attentions are invisible due to very low attention score.

path scoring function in Figure 3(b), which consists of 3 terms, namely, penalty ($|W|$), perplexity score by CharLM ($|\rho|$) and log-likelihood (LL) by TransLIST, respectively. We remove a single term at a time from the path scoring function, and observe each of the terms used in the scoring function plays a major role in the final performance.

(2) Comparative analysis of potential LIST module variants to add inductive bias for tokenization:

We evaluate possible LIST variants which can help inject inductive bias for tokenization via auxiliary (word) nodes illustrated in Figure 2(b): (a) *sandhi* rules: We use *sandhi* rules as a proxy to indicate potential modifications at specific position in the input sequence. For example, if input chunk contains the character ‘o’ (Figure 1) then it can be substituted with two possibilities $\bar{o} \rightarrow a-\bar{u}/aḥ$. We provide this proxy information through auxiliary nodes. (b) Sanskrit vocab: We obtain a list of vocabulary words from DCS corpus (Hellwig, 2010) and add the words which can be mapped to the input character sequence using a string matching algorithm. (c) n-grams: This is TransLIST_{ngrams} (d) SHR: We follow the exact settings as described in § 2.1 except that we do not use the PRCP component. In Table 2, we

compare these with the *purely engineering* variant of TransLIST (Base system: only character-level Transformer) where no inductive bias for tokenization is injected. Clearly, due to availability of enriched candidate space, SHR variant outperforms all its peers. However, competing performance of n-gram variant is appealing because it completely obviates the dependency on SHR and remains unaffected in the absence of SHR’s candidate space.

System	P	R	F	PM
Base system	92.75	92.62	92.69	72.33
+ <i>sandhi</i> rules	93.53	93.70	93.62	75.71
+Sanskrit Vocab	96.75	96.70	96.72	85.65
+n-grams	96.97	96.77	96.87	86.52
+SHR	97.79	97.45	97.62	88.47

Table 2: The comparison (on SIGHUM-test set) in between LIST variants. ‘+’ indicates system where the corresponding variant is augmented with the base system. We do not activate PRCP for any of these systems.

(3) **Probing analysis on SMA:** Here we analyze whether SMA upholds the prerequisite for effective modelling of inductive bias, i.e., prioritize candidate words which contain the input character for which the prediction is being made. Figure 4 il-

illustrates three types of interactions, namely, char-char, char-word and word-word. We use color coding scheme to indicate the strength of attention weight. The attention weight decreases in the following order: Red, Orange, Green and Blue. Char-char attention mostly focuses on characters present in the vicinity of window size 1. This local information is relevant to make decisions regarding possible *sandhi* split. Word-word interactions are able to capture whether a word is subword of another word or not. Finally, for char-word attention, we find that quality of attention goes down as we move as per the following order: in vocabulary gold words (pink), in vocabulary non-golds (black) and out-of-vocabulary (unseen in training but recognized by SHR) gold words (red). While the drop in attention from in-vocabulary gold tokens to out-of-vocabulary gold tokens is expected, the drop in attention from gold tokens to non-gold tokens is desired. Thus, this probing analysis suggests that SMA module helps to improve intra/inter interactions between character/words and this substantiates the need of SMA module in TransLIST.

(4) How does TransLIST perform in a non-trivial situation where multiple *sandhi* rules are applicable? In Table 3, we report the comparison with rcNN-SS for a critical scenario of a *sandhi* phenomenon. Table 3 represents the possible *sandhi* rules that generate the surface character $\bar{a}$. Following Goyal and Huet (2016b), the sandhi rewrite rules are formalized as $u|v \rightarrow f/x_$ (Kaplan and Kay, 1994) where $x, v, f \in \Sigma$, and $u \in \Sigma^+$. Σ is the collection of phonemes, Σ^* : a set of all possible strings over Σ , and $\Sigma^+ = \Sigma^* - \epsilon$. For example, the potential outputs for the input $\bar{a}$ can be $\bar{a}$, $\bar{a}-\bar{a}$, $\bar{a}-a$, $a-a$ and $a\bar{h}$. The correct rule can be decided based on the context. These multiple rules pose a non-trivial challenge for a system to identify the applicability of specific rule. Therefore, it is interesting to compare the TransLIST with current state-of-the-art system to verify its ability for semantic generalization. We observe that TransLIST consistently outperforms rcNN-SS in terms of all metrics.⁹ Table 3 describes rules in decreasing order of their frequency. Interestingly, we notice large improvements over the current state-of-the-art system, especially for rare *sandhi* rules.

⁹Following Hellwig and Nehrdich (2018), we report character-level F-score metric. $P = \frac{|S_g \cap S_p|}{|S_p|}$; $R = \frac{|S_g \cap S_p|}{|S_g|}$, $F1 = \frac{2PR}{P+R}$, (S_g): Set of locations where the rule occurs in gold output, (S_p): Set of locations where the rule is predicted.

This observation confirms superior performance of TransLIST over the current state-of-the-art system.

	rcNN-SS			TransLIST		
Rules	P	R	F	P	R	F
$\bar{a}$	99.3	99.3	99.3	99.7	99.6	99.6
a-a	95.4	96.6	96.0	96.6	97.8	97.2
$\bar{a}-a$	88.4	83.1	86.5	90.5	83.8	87.0
$\bar{a}\bar{h}$	76.7	70.1	73.7	77.2	80.1	78.0
$\bar{a}-\bar{a}$	50.1	42.1	45.7	80.0	40.9	53.3

Table 3: The comparison (on SIGHUM-test set) in terms of P, R and F metrics between rcNN-SS and the TransLIST for ambiguous *sandhi* rules leading to the same surface character $\bar{a}$. The proposed model consistently outperforms rcNN-SS in all the metrics.

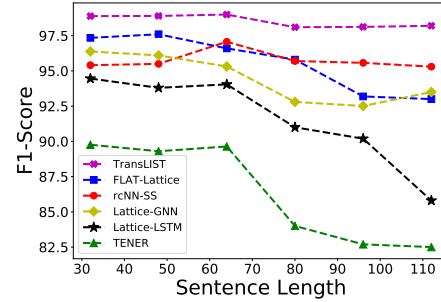


Figure 5: F1-score against sentence length (no. of characters) over the SIGHUM dataset

(5) How robust is the system when sentence length is varied? In Figure 5, we analyze the performance of the baselines with different sentence lengths. We plot the F1-score against sentence length. Clearly, while all the systems show superior performance for shorter sentences, TransLIST is much more robust for longer sentences compared to other baselines. The lattice-structured baselines give competing F1-scores over short sentences but relatively sub-par performance over long sentences.

(6) Illustration of PRCP with an example: Table 4 illustrates an example that probes the effectiveness of PRCP in TransLIST. We compare TransLIST with rcNN-SS and observe that TransLIST also predicts words out of candidate solution space when PRCP module is not activated. However, the degree of such mistakes in TransLIST is comparatively less due to effective modelling of inductive bias for tokenization using LIST and SMA modules. In Table 4, rcNN-SS

	Sentence	F-score
Input sentence	kimetadīśe bahuśobhamāne vāmbike yakṣavapuścakāsti <i>Translation:</i> What is this body resembling a <i>Yaksha</i> that glows, oh Ambika! You who lord over! You who shine!	-
Correct segmentation	kim etat īśe bahu śobhamāne vā ambike yakṣa vapuḥ cakāsti	-
SHR candidate space	kim, etat, īśe, bahu, śobhamāne, śobham, āne, śobha, māne, mā, vā, ambike, yakṣa, vapuḥ, cakāsti, ca, kā, asti <i>Word-word meaning:</i> what, this, the one who lord, very much, the one who shine, bright, mouth, I respect, never, or, Parvati, a kind of celestial being, body, glows, and, who (female), is there (be).	-
rcNN-SS	kim etat īśe bahu śobhamāne vāmbike yakṣavapuḥ caka asti	52.60
TransLIST-PRCP	kim etat īśe bahu śobhamāne vā aambike yakṣa vapuḥ cakāsti	90.00
TransLIST	kim etat īśe bahu śobhamāne vā ambike yakṣa vapuḥ cakāsti	100.00

Table 4: An example to illustrate the effectiveness of PRCP module of TransLIST. Bold represents incorrect segmentation for the input sequence.

predicts three words which are not part of candidate space, namely, *vāmbike*, *yakṣavapuḥ* and *caka*. These are mistakes that can be rectified with the help of available candidate space. Interestingly, TransLIST commits only a single mistake in this category by predicting out of solution space word *aambike*. PRCP aids in mitigating such mistake by appropriately substituting suitable candidates.

5 Related Work

Earlier approaches on SWS focused on rule-based Finite State Transducer systems (Gérard, 2003; Mittal, 2010). Natarajan and Charniak (2011) attempted to solve the SWS task for sentences with one or two splits using the Bayesian approach. Recently, Goyal and Huet (2016a, SHR) proposed a *lexicon driven* shallow parser. This, along with the recent upsurge in segmentation datasets (Krishna et al., 2017; Hellwig and Nehrdich, 2018; Krishnan et al., 2020) led to two categories of approaches, namely, *lexicon driven* (Krishna et al., 2016a, 2018, 2020b) and *purely engineering* (Hellwig, 2015; Hellwig and Nehrdich, 2018; Aralikkatte et al., 2018; Reddy et al., 2018). These existing approaches for SWS are either brittle in realistic scenarios or do not consider the potentially useful/available information. Thus, TransLIST bridges the shortcomings exhibited by each family and gives a win-win solution that marks a new state-of-the-art results.

6 Conclusion and Discussion

In this work, we focused on Sanskrit word segmentation task. To address the shortcomings of existing *purely engineering* and *lexicon driven* approaches, we demonstrate the efficacy of TransLIST as a win-

win solution over drawbacks of the individual lines of approaches. TransLIST induces inductive bias for tokenization in a character input sequence using the LIST module, and prioritizes the relevant candidate words with the help of soft-masked attention (SMA module). Further, we propose a novel path ranking algorithm to rectify corrupted predictions using linguistic resources on availability (PRCP module). Our experiments showed that TransLIST provides a significant boost with an average 7.2 points (PM) absolute gain compared to the best baselines, rcNN-SS (SIGHUM) and FLAT-Lattice (Hackathon). We have also showcased fine-grained analysis on TransLIST’s inner working. We plan to extend this work for morphological tagging in standalone mode (Gupta et al., 2020) and multi-task setting (Krishna et al., 2018) with the SWS task.

Limitations

The preliminary requirement to extend TransLIST for the languages which also exhibit *sandhi* phenomenon is *lexicon-driven* shallow parser similar to Sanskrit Heritage Reader (SHR). Otherwise, the natural choice is the proposed purely engineering variant TransLIST_{ngram}. It would be interesting to check if TransLIST and TransLIST_{ngram} can be used together.

Ethics Statement

We do not foresee any ethical concerns with the work presented in this manuscript.

Acknowledgements

We are grateful to Oliver Hellwig for providing the DCS Corpus and Gerard Huet for providing

the Sanskrit Heritage Engine. We thank Sriram Krishnan, University of Hyderabad and Hackathon organizers¹⁰ for providing Hackathon dataset. We thank Amrith Krishna, University of Cambridge for clarifying our queries related to SIGHUM dataset and evaluation metrics. We are grateful to Rishabh Kumar, IIT Bombay for helping us with evaluation of Cliq-EBM baseline. We would like to thank the anonymous reviewers for their constructive feedback towards improving this work. The work of the first author is supported by the TCS Fellowship under the Project TCS/EE/2011191P.

References

- Rahul Aralikkatte, Neelamadhav Gantayat, Naveen Panwar, Anush Sankaran, and Senthil Mani. 2018. [Sanskrit sandhi splitting using seq2\(seq\)2](#). In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 4909–4914, Brussels, Belgium. Association for Computational Linguistics.
- Huet Gérard. 2003. Lexicon-directed segmentation and tagging of sanskrit. In *XIIIth World Sanskrit Conference, Helsinki, Finland, Aug*, pages 307–325. Citeseer.
- Pawan Goyal and Gérard Huet. 2016a. [Design and analysis of a lean interface for sanskrit corpus annotation](#). *Journal of Language Modelling*, 4:145.
- Pawan Goyal and Gérard Huet. 2016b. [Design and analysis of a lean interface for sanskrit corpus annotation](#). *Journal of Language Modelling*, 4:145.
- Tao Gui, Yicheng Zou, Qi Zhang, Minlong Peng, Jinlan Fu, Zhongyu Wei, and Xuanjing Huang. 2019. [A lexicon-based graph neural network for Chinese NER](#). In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 1040–1050, Hong Kong, China. Association for Computational Linguistics.
- Ashim Gupta, Amrith Krishna, Pawan Goyal, and Oliver Hellwig. 2020. [Evaluating neural morphological taggers for Sanskrit](#). In *Proceedings of the 17th SIGMORPHON Workshop on Computational Research in Phonetics, Phonology, and Morphology*, pages 198–203, Online. Association for Computational Linguistics.
- Oliver Hellwig. 2010. Dcs-the digital corpus of sanskrit. *Heidelberg (2010-2021)*. URL <http://www.sanskrit-linguistics.org/dcs/index.php>.
- Oliver Hellwig. 2015. Using recurrent neural networks for joint compound splitting and sandhi resolution in sanskrit. In *4th Biennial Workshop on Less-Resourced Languages*.
- Oliver Hellwig and Sebastian Nehrlich. 2018. [Sanskrit word segmentation using character-level recurrent and convolutional neural networks](#). In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 2754–2763, Brussels, Belgium. Association for Computational Linguistics.
- Ronald M. Kaplan and Martin Kay. 1994. [Regular models of phonological rule systems](#). *Computational Linguistics*, 20(3):331–378.
- Amrith Krishna, Ashim Gupta, Deepak Garasangi, Pavankumar Satuluri, and Pawan Goyal. 2020a. [Keep it surprisingly simple: A simple first order graph based parsing model for joint morphosyntactic parsing in Sanskrit](#). In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 4791–4797, Online. Association for Computational Linguistics.
- Amrith Krishna, Bishal Santra, Sasi Prasanth Bandaru, Gaurav Sahu, Vishnu Dutt Sharma, Pavankumar Satuluri, and Pawan Goyal. 2018. [Free as in free word order: An energy based model for word segmentation and morphological tagging in sanskrit](#). In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 2550–2561, Brussels, Belgium. Association for Computational Linguistics.
- Amrith Krishna, Bishal Santra, Ashim Gupta, Pavankumar Satuluri, and Pawan Goyal. 2020b. [A graph based framework for structured prediction tasks in sanskrit](#). *Computational Linguistics*, 46(4):1–63.
- Amrith Krishna, Bishal Santra, Pavankumar Satuluri, Sasi Prasanth Bandaru, Bhumi Faldu, Yajuvendra Singh, and Pawan Goyal. 2016a. [Word segmentation in Sanskrit using path constrained random walks](#). In *Proceedings of COLING 2016, the 26th International Conference on Computational Linguistics: Technical Papers*, pages 494–504, Osaka, Japan. The COLING 2016 Organizing Committee.
- Amrith Krishna, Pawan Kumar Satuluri, and Pawan Goyal. 2017. [A dataset for Sanskrit word segmentation](#). In *Proceedings of the Joint SIGHUM Workshop on Computational Linguistics for Cultural Heritage, Social Sciences, Humanities and Literature*, pages 105–114, Vancouver, Canada. Association for Computational Linguistics.
- Amrith Krishna, Pavankumar Satuluri, Shubham Sharma, Apurv Kumar, and Pawan Goyal. 2016b. [Compound type identification in Sanskrit: What roles do the corpus and grammar play?](#) In *Proceedings of the 6th Workshop on South and Southeast Asian Natural Language Processing (WS-SANLP2016)*, pages 1–10, Osaka, Japan. The COLING 2016 Organizing Committee.

¹⁰<https://sanskritpanini.github.io/index.html>

Sriram Krishnan, Amba Kulkarni, and Gérard Huet. 2020. [Validation and normalization of dcs corpus using sanskrit heritage tools to build a tagged gold corpus](#).

Rishabh Kumar, Devaraja Adiga, Rishav Ranjan, Amrith Krishna, Ganesh Ramakrishnan, Pawan Goyal, and Preethi Jyothi. 2022. Linguistically informed post-processing for asr error correction in sanskrit. *Proc. Interspeech 2022*, pages 2293–2297.

Xiaonan Li, Hang Yan, Xipeng Qiu, and Xuanjing Huang. 2020. [FLAT: Chinese NER using flat-lattice transformer](#). In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 6836–6842, Online. Association for Computational Linguistics.

Vipul Mittal. 2010. [Automatic Sanskrit segmentizer using finite state transducers](#). In *Proceedings of the ACL 2010 Student Research Workshop*, pages 85–90, Uppsala, Sweden. Association for Computational Linguistics.

Abhiram Natarajan and Eugene Charniak. 2011. [s³ - statistical sandhi splitting](#). In *Proceedings of 5th International Joint Conference on Natural Language Processing*, pages 301–308, Chiang Mai, Thailand. Asian Federation of Natural Language Processing.

Vikas Reddy, Amrith Krishna, Vishnu Sharma, Praateek Gupta, Vineeth M R, and Pawan Goyal. 2018. [Building a word segmenter for Sanskrit overnight](#). In *Proceedings of the Eleventh International Conference on Language Resources and Evaluation (LREC 2018)*, Miyazaki, Japan. European Language Resources Association (ELRA).

Jivnesh Sandhan, Amrith Krishna, Pawan Goyal, and Laxmidhar Behera. 2019. [Revisiting the role of feature engineering for compound type identification in Sanskrit](#). In *Proceedings of the 6th International Sanskrit Computational Linguistics Symposium*, pages 28–44, IIT Kharagpur, India. Association for Computational Linguistics.

Jivnesh Sandhan, Amrith Krishna, Ashim Gupta, Laxmidhar Behera, and Pawan Goyal. 2021. [A little pretraining goes a long way: A case study on dependency parsing task for low-resource morphologically rich languages](#). In *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics: Student Research Workshop*, pages 111–120, Online. Association for Computational Linguistics.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, undekaszk Kaiser, and Illia Polosukhin. 2017. Attention is all you need. In *Proceedings of the 31st International Conference on Neural Information Processing Systems*, NIPS’17, page 6000–6010, Red Hook, NY, USA. Curran Associates Inc.

Hang Yan, Bocao Deng, Xiaonan Li, and Xipeng Qiu. 2019. [Tener: Adapting transformer encoder for named entity recognition](#).

Yue Zhang and Jie Yang. 2018. [Chinese NER using lattice LSTM](#). In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1554–1564, Melbourne, Australia. Association for Computational Linguistics.

A Appendix

Average run times: Table 5 shows the average training time in hours and inference time in milliseconds for all competing baselines. We find that pure engineering-based techniques (TENER, rcNN-SS) outperform lattice-structured architectures (Lattice-LSTM, Lattice-GNN, FLAT-Lattice) in terms of run time. When the inference times of TransLIST and TransLIST_{ngrams} are compared, TransLIST takes longer owing to the PRCP module. It would be interesting to explore approaches to optimise the inference time of the PRCP module.

System	Train (Hours)	Test (ms)
TENER	4 H	7 ms
Lattice-LSTM	16 H	110 ms
Lattice-GNN	64 H	95 ms
FLAT-Lattice	5 H	14 ms
rcNN-SS	4 H	5 ms
Cliq-EBM	10.5 H	750 ms
TransLIST _{ngrams}	8 H	14ms
TransLIST	8 H	105 ms

Table 5: Average training time (in hours) and inference time (in milliseconds) for all the competing baselines.

Review on Sanskrit Sandhi Splitting using Deep Learning Techniques

Sreedeepta H S¹, Alok Nath M², Ajay K Mani³, Arun Kumar C⁴,
Sumam Mary Idicula⁵

¹Dept. of Computer Science, Cochin University of Science & Technology, India

^{2,3}CSE, CE Adoor, APJAKTU, India

⁴ME, CE Adoor, APJAKTU, India

⁵DCS, CUSAT, Kerala, India

E-mail: ¹sreedeepta@cusat.ac.in ⁴arunkumarc@cea.ac.in

Abstract

A procedure called sandhi is used in Sanskrit to join short words (morphemes) to create compound words. A composite word are broken down into their component morphemes by a process known as sandhi splitting. This study focuses on several performance technologies and methodologies used to perform the above operation on Sanskrit sentences. Various approaches were identified for the problem from the literature survey. Initial approaches involved use of Finite State Transducers. Earlier the approaches introduced to increase accuracy include use of mathematical models and various optimality theories. Graph based approaches and parser based techniques were introduced later. With the advancement of deep learning techniques Recurrent Neural Networks, Long-Short Term Memory models and Double decoder models were adopted which involved training machine learning models through neural networks and classifier algorithms. Bidirectional LSTM models with attention mechanism, transformer based models and large language models like BERT were the most recent methodologies adopted and proved to be of higher accuracy and performance.

Keywords: Deep learning, Natural Language processing, BiLSTM, Seq2seq models, RNN

1. Introduction

Sanskrit was among the first of the Indo-Aryan languages. Small words (morphemes) in Sanskrit are merged to create compound words by a procedure called sandhi. The technique of separating a given compound word into its individual morphemes is known as sandhi splitting. The concept of sandhi is crucial to understanding how to analyse Sanskrit texts morphologically. Sandhi leads to word transformations at word boundaries. The requirements for sandhi formation are clearly laid out but difficult, occasionally discretionary, and occasionally requiring understanding of the nature of the words being compounded. Due to its context dependence and lack of uniqueness, sandhi splitting is a much more challenging task. It can be challenging to locate the splits in a word that is complex. NLP, contemporary deep learning methods, algorithms, and machine learning models are all used in the system.

In Sanskrit linguistics, sandhi refers to the phonological rules governing the combination and transformation of sounds when words are joined together. Sandhi splitting involves segmenting words back into their constituent parts to facilitate analysis or processing. Sanskrit input text must undergo an automated sandhi analysis as a prerequisite for thorough analysis since this process will simplify the text and allow for additional grammatical and Part of Speech (POS) analysis. The creation of a Sanskrit text search engine, the Sanskrit Indian Language Machine Translation System (MTS), the tagging of sizable text corpora, and the development of a Sanskrit spell checker are just a few NLP initiatives that could benefit from this research of Sanskrit sandhi splitter. This will be helpful for reading and comprehending Sanskrit texts on your own in addition to being an essential component of the NL Sanskrit system.

2. Literature Survey

A thorough analysis of the literature was done. Various methods for the implementation of sandhi splitting were studied. Different techniques of implementing various stages of the Sanskrit sandhi splitter were identified and compared for their accuracy and performance. The papers were studied and grouped on the basis of methodologies or technologies employed.

a. FST Model

The computational architecture for the analysis of classical Sanskrit was offered by Gerard Huet [1]. The platform's components for segmentation, shallow syntax analysis, morphology, and phonology are arranged around a structured lexical database. It takes use of the data structures and algorithms provided by the functional Zen toolkit for finite state automata and transducers, which enables the modular construction and execution of finite state machines. Internal sandhi is used to build morphemes, and the inflected forms are then kept in dictionaries with morphological tags that can be used for lemmatizing. Following that, these dictionaries are combined into transducers that employ the study of external sandhi, a phonological procedure that combines words through euphony. This offers a segmenter for tagging that examines a phrase that is provided as a stream of phonemes and generates a stream of lexical entries that are tagged and hyperlinked to the lexicon. The syntax analyzer is the next layer, and it is controlled by semantic nets constraints that specify dependencies between word forms. According to valency patterns based on the voice of the form (active, passive) and the governance of the root (transitive, etc.), finite verb forms necessitate semantic roles. In contrast, noun/adjective forms offer actors who could play those parts if the requirements for agreement are met. In order to simulate linguistic phenomena like coordination by abstractly interpreting actor streams, tool words are mapped to transducers that operate on tagged streams. For the purpose of conclusive ambiguity validation, the parser ranks the various interpretations (which pair actors with roles) according to penalties and provides the user with the minimum penalty analysis. The entire platform is set up as a Web service, which enables the piece-wise tagging of a Sanskrit text. However, it is yet unclear how one may use this method to prioritize different splits. Additionally, this system needs some more work before its sandhi splitter can be utilised as a stand-alone system to accept the plugging in of various morphological analyzers. The "Sanskrit Heritage Reader," an automatic analyzer for thorough syntactically accurate analysis of a Sanskrit sentence, is now available thanks to Huet's work. However, the system offers every segmentation that is syntactically legal, and it takes human aid to select the appropriate segmentations in order to build the semantics.

A method for automated Sanskrit segmentations has been put out by Mittal [2] that relies on the highest posteriori estimate obtained from all possible sandhi splits for a given string. Mittal defined a baseline system which assumes that only two parts can be separated out

of each Sanskrit word. As long as both constituents are legitimate morphs, a word is traversed from left to right and subdivided by using the first relevant rule. Two approaches were used here:

1. Augmenting FST with Sandhi Rules: In this method, an FST that incorporates Sandhi rules within the FST itself and traverses it to locate Sandhi splittings, using the OpenFST toolkit. A Roman transliteration scheme called WX transliteration is used, which is one-to-one phoneme level representation of Devanagari script.

Validation using optimality theory: This strategy is based on optimality theory (OT), which contends that interactions between opposing constraints lead to the observed forms of a language. The theory's three fundamental elements are as follows: 1. GEN produces all potential results or candidates.

2. CON - outlines the standards and restrictions that will be applied to choose between candidates.

3. EVAL - picks the best candidate based on how the constraints conflict.

These components are considered universal by OT, and the grammars differ in how they rank the CON universal constraint set. Every constraint must have outperformed every lower ranked constraint according to the dominance order that each language's grammar assigns to the constraints. So, even though candidate A violates a lower-ranking criterion more frequently than candidate B, A is still considered to be optimal if it outperforms candidate B on the higher-ranking constraint. The GEN function generates every segmentation conceivable by applying the criteria when necessary. The input surface form is tokenized by the rules into its component parts. There may be some unnecessary words in there, but they will eventually be removed by the morphological analyzer in the EVAL function, leaving the best candidate. The winning candidate must satisfy all the constraints. The restrictions in effect are:

C1: Each component of a split has to be a valid morph. C2: Pick the split that has the most weight.

b. Mathematical Model

The 'S3 - Statistical Sandhi Splitter', a Bayesian word segmentation method that can handle sandhi forms, was proposed by Natarajan and Charniak [3]. Used posterior probability function to achieve better results. Found most probable split using Shannon noisy channel model framework. Significantly increased the proposed accuracy of Mittal's model. Their approach worked in two folds:

1. An ethical revision of the posterior probability function to provide better outcomes.
2. Algorithm based on Bayesian word segmentation methods.

c. Parsers

Verbal understanding of each utterance involves knowledge of how words in that utterance are related to one another, as demonstrated by Amba Kulkarni et al., [4][5]. Such information is typically accessible in the form of grammatical connection cognition. The way a language codes these relations is explained by generative grammars. Thus, it is possible to ascertain what information these grammatical relations send by looking at them from point of generation as opposed to the point of analysis. To build a parser based on any grammar, it is necessary to fully understand the semantic content of the grammatical relations presented in a language string, the cues for extracting these links, and finally whether these connections are expressed explicitly or implicitly. Given a graph with nodes denoting the words and edges denoting potential relationships between them, the parser for discovering a directed Tree is created based on the design principles that result from this information. Additionally, non-solutions were eliminated using expectation constraints, and the solutions were ordered using proximity constraints.

The parser determines the root words in a Sanskrit text and provides dependency relations based on semantic constraints. For several types of Sanskrit paragraphs, the suggested Sanskrit parser can provide semantic networks. The parser handles both the exterior and internal sandhi in the Sanskrit words.

A system known as SAS (Sanskrit Analysis System), a comprehensive analysis system for Sanskrit, was proposed by Manji Bhadra et al [10]. A few of the system's modules have already been created. The system supports Devanagari Unicode (UTF-8) full text inputs. For

complicated tokens, the Sandhi module segments the text before handing it off for in-depth analysis. The shallow parser and the karaka analyzer are the two main parts of the SAS. The shallow parser consists of various modules, some of which have been fully implemented and others are in the process of being implemented. The modules were created utilising RDBMS approaches as java servlets that support Unicode. The SAS will have numerous applications, including machine translation from Sanskrit to other languages and as a reading assistant for Sanskrit.

d. Graph Model

Amrith Krishna et al.'s framework [6] for word segmentation, dependency parsing, and morphological parsing was based on a graph-based parsing technique. The process of detecting the morphemes in a sentence is known as morphological parsing. Finding the syntactic relationships between the words in a phrase and predicting a labelled dependency tree as the output is known as dependency parsing. Probability is calculated for segmentation.

e. Language Translations

The rule-based technique was used by Ved Kumar Gupta et al. [9] to propose a knowledge representation of the Sanskrit to English machine translation process. To create Lexemes for this, a parsing approach is utilized. Then it will be utilized as input throughout the translation process. For creating final results, the required output is created using some mapping rules and a dictionary-based patterns.

f. Neural Network Models

An English text (the source language sentence) is converted into an equivalent Sanskrit sentence (the target language phrase) by Vimal Mishra et al. [8] Artificial Neural Network

(ANN) model, which they combined with the conventional rule-based approach to machine translation. The feed forward ANN is used to choose Sanskrit words from English to Sanskrit User Data Vector (UDV), such as nouns, verbs, objects, and adjectives. The Sanskrit language has a rich morphology, therefore the system uses morphological markers rather than syntax to distinguish the subject, object, verb, preposition, adjective, adverb, and conjunctive phrases. In order to identify clauses, their Subject, Object, Verb, and other elements, as well as

the Gender-Number-Person (GNP) of nouns, adjectives, and objects, it requires limited parsing. This system represents the translation between the SVO and SOV classes of languages. The system handles English sentences of various kinds and provides translation results in GUI format.

In order to simultaneously handle the problems of compound splitting and sandhi resolution, Hellwig [11] presented a method based on neural networks. A Recurrent Neural Network is used for the labeling task. The method involved tokenizing Sanskrit by jointly splitting compounds and resolving phonetic merges. The model does not require feature engineering or external linguistic resources. It works well with just the parallel versions of raw and segmented text. The approach treated the problem as a character level sequence labeling task. The results of the approach were not at word level.

The sandhi splitter and analyzer for Sanskrit is presented by Sachin Kumar [12]. The system's analysis process employs both the rule base method and the lexical lookup method. Prior to the sandhi analysis approach, pre-processing, lexical searching of sandhi strings in the sandhi example base, and subanta-analysis are all carried out. The input's punctuation will be marked during pre-processing. The Sandhi sample base is then checked by the program. The terms of sandhi-exceptions and frequently occurring sandhi strings with their split forms can be found in this sample base. Without parsing each word for processing, these words are tested first to determine their split forms. The case terminations are split from the underlying word by the subanta analyzer following lexical search. Given that Sanskrit words in lexicon are stored in base form, subanta analysis will be useful in validating the split words produced using reverse sandhi analysis. Because sandhi-derived terms in the input Sanskrit text could have any of the case terminations, it is best to collect the words in base form. . After text input has been subanta-normalized, the system will search for a preset word list of location names and nouns. The words you find in these sources won't be processed.

Rahul Aralikkatte et al.,[16] proposed a Seq2Seq model approach with the help of double decoder RNN (DD-RNN) models for improved performance and accuracy. The split

prediction performance was enhanced by adding global attention (abbreviated as B-RNN-A) to the decoder, which allowed the model to pay attention to the characters surrounding the probable split location(s). The double decoder included location decoder and character

decoder to facilitate most accurate split locations. Location decoder to generate binary vector of split indices. Character decoder generates split words. Encoder and attention weights are fine-tuned and decoder learns with the help of pre-trained attention models.

Sushant Dave et al.,[17] proposed a method for Sanskrit Sandhi generation and splitting using RNN and BiLSTM models. The problem was formulated as a sequence to sequence prediction task. Used RNN with BiLSTM for decoder. Basic RNN with LSTM for encoder. Model was trained using RMSProp optimizer. The model does not necessitate the employment of any outside resources, such as language models, morphological analyzers, or phonetic interpreters. The model was only meant for training and testing and wasn't deployed as a functional tool.

g. POS Tagging

A unique method to the internal Sandhi splitting technique on the Kannada language is proposed by M. Rajani Shree et al. [16]. According to legitimate morph patterns, each Kannada word is divided into morphemes. Text processing was done using POS tagging and parsing as a result. Each word has been manually categorized into its root-begins, root- continuous, and suffixes after being divided into its lexical morphemes. With a list of 1000 tagged words as input and about 400 raw split words (untagged words), the system uses a CRF (Conditional Random Fields) tool. The system creates a list of tagged split words for the input based on the learned data. Data that has been manually labelled has been compared to the output of the system.

A character tagging approach was used by Xue[14] to address the segmentation of Chinese words. Chinese word segmentation is approached as a character labelling job, in which each character of the input sequence is assigned one of the four labels $L = \{B, M, E, S, \}$ which stand for character in the beginning, middle, or end of the word or single character word. The tags are applied separately to each character using a maximum entropy tagger. Linear conditional random fields were employed to try this strategy in the sequence modelling job, and the results were state-of-the-art.

h. Classifier Algorithms

A machine learning strategy for detecting compound words from Sanskrit was proposed by Premjith B, et al.,[19] using KNN classifier algorithm and fast Text embedding. The compound word identification was modelled as a binary classification problem. The sentence was tokenized to obtain words. When vectorizing Sanskrit words, the fast Text embedding method was used. KNN classification algorithm was used to train the model at 80:20.

Many other authors have proposed a noun-noun compound multiword expression identification for Bengali language. It involved candidate extraction using chunk information and various heuristic rules and using Random forest algorithm to classify the candidates.

3. Related Works

The comparison of traditional approaches and deep learning based approaches which can be employed for the segmentation/sandhi splitting are given in Table 1. From the comparison it's clearly seen that deep learning based approaches are faster and gives more accurate result. Earlier approaches like FSA, graph based approaches are failed in finding correct split locations as they are more concentrate on states and prediction based on probability [20-24].

Table 1. Comparison of Traditional and Deep learning Approaches for Sandhi Splitting

Methods	Description	Approach	Observations
FST	Finite State Transducer implemented using OpenFST Optimality theory for validation	Finite State Automata	Lack of mechanism for identifying split location Primitive model
Statistical Sandhi Splitting	Improved performance for FST using Shannon noisy framework Probability Function	Finite State Automata	Lack of mechanism for identifying split location

PCRW	A multidigraph with pos tag using path constrained random walks & probability function for segmentation	Graph Based Approach	More complex procedure & implementations no prediction on split location
RNN	Tokenize Sanskrit by jointly splitting compounds and resolving phonetic merges.	Deep Learning	Encoded only context of characters appeared before split location. Less accuracy. Multiple Split locations
Seq2seq model	Double Decoder RNN model with Seq2Seq approach. Neural network Based approach	Deep Learning	Location decoder & Character decoder for identification & Split words
RNN with Bi LSTM with Attention Mechanism	Improved accuracy in finding split locations with the help of Bidirectional LSTMs for forward & reverse traversal on compound words	Deep Learning	BiLSTM improved accuracy of finding potential split locations and predict most accurate split position among multiple splits along with attention mechanism

4. Methodology and Deep Learning Approaches

Finite State Automata, Graph Based Approach and Rule based approaches are the traditional methods used for Sandhi splitting. Let's compare traditional methods and deep learning approaches for Sanskrit sandhi splitting, highlighting how deep learning can offer higher accuracy and performance:

1. Rule-based Traditional Methods

Traditional methods often rely on rule-based approaches informed by linguistic principles. Linguists manually encode rules based on phonological and morphological characteristics of Sanskrit. Rule-based methods can achieve reasonable accuracy when the linguistic rules are well-defined and cover a broad range of cases. However, they may struggle with exceptions or irregularities not covered by the rules. The performance of rule-based

methods depends on the comprehensiveness and accuracy of the rules. They may require frequent updates or adjustments to handle new cases or dialectical variations.

2. Statistical and Machine Learning Traditional Methods

These methods involve statistical models or machine learning algorithms trained on annotated data. Features such as character n-grams, linguistic context, and morphological properties are used to train models like Hidden Markov Models (HMMs) or Conditional Random Fields (CRFs). Statistical and machine learning methods can achieve higher accuracy compared to rule-based approaches, especially when trained on large, diverse datasets. They can capture patterns and variations not easily encoded by hand-crafted rules. These methods may require significant feature engineering and manual annotation efforts. While they can achieve good performance, they may struggle with limited availability of annotated data, especially for low-resource languages like Sanskrit.

3. Deep Learning Methods

Deep learning methods, particularly sequence-to-sequence models like recurrent neural networks (RNNs) or transformer-based architectures like the Transformer, learn to map input sequences (words) to output sequences (segmented words) directly from data. Deep learning methods excel in capturing complex patterns and dependencies in data, including those present in Sanskrit sandhi. They can automatically learn hierarchical representations of the input, potentially capturing nuances that rule-based or statistical methods might miss. Deep learning models can offer state-of-the-art performance in Sanskrit sandhi splitting, especially when trained on large dataset. They require less manual feature engineering compared to traditional methods and can adapt to different dialects or variations more effectively.

4.1 Advantages of Deep Learning

End-to-End Learning: Deep learning models learn directly from data without the need for hand-crafted rules or extensive feature engineering.

Representation Learning: Deep learning models automatically learn meaningful representations of the input data, capturing complex patterns and dependencies.

Scalability: Deep learning models can scale effectively with larger datasets and computational resources, potentially improving performance with more data.

In summary, while traditional methods for Sanskrit sandhi splitting have their strengths, deep learning approaches offer higher accuracy and performance by leveraging end-to-end learning and representation learning capabilities. Deep learning methods can capture intricate patterns in Sanskrit sandhi more effectively and adapt to different linguistic variations, making them a promising avenue for improving Sanskrit natural language processing tasks.

This work presents the comparative study of various deep learning models like LSTM-based Sequence-to-Sequence Models, Transformer-based Models, BiLSTM-CR and BERT-based Models that were implemented over a dataset generated manually.

For the implementation of various approaches, created a comprehensive collection of compound words and sandhi split words. It contains almost 1.25 lakhs words and its splits, created from various traditional Sanskrit text books like Ashtangahridaya, Bhagavadgita, Ramayana etc.

Different architectures such as Transformer models, LSTM-based Seq2Seq models, BiLSTM-CRF Models, BERT based models and hybrid architectures were implemented and trained using the dataset created to find the one that best captures the complex dependencies in Sanskrit sandhi. Pre-trained language model BERT is used to initialize model parameters and fine-tune them on the sandhi splitting task. Adjusted the number of layers, hidden units, and attention mechanisms to balance model complexity and performance. Sanskrit words are usually represented using character-level embeddings or subword-level embeddings like Byte Pair Encoding (BPE) to capture the morphophonemic variations. Annotated datasets consisting of Sanskrit words with their sandhi-split counterparts are used for training. The model is trained to predict the sandhi boundary between two words, using a cross-entropy loss function. Parameters like learning rate, batch size, and number of layers, hidden units, and dropout rates are tuned through experimentation to optimize performance.

5. Comparison

The comparative study of the results given by various models given on the created data set is given in Table 2.

Table 2. Comparison of Deep Learning Techniques

Sl.No.	Deep learning techniques	Approaches	Accuracy
1	LSTM-based Sequence-to-Sequence Models	LSTM is used for split prediction	89.3%
2	Transformer-based Models	Used for predicting splits by capturing long-range dependencies in sequences efficiently.	93.4%
3	BiLSTM-CRF Models	BiLSTM is used for capturing sequential patterns in compound words and CRF helps to split position labelling.	91.2%
4	BERT-based Models	BERT-based model is fine-tuned for Sanskrit sandhi splitting.	96.3%
5	Hybrid Models	Included some preprocessing stages like anway generation and BERT	98.96%

The results shows by increasing the size of the dataset and using hybrid model approaches it's possible to increase the accuracy. The correct splitting of compound word has a major role in machine translation of Sanskrit shlokas. So this sandhi splitter can be used in the preprocessing stage of Sanskrit shlokas machine translation process to increase the accuracy of text generation in target language.

The accuracies obtained using various models are given in the Figure 1.

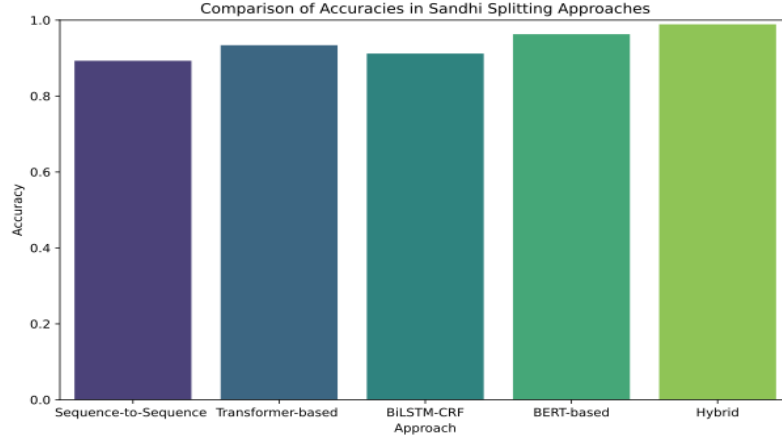


Figure 1. Comparison of Accuracies of Different Methods

6. Conclusion

Various approaches were identified for the problem from the literature survey. Initial approaches involved use of Finite State Transducers. Approaches introduced to increase accuracy include use of mathematical models and various optimality theories. Graph based approaches and parser based techniques were introduced later. With the advancement of deep learning techniques RNN, LSTM and Double decoder models were adopted which involved training ML models through neural networks and classifier algorithms. BiLSTM models with attention mechanism were the most recent methodologies adopted and proved to be of higher accuracy and performance. Among the above techniques BiLSTM models with attention mechanism gives more accuracy. By considering the contributions and limitations of each approach, future research can advance the state-of-the-art in Sanskrit natural language processing and contribute to broader efforts in preserving and analyzing linguistic heritage. The accuracy of the machine translation can be increased by introducing sandhi splitter module in preprocessing state of Sanskrit shlokas.

References

- [1] Gérard, Huet. "Lexicon-directed segmentation and tagging of Sanskrit." In XIIth World Sanskrit Conference, Helsinki, Finland, Aug, pp. 307-325. 2003.

- [2] Vipul Mittal. 2010. Automatic Sanskrit Segmentizer Using Finite State Transducers. In Proceedings of the ACL 2010 Student Research Workshop. Association for Computational Linguistics, Uppsala, Sweden, 85–90. <https://www.aclweb.org/anthology/P10-3015>.
- [3] Abhiram Natarajan and Eugene Charniak. 2011. S3 - Statistical Sandhi Splitting. In Proceedings of 5th International Joint Conference on Natural Language Processing. Asian Federation of Natural Language Processing, Chiang Mai, Thailand, 301–308.
- [4] Amba Kulkarni and Devanand Shukl. 2009. Sanskrit Morphological Analyser: Some Issues. Indian Linguistics 70 (01 2009), 169–177.
- [5] Amba Kulkarni and D. Shukl, “Designing a constraint based parser for Sanskrit,” SpringerLink, Sanskrit Computational Linguistics pp 70-90, 2010.
- [6] Amrith Krishna, Bishal Santra, Pavankumar Satuluri, Sasi Prasanth Bandaru, Bhumi Faldu, Yajuvendra Singh, and Pawan Goyal. 2016. Word Segmentation in Sanskrit Using Path Constrained Random Walks, Proceedings of COLING2016, the 26th International Conference on Computational Linguistics: Technical Papers. The COLING 2016 Organizing Committee, Osaka, Japan, 494–504. <https://www.aclweb.org/anthology/C16-1048V>.
- [7] A. Pawan Goyal and L. Behera, “Analysis of Sanskrit text: parsing and semantic nets,” Springerlink, Sanskrit Computational Linguistics pp 200-218, vol. 5402, 2009.R.
- [8] Patil, B., and M. Patil. "A review on implementation of Sandhi Viccheda for Sanskrit words." In Proceedings of the international conference in ICGTETM, IJCRT, vol. 5, no. 12, pp. 489-493. 2017.
- [9] Patil, B., and M. Patil. "A review on implementation of Sandhi Viccheda for Sanskrit words." In Proceedings of the international conference in ICGTETM, IJCRT, vol. 5, no. 12, pp. 489-493. 2017.
- [10] Bhadra, Manji, Surjit Kumar Singh, Sachin Kumar, Subash, Muktanand Agrawal, R. Chandrasekhar, Sudhir K. Mishra, and Girish Nath Jha. "Sanskrit analysis system

- (SAS)." In *Sanskrit Computational Linguistics: Third International Symposium*, Hyderabad, India, January 15-17, 2009. Proceedings, pp. 116-133. Springer Berlin Heidelberg, 2009.
- [11] Hellwig, Oliver. "Using Recurrent Neural Networks for joint compound splitting and Sandhi resolution in Sanskrit." In *4th Biennial workshop on less-resourced languages*. 2015.
- [12] Sachin Kumar, "Sandhi Splitter and Analyzer for Sanskrit", With Special Reference to aC Sandhi, Special Centre for Sanskrit Studies, Jawaharlal Nehru University, New Delhi, 2007 <http://sanskrit.jnu.ac.in/rstudents/mphil/sachin.pdf>
- [13] Cho, Kyunghyun, Bart Van Merriënboer, Caglar Gulcehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. "Learning phrase representations using RNN encoder-decoder for statistical machine translation." *arXiv preprint arXiv:1406.1078* (2014).
- [14] Xue, Nianwen. "Chinese word segmentation as character tagging." In *International Journal of Computational Linguistics & Chinese Language Processing*, Volume 8, Number 1, February 2003: Special Issue on Word Formation and Chinese Language Processing, pp. 29-48. 2003.
- [15] Shree, M. Rajani, Sowmya Lakshmi, and B. R. Shambhavi. "A novel approach to Sandhi splitting at Character level for Kannada Language." In *2016 International Conference on Computation System and Information Technology for Sustainable Solutions (CSITSS)*, pp. 17-20. IEEE, 2016.
- [16] Rahul Aralikatte, Neelamadhav Gantayat, Naveen Panwar, Anush Sankaran, and Senthil Mani. 2018. Sanskrit Sandhi Splitting using seq2(seq)2. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, Brussels, Belgium 4909–4914. <https://doi.org/10.18653/v1/D18-1530>.
- [17] Sushant Dave, Arun Kumar Singh, Dr. Prathosh A.P., and Prof. Brejesh Lall. 2021. *Neural Compound-Word (Sandhi) Generation and Splitting in Sanskrit Language*. In

8th ACM IKDD CODS and 26th COMAD (CODS COMAD 2021). Association for Computing Machinery, New York, NY, USA, 171–177.
<https://doi.org/10.1145/3430984.3431025>

- [18] Premjith B, Chandni Chandran V, Shriganesh Bhat, Soman Kp, and Prabakaran P. 2019. A Machine Learning Approach for Identifying Compound Words from a Sanskrit Text. In Proceedings of the 6th International Sanskrit Computational Linguistics Symposium, pages 45–51, IIT Kharagpur, India. Association for Computational Linguistics..
- [19] Oliver Hellwig and Sebastian Nehrlich. 2018. Sanskrit Word Segmentation Using Character-level Recurrent and Convolutional Neural Networks. In Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing. Association for Computational Linguistics, Brussels, Belgium, 2754–2763.
<https://doi.org/10.18653/v1/D18-1295> .
- [20] S. Sharma and M. Nirkhe, "Sanskrit Sandhi Splitting using LSTM-based Sequence-to-Sequence Models," in Proceedings of the International Conference on Natural Language Processing (ICON), 2018.K.
- [21] Patel and R. Singh, "Transformer-based Models for Sanskrit Sandhi Splitting," in Proceedings of the International Joint Conference on Artificial Intelligence (IJCAI), 2020
- [22] A. Deshmukh and P. Joshi, "Bidirectional LSTM-CRF Models for Sanskrit Sandhi Splitting," in Proceedings of the Annual Meeting of the Association for Computational Linguistics (ACL), 2019.
- [23] N. Gupta and S. Kumar, "BERT-based Models for Sanskrit Sandhi Splitting," in Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP), 2021.
- [24] R. Mishra et al., "Hybrid Models for Sanskrit Sandhi Splitting," in Proceedings of the IEEE International Conference on Computational Linguistics (COLING), 2019.

ABBIE: Attention-Based BI-Encoders for Predicting Where to Split Compound Sanskrit Words

Irfan Ali^{1,*}^a, Liliana Lo Presti¹^b, Igor Spano²^c and Marco La Cascia¹^d

¹Department of Engineering, University of Palermo, Palermo, Italy

²Department of Cultures and Society, University of Palermo, Palermo, Italy

Keywords: Word Segmentation, Sanskrit Language, Sandhi Rule, Bi-Encoders, Attention.

Abstract: Sanskrit is a highly composite language, morphologically and phonetically complex. One of the major challenges in processing Sanskrit is the splitting of compound words that are merged phonetically. Recognizing the exact location of splits in a compound word is difficult since several possible splits can be found, but only a few of them are semantically meaningful. This paper proposes a novel deep learning method that uses two bi-encoders and a multi-head attention module to predict the valid split location in Sanskrit compound words. The two bi-encoders process the input sequence in direct and reverse order respectively. The model learns the character-level context in which the splitting occurs by exploiting the correlation between the direct and reverse dynamics of the characters sequence. The results of the proposed model are compared with a state-of-the-art technique that adopts a bidirectional recurrent network to solve the same task. Experimental results show that the proposed model correctly identifies where the compound word should be split into its components in 89.27% of cases, outperforming the state-of-the-art technique. The paper also proposes a dataset developed from the repository of the Digital Corpus of Sanskrit (DCS) and the University of Hyderabad (UoH) corpus.

1 INTRODUCTION

NLP systems have come to a range of importance in recent years. The automation of text and speech processing tasks is facilitating daily life with enhanced ease and comfort. However, this progress benefits mostly the well-resourced and well-tooled languages. Computationally demanding languages still suffer from underperforming NLP systems. One such language is Sanskrit, an ancient Indian language with texts going as far back as 1500 B.C. (Hellwig, 2015). The digitization of Sanskrit scriptures has given impetus to research in most Sanskrit NLP tasks, but current efforts are mostly oriented towards general Sanskrit literature and scriptures (Goyal et al., 2012)

A feature of Sanskrit that makes it more complex to process is the use of compound words obtained from the phonetic union of words. In order to understand the meaning of the compound word, it is necessary to identify the words that compose it and, therefore, split the compound word into its compo-

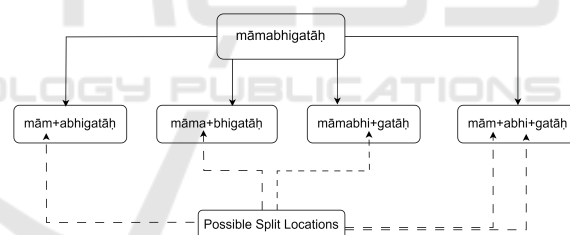


Figure 1: Different possible split locations of the compound word *māmabhigatāḥ* given by the standard Sandhi splitter. All splits are syntactically correct but only the first one is semantically meaningful.

nent words (see Figure 1).

Sanskrit word splitting is similar to other word segmentation tasks for Asian languages, such as Thai (Haruechaiyasak et al., 2008), Chinese, and Japanese (i.e., Kanji). Nowadays, most of the research is oriented towards Chinese and Japanese and focuses on enhancing the performance of single segmentation criterion. To reduce feature engineering efforts (Zheng et al., 2013; Pei et al., 2014; Cai and Zhao, 2016; Yao and Huang, 2016), in Chinese language, compound words are segmented through a sequence labeling process to assign labels to each character of the compound word and segment the component words.

^a <https://orcid.org/0009-0008-1664-5125>

^b <https://orcid.org/0000-0003-0833-4403>

^c <https://orcid.org/0000-0003-0304-0764>

^d <https://orcid.org/0000-0002-8766-6395>

*Corresponding author

In Sanskrit, the same process is generally accomplished by applying the Sandhi rule (Krishna et al., 2016). Besides Sandhi, a number of other morphological processes are involved in Sanskrit word formation, especially compounding and inflection. However, Sandhi deals explicitly with phonological and orthographic processes at word junctions or between words in a sentence. It is a process that changes the sounds and, on occasion, the structure of words, important for exact interpretation and generation of text in Sanskrit. Although compounding and inflection are also of prime importance for Sanskrit grammar, Sandhi is more crucial for it because it has a direct influence on the joining and pronunciation of words and hence their syntactic and semantic meaning. Segmenting Sanskrit words is not a trivial task because the Sandhi phenomenon causes phonetic transformations at word junctions. This not only results in word junctions being obscured, but the characters at the junctions are changed through deletion, insertion, and replacement operations. Learning how to predict the Sandhi split presents the additional challenge of not only correctly splitting compound words, but also predicting where to split them (Dave et al., 2021). The task is further complicated by the fact that the language is inflectional, with a large number of derivational affixes. Since Sanskrit compound words can be split at multiple locations, the split words will be syntactically correct but semantically meaningless. But till now researchers only consider the splitting of compound words that have one valid split location.

Figure 1 shows the possible split locations of the compound word *māmabhiḡatāḥ*. Although several splits are possible, only *mām+abhiḡatāḥ* is syntactically and semantically meaningful. Determining the semantically correct split for a word is context-dependent.

To account for this challenge, given in input a compound word, the method in (Dave et al., 2021) uses a bidirectional-LSTM to predict the Sandhi-window, which is a portion of the compound word wherein the Sandhi transformation has taken place. The model outputs a sequence where only the characters in the Sandhi-window are marked 1. All other characters are marked 0 (see Figure 2).

Similarly to (Dave et al., 2021), our model takes a Sanskrit compound word, i.e., a sequence of characters, as input. The model returns a sequence of 0 and 1 values, with 1 indicating the split location of the compound word. In contrast to (Dave et al., 2021), our novel model uses two BiLSTMs and a multi-head attention layer to predict the split location in the Sanskrit compound word. To the best of our knowledge this is the first work where two Bidirectional-LSTMs

act as encoders to capture the character-level contextual information of the Sanskrit compound word. These two encoders process the input sequence in direct and reverse order and, thus, will be named direct and reverse encoders respectively. As highlighted in (Sutskever et al., 2014), the processing order of the input sequence allows for capturing diverse information about the character-level contexts of each input character. Furthermore, multi-head attention layers focus on different parts of the sequence, i.e., short-term and long-term dependencies, to capture the correlations between the outputs of the direct and reverse encoders and predict the exact split point in the Sanskrit compound word. Using two parallel BiLSTMs, a network can learn from sequential data with more enriched, and subtle information captured, while higher model capacity is achieved at minimal overhead and computational complexity. This in turn helps improve the performance of predicting the location of Sanskrit compound word. To train and validate our model, we augmented the dataset used in (Dave et al., 2021) with new compound words and their respective splits collected from the Digital Corpus of Sanskrit (Hellwig, 2021). Incorrect splits, and words not following the Sandhi rule of compound word formation were manually discarded.

Overall, this paper proposes:

- A novel multi-headed attention-based bi-encoder model that exploits direct and reverse dynamics at a character level to predict the split location in compound Sanskrit words; The code implementation will be made publicly available to ease future comparisons.
- An enhanced dataset of compound Sanskrit words following the Sandhi rule with their respective split points. The dataset will be made publicly available to the research community¹.

In the following, we first describe the main motivation behind solving the described problem, then discuss related works and illustrate the proposed neural architecture. Finally, we present and discuss our experimental results and conclusions at the end.

2 MOTIVATION

Sandhi splitting is important for proper understanding and analysis of Sanskrit text, as it reveals the correct meaning of each word within the compound words. The difficulty in applying Sandhi split lies in the ability to apply the correct division of compound words.

¹Code and dataset available at <https://github.com/IrfanAliBabar/ABBIE-Sanskrit>

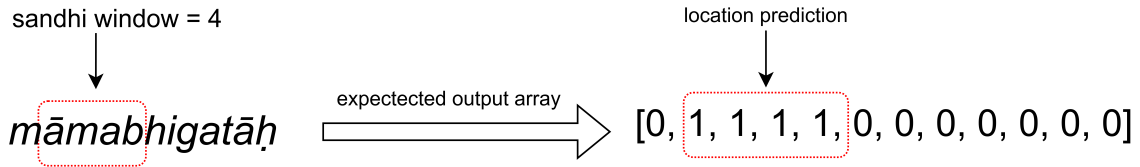


Figure 2: Sandhi window for predicting the location in a compound word.

We stress here that Sanskrit compound words can be divided at more than one point. Although the divided words will be syntactically correct, they can be semantically meaningless.

In this context, the work in (Patankar, 2023) highlights the need for tools that can effectively apply Sandhi splitting to ensure comprehensive morphological analysis. Such tools would enable computational linguistic research with better analysis and advanced models. Predicting the correct splitting location in Sanskrit compound words is important to develop better text understanding that supports digital preservation, revealing the cultural and historical significance of Sanskrit texts, and filling the resource gaps.

The few currently available Sandhi splitting tools use rule-based implementations (Linguistics, 2024) and have significant drawbacks in that they are rigid and cannot handle exceptions or variations. Using these tools requires a lot of time and resources to maintain and update these rules as new linguistic phenomena are discovered. One of the main issues is maintaining consistency among these rules, which requires a great linguistic expertise. Furthermore, the performance of a rule-based system with very diverse data sets may not be good if the rules are too narrow leading to inconsistent results and, under conditions of incomplete or incorrect rules, to errors that are very difficult to correct. Addressing these drawbacks with better techniques, mostly based on artificial intelligence, could improve the reliability of Sanskrit text analysis tools.

Better methods of Sandhi splitting will be of great help in language learning and teaching, making Sanskrit user-friendly for both teachers and students. Additionally, learning to predict the position of the Sandhi split can provide interesting linguistic insights into word formation in almost all Dravidian languages. Finally, modeling how words are formed in Sanskrit can provide an NLP framework for organizing words in other Indian languages.

3 RELATED WORK

Most of the NLP systems for Sanskrit word splitting combine *Panini's* phonetic and morphological rules

with lexical resources.

These systems are based on the application of formal methods (Huet, 2005; Goyal et al., 2007; Kulkarni and Shukl, 2009), or they use statistical approaches such as Dirichlet processes (Natarajan and Charniak, 2011), or finite-state methods (Mittal, 2010), graph queries (Krishna et al., 2016), or hybrid systems (Haruechaiyasak et al., 2008).

The biggest challenge to splitting words in Sanskrit is to find out the most semantically accurate word segmentation among all the possible splits of a compound word. (Krishna et al., 2016) solved this issue by modeling word segmentation as a query expansion task under a path-constrained random walks (PCRW) framework. They further fine-tuned their model by adding morphological information using Inductive Logic Programming (ILP). This resulted in significant improvement in the performance measures.

Several works have proposed the use of recurrent neural network to solve the problem. The paper (Hellwig, 2015) presents a neural network-based method that does compound splitting and Sandhi resolution jointly in Sanskrit text. The paper uses Long Short-Term Memory cells for labeling tasks in Sanskrit analysis. Hellwig proposed an alternative method of Sandhi resolution in Sanskrit by developing a classifier that depends on the gold standard string splits. The source sequence represents a string split in phonemes, while the target sequence presents transformations applied to each phoneme. A classifier will then be trained to correctly split compounds and apply the appropriate Sandhi resolution in the process, generating the corresponding Sandhi rule. Five possible transformations rules, namely *R1-5*, are defined that will guide the classification of phonemes in each string. Next, the paper (Hellwig and Nehrdich, 2018) proposes an end-to-end trained neural network for Sanskrit tokenization, which jointly performs compound splitting and resolves phonetic mergers (Sandhi), requiring neither feature engineering nor outside linguistic resources but working on parallel versions of raw and segmented text alone. The models are designed to work at a character level and allows word splitting in a sequence labeling framework for the Sanskrit language. Their best model uses convolutional and recurrent el-

ements with shortcut connections (rcNNshort).

The work in (Dave et al., 2021) also presents neural networks for splitting Sanskrit compound words, focusing specifically on Sandhi splitting. The work formulates a sequence-to-sequence prediction problem, employing recurrent neural networks (RNNs) in a two-step process. Without additional lexical resources or a priori information, the model takes a compound word as input and generates the split of compound words separated by a “+” character as output. This data-driven approach demonstrates superior performance compared to existing methods without needing extra lexical or morphological resources.

Some works have included attention to solve the problem. In (Aralikatte et al., 2018), the authors presented an attention-based deep learning method to determine the split position of compound words. Based on the predicted position of split, the method determines the Sandhi components by graphically segmenting the compound word. Compared to rule-based Sandhi models, knowing the split position allows to massively reduce the number of possible splits to check. In (Reddy et al., 2018), the model primarily identifies word splits and the correctness of unsandhied strings from a sandhied input string, which shall not take into account morphological and semantic details. The paper shows that the attention module improved the results significantly.

The problem of splitting compound words has also been studied for other languages, such as Chinese. In (Gong et al., 2019), the authors described a model that automatically switches between several segmentation criteria to improve the segmentation process with the assistance of multiple LSTM networks and a switcher. The model mainly follows two solution approaches: 1. Several LSTM cells that represent the criteria of segmentation and a switcher for routing between the LSTMs. 2. Switch-LSTM offers a more flexible solution to the multi-criteria Chinese word segmentation problem and supports knowledge transfer to the new criteria. The use of multiple segmentation criteria is important not only for enhanced performance but also for knowledge transfer and flexibility, leading to improvements in multi-criteria Chinese word segmentation.

The work in (Chen et al., 2015) addressed Chinese word segmentation as a sequence labeling task and compared several stacked bidirectional recurrent architectures. A final sentence-level likelihood layer (Collobert et al., 2011) is added to maximize the transition score for the target sequence represented using BMES encoding. Their best model was based on a single-layer bidirectional LSTM with bigrams of pre-trained character embeddings supplied as inputs

Our proposed architecture is totally different from all the models mentioned above. It uses two bi-encoders to extract contextual information from the input sequence and the reverse input sequence. It can capture different types of relationships and dependencies in the character-level input sequence through the use of a multi-head attention module that correlates the outputs of the two bi-encoders to predict the accurate split position in the Sanskrit compound word.

3.1 Existing Tools

Three prominent Sandhi splitter tools are available in the open domain. These are: (i) JNU splitter (Kumar, 2007), (ii) UoH splitter (Kumar et al., 2010), and finally, (iii) INRIA Sanskrit Reader Companion (Huet, 2003) (Goyal and Huet, 2013). All these tools addressed the challenge of splitting differently, but they work on a principle that is essentially the same. For a given compound word, an exhaustive set of rules is applied to every character thus yielding an extensive set of possible word splits. Subsequently, a morpheme dictionary of Sanskrit words, together with a variety of heuristics, is utilized to remove infeasible combinations of splits. However, none of these approaches effectively solves the inherent problem of first identifying the location of the split to which the rules need to be applied. This would help narrow the range of applicable rules and possibly produce more accurate splits.

4 PROPOSED NEURAL ARCHITECTURE: ABBIE

Sanskrit compound word splitting can be viewed as a sequence-to-sequence problem where the input is the character sequence of the compound word, and the output is a sequence of the same length where each element takes values in $[0 \dots 1]$ and indicates whether a splitting occurs (1) or not (0). Therefore, all elements in these output sequences are 0 except the elements corresponding to the predicted splitting positions which are set to 1.

To make the prediction more robust, we consider a splitting window of size 4 in which the splitting can occur. The split occurs after the second element takes on the value 1 within the window. For example, if the inputs are characters of the compound word “*māmabhigatāḥ*”, the expected output will be the sequence $[0, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0]$. So the split occurs after the second “*m*” of the compound word.

More formally, given a compound word $X = \{x_1, x_2, x_3, \dots, x_T\}$, our model analyzes the input se-

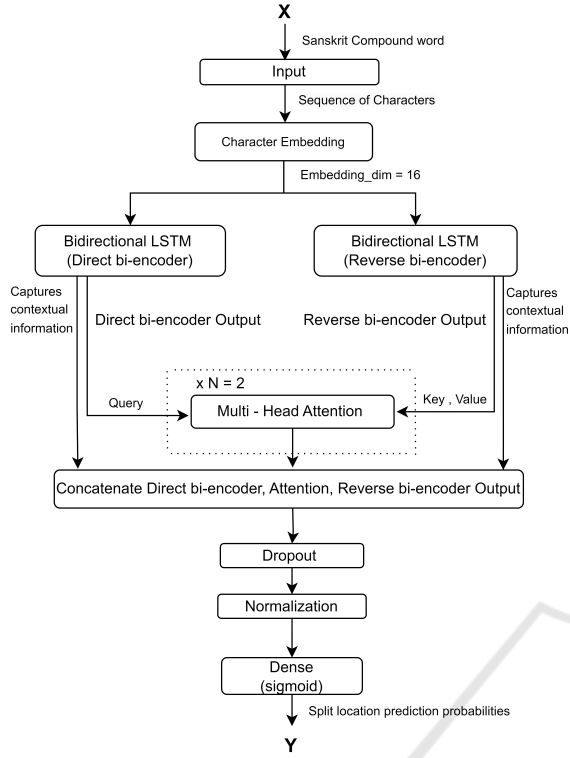


Figure 3: Architecture of Sanskrit Compound Word Split Position Prediction Model.

quence X in direct and reverse order and, through attention mechanisms aimed at correlating the hidden states of the recurrent layers, for each input character x_t provides in output $y_t \in L = \{0, 1\}$. The label y_t indicates whether the character is within the split window. Once the split window is calculated, the split position is identified as being between the second and third elements in the window.

To solve the Sandhi splitting problem, we propose ABBIE, whose architecture includes three main components:

1. a character Embedding Layer;
2. two Bi-LSTMs to encode character-level contextual information of the forward (direct) and backward (reverse) sequence;
3. multi-head attention layers to focus on different parts of the sequence (such as short-term and long-term dependencies) and capture more information.

Figure 3 shows the proposed architecture to predict the split location in the compound Sanskrit word. In the following, we provide a detailed description of each component.

4.1 Embedding Layer

Characters of the input sequence belongs to an alphabet (set of language symbols) of size V . We used a one-hot-encoding strategy to represent the characters of the given alphabet. Then, our model maps the language symbols into a learned embedding space.

Given an input sequence $X \in \mathbb{R}^{T \times V}$ where T is the sequence length, the embedding is achieved by using an Embedding Matrix $E \in \mathbb{R}^{V \times d}$ with d representing the embedding dimension. The embedding operation produces the embedded input E_X and can be written as:

$$E_X = XE, \quad \text{where } E_X \in \mathbb{R}^{T \times d}. \quad (1)$$

4.2 Direct and Reverse Bi-Encoders

We used two Bi-LSTMs (Hochreiter and Schmidhuber, 1997) as bi-encoders, which typically refers to an architecture encoding input sequences in both forward and backward directions to capture full information about the context. This makes information from the whole sequence in the past and future available to the model and very useful for tasks where context is important.

In particular, the first bi-LSTM, named *direct encoder*, processes the given input sequence as it is and, at each time t , computes:

$$\text{Forward LSTM: } \vec{h}_t = \text{LSTM}_{\text{fw}}(x_t, \vec{h}_{t-1}) \quad (2)$$

$$\text{Backward LSTM: } \overleftarrow{h}_t = \text{LSTM}_{\text{bw}}(x_t, \overleftarrow{h}_{t+1}) \quad (3)$$

The output of the first bidirectional-LSTM, h_t^{direct} , is the concatenation of the forward and backward LSTM outputs:

$$h_t^{\text{direct}} = [\vec{h}_t, \overleftarrow{h}_t], \quad \text{where } h_t^{\text{direct}} \in \mathbb{R}^{2n} \quad (4)$$

and n is the number of cell units of the LSTM layer. Over the entire input sequence, the first bi-LSTM produce $H^{\text{direct}} \in \mathbb{R}^{T \times 2n}$, whose rows are the vectors h_t^{direct} for $t \in [1, \dots, T]$.

The second bi-LSTM, named *reverse encoder*, processes the input sequence backward. At each time t , we consider the reversed input x'_t , and the second bi-LSTM will compute:

$$\text{Forward LSTM: } \vec{h}'_t = \text{LSTM}'_{\text{fw}}(x'_t, \vec{h}'_{t-1}) \quad (5)$$

$$\text{Backward LSTM: } \overleftarrow{h}'_t = \text{LSTM}'_{\text{bw}}(x'_t, \overleftarrow{h}'_{t+1}) \quad (6)$$

Similar to the first bi-LSTM, the output of the second bi-LSTM, h_t^{reverse} , is the concatenation of the outputs of its forward and backward LSTMs:

$$h_t^{\text{reverse}} = [\vec{h}'_t, \overleftarrow{h}'_t], \quad \text{where } h_t^{\text{reverse}} \in \mathbb{R}^{2n}. \quad (7)$$

Over the entire reversed input sequence, the second bi-LSTM produce $H^{\text{reverse}} \in \mathbb{R}^{T \times 2n}$, whose rows are the vectors h_t^{reverse} for $t \in [1, \dots, T]$.

Hyperbolic tangent (tanh) activation function is used by all the LSTMs.

Using two bidirectional LSTMs that separately process the input sequence in different order (direct and reverse) has advantages over using a single Bi-LSTM with $2n$ units. The paper in (Sutskever et al., 2014) shows that the order (direct and reverse) of processing a sequence enables the LSTM to learn different information. In particular, a single Bi-LSTM learns the left and right context information of each input character. This will lead to a nuanced understanding of the preceding and subsequent sequence of characters at each sequential element.

In the proposed approach, we use two Bi-LSTMs. The direct encoder acquires the left (LSTM_{fw}) and right (LSTM_{bw}) context of each character of the sequence X . The reverse encoder analyzes the input sequence in reverse order, $\text{reverse}(X)$. To put it simply, the reverse encoder learns the right (LSTM_{fw}) and left (LSTM_{bw}) context of the original (direct) sequence X . This is important in the subsequent processing stages when the outputs of the two bi-encoders is used in the multi-head attention layer.

Furthermore, two Bi-LSTMs with fewer units each provide better regularization and generalization than a single Bi-LSTM with many units. At the same time, independent regularization after each bi-encoder, especially the use of dropout, reduces the chances of overfitting. This effectively balances the complexity involved in splitting Sanskrit compounds and improves the overall performance of the model.

4.3 Multi-Head Attention Layers

One of the mechanisms incorporated into neural networks for natural language processing tasks, more particularly in the Transformer architecture, is the Multi-Head Attention (Vaswani et al., 2017). We utilize this mechanism with the bi-LSTM outputs to help our model learn to focus on different parts of the input sequence simultaneously and learn various aspects of relationships between different positions of the sequence. In particular, the multi-head attention layers will correlate different information about left and right contexts at a character-level. In the case of Sanskrit Sandhi splitting, this mechanism will help the model attend to different parts of the compound word to correctly identify the split positions.

Attention transforms inputs into three different vectors: query Q , key K , and value V . It is well known that the attention mechanism computes (Vaswani

et al., 2017):

$$\text{Attention}(Q, K, V) = \text{Softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V. \quad (8)$$

The idea behind attention is to compare a query vector against a set of key vectors to compute attention scores, which are used to create a weighted sum of the value vectors. The Softmax function is applied to the product $QK^\top$ to obtain a set of normalized attention weights to determine how relevant each position is relative to every other position in computing the output. The scaling factor $\sqrt{d_k}$ is used to prevent the dot product from being too large in magnitude, leading to unstable training and slow convergence.

In our model, the attention mechanism considers the outputs of the two bi-LSTMs as follows:

$$Q = H^{\text{direct}} \quad (9)$$

$$K = H^{\text{reverse}} \quad (10)$$

$$V = H^{\text{reverse}} \quad (11)$$

Multiple attention heads allow to capture diverse kinds of relationships and dependencies in the data, which is very helpful in a complex task such as predicting the split position. In particular,

$$\text{MultiHead}(Q, K, V) = [\text{head}_1, \dots, \text{head}_H]W^O \quad (12)$$

where

$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

and W^O is the output weight matrix.

In our model, the multi-head attention output A can be written as:

$$A = \text{MultiHead}(H^{\text{direct}}, H^{\text{inverse}}, H^{\text{inverse}}). \quad (13)$$

4.4 Target Sequence

The outputs of the two bi-encoders and of the multi-head attention module are concatenated:

$$H^{\text{merged}} = \text{Concat}\left(H^{\text{direct}}, A, H^{\text{reverse}}\right) \quad (14)$$

where $H^{\text{merged}} \in \mathbb{R}^{T \times 6n}$, and n is the hidden state size.

In this way, H^{merged} gives a richer, more informative representation of the input sequence that is useful to predict more accurately where the split occurs in the compound word. This concatenation guarantees that both the local context and the global dependencies, which the attention mechanism would capture, will be available to the dense layer (with one neuron), which is used to predict the splitting window. A sigmoid activation function is used by the dense layer.

4.5 Detecting the Split Location

Our main objective is to predict the most likely split location of a compound Sanskrit word based on the output of the model. The model gives an output sequence in which values can range between 0 and 1. As already stated, the model is trained so to predict a splitting window of length $input_length = 4$.

The challenge lies in how to exploit the sequence of output values for the identification of the most probable window where the actual split occurs. Thus, we use a sliding window over the output sequence and select the window with the highest sum of values. This sliding window helps in smoothing the predictions and makes sure it considers a sequence of characters as a potential split location. Thus, for every position j in the sequence, we compute a sum of model's output values for the window starting at position j , ending at position $j + input_length - 1$.

The starting position of this window, max_start is considered as the most likely starting point of the window split. Once the window split is found, the predicted split arises in the middle of the splitting window. In our implementation, being $input_length = 4$, the split arises after the second element taking the value 1 within the window.

5 TRAINING AND IMPLEMENTATION DETAILS

ABBIE has been implemented in Python 3.10.14 with Keras API running on the TensorFlow backend. We used a character embedding size of 16, bi-directional encoders with 128 units. A dropout layer of 0.5 is applied after each Bi-LSTM. We used 2 attention heads in the multi-head attention layer and dropout layer 0.5 after the multi-head attention layer and the normalization layer. We used batches of size 64, and trained the model for 40 epochs by the Adam optimizer using a workstation equipped with an NVIDIA GeForce RTX 2070 GPU. The total number of trainable parameters in our model is 150,737. We adopted the mean squared error (MSE) as loss function.

6 EXPERIMENTAL RESULTS

In this section, we discuss data preparation, accuracy metric for our model, evaluation and comparison of our results with RNN-BiLSTM architecture (Dave et al., 2021). We also present ablation studies by comparing different variants of our model.

6.1 Dataset

We developed a new dataset containing a total number of compound words equal to 96535 from different books of the Digital Corpus of Sanskrit repository (Hellwig, 2021). We combined the dataset from the University of Hyderabad corpus (UoH, 2024), comprising 77842, with the data we collected. Overall, the full number of samples in the merged data before preprocessing was 174377. We pre-processed the merged data to remove duplicates. We discarded all compounds with more than two splits, incorrect splits, and words not following the Sandhi rules of compound word formation. Overall, the final dataset includes 99900 samples after the pre-processing. 80% of the data was used for training the model, and the remaining for testing. 10% of the training data is used for validation of the model. The dataset, originally in the IAST (International Alphabet of Sanskrit Transliteration) format (Hellwig, 2021), is pre-processed to convert to SLP1 (Sanskrit Library Phonetic) (Learn Sanskrit, 2024), a transliteration scheme better suited for computational analysis due to its phonetic nature.

6.2 Split Location Prediction Accuracy

To measure the accuracy of our model, we compare the predicted split location j with the true one j^* . Here, j indicates the character location within the compound word after which the split occurs.

The Split Location Prediction Accuracy is calculated as:

$$Accuracy = \frac{\sum_{i=1}^N \delta(j, j^*)}{N} \quad (15)$$

where the function $\delta(j, j^*)$ returns 1 if $j = j^*$ and 0 otherwise. N is the number of compounds in the test set.

6.3 Results and Comparison

In Tables 1 and 2 we compare our model to the RNN-BiLSTM model presented in (Dave et al., 2021). We trained our ABBIE model by minimizing the MSE loss function on the dataset in (Dave et al., 2021). As shown in Table 1, the RNN-BiLSTM achieves an accuracy value of 92.3%, and our ABBIE model achieves a higher accuracy value of 93.1% on the same dataset and minimizing the same loss function.

However, the dataset in (Dave et al., 2021) is limited and smaller than ours. Our dataset is built from various religious books, poems, etc, and contains complex compound words as well. As shown in Table 2, on our dataset, the RNN-BiLSTM achieves a

lower accuracy value, of 87%, suggesting that the proposed dataset is more challenging than the one proposed in (Dave et al., 2021). We stress here that we used the publicly available implementation of (Dave et al., 2021) to train the RNN-BiLSTM on our dataset, where the MSE loss function is minimized as well. Overall, compared with RNN-BiLSTM, ABBIE achieves superior performance. In particular, ABBIE achieves an increase in the accuracy value of about 2.3% on equal terms of data and experimental protocol.

To study the effect of adopting different loss functions, we train our model, ABBIE, on our dataset by using Mean Squared Error (MSE) and Binary Cross-Entropy (BCE) functions. As shown in Table 2, when using BCE, ABBIE achieves an accuracy value of 87.7%. Instead, when using MSE, ABBIE achieves an accuracy value of 89.27%. Overall, for our problem, an error-based loss function seems to be more effective.

Figs. 4 and 5 help to visualize how attention values are distributed across different characters in a (padded) compound word. For each character of the word (horizontal axes), the figures show the 128-dimensional attention vector obtained from the Multi-Head attention layer by Eq. 13. Under each figure the True split window and the predicted split window are also reported. The red dashed vertical line represents the True split location, while the green vertical line represents the predicted one.

In Fig. 4, for the compound word “bhArasADhanam”, the predicted split position (green dashed line) exactly corresponds to the true split position (red dashed line), which is between “bhAra” and “sAdhanam”. This means the model has learned to focus its attention on the correct characters within the sequence. A subset of lines, especially in the middle of the figure, shows focused attention close to the true split position of higher intensity (blue or red regions) around the respective positions. The presence of more lines with high intensity at positions around the split gives a strong and clear signal about the position, reducing the ambiguity and hence ensuring correctness. This behavior indicates that the model learns to identify key features indicating the presence of a split position. On the other hand, there is some redundancy since several elements of the attention vectors are observed to contribute to the same prediction, likely adding to robustness.

In Fig. 5, for the compound word “kalApA-VAtAH”, the predicted split locations are far from the true split locations that are between “kalApA” and “VAtAH”. This is clear from the misalignment of the green and red dashed lines. The attention heads do

Table 1: Comparison with the state-of-the-art on the dataset (Dave et al., 2021).

Model	Accuracy
RNN-BiLSTM (Dave et al., 2021)	92.3%
ABBIE (ours)	93.1%

Table 2: Comparison with the state-of-the-art on our dataset.

Model	Accuracy
RNN-BiLSTM (Dave et al., 2021)	87.0%
ABBIE - MSE (ours)	89.27%
ABBIE - BCE (ours)	87.7%

not pay proper attention to the true split locations, either diffused or misaligned, which drives the model to attend to other positions. Some elements of the attention vector appear to focus on higher intensities around the position of a true split but do not give sharp attention. Attention is diffused across neighboring positions. Some lines show a broader attention to irrelevant positions, which might result in the incorrect split position prediction. The true split window clearly identifies the start of the correct split position (1 at the true split) and the predicted split window deviates significantly, with earlier activations (1s) that align with the incorrect split.

These heatmaps indicate that not all attention features contribute the same in predicting the split window. Some features show sharper focus at critical positions, whereas others distribute their attention over the sequence. This also aligns with the behavior of a multi-head attention mechanism that allows to learn aspects complementary to one another when attending to the input.

6.4 Ablation Studies

To better understand the contribution of each component to the ABBIE model, we trained and compared different variants of our model by using the MSE function and on equal terms of experimental protocol. Results are summarized in Table 3 and discussed in details in the following.

To assess the contribution of the multi-head attention module, we trained the “Two Bi-Encoders” model. In practice, this model concatenates the out-

Table 3: Ablation Study. The table shows the contribution of the multi-head attention module and of the two bi-encoders to the model accuracy.

Model	Accuracy
Two Bi-Encoders	86.5%
One Bi-Encoder + Attention	87.5%
ABBIE (ours)	89.27%

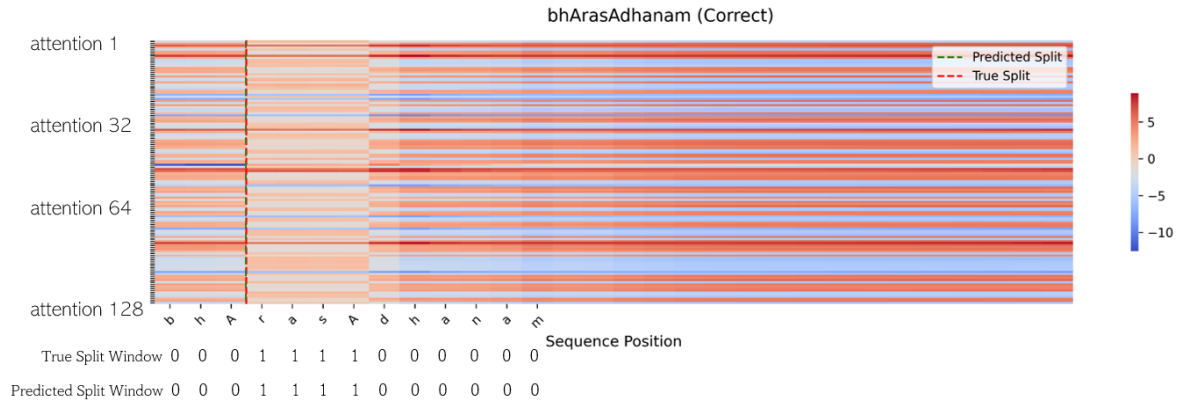


Figure 4: Heatmap for Correct Split Window Prediction.

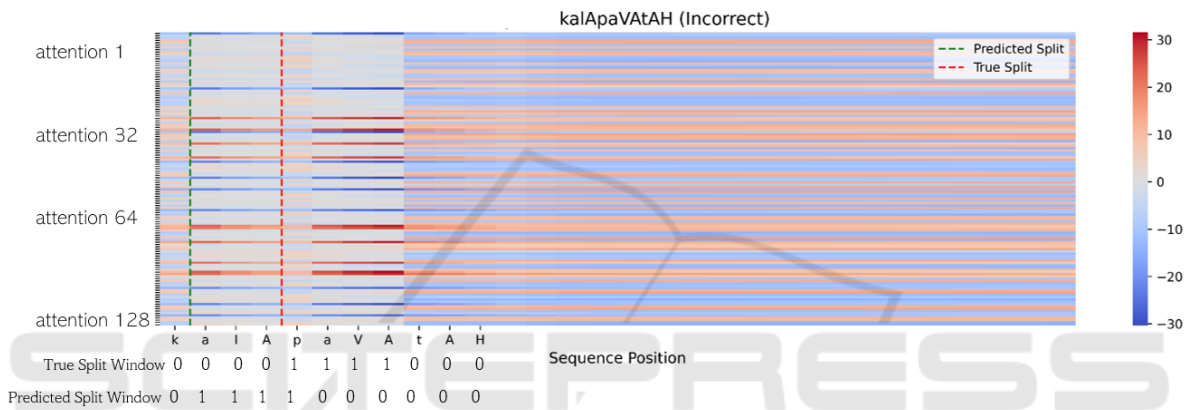


Figure 5: Heatmap for Incorrect Split Window Prediction.

puts of the direct and reverse bi-LSTMs and uses a dense layer to output the target sequence. No attention mechanism is used in this model. The modified model achieves an accuracy of 86.5% showing a clear reduction of 2.8% compared to the ABBIE model. The results are also comparable to the one reached by the RNN-BiLSTM model, which does not include any attention mechanism as well. This result suggests that multi-head attention plays a crucial role in capturing the most relevant information in the compound word, thereby enhancing the accuracy of the split location prediction.

To assess the contribution of using direct and reverse Bi-Encoders, we trained the “*One Bi-Encoder + Attention*” model. This model only includes a single Bi-directional LSTM. The output of the direct Bi-Encoder feeds the multi-head attention module. The model achieves an accuracy of 87.5%, showing a reduction of 1.8% compared to ABBIE. The accuracy value is slightly superior to the one reached by the model in (Dave et al., 2021). The result also confirms the findings of (Sutskever et al., 2014), which can be summarized here by stating that the direct and reverse Encoders capture different contextual informa-

tion. Moreover, the results highlight the importance of multi-head attention in comparing the direct and reverse dynamics of the input sequence to predict the Sandhi split location.

7 CONCLUSION AND FUTURE WORK

In this research work, we focus on the problem of predicting where to split compound Sanskrit words whose formation follows the Sandhi rule without any use of external resources such as phonetic or morphological analyzers. To address the problem, we propose a novel deep learning method that uses two Bi-LSTMs to encode character-level contextual information for direct and reverse sequence. The two bi-encoders are used with a multi-head attention module to focus on the different parts of the compound word and predict where the split arises. We also collected an enhanced dataset that augment the UoH dataset with challenging compound words from the Digital Corpus of Sanskrit repository. We compared the per-

formance of our proposed ABBIE model with other RNN based architecture.

Overall, this paper shows that bidirectional encoders on the direct and reverse input sequences can be used together with an attention-based module to get better contextual information on the input sequence.

The main limitation of our work, as well as of all other works on the topic, is that it considers compound words having only one Sandhi split. Despite we could use our model recursively until no split is found anymore, in future work, we intend to enhance our model to also consider the splitting of compound words which have more than one valid split locations integrate with the morphological analyzer and observe the common error patterns.

ACKNOWLEDGMENTS

This work is partially supported by the PNRR MUR Project ITSERR (Italian Strengthening of the ESFRI RI RESILIENCE) (IR0000014, CUP B53C22001770006).

REFERENCES

- Aralikatte, R., Gantayat, N., Panwar, N., Sankaran, A., and Mani, S. (2018). Sanskrit sandhi splitting using seq2(seq)². *arXiv preprint arXiv:1801.00428*.
- Cai, D. and Zhao, H. (2016). Neural word segmentation learning for chinese. *arXiv preprint arXiv:1606.04300*.
- Chen, X., Qiu, X., Zhu, C., Liu, P., and Huang, X.-J. (2015). Long short-term memory neural networks for chinese word segmentation. In *Proceedings of the 2015 conference on empirical methods in natural language processing*, pages 1197–1206.
- Collobert, R., Weston, J., Bottou, L., Karlen, M., Kavukcuoglu, K., and Kuksa, P. (2011). Natural language processing (almost) from scratch. *Journal of machine learning research*, 12:2493–2537.
- Dave, S., Singh, A. K., AP, D. P., and Lall, P. B. (2021). Neural compound-word (sandhi) generation and splitting in sanskrit language. In *Proceedings of the 3rd ACM India Joint International Conference on Data Science & Management of Data (8th ACM IKDD CODS & 26th COMAD)*, pages 171–177.
- Gong, J., Chen, X., Gui, T., and Qiu, X. (2019). Switch-lstms for multi-criteria chinese word segmentation. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 33, pages 6457–6464.
- Goyal, P., Arora, V., and Behera, L. (2007). Analysis of sanskrit text: Parsing and semantic relations. In *International Sanskrit Computational Linguistics Symposium*, pages 200–218. Springer.
- Goyal, P. and Huet, G. (2013). Completeness analysis of a sanskrit reader. In *International Symposium on Sanskrit Computational Linguistics*, pages 130–171.
- Goyal, P., Huet, G., Kulkarni, A., Scharf, P., and Bunker, R. (2012). A distributed platform for sanskrit processing. In *Proceedings of COLING 2012*, pages 1011–1028.
- Haruechaiyasak, C., Kongyoung, S., and Dailey, M. (2008). A comparative study on thai word segmentation approaches. In *2008 5th International Conference on Electrical Engineering/Electronics, Computer, Telecommunications and Information Technology*, volume 1, pages 125–128. IEEE.
- Hellwig, O. (2010–2021). *The Digital Corpus of Sanskrit (DCS)*.
- Hellwig, O. (2015). Using recurrent neural networks for joint compound splitting and sandhi resolution in sanskrit. In *4th Biennial workshop on less-resourced languages*.
- Hellwig, O. and Nehrdich, S. (2018). Sanskrit word segmentation using character-level recurrent and convolutional neural networks. In *Proceedings of the 2018 conference on empirical methods in natural language processing*, pages 2754–2763.
- Hochreiter, S. and Schmidhuber, J. (1997). Long short-term memory. *Neural computation*, 9(8):1735–1780.
- Huet, G. (2003). Towards computational processing of sanskrit. In *International Conference on Natural Language Processing (ICON)*. CiteSeer.
- Huet, G. (2005). A functional toolkit for morphological and phonological processing, application to a sanskrit tagger. *Journal of Functional Programming*, 15(4):573–614.
- Krishna, A., Santra, B., Satuluri, P., Bandaru, S. P., Faldu, B., Singh, Y., and Goyal, P. (2016). Word segmentation in sanskrit using path constrained random walks. In *Proceedings of COLING 2016, the 26th International Conference on Computational Linguistics: Technical Papers*, pages 494–504.
- Kulkarni, A. and Shukl, D. (2009). Sanskrit morphological analyser: Some issues. *Indian Linguistics*, 70(1-4):169–177.
- Kumar, A., Mittal, V., and Kulkarni, A. (2010). Sanskrit compound processor. In *Sanskrit Computational Linguistics: 4th International Symposium*, pages 57–69, New Delhi, India. Springer.
- Kumar, S. (2007). Sandhi splitter and analyzer for sanskrit (with reference to ac sandhi). Submitted, 2007.
- Learn Sanskrit (2024). Sanscript: Sanskrit transliteration. Accessed: 2024-08-08.
- Linguistics, S. C. (2002-24). Sanskrit computational linguistics.
- Mittal, V. (2010). Automatic sanskrit segmentizer using finite state transducers. In *Proceedings of the ACL 2010 Student Research Workshop*, pages 85–90.
- Natarajan, A. and Charniak, E. (2011). s3-statistical sandhi splitting. In *Proceedings of 5th international joint conference on natural language processing*, pages 301–308.
- Patankar, M. P. S. (2023). Exploring the intricacies of sandhi in sanskrit: phonological rules and linguistic

- significance. *International Journal of Applied Engineering & Technology*.
- Pei, W., Ge, T., and Chang, B. (2014). Max-margin tensor neural network for chinese word segmentation. In *Proceedings of the 52nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 293–303.
- Reddy, V., Krishna, A., Sharma, V. D., Gupta, P., Goyal, P., et al. (2018). Building a word segmenter for sanskrit overnight. *arXiv preprint arXiv:1802.06185*.
- Sutskever, I., Vinyals, O., and Le, Q. V. (2014). Sequence to sequence learning with neural networks. *Advances in neural information processing systems*, 27.
- UoH (2024). Sanskrit corpus.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. (2017). Attention is all you need. *Advances in neural information processing systems*, 30.
- Yao, Y. and Huang, Z. (2016). Bi-directional lstm recurrent neural network for chinese word segmentation. In *Neural Information Processing: 23rd International Conference, ICONIP 2016, Kyoto, Japan, October 16–21, 2016, Proceedings, Part IV 23*, pages 345–353. Springer.
- Zheng, X., Chen, H., and Xu, T. (2013). Deep learning for chinese word segmentation and pos tagging. In *Proceedings of the 2013 conference on empirical methods in natural language processing*, pages 647–657.



One Model is All You Need: ByT5-Sanskrit, a Unified Model for Sanskrit NLP Tasks

Sebastian Nehrdich^{1,3}, Oliver Hellwig², and Kurt Keutzer³

¹Heinrich Heine University Düsseldorf,

²University of Zürich, Department of Comparative Language Science

³University of California, Berkeley, Berkeley Artificial Intelligence Research (BAIR)

Abstract

Morphologically rich languages are notoriously challenging to process for downstream NLP applications. This paper presents a new pre-trained language model, ByT5-Sanskrit, designed for NLP applications involving the morphologically rich language Sanskrit. We evaluate ByT5-Sanskrit on established Sanskrit word segmentation tasks, where it outperforms previous data-driven approaches by a considerable margin and matches the performance of the current best lexicon-based model. It is easier to deploy and more robust to data not covered by external linguistic resources. It also achieves new state-of-the-art results in Vedic Sanskrit dependency parsing and OCR post-correction tasks. Additionally, based on the Digital Corpus of Sanskrit, we introduce a novel multitask dataset for the joint training of Sanskrit word segmentation, lemmatization, and morphosyntactic tagging tasks. We fine-tune ByT5-Sanskrit on this dataset, creating a versatile multitask model for various downstream Sanskrit applications. We have used this model in Sanskrit linguistic annotation projects, in information retrieval setups, and as a preprocessing step in a Sanskrit machine translation pipeline. We also show that our approach yields new best scores for lemmatization and dependency parsing of other morphologically rich languages. We thus demonstrate that byte-level pretrained language models can achieve excellent performance for morphologically rich languages, outperforming tokenizer-based models and presenting an important vector of exploration when constructing NLP pipelines for such languages.

1 Introduction

It is generally acknowledged that morphologically rich languages (MRL) are challenging for NLP (Tsarfaty et al., 2020). While language modeling has addressed this challenge, e.g. by integrating subword information (see e.g. Bojanowski et al., 2017), there is surprisingly little systematic research on how efficient models for low-level tasks

such as tokenization, lemmatization, morphosyntactic analysis, and dependency parsing can be designed for MRLs. Access to this low-level information is relevant for downstream tasks such as information retrieval and question answering, as well as for linguistic and literary studies.

In this paper, we introduce a unified model that jointly performs these tasks for Sanskrit, an ancient South-Asian MRL, which has been continuously attested since 1,300 BCE. Vedic, its archaic level, primarily focusses on the description of the Soma and the fire sacrifice. Starting around 300 BCE, the majority of Sanskrit literature was composed in classical Sanskrit, encompassing a vast array of domains from religious hymns to scientific and narrative texts (see Table 2). Linguistic processing of Sanskrit poses challenges due to its rich morphology and vocabulary, free word order, heavy compounding, and particularly due to the phonetic merging of individual words into longer strings (Sandhi; see e.g. Gupta et al. 2020), as can be observed in this example:

yuvoh̐ hi mātā aditiḥ
your indeed mother Aditi
Aditi is indeed your mother.

With Sandhi: yuvorhi mātāditiḥ

Here, the words *yuvoh̐* and *hi* as well as *mātā* and *aditiḥ* are merged into longer strings, thereby changing their contact phonemes ($h+h \rightarrow rh$, $\bar{a}+a \rightarrow \bar{a}$). While the synthesis of Sandhi is deterministic, its analysis is not, as the new phoneme $\bar{a}$ in *mātāditiḥ* could also arise from $a+a$, $a+\bar{a}$ or $\bar{a}+\bar{a}$. As a consequence, Sanskrit word segmentation (SWS) needs to be performed in order to enable tasks such as lemmatization, morphosyntactic tagging, and dependency parsing.

We propose a framework in which we pretrain a character-level Sanskrit language model based on ByT5 on a large body of Sanskrit data before jointly fine-tuning it on a number of downstream NLP tasks, which we reformulate as sequence-

generation tasks. This paradigm brings large performance gains, leading to new SOTA results on established Sanskrit NLP benchmarks. We emphasize creating a system that is as simple as possible to train and deploy, without depending on complex pre- or postprocessing steps and retaining high performance on data that shows challenges such as OCR mistakes or the use of non-standard language not sufficiently covered by available linguistic resources.

In concrete terms, we achieve a **gain of 8.8 points on perfect sentence matching score (PM) for the Hackathon SWS benchmark** compared to the current state-of-the-art, while we come close by 0.13 in performance on the SIGHUM dataset to the currently best performing lexicon-driven model. **We achieve 4.88 points improvement on the SWS DCS 2018 benchmark.** On Vedic dependency parsing, we achieve **2.18 points improvement on UAS and 2.60 points on LAS** compared to the current state-of-the-art. On OCR post-correction, we outperform the currently best approach by **0.29 lower CER and 3.16 lower WER**. We also show that our approach yields the best performance on lemmatization and dependency parsing for three other MLR languages.

We also present a novel dataset for the training and evaluation of three central Sanskrit NLP tasks based on the Digital Corpus of Sanskrit (DCS): Word segmentation, lemmatization, and morphosyntactic tagging. We show that our pretrained model outperforms other baselines on these new tasks. We also demonstrate that jointly training on the tasks of SWS, lemmatization, and morphosyntactic tagging on top of the pretrained language model leads to the best performance. This enables the deployment of one single model without dependence on external linguistic resources to handle all relevant NLP tasks for annotated Sanskrit corpus building with the best performance. We show that training and evaluating this model on pseudo-paragraph-level, where multiple sentences are predicted at once, gives a distinct performance advantage due to the available contextual information.

In Section 2, we give an overview of the relevant research literature. In Section 3, we discuss the pretraining and fine-tuning datasets used in this paper. Section 4 introduces the layout of our proposed multitask framework. In Section 5, we first evaluate the model on established Sanskrit word

segmentation, Vedic Sanskrit dependency parsing, OCR post-correction tasks, as well as on other MLR languages, and then present the performance of the unified model trained on the new dataset. We also perform a detailed manual analysis of the error patterns of the multitask model. We make the code, all relevant datasets, the pretrained base model as well as the fine-tuned multitask model available under the Apache license 2.0 at <https://github.com/sebastian-nehrdich/byt5-sanskrit-analyzers>.

2 Related Research

The pretrain-fine-tune paradigm, where a pretrained language model (PLM) trained on a large corpus of unlabeled data is subsequently fine-tuned on a smaller dataset of task-specific labeled data, is the de-facto standard approach for NLP tasks such as part-of-speech and morphosyntactic tagging, sentence classification, and many more since the publication of the encoder-only approaches BERT (Devlin et al., 2019) and ELMo (Peters et al., 2018) in 2018. When it comes to morphologically rich languages, the good performance of this paradigm is demonstrated for Turkish in Özçift et al. (2021), while Bamman and Burns (2020) and Nehrlich and Hellwig (2022) show the superior performance of BERT on linguistic annotation tasks for the morphologically rich classical language Latin.

T5 (Raffel et al., 2019) introduced a new pre-training paradigm where both encoder and decoder are trained. This encoder-decoder architecture enables the fine-tuning of the same base model on diverse tasks such as translation, question answering, and text classification with the same hyperparameters and loss function. Sanh et al. (2022) further show how the T5 paradigm can be used efficiently in a multitask setup with large variation between the different tasks. For morphologically rich languages, language models that make use of character-level information show superior performance to those operating on word-level alone (Gerz et al., 2018). While a number of openly available pretrained language models exist, only Xue et al. (2021) followed a tokenizer-free byte-level approach, resulting in strong performance on linguistic tasks and achieving the best performance on the morphological inflection task.

Most approaches to Sanskrit NLP tasks such as Sanskrit word segmentation (SWS) can be broadly separated into two groups: lexicon-based and data-

driven. For a recent, comprehensive overview of the relevant literature, see [Sandhan et al. \(2022\)](#). Lexicon-driven approaches rely on external linguistic resources, while data-driven approaches learn from data alone and are therefore less complex to train and deploy. The main drawback of data-driven approaches is that they cannot access latent knowledge contained in lexical resources. [Sandhan et al. \(2022\)](#) combine lexicon-based and data-driven aspects, formulating SWS as a character-level sequence labeling task that uses lexical information whenever available. [Krishna et al. \(2020\)](#) presents a lexicon-based multitask model that handles SWS, morphological parsing, dependency parsing, syntactic linearization, and prosodic linearization. To our knowledge, this is the only other published multitask approach to central Sanskrit NLP tasks.

Pretrained language models supporting Sanskrit are available, but they are not yet widely used for Sanskrit linguistic tasks. [Conneau and Lample \(2019\)](#) included Sanskrit data in its pretraining setup. [Hellwig et al. \(2023\)](#) trained and evaluated encoder-only PLMs for the task of Vedic Sanskrit dependency parsing, coming to the conclusion that they do not offer clear advantages in performance yet due to the comparatively small amount of training data used.

3 Data

For pretraining, we use the Sanskrit data of the Sangraha dataset ([Khan et al., 2024](#)) as a basis, which mostly consists of data gained by a comprehensive OCR effort of the Sanskrit-related literature available at the Internet Archive¹. We only use the language-verified split of this dataset and none of the synthetic data. We decided to use this noisy OCR-based dataset following the observation made in [Bamman and Burns \(2020\)](#), where a PLM for Latin trained on a noisy corpus consisting of largely OCR'd data achieved new state of the art results on Latin POS tagging tasks. We augment this data with high-quality human input Sanskrit data from the GRETEL collection² and the Digital Sanskrit Buddhist Canon.³ The statistics of the dataset are shown in Table 1.

We use IAST transliteration for pretraining as well as all of the fine-tuning tasks, as this yields

Source	Number of Characters
IndicLLMSuite	5,173,251,798
GRETEL	253,712,457
DSBC	2,473,226

Table 1: Composition of the pretraining dataset. Number of characters is measured in character count in IAST roman transliteration.

Category	Number of Characters
Epics	9,814,868
Vedic	7,211,586
Science	6,299,576
Purāṇa	4,682,010
Poetry	2,028,535
Buddhist	1,762,012
other	2,728,511

Table 2: Distribution of the fine-tuning data according to different categories. Number of characters is measured in character count in IAST roman transliteration.

clear efficiency advantages compared to Devanagari when training on the individual byte level, with half the bytes needed. While other transliteration schemes such as SLP1 offer further small gains in efficiency, we decided against using them as the human readability advantages of IAST lead to less overhead during training and evaluation, as well as less complex deployment pipelines.

3.1 Fine-tuning Dataset

The fine-tuning data utilized in this study for the SWS, lemmatization, morphological tagging, and dependency parsing tasks comes from the Digital Corpus of Sanskrit (DCS; [Hellwig 2010–2024](#)), a collection of classical and Vedic texts with manually validated lexical and morphosyntactic annotations. For some Vedic texts, the DCS also provides manually validated syntactic annotations ([Hellwig et al., 2023](#)). The complete annotation is available as text files in CoNLL-U format,⁴ serving as input for the multitask and dependency parsing models described in this paper. We use a snapshot of the DCS dataset from April 2024. Table 2 gives an overview of the DCS fine-tuning data, showing its bias towards narrative (epics, Purāṇas), Vedic, and scientific texts.

¹archive.org

²<https://gretel.sub.uni-goettingen.de/gretel.html>

³<https://www.dsbcproject.org/>

⁴<https://github.com/OliverHellwig/sanskrit/tree/master/dcs/data/conllu>

4 Proposed Method

We propose the combination of the following paradigms in order to generate an efficient, high-performing end-to-end framework for various Sanskrit NLP tasks: We first pretrain a byte-level Sanskrit PLM based on the ByT5 architecture, which is distributed under the Apache license 2.0, overcoming the limitation of lack of access to latent information for data-driven approaches. Then, we reformulate the central Sanskrit NLP tasks of word segmentation, lemmatization, and morphosyntactic tagging as sequence generation tasks, using a novel serialization strategy. In order to distinguish between the different tasks, we use prefix letters at the beginning of the input sequence to indicate the task. “S” for segmentation, “L” for lemmatization, and “M” for morphosyntactic tagging. Inspired by T0 (Sanh et al., 2022), we combine these tasks into a unified multitask setup, enabling the fine-tuning of a single model to handle all of them simultaneously. The schema of this approach is demonstrated in Figure 2.

The full morphosyntactic tags of the DCS consume on average 46 characters, making their prediction with a byte-level LM challenging. We therefore propose a serialization strategy by manually mapping the morphosyntactic tags to unused letter combinations of the IAST alphabet, reducing the number of needed tokens per tag significantly. The full tags can be restored based on this mapping without information loss. Figure 1 demonstrates this process. On average, the compression ratio of this method is 0.14.

5 Experiments

Models were trained on GPU nodes of 8 NVIDIA A6000 48GB GPUs. The time needed for pre-training was one week, while the fine-tuning runs varied between 2 and 8 hours. The joint multitask model took 32 hours to finetune. We leveraged the DeepSpeed library <https://www.deepspeed.ai/> for training in half precision bf16, and for making efficient use of the multi-GPU setup.

For the foundation model, we further pretrain a ByT5 model (Xue et al., 2021) in the “base” configuration with 582M parameters on the entire dataset for 100,000 steps with a batch size of 512 and a sequence length of 512. The resulting model is called ByT5-Sanskrit in this paper. According to the scaling laws presented in Hoffmann et al.

(2022), the optimal number of parameters for our training dataset size of 6.5B tokens is about 325M parameters. This matches the ByT5 “small” configuration with 300M parameters. We decided to train a model one category larger than that to ensure we get optimal performance.

5.1 Evaluation on Previous Sanskrit Word Segmentation Tasks

In order to examine how ByT5-Sanskrit performs in comparison to other baselines, we fine-tune it on a selection of different previously established Sanskrit word segmentation tasks, each of which used its own dataset.

The SIGHUM and Hackathon datasets are adapted from Sandhan et al. (2022). DCS 2018 is the dataset presented in Hellwig and Nehrdich (2018). rcNN-SS denotes a character-based segmentation algorithm that performs joint compound and Sandhi splitting using a combination of recurrent and convolutional operations (Hellwig and Nehrdich, 2018). TransLIST is the model described in Sandhan et al. (2022), which uses a combination of character-level and lexicon-based word input with a transformer model.

The results of our comparison are shown in Table 3. Since TransLIST, due to its elaborate preprocessing pipeline, is not compatible with the DCS 2018 dataset, we cannot evaluate it in that setting. On DCS 2018 and Hackathon, ByT5-Sanskrit outperforms the existing best baselines with a very considerable margin, while it comes close to the best-performing lexicon-based model, TransLIST, on the SIGHUM dataset. The results show that ByT5-Sanskrit successfully learns latent features of the Sanskrit language and achieves very strong performance without relying on lexical resources. The performance gain on the Hackathon task is especially noteworthy, as this task has the smallest train split of the three with 89k samples, indicating that fine-tuning ByT5-Sanskrit is very sample efficient.

Compared to ByT5-Sanskrit, TransLIST shows more variation in performance between SIGHUM and Hackathon, indicating that the quality of data preprocessing determines the quality of the outcome for TransLIST to a significant degree. ByT5-Sanskrit on the other hand shows consistent performance improvements on all three tasks, making elaborate lexical pre-processing unnecessary to reach competitive performance.



Figure 1: Serialization for the morphosyntactic tagging task. The abbreviated tags are highlighted in red. We use spaces as separation token between words.

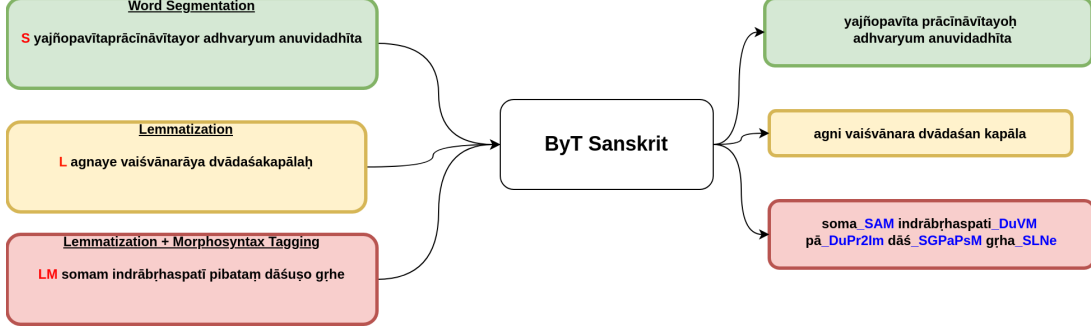


Figure 2: Sanskrit Multitask Formulation: All tasks are converted into sequence-generation tasks. For each task, we prepend prompt tokens (S, L, LM, here marked in red) in order to enable the model to distinguish between tasks. For efficient training and inference, we use a novel serialization strategy to compress the morphosyntactic tags into as few characters as possible, here marked in blue.

Model	DCS 2018 (509k samples)	SIGHUM (99k samples)	Hackathon (89k samples)
rcNN-SS	85.2	87.08	77.62
TransLIST	-	93.97	85.47
ByT5-Sanskrit	90.11	93.83	94.29

Table 3: Sentence level perfect matches on previous Sanskrit word segmentation tasks. Results for rcNN-SS and TransLIST are reported based on the respective publications. Due to data incompatibility, we cannot evaluate TransLIST on the DCS 2018 task. Size of training dataset in parentheses.

5.2 Vedic Dependency Parsing

We also evaluate the performance of ByT5-Sanskrit on the Vedic Sanskrit dependency parsing task. We follow the serialization strategy of Lin et al. (2022) and reformulate dependency parsing as a sequence generation task. We compare our results against the biaffine architecture (Dozat and Manning, 2016) in its best performing configuration as presented in Hellwig et al. (2023). Using the latest version of the dependency annotated data of the DCS for our experiments, we extract a total number of 24,807 sentences with gold dependency, part of speech, and morphosyntactic annotation. We use 90% for training, and 5% for each evaluation and testing. Following the setup in Hellwig et al. (2023), we exclude R̥gvedic data from the test and evaluation

Setting	Biaffine		ByT5-Sanskrit	
	UAS	LAS	UAS	LAS
None	77.68	70.67	86.54	81.54
ALL	86.86	81.98	89.04	84.58

Table 4: UAS and LAS for the Vedic dependency parsing. “None”: only surface forms used; “ALL”: all linguistic gold information used

split and apply the augmentation strategy of randomly concatenating up to four sentences from the training set. Moreover, we replace the POS and morphosyntactic information of old Vedic citations (mantras) with a special tag. The biaffine model and ByT5-Sanskrit are trained and evaluated on the same data. We use 50 epochs for training. We evaluate both models in two settings: One without any additional linguistic information, using only the surface form of a word (None), and one where all available linguistic features (POS tags, morphosyntax, punctuation) are used (All).

The results in Table 4 show significant performance improvements of 2.18 in UAS and 2.60 in LAS over the biaffine baseline. Especially noteworthy is the observation that the ByT5-Sanskrit-based parser without any additional linguistic information comes close to the performance of the biaffine parser with support of gold data. These

results are in line with the observations made in [Nehrdich and Hellwig \(2022\)](#), where the addition of a strong Latin PLM boosted dependency parsing performance very significantly on three different Latin treebanks, with configurations based on the Latin PLM alone matching those that make use of gold annotation without the PLM.

5.3 Sanskrit OCR Post-correction

We also evaluate our model on the task of Sanskrit OCR post-correction as defined in [Maheshwari et al. \(2022\)](#). Our results are presented in Table 6. We fine-tune ByT5-Sanskrit with a sequence length of 512. The results show that ByT5-Sanskrit also achieves the best performance on this task.

5.4 Lemmatization and Dependency Parsing on other MLR Languages

In order to test whether our proposed framework generalizes to other MLR languages, we conduct experiments on lemmatization and dependency parsing for three MLR languages: Bulgarian, Romanian, and Turkish. The data is taken from the Universal Dependency ([Nivre et al., 2016](#)) 2.2 release. As the base model for finetuning, we use ByT5 in the "base" configuration without further pretraining. We show the results in Table 5. Since our serialization strategy requires language expertise, we cannot evaluate our framework on morphosyntactic tagging for these languages. We compare our approach against the current best baseline UDPipe ([Kondratyuk and Straka, 2019](#)) for lemmatization, for dependency parsing we also compare against DPSG ([Lin et al., 2022](#)), since their approach reaches the currently best results on these languages and is structurally very similar to our, with the main difference being that we use a byte-level PLM, while they use the tokenizer-based PLM mT5. The results show that our approach outperforms the previous best baselines on lemmatization for two languages, while outperforming the previous baselines on dependency parsing for all languages. This shows that the performance advantages of byte-level PLMs generalize to other morphologically rich languages.

5.5 Joint Sanskrit Word Segmentation, Lemmatization and Morpho-syntax Tagging Task

We use a snapshot of the DCS from April 2024 as the basis for our experiments with a total number of 601,403 sentences. We hold back 8,190 sentences

Language	Lemma Acc	Dep. Parsing LAS
Turkish IMST (UDPipe)	96.01	67.56
Turkish IMST (Ours, ByT5)	97.94	77.00
Romanian RRT (UDPipe)	98.41	86.74
Romanian RRT (DPSG, mT5)	-	88.76
Romanian RRT (Ours, ByT5)	98.15	91.16
Bulgarian BTB (UDPipe)	97.94	90.35
Bulgarian BTB (DPSG, mT5)	-	93.92
Bulgarian BTB (Ours, ByT5)	98.51	94.11

Table 5: Lemmatization and dependency parsing results on three other MLR languages based on ByT5 base. The UDPipe results are reported based on [Kondratyuk and Straka \(2019\)](#), DPSG based on [Lin et al. \(2022\)](#).

Model	CER	WER
ByT5-Small	2.98	23.19
ByT5-Sanskrit	2.69	20.03

Table 6: CER and WER results for the Sanskrit OCR post-correction task. ByT5-Small are the results as presented in [Maheshwari et al. \(2022\)](#).

for evaluation and 8,398 sentences for testing. We keep the original order of the sentences, ensuring that this data can also be used to train models on longer sections of text. The DCS presents a challenge for our word segmentation model because the forms without Sandhi were not consistently recorded during the initial annotation of the DCS. This incomplete annotation affects 65.8% of all words in the DCS, primarily from classical Sanskrit, whereas unsandhied forms are recorded for most Vedic and some Buddhist texts. When generating conllu files from the DCS, a heuristic that supplements missing unsandhied forms is employed to address this inconsistency. While this heuristic can occasionally produce morphologically correct but unattested nominal forms (e.g., generating *hṛdayataḥ* instead of the attested *hṛdayāt* for the ablative singular of *hṛdaya*- ‘heart’), a cursory examination suggests such cases are infrequent (1-3% of all words). Since we believe that the heuristically generated forms can nonetheless be useful for the training, we include them in the training set and prepend a special flag “R” at the beginning of each line containing such forms. The test and validation splits do not contain any reconstructed forms and are therefore strongly biased towards Vedic texts. This makes the annotation tasks more challenging because Vedic texts are underrepresented in the DCS (see Table 2).

Since the tasks can be combined arbitrarily, we

Task	mT5	ByT5		ByT5-Sanskrit	
	Sen	Sen	Par	Sen	Par
S	76.09	83.71	87.21	84.61	88.21
L	68.27	77.99	82.05	79.88	83.96
S+M	49.94	60.93	71.50	63.86	74.38
L+M	49.23	59.28	69.40	62.00	72.33
S+L+M	49.10	58.75	71.92	61.27	74.31

Table 7: Sentence level perfect match results for the multitask experiment. "S" denotes the task of Sanskrit word segmentation, "L" the task of lemmatization, and "M" the task of morphosyntax tagging.

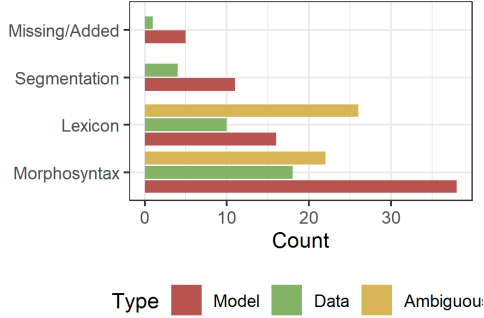


Figure 3: Results of the detailed error analysis

decided to limit the experiments to a number of settings with real-world relevance. We provide the data on sentence- and pseudo-paragraph level. Pseudo-paragraphs are constructed by concatenating adjunct sentences with a length of up to 512 characters, giving the model the possibility to utilize contextual information.

We show the results in Table 7. As the lexical resources used for Sandhan et al. (2022) and Krishna et al. (2020) are not compatible with our dataset, we cannot evaluate their performance in this setting. Due to resource constraints, we could not evaluate mT5 on paragraph level. ByT-Sanskrit outperforms all other models on these tasks. The visible gains compared to ByT5 indicate that even in a multi-task setup with a large fine-tuning dataset, the prior knowledge from the pretraining stage brings distinct performance advantages. The weaker performance of mT5 shows that tokenizer-based models don't perform as well in this setting. All models perform better on pseudo-paragraph level, showing that contextual information beyond sentence boundaries is crucial for Sanskrit linguistic tasks and should be used wherever possible. The visible performance drop that occurs when including morphosyntactic tagging can be explained by the fact that these tags are often ambiguous, as will be discussed in the error analysis below.

5.6 Error analysis

For a detailed error analysis, one author of this paper inspected 150 randomly drawn sentences in which the model result differs from ground truth. The differences were categorized into three classes: (1) errors in the ground truth data which were corrected by the model; (2) ambiguous cases where both ground truth and model result are acceptable; and (3) model errors. These three classes were further subsetted with fine-grained error labels. Most basically, the segmentation of a string may have failed or parts of the analysis is missing or added. Figure 3 shows that these cases constitute only a small part of all errors. Notably, missegmentations as well as missing words are also present in the ground truth, e.g. when a meaningful linguistic analysis of a string was impossible.

Most differences are observed at the lexical and morphological levels. The pattern of model errors aligns with the summary in Gupta et al. (2020), primarily relating to nominal endings that denote more than one case. For instance, an error is seen in the phrase *uttamāyā diśaḥ* 'from the highest region', where both the correctly reconstructed ending *-āyāḥ* in *uttamāyāḥ* and *-aḥ* in *diśaḥ* can signify a genitive or ablative singular. However, the sentential context of this phrase unambiguously indicates an ablative interpretation. Errors in the analysis of verbal morphology are less common and typically occur with unusual and rare forms. An example is *āvarvrataḥ*, the genitive singular of a participle derived from the intensive of the verb *ā vart-* 'revolve' ('of someone who rotates intensively'). The model misinterpreted this complex form as the nominative singular of a newly coined noun *āvarvrata-*.

Both gold and silver exhibit a significant number of ambiguities. Consider the following phrase:

apareṇa *śālāyāḥ*
 western-ADJ.INS.SG hall-GEN/ABL.SG
 to_the_west_of-PREP
 'to the west of the hall'

Here, *apareṇa* is an example of a lexical ambiguity. While, at the level of morphosyntax, the word is the instrumental singular of the adjective *apara-* 'western', it can be argued that this word became grammaticalized in Vedic, as was shown for Vedic *madhye* 'in the middle' > Hindi *mem* 'in' (Reinöhl, 2016). Although the grammaticalized reading is preferred in the DCS, analyzing *apareṇa* as an inflected adjective is perfectly valid, given our

limited knowledge about the temporal dynamics of grammaticalization processes. Similarly, while the genitive reading of *śālāyāḥ* is the preferred analysis in the DCS, the ablative cannot be ruled out here, leading to a morphological ambiguity.

Beyond this, lexical ambiguities in the DCS primarily arise from compound splitting. The DCS follows major Sanskrit dictionaries in not splitting compounds deemed to have non-compositional meanings. Given the lexical transparency of most Sanskrit compounds, splitting them into their constituent parts is often a justifiable approach. For instance, the DCS keeps the compound *ādikaraḥ* “creator” intact, but our model reasonably splits it into *ādi-karaḥ* “beginning-maker”.

In about one third of all cases, our model corrects a wrong analysis in the DCS. One case is *dvādaśa-kapālam* ‘(consisting of) twelve cups’ where the first word is the compound form of the numeral *dvādaśan-* ‘twelve’, but not, as recorded in the DCS, of the adjective *dvādaśa-* ‘twelfth’. Apparently our model achieves a quality high enough to be usable for error detection in the ground truth data. While retraining ByT5-Sanskrit with corrected data is not likely to improve its quality, such error correction may nevertheless be useful for linguistic studies.

Overall, more than half of the 150 sentences inspected (80 or 53.3%) revealed errors in the source data or alternative valid readings. Together with the bias in the test data (see Section 5.5), this result indicates that the predictive quality of our model is higher than indicated by the numbers in Table 7.

5.7 Ablation Study

To assess the impact of joint training on multiple tasks, we conducted an ablation study where we fine-tuned ByT5-Sanskrit on selected tasks individually. The results, presented in the upper half of Table 8, clearly demonstrate that individual task training diminishes performance for both segmentation and lemmatization tasks. This confirms that transfer learning across different tasks contributes to enhanced overall performance.

A second experiment evaluated the effect of removing samples containing reconstructed surface forms (refer to Section 5.5). This condition reduced the training sample size to merely 26.22% of the original data, effectively serving as an ablation experiment probing the dataset size. Despite a noticeable negative impact on performance (as seen

Task	Sentence PM
Segmentation only	83.52 (-1.09)
Lemmatization only	77.52 (-2.35)
Segmentation only w/o rec.	81.58 (-3.03)
Lemmatization only w/o rec.	76.85 (-3.03)

Table 8: Ablation Study where we fine-tune ByT-Sanskrit on individual tasks separately to show the performance difference to the multitask setup. W/o rec. indicates the setting where reconstructed forms (see Section 5.5) are not used in the training dataset. Results are given in sentence level perfect matches.

in the lower half of Table 8), the effect was less pronounced than we anticipated. We hypothesize that this behavior can primarily be attributed to the strong priors of the ByT5-Sanskrit model. Concurrently, removing sentences with reconstructed forms from training rendered the distributions of training and test data, which exclusively contain such sentences, more similar. This suggests that our approach is viable even for languages with a limited amount of labeled training data.

6 Conclusion and Future Work

We have demonstrated that by pretraining a byte-level language model on a large collection of mostly noisy data, new state-of-the-art results for Sanskrit word segmentation, Vedic dependency parsing, and OCR post-correction are achieved, closing the performance gap between lexicon-based and data-driven Sanskrit NLP approaches. We further demonstrated that this pretrained language model can be used as a basis for a multitask model that handles word segmentation, lemmatization, and morphosyntactic tagging jointly with high accuracy. We further demonstrated that this multitask model benefits greatly from training and inference on pseudo-paragraph-level. For the joint fine-tuning on these tasks, we presented a novel dataset. The resulting unified model, being independent of external linguistic resources, is simple to deploy and is already used for Sanskrit corpus annotation projects as well as in information retrieval and machine translation setups. We also showed that our approach generalizes to other morphologically rich languages, where the application of a byte-level PLM yields best results for two languages on lemmatization and for three languages on dependency parsing. We thus establish that byte-level PLMs are a crucial vector of exploration when building NLP pipelines for MLR languages.

7 Limitations

Our model currently does not adequately address the homonymy of words. In the DCS, 7.5% of all lemmata, or tokenized words, have at least one homonym. These homonyms account for a significant 57.5% of all tokenized words. However, this percentage is somewhat misleading. The primary contributors to this high rate of homonymy are indeclinable words such as *ca* ‘and’ and *iti* ‘thus’. These words are used as nouns in grammatical literature, most notably in Pāṇini’s Aṣṭādhyāyī, where their case endings indicate grammatical uses (e.g., at Aṣṭādhyāyī 1.1.16: ... *itau anārṣe* ‘in front of (the particle) *iti* in non-Vedic texts’). In non-grammatical texts, these words almost always have their non-technical meaning. Similar considerations apply to the use of nominalized verbal roots in grammatical texts.

There are more problematic, but less frequent cases. For instance, the word *veda* has four different lemmata recorded in the DCS: (1) the famous text collection of the same name, (2) ‘finding, obtaining’, (3) a small broom, and (4) the name of a man. At least homonyms (1) and (3) are regularly attested in Vedic and classical Sanskrit. Merging them into one lemma is lexicographically inadequate: while (1) and (2) may be etymologically related, (3) and probably also (4) are not (see Mayrhofer, 1992, 579-581). However, the context of their occurrence typically indicates very clearly which of the lemmata is meant.

To address this issue, we plan to mark lemmata with homonyms by numeric affixes in future versions of our model.

References

- David Bamman and Patrick J. Burns. 2020. [Latin bert: A contextual language model for classical philology](#). *ArXiv*, abs/2009.10053.
- Piotr Bojanowski, Edouard Grave, Armand Joulin, and Tomas Mikolov. 2017. [Enriching word vectors with subword information](#). *Transactions of the Association for Computational Linguistics*, 5:135–146.
- Alexis Conneau and Guillaume Lample. 2019. *Cross-lingual language model pretraining*. Curran Associates Inc., Red Hook, NY, USA.
- Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. [Bert: Pre-training of deep bidirectional transformers for language understanding](#). In *North American Chapter of the Association for Computational Linguistics*.
- Timothy Dozat and Christopher D. Manning. 2016. [Deep biaffine attention for neural dependency parsing](#). *ArXiv*, abs/1611.01734.
- Daniela Gerz, Ivan Vulić, Edoardo Ponti, Jason Naradowsky, Roi Reichart, and Anna Korhonen. 2018. [Language Modeling for Morphologically Rich Languages: Character-Aware Modeling for Word-Level Prediction](#). *Transactions of the Association for Computational Linguistics*, 6:451–465.
- Ashim Gupta, Amrith Krishna, Pawan Goyal, and Oliver Hellwig. 2020. Evaluating neural morphological taggers for Sanskrit. In *Proceedings of the 17th SIG-MORPHON Workshop on Computational Research in Phonetics, Phonology, and Morphology*, pages 198–203, Online. Association for Computational Linguistics.
- Oliver Hellwig. 2010–2024. [DCS - The Digital Corpus of Sanskrit](#).
- Oliver Hellwig and Sebastian Nehrlich. 2018. [Sanskrit word segmentation using character-level recurrent and convolutional neural networks](#). In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 2754–2763, Brussels, Belgium. Association for Computational Linguistics.
- Oliver Hellwig, Sebastian Nehrlich, and Sven Sellmer. 2023. Data-driven dependency parsing of Vedic Sanskrit. *Language Resources and Evaluation*, 57:1173–1206.
- Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, Tom Hennigan, Eric Noland, Katie Millican, George van den Driessche, Bogdan Damoc, Aurelia Guy, Simon Osindero, Karen Simonyan, Erich Elsen, Jack W. Rae, Oriol Vinyals, and L. Sifre. 2022. [Training compute-optimal large language models](#). *ArXiv*, abs/2203.15556.
- Mohammed Safi Ur Rahman Khan, Priyam Mehta, Ananth Sankar, Umashankar Kumaravelan, Sumanth Doddapaneni, Suriyaprasaad G, Varun Balan G, Sparsh Jain, Anoop Kunchukuttan, Pratyush Kumar, Raj Dabre, and Mitesh M. Khapra. 2024. [IndicLLM-suite: A blueprint for creating pre-training and fine-tuning datasets for Indian languages](#).
- D. Kondratyuk and M. Straka. 2019. [75 languages, 1 model: Parsing universal dependencies universally](#). In *Conference on Empirical Methods in Natural Language Processing*.
- Amrith Krishna, Bishal Santra, Ashim Gupta, Pavankumar Satuluri, and Pawan Goyal. 2020. [A graph-based framework for structured prediction tasks in Sanskrit](#). *Computational Linguistics*, 46(4):785–845.
- Boda Lin, Zijun Yao, Jiaxin Shi, Shulin Cao, Binghao Tang, Si Li, Yong Luo, Juanzi Li, and Lei Hou. 2022. [Dependency parsing via sequence generation](#). In

- Conference on Empirical Methods in Natural Language Processing.*
- Ayush Maheshwari, Nikhil Singh, Amrith Krishna, and Ganesh Ramakrishnan. 2022. A benchmark and dataset for post-OCR text correction in Sanskrit. In *Findings of the Association for Computational Linguistics: EMNLP 2022*, pages 6258–6265, Online. Association for Computational Linguistics.
- Manfred Mayrhofer. 1992. *Etymologisches Wörterbuch des Altindoarischen. I. Band.* Carl Winter, Heidelberg.
- Sebastian Nehrdich and Oliver Hellwig. 2022. [Accurate dependency parsing and tagging of Latin](#). In *Proceedings of the Second Workshop on Language Technologies for Historical and Ancient Languages*, pages 20–25, Marseille, France. European Language Resources Association.
- Joakim Nivre, Marie-Catherine de Marneffe, Filip Ginter, Yoav Goldberg, Jan Hajič, Christopher D. Manning, Ryan McDonald, Slav Petrov, Sampo Pyysalo, Natalia Silveira, Reut Tsarfaty, and Daniel Zeman. 2016. [Universal Dependencies v1: A multilingual treebank collection](#). In *Proceedings of the Tenth International Conference on Language Resources and Evaluation (LREC’16)*, pages 1659–1666, Portorož, Slovenia. European Language Resources Association (ELRA).
- Matthew E. Peters, Mark Neumann, Mohit Iyyer, Matt Gardner, Christopher Clark, Kenton Lee, and Luke Zettlemoyer. 2018. [Deep contextualized word representations](#). *ArXiv*, abs/1802.05365.
- Colin Raffel, Noam M. Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. 2019. [Exploring the limits of transfer learning with a unified text-to-text transformer](#). *J. Mach. Learn. Res.*, 21:140:1–140:67.
- Uta Reinöhl. 2016. *Grammaticalization and the Rise of Configurationality in Indo-Aryan*. Oxford University Press, Oxford, UK.
- Jivnesh Sandhan, Rathin Singha, N. Venkat Rao, Suwendu Samanta, Laxmidhar Behera, and Pawan Goyal. 2022. [TransLIST: A transformer-based linguistically informed Sanskrit tokenizer](#). *ArXiv*, abs/2210.11753.
- Victor Sanh, Albert Webson, Colin Raffel, Stephen Bach, Lintang Sutawika, Zaid Alyafeai, Antoine Chaffin, Arnaud Stiegler, Arun Raja, Manan Dey, M Saiful Bari, Canwen Xu, Urmish Thakker, Shanya Sharma Sharma, Eliza Szczechla, Taewoon Kim, Gunjan Chhablani, Nihal Nayak, Debajyoti Datta, Jonathan Chang, Mike Tian-Jian Jiang, Han Wang, Matteo Manica, Sheng Shen, Zheng Xin Yong, Harshit Pandey, Rachel Bawden, Thomas Wang, Trishala Neeraj, Jos Rozen, Abheesht Sharma, Andrea Santilli, Thibault Fevry, Jason Alan Fries, Ryan Teehan, Teven Le Scao, Stella Biderman, Leo Gao, Thomas Wolf, and Alexander M Rush. 2022. [Multi-task prompted training enables zero-shot task generalization](#). In *International Conference on Learning Representations*.
- Reut Tsarfaty, Dan Bareket, Stav Klein, and Amit Seker. 2020. [From SPMRL to NMRL: What did we learn \(and unlearn\) in a decade of parsing morphologically-rich languages \(MRLs\)?](#) In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 7396–7408, Online. Association for Computational Linguistics.
- Linting Xue, Aditya Barua, Noah Constant, Rami Al-Rfou, Sharan Narang, Mihir Kale, Adam Roberts, and Colin Raffel. 2021. [ByT5: Towards a token-free future with pre-trained byte-to-byte models](#). *Transactions of the Association for Computational Linguistics*, 10:291–306.
- Akın Özçift, Kamil Akarsu, Fatma Yumuk, and Cevhernur Söylemez. 2021. [Advancing natural language processing \(NLP\) applications of morphologically rich languages with bidirectional encoder representations from transformers \(BERT\): an empirical case study for Turkish](#). *Automatika*, 62(2):226–238.

SanskritShala: A Neural Sanskrit NLP Toolkit with Web-Based Interface for Pedagogical and Annotation Purposes

Jivnesh Sandhan¹, Anshul Agarwal¹, Laxmidhar Behera^{1,3},
Tushar Sandhan¹ and Pawan Goyal²

¹IIT Kanpur, ²IIT Kharagpur, ³IIT Mandi
jivnesh@iitk.ac.in, pawang@cse.iitkgp.ac.in

Abstract

We present a neural Sanskrit Natural Language Processing (NLP) toolkit named SanskritShala¹ to facilitate computational linguistic analyses for several tasks such as word segmentation, morphological tagging, dependency parsing, and compound type identification. Our systems currently report state-of-the-art performance on available benchmark datasets for all tasks. SanskritShala is deployed as a web-based application, which allows a user to get real-time analysis for the given input. It is built with easy-to-use interactive data annotation features that allow annotators to correct the system predictions when it makes mistakes. We publicly release the source codes of the 4 modules included in the toolkit, 7 word embedding models that have been trained on publicly available Sanskrit corpora and multiple annotated datasets such as word similarity, relatedness, categorization, analogy prediction to assess intrinsic properties of word embeddings. So far as we know, this is the first neural-based Sanskrit NLP toolkit that has a web-based interface and a number of NLP modules. We are sure that the people who are willing to work with Sanskrit will find it useful for pedagogical and annotative purposes. SanskritShala is available at: <https://cnerg.iitkgp.ac.in/sanskritshala>. The demo video of our platform can be accessed at: <https://youtu.be/x0X31Y9k0mw4>.

1 Introduction

Sanskrit is a culture-bearing and knowledge-preserving language of ancient India. Digitization has come a long way, making it easy for people to access ancient Sanskrit manuscripts (Goyal et al., 2012; Adiga et al., 2021). However, we find that the utility of these digitized manuscripts is limited due to the user’s lack of language expertise and various linguistic phenomena exhibited by the language. This motivates us to investigate how we

can utilize natural language technologies to make Sanskrit texts more accessible.

The aim of this research is to create neural-based Sanskrit NLP systems that are accessible through a user-friendly web interface. The Sanskrit language presents a range of challenges for building deep learning solutions, such as the *sandhi* phenomenon, a rich morphology, frequent compounding, flexible word order, and limited resources (Sandhan et al., 2022d; Krishna et al., 2021; Sandhan et al., 2021, 2019). To overcome these challenges, 4 preliminary tasks were identified as essential for processing Sanskrit texts: word segmentation, morphological tagging, dependency parsing, and compound type identification. The word segmentation task is complicated by the *sandhi* phenomenon, which transforms the word boundaries (Sandhan et al., 2022d). The lack of robust morphological analyzers makes it challenging to extract morphological information, which is crucial for dependency parsing. Similarly, dependency information is essential for several downstream tasks such as word order linearisation (Krishna et al., 2019) which helps to decode possible interpretation of the poetic composition. Additionally, the ubiquitous nature of compounding in Sanskrit is difficult due to the implicitly encoded semantic relationship between its constituents (Sandhan et al., 2022c). These 4 tasks can be viewed as a preliminary requirement for developing robust NLP technology for Sanskrit. Thus, we develop novel neural-based linguistically informed architectures for all 4 tasks, reporting state-of-the-art performance on Sanskrit benchmark datasets (Sandhan et al., 2022c,d,a).

In this work, we introduce a neural Sanskrit NLP toolkit named SanskritShala² to assist computational linguistic analyses involving multiple tasks such as word segmentation, morphological tagging, dependency parsing, and compound type identification. SanskritShala is also deployed as a web

¹It means ‘a school of Sanskrit’.

²Roughly, it can be translated as ‘a school of Sanskrit’.

application that enables users to input text and gain real-time linguistic analysis from our pretrained systems. It is also equipped with user-friendly interactive data annotation capabilities that allow annotators to rectify the system when it makes errors. It provides the following benefits: (1) A user with no prior experience with deep learning can utilise it for educational purposes. (2) It can function as a semi-supervised annotation tool that requires human oversight for erroneous corrections. We publicly release the source code of the 4 modules included in the toolkit, 7 word embedding models that have been trained on publicly available Sanskrit corpora and multiple annotated datasets such as word similarity, relatedness, categorization, analogy prediction to measure the word embeddings' quality. To the best of our knowledge, this is the first neural-based Sanskrit NLP toolkit that contains a variety of NLP modules integrated with a web-based interface.

Summarily, our key contributions are as follows:

- We introduce the first neural Sanskrit NLP toolkit to facilitate automatic linguistic analyses for 4 downstream tasks (§4).
- We release 7 pretrained Sanskrit embeddings and suit of 4 intrinsic evaluation datasets to measure the word embeddings' quality (§5).
- We integrate SanskritShala with a user-friendly web-based interface which is helpful for pedagogical purposes and in developing annotated datasets (§5).
- We publicly release codebase and datasets of all the modules of SanskritShala which currently mark the state-of-the-art results.³

2 Related Work on Sanskrit NLP Tools

Recently, the Sanskrit Computational Linguistics (SCL) field has seen significant growth in building web-based tools to help understand Sanskrit texts. Goyal and Huet (2016a) introduced the Sanskrit Heritage Reader (SHR), a lexicon-driven shallow parser that aids in the selection of segmentation solutions. Samsādhani is another web-based tool consisting of various rule-based modules (Kulkarni and Sharma, 2019; Kulkarni et al., 2020; Sriram et al., 2023). Recently, Terdalkar and Bhattacharya (2021, 2022) introduced a web-based annotation

tool for knowledge-graph construction and a metrical analysis.

In short, tools for NLP can be divided into two groups: rule-based and annotation tools. Rule-based tools have limitations such as not providing a final solution, limited vocabulary coverage, and lacking user-friendly annotation features. Annotation tools, on the other hand, do not have the recommendations of rule-based systems, relying solely on annotators. To address these limitations, a web-based annotation framework called SHR++ (Krishna et al., 2020c) was proposed. It combines the strengths of both types of tools by offering all possible solutions from rule-based system SHR for tasks like word segmentation and morphological tagging, allowing annotators to choose the best solution rather than starting from scratch.

Our proposal, SanskritShala, goes a step further by integrating a neural-based NLP toolkit that combines state-of-the-art neural-based pre-trained models with rule-based suggestions through a web-based interface. Each module of SanskritShala is trained to predict the solutions from the exhaustive candidate solution space generated by rule-based systems. Hence, it makes predictions in real time using neural-based models that have already been trained. Thus, a complete solution is shown to the users / annotators, which was not possible in any of the previous attempts.

Further, annotators can easily correct the mispredictions of the system with the help of user-friendly web-based interface. This would significantly reduce the overall cognitive load of the annotators. To the best of our knowledge, SanskritShala is the first NLP toolkit available for a range of tasks with a user friendly annotation interface integrated with the neural-based modules.

3 About Sanskrit

Sanskrit is an ancient language known for its cultural significance and knowledge preservation. However, it presents challenges for deep learning due to its morphological complexity, compounding, free word order, and lack of resources. Sanskrit's intricate grammar, with its combination of roots, prefixes, and suffixes, requires advanced algorithms to analyze and understand. Compounding adds another layer of complexity as multiple words combine to form new words with unique meanings (Krishna et al., 2016; Sandhan et al., 2022c). The free word order in Sanskrit complicates tasks

³<https://github.com/Jivnesh/SanskritShala>

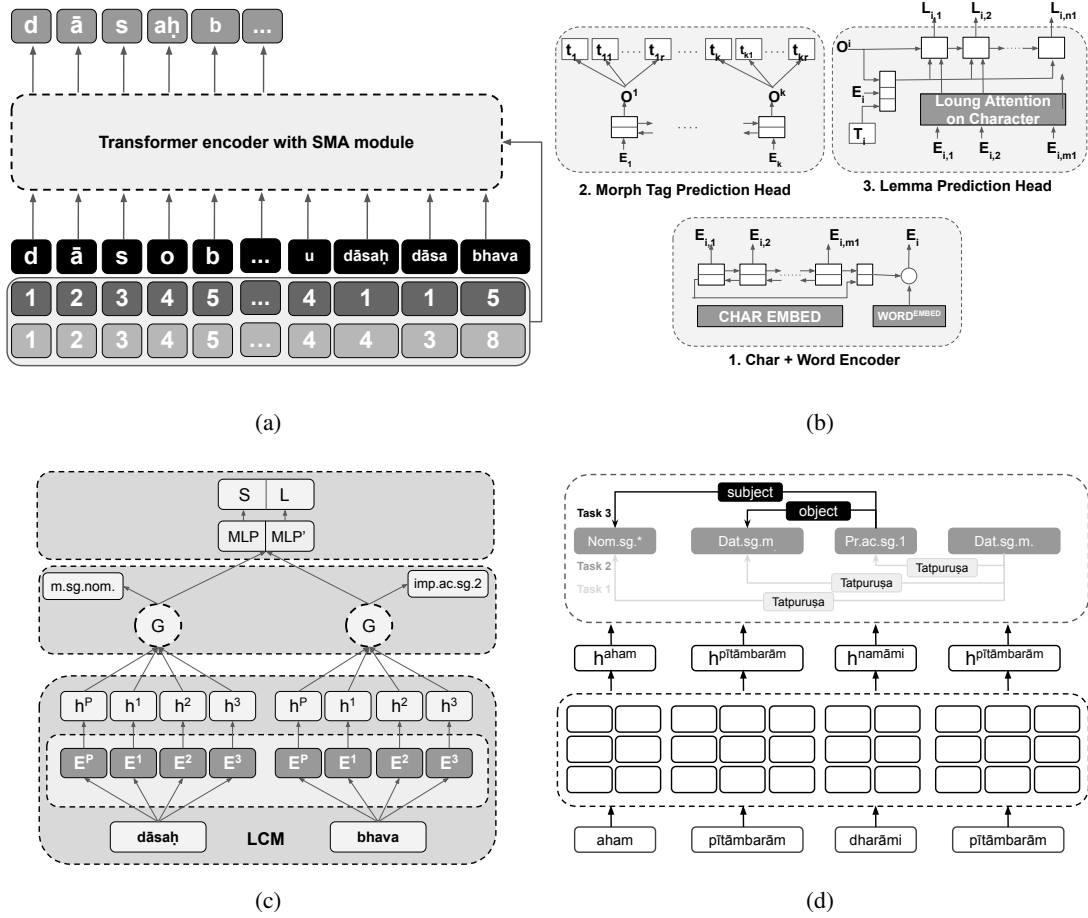


Figure 1: (a) Toy illustration of the TransLIST system. “*dāsobhava*”. Translation: “Become a servant.” (b) LemmaTag architecture in which multi-task learning formulation is leveraged to predict morphological tags and lemmas by employing bidirectional RNNs with character-level and word-level representations. (c) Proposed ensembled architecture for dependency parsing integrated with the LCM pretraining. LCM is acronym for three auxiliary tasks: Lemma prediction, Case prediction and Morphological tag prediction. (d) Toy example illustrating the context-sensitive multi-task learning system: “*aham pīta-ambaram dharāmi*” (Translation: “I wear a yellow cloth”) where ‘*pīta-ambaram*’ is a compound having *Tatpuruṣa* semantic class according to the context presented.

like parsing and understanding, requiring models to comprehend meaning regardless of word placement (Krishna et al., 2023, 2019). Moreover, Sanskrit is considered a low-resource language, lacking extensive datasets and pre-trained models (Sandhan et al., 2021). Overcoming these challenges necessitates linguistic expertise, computational techniques, and sufficient language resources. Developing specialized models to handle Sanskrit’s morphology, compounding, and word order is essential. Creating annotated datasets, lexicons, and corpora will also contribute to advancing research and applications in Sanskrit (Sandhan et al., 2022b, 2023). Despite the obstacles, utilizing deep learning to explore Sanskrit benefits the preservation of cultural heritage and facilitates a deeper understanding of India’s literature and philosophy, while also push-

ing the boundaries of natural language processing.

4 A Neural NLP Sanskrit Toolkit

In this section, we describe SanskritShala, which is a neural Sanskrit NLP toolkit designed to aid computational linguistic analysis including various tasks, such as word segmentation, morphological tagging, dependency parsing, and compound type identification. It is also available as a web application that allows users to input text and obtain real-time linguistic analysis from our pretrained algorithms. We elucidate SanskritShala by first elaborating on its key modules.

Word Tokenizer: Earlier *lexicon-driven* systems for Sanskrit word segmentation (SWS) rely on Sanskrit Heritage Reader (Goyal and Huet, 2016b,

SHR), a rule-based system, to obtain the exhaustive solution space for segmentation, followed by diverse approaches to find the most valid solution. However, these systems are rendered moot while stumbling out-of-vocabulary words. Later, *data-driven* systems for SWS are built using the most recent techniques in deep learning, but can not utilize the available candidate solution space. To overcome the drawbacks of both lines of modelling, we build a **Transformer-based Linguistically-Informed Sanskrit Tokenizer** (TransLIST) (Sandhan et al., 2022d) containing (1) a component that encodes the character-level and word-level potential candidate solutions, which tackles *sandhi* scenario typical to SWS and is compatible with partially available candidate solution space, (2) a novel soft-masked attention for prioritizing selected set of candidates and (3) a novel path ranking module to correct the mispredictions. Figure 1(a) illustrates the TransLIST architecture, where the candidate solutions obtained from SHR are used as auxiliary information. In terms of the perfect match (PM) metric, TransLIST outperforms with 93.97 PM compared to the state-of-the-art (Hellwig and Nehrdich, 2018) with 87.08 PM.

Morphological Tagger: Sanskrit is a morphologically-rich fusional Indian language with 40,000 possible labels for inflectional morphology (Krishna et al., 2020b; Gupta et al., 2020), where homonymy and syncretism are predominant (Krishna et al., 2018). We train a neural-based architecture (Kondratyuk et al., 2018, LemmaTag) on Sanskrit dataset (Krishnan et al., 2020). Figure 1(b) illustrates the system architecture in which multi-task learning formulation is leveraged to predict morphological tags and lemmas by employing bidirectional RNNs with character-level and word-level representations. Our system trained on the Sanskrit dataset stands first with 69.3 F1-score compared to the second position with 69.1 F1-score on the Hackathon dataset (Krishnan et al., 2020) leaderboard.⁴

Dependency Parser: Due to labelled data bottleneck, we focus on low-resource techniques for Sanskrit dependency parsing. Numerous strategies are tailored to improve task-specific performance in low-resource scenarios. Although these strategies are well-known to the NLP community, it is

not obvious to choose the best-performing ensemble of these methods for a low-resource language of interest, and not much effort has been given to gauging the usefulness of these methods. We investigate 5 low-resource strategies in our ensembled Sanskrit parser (Sandhan et al., 2022a): data augmentation, multi-task learning, sequential transfer learning, pretraining, cross/mono-lingual and self-training. Figure 1(c) shows our ensembled system, which supersedes with 88.67 Unlabelled Attached Score (UAS) compared to the state-of-the-art (Krishna et al., 2020a) with 87.46 UAS for Sanskrit and shows on par performance in terms of Labelled Attached Score.

Sanskrit Compound Type Identifier (SaCTI) is a multi-class classification task that identifies semantic relationships between the components of a compound. Prior methods only used the lexical information from the constituents and did not take into account the most crucial syntactic and contextual information for SaCTI. However, the SaCTI task is difficult mainly due to the implicitly encrypted context-dependent semantic relationship between the compound’s constituents. Thus, we introduce a novel multi-task learning approach (Sandhan et al., 2022c) (Figure 1(d)) which includes contextual information and enhances the complementary syntactic information employing morphological parsing and dependency parsing as two auxiliary tasks. SaCTI outperforms with 81.7 F1-score compared to the state-of-the-art by Krishna et al. (2016) with 74.0 F1-score.

5 Sanskrit Resources in SanskritShala

In this section, we describe 7 word embeddings pre-trained on Sanskrit corpora and suit of 4 intrinsic tasks datasets to assess the quality of word embeddings, followed by the description of web interface.

Pretrained word embeddings for Sanskrit:

There are two types of embedding methods: static and contextualized. Table 1 shows how they are categorized based on the smallest unit of input to the embedding model, such as character, subword, or token level. The paper focuses on two token-level word embeddings: Mikolov et al. (2013, word2vec) and Pennington et al. (2014, GloVe). Word2vec is the foundation for all subsequent embeddings and works on the local context window, while GloVe considers the global context. To address the OOV issue, subword (Wieting et al., 2016; Bojanowski

⁴Hackathon leaderboard: <https://competitions.codalab.org/competitions/35744#results>

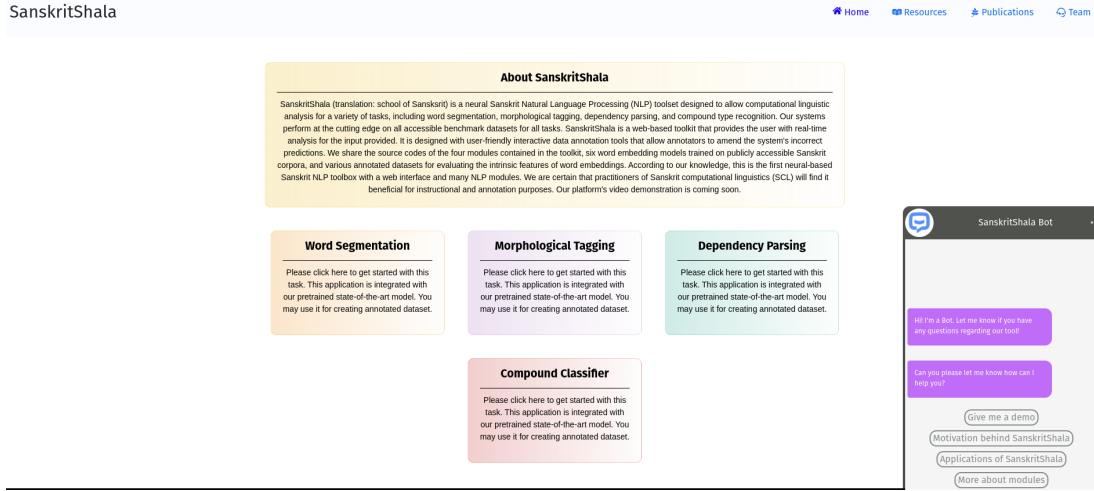


Figure 2: The web interface of the SanskritShala. At the bottom right, a rule-based chatbot is added to navigate users on the platform to give users a user-friendly experience.

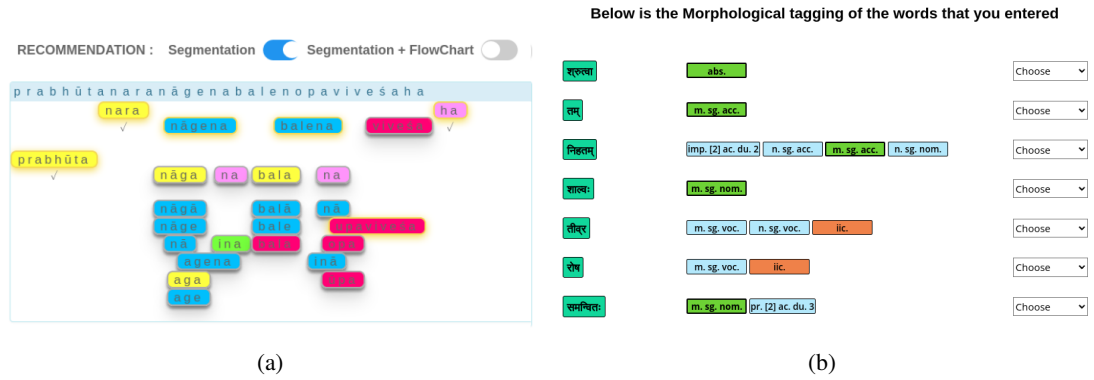


Figure 3: (a) The candidate solution space generated by SHR for the word segmentation task and the predicted solution by our pretrained model is recommended for the sequence 'prabhūtanānāgenā balenopaviveśa ha' using a yellow highlight. (b) Morphological Tagger: For each word, we show possible morphological analyses suggested by SHR as well as our system prediction in green if it falls in SHR's candidate space, otherwise in orange.

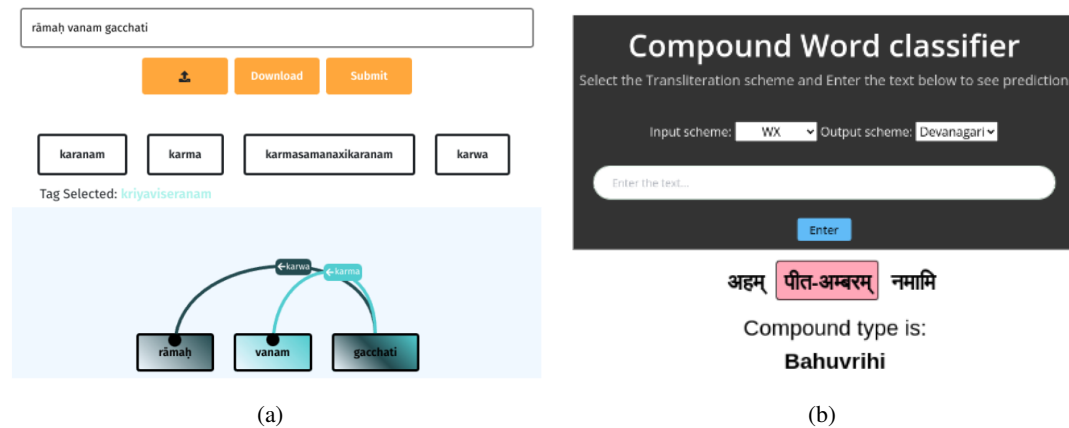


Figure 4: (a) Dependency parser: Interactive module for the dependency parsing task which directly loads predicted dependency trees from our pretrain model and allows user to correct mispredictions using our interactive interface. (b) Illustration of compound identifier

Class	Input type	Systems
Static	character	charLM
	subword	fastText
	token	word2vec, gloVe, LCM
Contextualized	character	ELMo
	subword	ALBERT

Table 1: Overview of Sanskrit pretrained embeddings.

et al., 2017; Heinzerling and Strube, 2018) and character-level (Kim et al., 2016; Jozefowicz et al., 2016) modeling have been proposed. We also explore two contextualized embeddings: ELMo (Peters et al., 2018) and ALBERT (Lan et al., 2020), a lighter version of BERT. We trained these 6 embedding methods on Sanskrit corpora and made the pretrained models publicly available (Sandhan et al., 2023).⁵ The following section describes our proposed pretraining for low-resource settings.

LCM Pretraining: We propose a supervised pretraining, which automatically leverages morphological information using the pretrained encoders. In a nutshell, LCM integrates word representations from multiple encoders trained on three independent auxiliary tasks into the encoder of the neural dependency parser. LCM is acronym for three auxiliary tasks: Lemma prediction, Case prediction and Morphological tag prediction. LCM follows a pipeline-based approach consisting of two steps: pretraining and integration. Pretraining uses a sequence labelling paradigm and trains encoders for three independent auxiliary tasks. Later, these pretrained encoders are combined with the encoder of the neural parser via a gating mechanism similar to Sato et al. (2017). The LCM consists of three sequence labelling-based auxiliary tasks, namely, predicting the dependency label between a modifier-modified pair (LT), the monolithic morphological label (MT), and the case attribute of each nominal (CT). We encourage readers to refer Sandhan et al. (2021, LCM) for more details.

Datasets: The quality of word embedding spaces is evaluated through intrinsic and extrinsic methods. This study focuses on intrinsic evaluation, which involves assessing semantic and syntactic information in the words without testing on NLP applications. It is based on works such as Mikolov et al. (2013) and Baroni et al. (2014). These evaluations require a query inventory containing a query word

and a related target word. However, such query inventories are not readily available for Sanskrit. To address this, we annotated query inventories for 4 intrinsic tasks: analogy prediction, synonym detection, relatedness, and concept categorization. The inventories were constructed using resources such as Sanskrit WordNet (Kulkarni, 2017), Amarakoṣa (Nair and Kulkarni, 2010), and Sanskrit Heritage Reader (Goyal and Huet, 2016b; Huet and Goyal, 2013).

Web Interface: Figure 2 shows our Sanskrit-Shala toolkit that offers interactive web-based predictions for various NLP tasks. The toolkit is built using React framework, which makes it user-friendly and easy to use. One of the tasks it handles is the word segmentation task, which is built on top of the web-based application called SHR++. The SHR++ demonstration is depicted in Figure 3(a). The user inputs a Sanskrit string, which is then sent in real-time to SHR for potential word splits. The system prediction is then obtained from the pretrained word tokenizer. The human annotator is presented with the candidate solution space, with the system prediction highlighted in yellow. The toolkit also features a flask-based application for morphological tagging, which takes user input and scrapes possible morphological tags for each word using SHR. As shown in Figure 3(b), the predictions of the pretrained morphological tagger are displayed in green or orange, depending on whether they are present in the candidate solution of SHR or not. The user can also add a new tag if the actual tag is missing in the SHR solution space or the system’s prediction. For the dependency parsing module, we have built a react-based front-end. The user input is passed to the pretrained model to generate a dependency structure. As illustrated in Figure 4(a), the front-end automatically loads the predicted dependency tree and allows the user to make corrections if there are any mispredictions. Additionally, Figure 4(b) shows a flask-based application for the compound type identifier, where users can give input to the system through its web interface. The final annotations can be downloaded after each individual module. We plan to maintain the progress of Sanskrit NLP and offer an overview of available datasets and existing state-of-the-art via the leaderboard for various tasks.

Interactive Chatbot: SanskritShala-bot is a rule-based chatbot that makes it easy to automate simple

⁵<https://github.com/Jivnesh/SanskritShala/tree/master/EvalSan>

and repetitive user requests, like answering frequently asked questions and directing users to relevant resources. It is also easier to set up and maintain than AI-powered chatbots, which are more complicated. SanskritShala-bot is useful for addressing frequently asked standard queries. It helps familiarize users with the platform by providing them with information and guidance on how to use it. It can answer questions about the platform’s features, help users find their way around it, and explain step-by-step how to do certain tasks. This can make it easier for users to get started and leading to a better user experience.

6 Conclusion

We present the first neural-based Sanskrit NLP toolkit, SanskritShala which facilitates diverse linguistic analysis for tasks such as word segmentation, morphological tagging, dependency parsing and compound type identification. It is set up as a web-based application to make the toolkit easier to use for teaching and annotating. All the codebase, datasets and web-based applications are publicly available. We also release word embedding models trained on publicly available Sanskrit corpora and various annotated datasets for 4 intrinsic evaluation tasks to assess the intrinsic properties of word embeddings. We strongly believe that our toolkit will benefit people who are willing to work with Sanskrit and will eventually accelerate the Sanskrit NLP research.

Limitations

We plan to extend SanskritShala by integrating more downstream tasks such as Post-OCR correction, named entity recognition, verse recommendation, word order linearisation, and machine translation. Improving the performance of existing tasks would be important. For example, the current dependency parser is very fragile (performance drops by 50%) in the poetry domain.

Ethics Statement

Our work involves the development of a platform for annotating Sanskrit text. We believe that this platform will be useful for people who are willing to work with Sanskrit for research and educational purposes. We have ensured that our platform is designed ethically and responsibly. We do not foresee any harmful effects of our platform on any community. However, we caution users to use the plat-

form carefully as our pretrained models are not perfect, and errors can occur in the annotation process. All our systems are built using publicly available benchmark datasets, and we have released all our pretrained models and source codes publicly for future research. We are committed to transparency and open access in our work, and we believe that sharing our resources will benefit the wider NLP community. We also acknowledge that NLP research can have potential ethical implications, particularly in areas such as data privacy, bias and discrimination. We are committed to continuing to consider these ethical implications as we develop our platform, and we welcome feedback from the community on how we can improve our ethical practices.

Acknowledgements

We are thankful to Oliver Hellwig for the DCS dataset and Gerard Huet for the Sanskrit Heritage Engine. We are grateful to Hackathon organizers⁶ who encouraged us to build the best performing morphological tagger. We thank Amrith Krishna, Uniphore for providing the SHR++ interface as a starting point for the web interface of SanskritShala. We appreciate the assistance of Bishal Santra and Suman Chakraborty, IIT Kharagpur in deploying SanskritShala on the IIT Kharagpur server. We are grateful to Hritik Sharma, IIT Kanpur for helping us build a React-based front-end for SanskritShala. We are thankful to Hrishikesh Terdalkar, IIT Kanpur for helpful discussions on deploying systems. We appreciate the anonymous reviewers’ insightful suggestions for enhancing this work. We’d like to say thanks to everyone who helped us make the different neural models for SanskritShala. The work was supported in part by the National Language Translation Mission (NLTM): Bhashini project by Government of India. The work of the first author is supported by the TCS Fellowship under the Project TCS/EE/2011191P.

References

Devaraja Adiga, Rishabh Kumar, Amrith Krishna, Preethi Jyothi, Ganesh Ramakrishnan, and Pawan Goyal. 2021. [Automatic speech recognition in Sanskrit: A new speech corpus and modelling insights](#). In *Findings of the Association for Computational Linguistics: ACL-IJCNLP 2021*, pages 5039–5050, Online. Association for Computational Linguistics.

⁶<https://sanskritpanini.github.io/index.html>

- Marco Baroni, Georgiana Dinu, and Germán Kruszewski. 2014. [Don't count, predict! a systematic comparison of context-counting vs. context-predicting semantic vectors](#). In *Proceedings of the 52nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 238–247, Baltimore, Maryland. Association for Computational Linguistics.
- Piotr Bojanowski, Edouard Grave, Armand Joulin, and Tomas Mikolov. 2017. [Enriching word vectors with subword information](#). *Transactions of the Association for Computational Linguistics*, 5:135–146.
- Pawan Goyal, Gérard Huet, Amba Kulkarni, Peter Scharf, and Ralph Bunker. 2012. [A distributed platform for Sanskrit processing](#). In *Proceedings of COLING 2012*, pages 1011–1028, Mumbai, India. The COLING 2012 Organizing Committee.
- Pawan Goyal and Gérard Huet. 2016a. [Design and analysis of a lean interface for sanskrit corpus annotation](#). *Journal of Language Modelling*, 4:145.
- Pawan Goyal and Gérard Huet. 2016b. [Design and analysis of a lean interface for sanskrit corpus annotation](#). *Journal of Language Modelling*, 4:145.
- Ashim Gupta, Amrith Krishna, Pawan Goyal, and Oliver Hellwig. 2020. [Evaluating neural morphological taggers for Sanskrit](#). In *Proceedings of the 17th SIGMORPHON Workshop on Computational Research in Phonetics, Phonology, and Morphology*, pages 198–203, Online. Association for Computational Linguistics.
- Benjamin Heinzerling and Michael Strube. 2018. [BPEmb: Tokenization-free pre-trained subword embeddings in 275 languages](#). In *Proceedings of the Eleventh International Conference on Language Resources and Evaluation (LREC 2018)*, Miyazaki, Japan. European Language Resources Association (ELRA).
- Oliver Hellwig and Sebastian Nehrlich. 2018. [Sanskrit word segmentation using character-level recurrent and convolutional neural networks](#). In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 2754–2763, Brussels, Belgium. Association for Computational Linguistics.
- Gérard Huet and Pawan Goyal. 2013. Design of a lean interface for sanskrit corpus annotation.
- Rafal Jozefowicz, Oriol Vinyals, Mike Schuster, Noam Shazeer, and Yonghui Wu. 2016. [Exploring the limits of language modeling](#).
- Yoon Kim, Yacine Jernite, David Sontag, and Alexander M. Rush. 2016. Character-aware neural language models. In *Proceedings of the Thirtieth AAAI Conference on Artificial Intelligence, AAAI'16*, page 2741–2749. AAAI Press.
- Daniel Kondratyuk, Tomáš Gavenčák, Milan Straka, and Jan Hajič. 2018. [LemmaTag: Jointly tagging and lemmatizing for morphologically rich languages with BRNNs](#). In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 4921–4928, Brussels, Belgium. Association for Computational Linguistics.
- Amrith Krishna, Ashim Gupta, Deepak Garasangi, Jeevnesh Sandhan, Pavankumar Satuluri, and Pawan Goyal. 2023. [Neural approaches for data driven dependency parsing in Sanskrit](#). In *Proceedings of the Computational Sanskrit & Digital Humanities: Selected papers presented at the 18th World Sanskrit Conference*, pages 1–20, Canberra, Australia (Online mode). Association for Computational Linguistics.
- Amrith Krishna, Ashim Gupta, Deepak Garasangi, Pavankumar Satuluri, and Pawan Goyal. 2020a. [Keep it surprisingly simple: A simple first order graph based parsing model for joint morphosyntactic parsing in Sanskrit](#). In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 4791–4797, Online. Association for Computational Linguistics.
- Amrith Krishna, Bishal Santra, Sasi Prasanth Bandaru, Gaurav Sahu, Vishnu Dutt Sharma, Pavankumar Satuluri, and Pawan Goyal. 2018. [Free as in free word order: An energy based model for word segmentation and morphological tagging in Sanskrit](#). In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 2550–2561, Brussels, Belgium. Association for Computational Linguistics.
- Amrith Krishna, Bishal Santra, Ashim Gupta, Pavankumar Satuluri, and Pawan Goyal. 2020b. [A graph-based framework for structured prediction tasks in Sanskrit](#). *Computational Linguistics*, 46(4):785–845.
- Amrith Krishna, Bishal Santra, Ashim Gupta, Pavankumar Satuluri, and Pawan Goyal. 2021. [A Graph-Based Framework for Structured Prediction Tasks in Sanskrit](#). *Computational Linguistics*, 46(4):785–845.
- Amrith Krishna, Pavankumar Satuluri, Shubham Sharma, Apurv Kumar, and Pawan Goyal. 2016. [Compound type identification in Sanskrit: What roles do the corpus and grammar play?](#) In *Proceedings of the 6th Workshop on South and Southeast Asian Natural Language Processing (WSSANLP2016)*, pages 1–10, Osaka, Japan. The COLING 2016 Organizing Committee.
- Amrith Krishna, Vishnu Sharma, Bishal Santra, Aishik Chakraborty, Pavankumar Satuluri, and Pawan Goyal. 2019. [Poetry to prose conversion in Sanskrit as a linearisation task: A case for low-resource languages](#). In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 1160–1166, Florence, Italy. Association for Computational Linguistics.
- Amrith Krishna, Shiv Vidhyut, Dilpreet Chawla, Sruti Sambhavi, and Pawan Goyal. 2020c. [SHR++: An](#)

- interface for morpho-syntactic annotation of Sanskrit corpora. In *Proceedings of the Twelfth Language Resources and Evaluation Conference*, pages 7069–7076, Marseille, France. European Language Resources Association.
- Sriram Krishnan, Amba Kulkarni, and Gérard Huet. 2020. [Validation and normalization of dcs corpus using sanskrit heritage tools to build a tagged gold corpus](#).
- Amba Kulkarni, Pavankumar Satuluri, Sanjeev Panchal, Malay Maity, and Amruta Malvade. 2020. [Dependency relations for Sanskrit parsing and treebank](#). In *Proceedings of the 19th International Workshop on Treebanks and Linguistic Theories*, pages 135–150, Düsseldorf, Germany. Association for Computational Linguistics.
- Amba Kulkarni and Dipti Sharma. 2019. [Pāṇinian syntactico-semantic relation labels](#). In *Proceedings of the Fifth International Conference on Dependency Linguistics (Depling, SyntaxFest 2019)*, pages 198–208, Paris, France. Association for Computational Linguistics.
- Malhar Kulkarni. 2017. [Sanskrit WordNet at Indian Institute of Technology \(IITB\) Mumbai](#), pages 231–241. Springer Singapore, Singapore.
- Zhenzhong Lan, Mingda Chen, Sebastian Goodman, Kevin Gimpel, Piyush Sharma, and Radu Soricut. 2020. [Albert: A lite bert for self-supervised learning of language representations](#). In *International Conference on Learning Representations*.
- Tomas Mikolov, Ilya Sutskever, Kai Chen, Greg S Corrado, and Jeff Dean. 2013. Distributed representations of words and phrases and their compositionality. *Advances in neural information processing systems*, 26:3111–3119.
- Sivaja Nair and Amba Kulkarni. 2010. [The knowledge structure in amarakosa](#). pages 173–189.
- Jeffrey Pennington, Richard Socher, and Christopher Manning. 2014. [GloVe: Global vectors for word representation](#). In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1532–1543, Doha, Qatar. Association for Computational Linguistics.
- Matthew Peters, Mark Neumann, Mohit Iyyer, Matt Gardner, Christopher Clark, Kenton Lee, and Luke Zettlemoyer. 2018. [Deep contextualized word representations](#). In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long Papers)*, pages 2227–2237, New Orleans, Louisiana. Association for Computational Linguistics.
- Jivnesh Sandhan, Laxmidhar Behera, and Pawan Goyal. 2022a. [Systematic investigation of strategies tailored for low-resource settings for sanskrit dependency parsing](#).
- Jivnesh Sandhan, Ayush Daksh, Om Adideva Paranjay, Laxmidhar Behera, and Pawan Goyal. 2022b. [Prabhupadavani: A code-mixed speech translation data for 25 languages](#). In *Proceedings of the 6th Joint SIGHUM Workshop on Computational Linguistics for Cultural Heritage, Social Sciences, Humanities and Literature*, pages 24–29, Gyeongju, Republic of Korea. International Conference on Computational Linguistics.
- Jivnesh Sandhan, Ashish Gupta, Hrishikesh Terdalkar, Tushar Sandhan, Suvendu Samanta, Laxmidhar Behera, and Pawan Goyal. 2022c. [A novel multi-task learning approach for context-sensitive compound type identification in Sanskrit](#). In *Proceedings of the 29th International Conference on Computational Linguistics*, pages 4071–4083, Gyeongju, Republic of Korea. International Committee on Computational Linguistics.
- Jivnesh Sandhan, Amrith Krishna, Pawan Goyal, and Laxmidhar Behera. 2019. [Revisiting the role of feature engineering for compound type identification in Sanskrit](#). In *Proceedings of the 6th International Sanskrit Computational Linguistics Symposium*, pages 28–44, IIT Kharagpur, India. Association for Computational Linguistics.
- Jivnesh Sandhan, Amrith Krishna, Ashim Gupta, Laxmidhar Behera, and Pawan Goyal. 2021. [A little pretraining goes a long way: A case study on dependency parsing task for low-resource morphologically rich languages](#). In *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics: Student Research Workshop*, pages 111–120, Online. Association for Computational Linguistics.
- Jivnesh Sandhan, Om Adideva Paranjay, Komal Dighumathi, Laxmidhar Behera, and Pawan Goyal. 2023. [Evaluating neural word embeddings for Sanskrit](#). In *Proceedings of the Computational Sanskrit & Digital Humanities: Selected papers presented at the 18th World Sanskrit Conference*, pages 21–37, Canberra, Australia (Online mode). Association for Computational Linguistics.
- Jivnesh Sandhan, Rathin Singha, Narein Rao, Suvendu Samanta, Laxmidhar Behera, and Pawan Goyal. 2022d. [Translist: A transformer-based linguistically informed sanskrit tokenizer](#).
- Motoki Sato, Hitoshi Manabe, Hiroshi Noji, and Yuji Matsumoto. 2017. [Adversarial training for cross-domain Universal Dependency parsing](#). In *Proceedings of the CoNLL 2017 Shared Task: Multilingual Parsing from Raw Text to Universal Dependencies*, pages 71–79, Vancouver, Canada. Association for Computational Linguistics.
- Krishnan Sriram, Amba Kulkarni, and Gérard Huet. 2023. [Validation and normalization of DCS corpus and development of the Sanskrit heritage engine’s segmenter](#). In *Proceedings of the Computational*

Sanskrit & Digital Humanities: Selected papers presented at the 18th World Sanskrit Conference, pages 38–58, Canberra, Australia (Online mode). Association for Computational Linguistics.

Hrishikesh Terdalkar and Arnab Bhattacharya. 2021. [Sangrahaka: A tool for annotating and querying knowledge graphs](#). In *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, ESEC/FSE 2021, page 1520–1524, New York, NY, USA. Association for Computing Machinery.

Hrishikesh Terdalkar and Arnab Bhattacharya. 2022. [Chandojnanam: A sanskrit meter identification and utilization system](#).

John Wieting, Mohit Bansal, Kevin Gimpel, and Karen Livescu. 2016. [Charagram: Embedding words and sentences via character n-grams](#). In *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*, pages 1504–1515, Austin, Texas. Association for Computational Linguistics.

A Benchmark and Dataset for Post-OCR text correction in Sanskrit

Ayush Maheshwari¹, Nikhil Singh*, Amrith Krishna² and Ganesh Ramakrishnan¹

{ayusham, ganesh}@cse.iitb.ac.in,
{nikhil3198, krishnamrith12}@gmail.com

¹Indian Institute of Technology Bombay, ²Uniphore

Abstract

Sanskrit is a classical language with about 30 million extant manuscripts fit for digitisation, available in written, printed or scanned-image forms. However, it is still considered to be a low-resource language when it comes to available digital resources. In this work, we release a post-OCR text correction dataset containing around 218,000 sentences, with 1.5 million words, from 30 different books. Texts in Sanskrit are known to be diverse in terms of their linguistic and stylistic usage since Sanskrit was the ‘lingua franca’ for discourse in the Indian subcontinent for about 3 millennia. Keeping this in mind, we release a multi-domain dataset, from areas as diverse as astronomy, medicine and mathematics, with some of them as old as 18 centuries. Further, we release multiple strong baselines as benchmarks for the task, based on pre-trained Seq2Seq language models. We find that our best-performing model, consisting of byte level tokenization in conjunction with phonetic encoding (Byt5+SLP1), yields a 23% point increase over the OCR output in terms of word and character error rates. Moreover, we perform extensive experiments in evaluating these models on their performance and analyse common causes of mispredictions both at the graphemic and lexical levels. Our code and dataset is publicly available at <https://github.com/ayushbits/pe-ocr-sanskrit>.

1 Introduction

Post-OCR text correction is a crucial post-processing step employed for correcting errors from the predictions of Optical Character Recognition (OCR) systems (Rijhwani et al., 2021a). A post-OCR corrector leverages the distributional information encoded in language models that aims to not only handle the systemic errors introduced

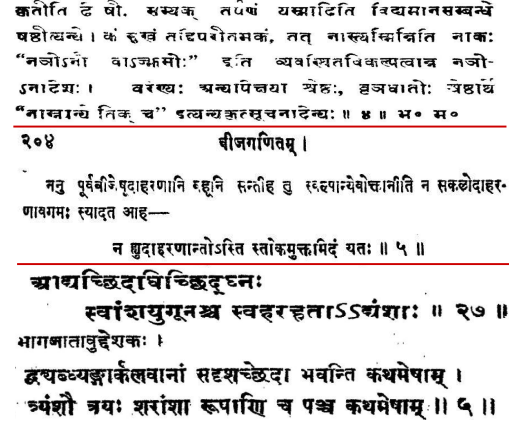


Figure 1: Image samples from different pages of our dataset.

by the OCR engine but also to predict meaningful and fluent sequences based on the context (Saluja et al., 2019). For a language like Sanskrit, the sources of OCR errors are diverse, owing to the availability of printed historical documents that vary vastly on a number of factors such as scan quality, book layout, typefaces, the orthographic similarity of letters in the alphabet, *etc.* (see Figure 1). Moreover, processing texts in Sanskrit is often challenging as the language is morphologically rich, lexically productive, follows relatively free-word order and is a low-resource language with limited available machine-readable corpora (Krishna et al., 2021).

In this work, we release a large Sanskrit post-correction dataset of more than 218,000 manually verified sentences, consisting of 1.5 million words. Our dataset consists of sentences from 30 books from domains as diverse as philosophy, literature, astronomy, medicine, mathematics *etc.* Figure 1 shows a sample of the scanned images from these books, from which we obtained our dataset. Further, the sample clearly demonstrates the diversity in some of the aforementioned factors affecting the quality of the OCR predictions. Histori-

*Work done while interning at IIT Bombay.

cally, depending on the region or the time period in which it was used, several writing systems and scripts were employed for writing Sanskrit. However, the advent of the printing press largely standardized the use of the ‘Devanāgarī’ script as the default writing system for Sanskrit.

We additionally release a set of strong seq2seq baselines to benchmark for the task, including a CopyNet based LSTM model (Gu et al., 2016) and four pretrained seq2seq systems (LMs). We find that all the pretrained-LM based baselines improve over the predictions from the original OCR. The best model which invokes byte level tokenization, viz., ByT5 (Xue et al., 2022), in conjunction with phonetic encoding (SLP1), among these benchmarks (all described in § 3) reports a character and word error rates of 2.98% and 23.19% respectively, as against that of 3.89% and 30.23% from the original OCR. This is primarily due to the ability of byte-level tokenizers to learn arbitrarily longer text in a setting where the frequency of words is low and out-of-vocabulary words are high (§2). Moreover, this goes well with the fact that the writings in Sanskrit follow a phonemic orthography, i.e. phonemes have a direct one-to-one correspondence with the orthographic symbols. We identify that most errors arising from the original OCR are from mispredictions in word boundary detection, diacritics and orthographically similar characters. Further, as the performance of the post-OCR text correction system is highly dependent on the predictions of the OCR, we also release the test dataset used for testing our current OCR. The test dataset consists of 500 images and their corresponding text, which can be used to benchmark an OCR, prior to using its predictions for post-OCR text correction.

2 Dataset

Sanskrit used to be the ‘lingua franca’ for scholarly discourse in the Indian subcontinent for about three millennia and the classical language is still in sustenance in the region. It is estimated that as many as 30 million extant documents, more than that in Greek and Latin combined, are fit for digitisation in Sanskrit (Goyal et al., 2012; Adiga et al., 2021). The current corpus is released as part of our attempt at large scale digitisation of old manuscripts in Sanskrit. Our corpus contains about 30 books, a subset of 103 books in our digitisation pipeline. These books were originally

Devanāgarī	क्	क	का	कि	की	कृ	क्क	अ
Romanized	k	ka	kā	ki	kī	kr̥	kka	a

Figure 2: Devanāgarī and Romanised representation of ‘k’ followed by different vowels. Conjunct consonant (‘kka’) may also have separate symbols in Devanāgarī.

Split	# sentences	# words
Train	208,173	1,444,913
Validation	5000	34,762
Test	5000	34,705

Table 1: Number of words and sentences in the dataset split.

published at least a century ago, and are manually verified to have no copyright issues. We consider printed versions of these books, most of them reprinted in the first half of the twentieth century. While, these books are widely accessible to the public via libraries and academic institutions, we manually had to scan several of them as part of its digitisation process. These books vary widely in their vocabulary and stylistic usage owing to the differences in the domain and the original time period of publication, where the latter can be as old as the fifth century AD.

We release a multi-domain dataset from 30 different books and have 218,000 manually verified sentences in it. The share of each book in the corpus amounts to 3.33% on average with a variance of 4.09, in terms of the number of pages. The corpus consists of more than 1.5 million tokens, with an average frequency of 2.59, and has a vocabulary of 581,445 unique words. Further, 88% has a frequency of one and more than 96% of the words appear less than 5 times. Such a frequency distribution of tokens in Sanskrit corpora is common, given the morphological richness and lexical productivity (due to compounding) in Sanskrit (Krishna et al., 2017; Hellwig, 2010-2016). Further, the average word length is 10.4 characters. In Table 1, we present count of sentences and tokens in our train-test-val split. Of the 20,738 words in the test data vocabulary, 54.53% of those are out of vocabulary. Earlier approaches to post-OCR text correction have employed lexicon-driven approaches for several languages, though such approaches without a wide coverage lexicon might be challenging for a language like Sanskrit (Bassil and Alwani, 2012; Carlson and Fette, 2007).

Prior work in post-OCR text correction in Sanskrit (Krishna et al., 2018) focused on texts written in IAST or Romanized Sanskrit (Monier-Williams et al., 1899), while the current dataset is focused on Devanāgarī. All the books we consider here use Devanāgarī script as the writing system. While Devanāgarī is in existence since the fourth century CE (State, 1896), it has become the primary standard for writing Sanskrit, and several other languages, with the advent of the printing press in India. The script consists of 47 primary characters and is a left-to-right abiguda, where contiguous consonant-vowel sequences are treated as unitary units. As shown in Figure 2, the vowels following the consonants (k in the figure), are treated as secondary units. These units are expressed as ‘mātras’ in the writing system, which essentially are diacritic markers. These markers may appear before, after, above or below an orthographic consonant symbol as shown in Figure 2. The same vowel, say ‘a’ in the romanised script, is written as a different character when used independently as a primary unit. Similarly, conjunct consonants, like ‘kka’ in the figure, would also result in a different orthographic unit, leading to an increase in possible output units for the original OCR. Moreover, these orthographically similar units can also be confusing to an OCR system.

2.1 OCR Editing Process

The current work is part of an OCR project that aims to digitize hundreds of Sanskrit books present in scanned image format¹. The dataset is an outcome of a publicly funded project, primarily carried out by researchers at IIT Bombay. The project currently has 103 books in its pipeline. Our dataset consists of books primarily from philosophy, literature, mathematics, medicine and astronomy. List of books is provided in Figure 4 in appendix.

To aid the process of correction of OCR output, we developed an open-source post-OCR editing tool (Maheshwari et al., 2022) that reduces the cognitive and editing load of the users and increases the speed of text correction. The in-house developed tool is used for OCR correction, verification and proofreading. Currently, 14 experts contribute to various stages of the digitisation process. These experts are either linguists,

trained specifically in Sanskrit linguistics, or computational linguists, and seven of them are working full time for the project. Each page in the book passes through a three step process, and a separate expert oversees each step. The 3 steps are: 1) Manual correction/post-editing of OCR prediction by looking at the original scanned image, 2) Verification of the corrected text performed in the previous step, and 3) Proofreading of the text to check for obvious errors. Verification is primarily aimed at maintaining fidelity of the corrected text to the scanned lines and proofreading is aimed at ensuring linguistic and semantic correctness of the text.

3 System Descriptions

OCR post-correction is a text correction task which can be formalised as a monotone seq2seq model (Schnober et al., 2016). We use an encoder-decoder framework that takes predictions from an OCR as its input. While we use multiple pre-trained seq2seq models as our baselines, none of these has Sanskrit as one of their languages. However, Devanāgarī script is employed in other languages, such as Hindi, which are present in these models. Secondly, unicode encoding of Devanāgarī often poses several challenges owing to the variable byte length employed per character for encoding. Hence, we losslessly transliterate the text into SLP1, an ASCII-based case-sensitive transliteration scheme in our experiments.

Baseline OCR Model : Our baseline OCR model is an OCR engine that uses the Tesseract OCR (Smith, 2007). The model is fine-tuned upon 20,000 synthetically-created images with the Sanskrit language flag. We release our OCR test set along with the post-OCR correction dataset.

CopyNet (Gu et al., 2016): uses a copying mechanism in an LSTM-based seq2seq framework to leverage the (partial) overlap between input and output strings. The model consists of 3 LSTM modules stacked on top of each other for both the encoder and decoder. Following Krishna et al. (2018), we use BPE for learning the vocabulary, which has shown the ability to handle corpora with ‘rare words’ (Sennrich et al., 2016).

mBART (Liu et al., 2020) is a multilingual variant of the BART, both of which are seq2seq models. It has an autoregressive decoder and a BERT-based encoder. We used mBART-50 (Tang et al., 2020), specifically its *HuggingFace* implementa-

¹Project website: <https://www.cse.iitb.ac.in/~ocr/>

tion (large), in our experiments which has been trained on large monolingual corpus of 50 languages. Here, we use text in its original form as well as in the transliterated SLP1 form.

mT5 (Xue et al., 2021) is a multilingual variant of T5 (Raffel et al., 2020), trained on 107 languages. T5 is a seq2seq text generation model, pretrained on a mixture of supervised and unsupervised tasks using a span-corruption objective. In experiments, we employ the mT5 base model from *HuggingFace* along with BPE tokenization and use both devānagari and SLP1 encoded text.

ByT5 (Xue et al., 2022): Given that we have a corpus with mostly ‘rare words’ (c.f., section 2), any unseen set in Sanskrit will suffer from out-of-vocabulary words. A natural solution is to tokenize words at a character level where each character is represented by UTF-8 bytes. ByT5 is a variant of mT5 except that model is fed with a fixed 256 byte values. In experiments, we use ByT5 small model from *HuggingFace* with byte tokenizer and SLP1 encoded text.

IndicBART (Dabre et al., 2021): It is a multilingual BART-based model trained on 11 Indic-family languages in devānagari script. The model is roughly half the size of mT5 model.

4 Experiments and Results

In Table 2, we present the macro-averaged Word Error Rate (WER) and Character Error Rate (CER) for each of our baseline systems. The predictions directly from OCR report a CER and WER of 3.89% and 30.23% respectively. In our experiments, CopyNet’s predictions worsen as per both our metrics, resulting in a CER and WER of 13.25% and 50.38% respectively. However, all of the pre-trained language model configurations employed for post-OCR correction improves over the original OCR predictions. In general, we find that use of Devanāgarī scripts instead of SLP1 to encode text in Sanskrit, results in improved performance for the task. However, with ByT5, we find that our model produces truncated outputs mostly due to an increase in sequence length in ByT5 due to the byte-level vocabulary used in it. The output from ByT5-Dev has a CER of 6.17% and a WER of 27.72%, higher than that of ByT5-SLP1, when a sequence length of 1024 was used. Even though the CER is further reduced to 4.59 (from 6.17) for ByT5-Dev, when its maximum sequence length is reconfigured to 2048, it still does

Encoding	Model	CER	WER
Dev	OCR	3.89	30.23
Dev	mBART	3.50 (+10)	26.11 (+13.7)
SLP1	mBART	3.71 (+4.5)	26.60 (+12)
Dev	IndicBART	3.55 (+8.7)	25.73 (+14.9)
Dev	CopyNet	13.25 (-240)	50.38 (-66)
SLP1	mT5	3.53 (+9.2)	26.47 (+12.5)
Dev	mT5	3.34 (+14.1)	25.57 (+15.4)
SLP1	ByT5	2.98 (+23.4)	23.19 (+23.3)

Table 2: CER and WER (lower is better) on Post-OCR correction task for different encoding schemes and model. Numbers in brackets () corresponds to percentage improvement over OCR model output (top row). All methods are evaluated on devānagari text and all models except ByT5 uses BPE tokenizer. Dev refers to devānagari.

not outperform ByT5-SLP1 (refer Table 3). The use of SLP1 encoding for Sanskrit converts them to ASCII sequences, thereby reducing the overall sequence length for input. The ByT5 configuration with SLP1 encoding of text currently yields the best outcome in our experiments. We discuss the impact of different sequence length and memory overheads between Dev and SLP1 variants of ByT5 in Appendix A.

We observe three primary sources of errors from the original OCR predictions, namely, word and sentence boundary identification owing to missing or extraneous prediction of space and sentence boundary markers, mispredictions due to mātra or diacritics, and mispredictions arising out of orthographically similar characters. All of these cumulatively contribute to 61.76% of the character-level errors. Boundary detection at the word level and at the sentence level, identified by a space marker or by sentence terminating punctuation, contributes to 26.96% of the OCR errors. 89.3% of the boundary detection errors arise out of identifying word boundaries. Similarly, errors due to incorrect or missing mātras (diacritics) contribute to 22.41% of all the errors in OCR. In Sanskrit, these mātras are generally secondary vowels following a consonant, a phenomenon common in abiguda writing systems (§2). Mispredictions specifically due to orthographically similar characters contribute to 12.39% of the total errors. With ByT5, the best performing model we report, we find an error reduction of 44.45%, 37.74% and 14.16% reduction in boundary identification, mātra prediction

and error corrections from orthographically similar predictions respectively.

As a further analysis, we collect the most frequent 300 tokens in the corpus, with at least three letters for a word, and find a total of 2875 occurrences in the ground truth corpus. Among these frequent tokens, OCR predicts each of them correctly at least once. Further, to identify possible similar tokens predicted instead of the correct token, we use the Ratcliff pattern recognition algorithm (Black, 2004), with a matching ratio of 0.6. Here, we find that 8.27% of the token occurrences among the most frequent tokens do not have a corresponding prediction that satisfies the matching criteria. With ByT5, this number is reduced by 2.09%. Moreover, in both OCR predictions and ByT5 based predictions, we mostly find unique patterns in mispredictions for each token and are not able to find any consistent or systemic patterns for each token.

4.1 Experiments on out of domain test dataset

Similar to prior works (Rijhwani et al., 2021b; Krishna et al., 2017), we ensure that there is no sentence-level (sequence) overlap between the train, test and validation split. Though there is an overlap in terms of the books, we ensure that none of the test-data sentences are seen during training. To test the generalizability of our models, we use an out-of-domain test data comes from a completely new book, Brihat-samhita, not included in any of the train-test-validation splits. We also release a new out-of-domain test dataset which comes from a text that is not part of any of the 30 texts included in our dataset. Figure 3 shows the performance of all the OCR, ByT5 and the mT5 systems for this dataset. Here, ByT5 has shown to significantly reduce the CER and WER from the OCR outputs.

5 Conclusion

We release a dataset consisting of 218,000 sentences from 30 books for Sanskrit Post-OCR text correction. We also release a set of strong baselines as a benchmark, which currently shows consistent and significant improvements from the OCR predictions, both on the in-domain test data and out-of-domain test data. All our baselines, in spite of not seeing Sanskrit during pretraining, have shown to generalise well for the task. While

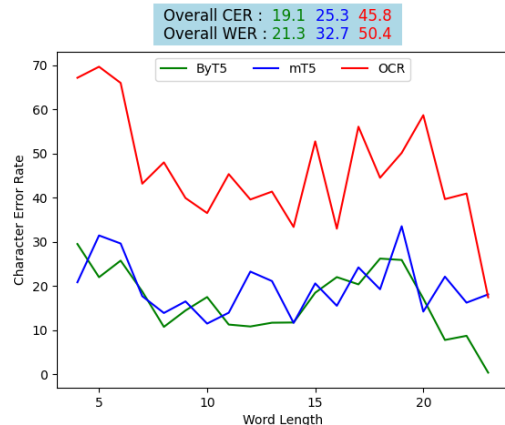


Figure 3: Comparison of CER with different word lengths on an out-of-corpus test set of 500 sentences.

using Devanāgarī Unicode encoding for our experiments has shown to perform better than using SLP1 for multiple baselines, SLP1-based encoding on ByT5 gives the best performance overall.

6 Limitations

A major limitation with the current baselines is the mispredictions happening at the word level. Here, of the mispredicted words by ByT5-SLP, our best performing model, 71.17% are not even valid words in Sanskrit. None of our pretrained models currently are lexically or morphologically aware resulting in the formation of invalid words in the language. Moreover, owing to the low-resource nature of the language, none of the pretrained language models we employed used Sanskrit for its pretraining. An immediate challenge with outputs of our post-OCR text correction would be the use of these predictions for downstream tasks, which are heavily reliant on rule-based morphological analysis of these words (Krishna et al., 2021). We plan to incorporate morphologically aware self-training approaches and dynamic markup decoding (De Cao et al., 2021) which can incorporate various valid inflected forms of a stem in a trie to handle such scenarios.

7 Acknowledgements

We thank anonymous reviewers for providing constructive feedback. Ayush Maheshwari is supported by a Fellowship from Ekal Foundation (www.ekal.org). Ganesh Ramakrishnan is grateful to NLTm OCR Bhashini project as well as the IIT Bombay Institute Chair Professorship for their support and sponsorship.

References

- Devaraja Adiga, Rishabh Kumar, Amrith Krishna, Preethi Jyothi, Ganesh Ramakrishnan, and Pawan Goyal. 2021. Automatic speech recognition in sanskrit: A new speech corpus and modelling insights. In *Findings of the Association for Computational Linguistics: ACL-IJCNLP 2021*, pages 5039–5050.
- Youssef Bassil and Mohammad Alwani. 2012. Ocr context-sensitive error correction based on google web 1t 5-gram data set. *arXiv preprint arXiv:1204.0188*.
- Paul E Black. 2004. Ratcliff/obershelp pattern recognition. *Dictionary of algorithms and data structures*, 17.
- Andrew Carlson and Ian Fette. 2007. Memory-based context-sensitive spelling correction at web scale. In *Sixth International Conference on Machine Learning and Applications (ICMLA 2007)*, pages 166–171. IEEE.
- Raj Dabre, Himani Shrotriya, Anoop Kunchukuttan, Ratish Puduppully, Mitesh M Khapra, and Pratyush Kumar. 2021. Indicbart: A pre-trained model for natural language generation of indic languages. *arXiv preprint arXiv:2109.02903*.
- Nicola De Cao, Gautier Izacard, Sebastian Riedel, and Fabio Petroni. 2021. [Autoregressive entity retrieval](#). In *International Conference on Learning Representations*.
- Pawan Goyal, Gérard Huet, Amba Kulkarni, Peter Scharf, and Ralph Bunker. 2012. [A distributed platform for Sanskrit processing](#). In *Proceedings of COLING 2012*, pages 1011–1028, Mumbai, India. The COLING 2012 Organizing Committee.
- Jiatao Gu, Zhengdong Lu, Hang Li, and Victor OK Li. 2016. Incorporating copying mechanism in sequence-to-sequence learning. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1631–1640.
- Oliver Hellwig. 2010-2016. *DCS - The Digital Corpus of Sanskrit*. Berlin.
- Amrith Krishna, Bodhisattwa P Majumder, Rajesh Bhat, and Pawan Goyal. 2018. Upcycle your ocr: Reusing ocrs for post-ocr text correction in romanised sanskrit. In *Proceedings of the 22nd Conference on Computational Natural Language Learning*, pages 345–355.
- Amrith Krishna, Bishal Santra, Ashim Gupta, Pavankumar Satuluri, and Pawan Goyal. 2021. [A Graph-Based Framework for Structured Pre diction Tasks in Sanskrit](#). *Computational Linguistics*, 46(4):785–845.
- Amrith Krishna, Pawan Kumar Satuluri, and Pawan Goyal. 2017. [A dataset for Sanskrit word segmentation](#). In *Proceedings of the Joint SIGHUM Workshop on Computational Linguistics for Cultural Heritage, Social Sciences, Humanities and Literature*, pages 105–114, Vancouver, Canada. Association for Computational Linguistics.
- Yinhan Liu, Jiatao Gu, Naman Goyal, Xian Li, Sergey Edunov, Marjan Ghazvininejad, Mike Lewis, and Luke Zettlemoyer. 2020. Multilingual denoising pre-training for neural machine translation. *Transactions of the Association for Computational Linguistics*, 8:726–742.
- Ayush Maheshwari, Ajay Ravindran, Venkatapathy Subramanian, Akshay Jalan, and Ganesh Ramakrishnan. 2022. Udaan-machine learning based post-editing tool for document translation. *arXiv preprint arXiv:2203.01644*.
- Monier Monier-Williams, Ernst Leumann, and Carl Cappeller. 1899. A sanskrit-english dictionary: etymologically and philologically arranged with special reference to cognate indo-european languages.
- Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, Peter J Liu, et al. 2020. Exploring the limits of transfer learning with a unified text-to-text transformer. *J. Mach. Learn. Res.*, 21(140):1–67.
- Shruti Rijhwani, Daisy Rosenblum, Antonios Anastopoulos, and Graham Neubig. 2021a. Lexically aware semi-supervised learning for ocr post-correction. *Transactions of the Association for Computational Linguistics*, 9:1285–1302.
- Shruti Rijhwani, Daisy Rosenblum, Antonios Anastopoulos, and Graham Neubig. 2021b. [Lexically aware semi-supervised learning for OCR post-correction](#). *Transactions of the Association for Computational Linguistics*, 9:1285–1302.
- Rohit Saluja, Ayush Maheshwari, Ganesh Ramakrishnan, Parag Chaudhuri, and Mark Carman. 2019. Ocr on-the-go: Robust end-to-end systems for reading license plates & street signs. In *2019 International Conference on Document Analysis and Recognition (ICDAR)*, pages 154–159. IEEE.
- Carsten Schnober, Steffen Eger, Erik-Lân Do Dinh, and Iryna Gurevych. 2016. [Still not there? comparing traditional sequence-to-sequence models to encoder-decoder neural networks on monotone string translation tasks](#). In *Proceedings of COLING 2016, the 26th International Conference on Computational Linguistics: Technical Papers*, pages 1703–1714, Osaka, Japan. The COLING 2016 Organizing Committee.
- Rico Sennrich, Barry Haddow, and Alexandra Birch. 2016. [Neural machine translation of rare words with subword units](#). In *Proceedings of the 54th Annual Meeting of the Association for Computational*

Linguistics (Volume 1: Long Papers), pages 1715–1725, Berlin, Germany. Association for Computational Linguistics.

Ray Smith. 2007. An overview of the tesseract ocr engine. In *Ninth international conference on document analysis and recognition (ICDAR 2007)*, volume 2, pages 629–633. IEEE.

Bombay (India : State). 1896. *Gazetteer of the Bombay Presidency*. v. 1, pt. 1. Printed at the Government Central Press.

Yuqing Tang, Chau Tran, Xian Li, Peng-Jen Chen, Naman Goyal, Vishrav Chaudhary, Jiatao Gu, and Angela Fan. 2020. Multilingual translation with extensible multilingual pretraining and finetuning. *arXiv preprint arXiv:2008.00401*.

Linting Xue, Aditya Barua, Noah Constant, Rami Al-Rfou, Sharan Narang, Mihir Kale, Adam Roberts, and Colin Raffel. 2022. Byt5: Towards a token-free future with pre-trained byte-to-byte models. *Transactions of the Association for Computational Linguistics*, 10:291–306.

Linting Xue, Noah Constant, Adam Roberts, Mihir Kale, Rami Al-Rfou, Aditya Siddhant, Aditya Barua, and Colin Raffel. 2021. mt5: A massively multilingual pre-trained text-to-text transformer. In *NAACL-HLT*.

Max token	2048		1024	
Model	CER	WER	CER	WER
OCR	4.02	30.7	4.67	30.02
ByT5-Dev	4.59	26.6	13.12	38.6
ByT5-SLP1	3.05	23.5	3.49	25.3

Table 3: ByT5-Dev and SLP1 models are trained with different maximum token length. Max token size of 2048 is equivalent to 135 characters while max token size of 1024 is equivalent to 56 characters. We truncated the maximum character length in the test set corresponding to each experiment.

Appendix

A Impact of difference sequence length

ByT5 splits each character into bytes. Since the unicode encoding of Devanāgarī characters typically have higher byte lengths, an input sequence in ByT5 often tends to be shorter, affecting contextual information in longer sentences. We present corresponding results in Table 3.

B List of Books

We present list of books used in our dataset in Table 4.

Book Name	Genre
Uttararamacharita by Bhavabhuti - commentary by Veeraraghava	Arts
Grahalaghava of Ganesh Daivajna	Astronomy
Suryasiddhanta of Ranganatha	Astronomy
Mahabhaskariyam	Astronomy
Aryabhatiya Bhashya of Gargyakerala Nilakantha Samabasiva Sastri K. Vol 1	Astronomy
Aryabhatiya Bhashya of Gargyakerala Nilakantha Samabasiva Sastri K. Vol 2	Astronomy
Aryabhatiya Bhashya of Gargyakerala Nilakantha Samabasiva Sastri K. Vol 3	Astronomy
Karana-Kutuhalam	Astronomy
LaghuManasa	Astronomy
Aryabhatiya commentary by Suryadeva Yajvan	Astronomy
Khandakhadyaka	Astronomy
Ganak Tarangini	Astronomy
Bijaganita with Navankjura-Apte	Mathematics
Bijaganitavatamksa of Narayan Shukla	Mathematics
Bijaganita with Bijankura	Mathematics
Ganitakaumudi of Narayanapandita (vol. 1)	Mathematics
Rekhaganita of Jagannatha Vol. 2	Mathematics
Bijaganita by Tr Abhyankar	Mathematics
Laghubhaskariyam Part 2	Mathematics
Rekhaganita of Jagannatha Vol. 1	Mathematics
Lilavati with kriyakramakri	Mathematics
Patiganita of Sridhara	Mathematics
Brahmasphutasiddhanta of Brahmagupta	Mathematics
Laghubhaskariyam	Mathematics
Hathayogapradipika by Svamarama	Medicine
Mimamsanyayaprakasha by Aapdeva	Philosophy
Shastradipika by Parthasarthi	Philosophy
Shabdashaktiprakashika by Jagdishtarkalankara	Philosophy
Prakaranapanchika-Shalikanatha	Philosophy

Table 4: List of books used in the experiments.



PDF Download
3729530.pdf
15 February 2026
Total Citations: 1
Total Downloads: 889

 Latest updates: <https://dl.acm.org/doi/10.1145/3729530>

SURVEY

Sandarśana: A Survey on Sanskrit Computational Linguistics and Digital Infrastructure for Sanskrit

ANAGHA PRADEEP, International Institute of Information Technology,
Hyderabad, Hyderabad, TG, India

RADHIKA MAMIDI, International Institute of Information Technology,
Hyderabad, Hyderabad, TG, India

Open Access Support provided by:

International Institute of Information Technology, Hyderabad

Published: 06 May 2025
Online AM: 14 April 2025
Accepted: 01 April 2025
Revised: 01 March 2025
Received: 03 April 2024

[Citation in BibTeX format](#)

Sandarśana: A Survey on Sanskrit Computational Linguistics and Digital Infrastructure for Sanskrit

ANAGHA PRADEEP, Language Technology Research Center, International Institute of Information Technology Hyderabad, Gachibowli, India

RADHIKA MAMIDI, Language Technology Research Center, International Institute of Information Technology Hyderabad, Gachibowli, India

Computational Linguistics is an interdisciplinary field of computer science and linguistics that focuses on designing computational models and algorithms for processing, analyzing, and generating human language. Over recent years, this field has made substantial progress. While its primary emphasis tends to center around widely spoken languages, there is equal importance in investigating languages that are not commonly spoken but have contributed immensely to the literature, culture, and philosophy of the society. Thus, this survey article comprehensively delves into the exploration of computational tasks undertaken for Sanskrit, an ancient language of the Indian sub-continent steeped in a wealth of literary heritage. The purpose of this study is to provide an overview of the progress made thus far in the computational analysis of Sanskrit, while also reviewing the current digital infrastructure that supports these efforts. Additionally, our study also identifies potential avenues for future research, serving as a reference for anyone interested in advancing their exploration in this field.

CCS Concepts: • **Computing methodologies** → **Natural language processing**;

Additional Key Words and Phrases: Sanskrit computational linguistics, Pāṇini, Aṣṭādhyāyī

ACM Reference Format:

Anagha Pradeep and Radhika Mamidi. 2025. Sandarśana: A Survey on Sanskrit Computational Linguistics and Digital Infrastructure for Sanskrit. *ACM Comput. Surv.* 57, 10, Article 254 (May 2025), 38 pages. <https://doi.org/10.1145/3729530>

1 Introduction

Linguists around the world have always been eager to analyse and frame theories on language learning. While Western scholars formulated the “universal grammar” theory [Hoque 2021], the Indian linguists were fascinated by the intricacies of how information is encoded in language, ensuring alignment between the speaker’s intentions and the listener’s understanding. Despite their differing approaches, both schools of thought underscored the significance of grammar in the process of language learning [Bharati et al. 2002].

Authors’ Contact Information: Anagha Pradeep, Language Technology Research Center, International Institute of Information Technology Hyderabad, Gachibowli, Telangana, India; e-mail: anagha.pradeep@research.iiit.ac.in; Radhika Mamidi, Language Technology Research Center, International Institute of Information Technology Hyderabad, Gachibowli, Telangana, India; e-mail: radhika.mamidi@iiit.ac.in.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from [permissions@acm.org](https://permissions.acm.org).

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM 0360-0300/2025/05-ART254

<https://doi.org/10.1145/3729530>

Sanskrit, an ancient classical language of India within the Indo-Aryan language family, is the repository of ancient Indian culture, heritage, practices, and diverse knowledge systems such as astronomy, economics, mathematics, medicine, literature, and philosophy. Profoundly intertwined with the intellectual and cultural legacy of India, Sanskrit proficiency is essential for individuals interested in exploring this rich heritage. Therefore, the creation of computational tools for Sanskrit becomes imperative, serving as a valuable resource for enthusiasts aiming at deciphering texts related to the **Indian Knowledge System (IKS)**. Not only that, Sanskrit exhibits linguistic evolution from its Vedic origins to the form meticulously described by the renowned grammarian Pāṇini in his masterpiece, the *Aṣṭādhyāyī*. Sanskrit is particularly distinctive among world languages due to the fact that Pāṇini's grammar (formulated around the 5th century BCE), was remarkably thorough at a time when the language had not yet been written (to the best of our knowledge). It systematically codified the phonemic system, morphological constructions, and syntactico-semantic structures in one comprehensive framework, establishing Classical Sanskrit as the standardized register for communication. Not only was this grammar highly descriptive, but it also became de facto prescriptive to such an extent that, from Vālmiki's *Rāmāyaṇa* onward, all classical Sanskrit literary works largely adhere to the rules outlined in this grammatical framework, resulting in nearly 25 centuries of homogeneous literature. Therefore, while the language itself became specifically suited for linguistic analysis, the Pāṇinian system gave a framework to develop original computational linguistic tools for such an analysis. Moreover, Pāṇini's work not only lays down a structural framework for Classical Sanskrit (*samskr̥ta*), it also encompasses insights into Vedic usages, demonstrating the differences between them [Huet 2003b].

Along with *Aṣṭādhyāyī*, the grammatical tradition of Sanskrit is also enriched by Kātyāyana's *Vārttikas*, and Patañjali's *Mahābhāṣya*. Additionally, we also have the three schools of philosophy viz Nyāya, Vyākaraṇa, and Mīmāṃsā that contribute valuable theories of verbal cognition (*Śābdabodha*) supporting the semantics of Sanskrit at a deeper level. These linguistic theories and frameworks have significant relevance to the field of computational linguistics [Bharati and Kulkarni 2007].

Thus, leveraging Sanskrit's rich resources and its robust grammatical framework, numerous Sanskrit scholars, computational linguists, and computer scientists have worked diligently to create computational tools for analyzing Sanskrit (and other Indian languages). This collective effort has given rise to a new subfield called **Sanskrit Computational Linguistics (SCLs)**, with Indian Cultural Heritage being its main application area (An overview of these tasks has been given in Figure 1).

This article seeks to outline a comprehensive overview of the wide range of computational tasks performed for Sanskrit. The exploration spans from rule-based methodologies that draw inspiration from the grammatical tradition to various Machine Learning experiments employing Sanskrit corpora. It also discusses the spectrum of digital infrastructure that Sanskrit has and subsequently identifies areas in which further research and refinement can be pursued. Although this survey article attempts to cover a wide range of tasks concerning Sanskrit processing by computer, it does not cover speech understanding and written character recognition.

2 Sanskrit Computational Linguistics: A Brief Background

Commencing in the 1980s, efforts were made to digitize Sanskrit texts. One notable achievement during this period was the creation of the first large collection of eighty texts by TITUS.¹ Subsequently, initiatives like The **Göttingen Register of Electronic Texts in Indian Languages (GRETEL)**² and the development of digital dictionaries emerged, driving efforts to digitize a larger

¹<http://titus.uni-frankfurt.de/indexe.htm>

²<https://gretel.sub.uni-goettingen.de/gretel.html>

number of Sanskrit texts. Recognizing the importance of computational tools for Indian languages, the Indian Ministry of Communications and Information Technology actively advocated for their development starting with the **Technology Development for Indian Languages Program (TDIL)** [Goyal et al. 2012]. During the year 1999 - 2000 a significant initiative known as SanskNet was initiated by the Rashtriya Sanskrit Vidyapeetha in Tirupati, led by K. V. Ramakrishnamacharyulu. SanskNet aimed at unite traditional learning institutes, oriental research centers, manuscript repositories, and libraries to create a digital repository of Sanskrit texts accessible online to researchers [Kulkarni 2016]. Early efforts to mechanically process Sanskrit texts were very few and isolated. These included Pushpak Bhattacharyya's Sanskrit parser, developed using integer programming as part of his M.Tech thesis at IIT Kanpur. Then there was Lakshmitatachar's verbal cognition generator for Bhandarkar's Sanskrit primer which was developed at the Academy of Sanskrit Research, Melkote³ in the early 1990s [Goyal et al. 2012]. In light of predominant linguistic theories favoring European languages and their structures, a novel approach to process Indian languages emerged with the "Akshar Bharati" group around 1995. This innovative approach adopted the Pāṇinian framework, providing a solution for Indian languages, and began with the digitization of approximately 475 books. Then there was an initiative by His Holiness Dr. Shiva-murthy Swamiji of Sri Taralabalu Jagadguru Brihanmath, Sirigere (Karnataka) to simulate Pāṇini's Aṣṭādhyāyī through a software known as "Ganaka Ashtadhyayi".⁴ Building on these early efforts, numerous initiatives have been launched to digitize and process Sanskrit texts, particularly given the significance of Pāṇinian grammar in shaping the linguistic understanding of this language [Goyal et al. 2012].

3 Challenges in Processing Sanskrit

Indian languages, including Sanskrit, exhibit a relatively flexible word order, where the position of words in a sentence does not convey information about their thematic role. Instead, semantic relationships between words are specified by declension suffixes [Bharati et al. 2002]. The Pāṇinian grammar is characterized by its syntactico-semantic nature. The adoption of such a grammatical framework would simplify the analysis of languages with free word order. Therefore, the kāraka⁵ rules articulated by Pāṇini play a crucial role in the analysis of Sanskrit and have been utilized by systems, especially in the parsing algorithms. However, even before moving to the syntactic analysis of Sanskrit texts, there are challenges to be dealt with at the morphological level. This includes linguistic features such as

- (1) Sandhi: The dominance of oral tradition necessitated the development of smoother and more fluid speech, leading to the emergence of the concept of "Sandhi" [Girrbach and Li 2022]. Sandhi refers to the phonological changes that occur either at morph or word boundaries [Natarajan and Charniak 2011]. This phenomenon is also common across other Indo-Aryan and Dravidian languages, as well as other languages in oral expression. Sandhi can be internal and external. Internal sandhi occurs at morph boundaries. For instance:

– Sanskrit: rāma + ina = rāmeṇa

While external sandhi occurs at word boundaries [Natarajan and Charniak 2011]. For example:

– Sanskrit: rāmaḥ + vighrahavān = rāmo vighrahavān

Sanskrit systematically encompasses both internal and external sandhi, and these phenomena can be found extensively throughout the language.

³<https://vedavid.org/ASR/>

⁴<https://www.taralabalu.org/panini/>

⁵Syntactico-semantic (or semantic-syntactic) relations between the verbals and other related constituents in a sentence [Bharati and Sangal 1993].

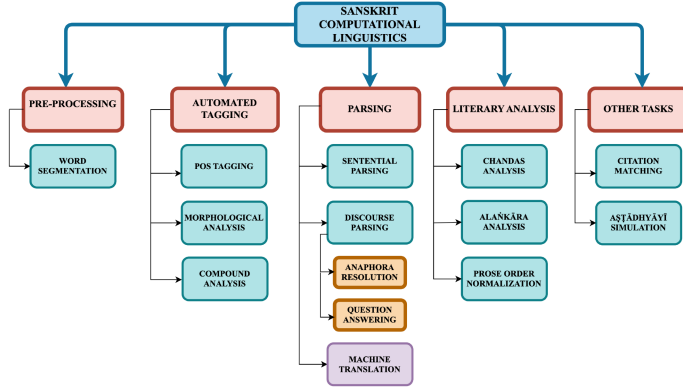


Fig. 1. Overview of tasks in Sanskrit computational linguistics.

- (2) *Samāsa*: The merging of two or more words with distinct meanings into a single word expressing a unified meaning in a concise manner is known as *samāsa* or compounding. Compounding is a highly occurring phenomenon in Sanskrit. Compounds are similar to multi-word expressions and often do not have individual dictionary entries despite being treated as single units. The fact that they can be iterated as needed means that the language’s words are not confined to a finite set, which makes morphology and syntax mutually recursive rather than separate layers for Sanskrit. Furthermore, during compound formations the phonological transformations (*sandhi*) between the compound components are a must. For example: *rāma + ālayaḥ = rāmālayaḥ* (Rama’s abode)

The word “*rāmālayaḥ*” is a compound (*rāmasya ālayaḥ*), for which the *sandhi* operation between the word boundaries (*a* and *ā*) is necessary.

The abundance of such morpho-phonemic and morphotactic features adds complexity to the analysis of Sanskrit. Nevertheless, considerable attempts have been made to resolve these challenges and to process Sanskrit texts.

Our article begins by exploring the most prominent pre-processing task for Sanskrit, which is word segmentation. This is followed by examining various taggers for Sanskrit including **part-of-speech (POS)** tagging, morphological tagging, and compound tagging. Subsequently, we delve into discussions about existing parsers. We discuss both sentence level parsers, and also applications of discourse parsing such as Anaphora Resolution and Question Answering. Followed by this is a section on Machine Translation. We then discuss tasks that are very specific to the analysis of Sanskrit literature such as Chandas Analysis, Alaṅkāra analysis and Prose Order Normalization. This is followed by sections on Citation Matching and Aṣṭādhyāyī Simulation. In the section on Aṣṭādhyāyī Simulation, we discuss how the grammatical treatise of Pāṇini has been made use of for simple generation tasks.⁶ Additionally, the article has a section completely devoted to the digital infrastructure for Sanskrit such as computational platforms, digital libraries and repositories for training software.⁷ In the final section, we examine potential avenues for further research and refinement in the field of SCL.

4 Word Segmentation

Sanskrit word segmentation involves compound segmentation along with incorporating mandatory *sandhi* splitting across words. However, resolving *sandhi* is non-deterministic in nature which

⁶A flowchart representing a brief overview of all the tasks has been provided in Figure 1.

⁷The datasets used for each task have been discussed in Section 12.2.

makes segmentation a complex task in Sanskrit. As the initial stride toward meaningful Sanskrit processing starts with segmentation, we have several segmenters spanning from rule-based to statistical and neural network-based. We will go through them.

Sanskrit Heritage Site (SH) - Huet [2003a] has developed a segmenter using the Zen toolkit [Huet 2005]. This toolkit is widely used in computational linguistics as it uses the data structure of annotated tries (lexical trees) to represent the lexicon in the form of **Finite State Transducers (FST)** [Goyal and Huet 2013]. The segmenter deals with both internal and external sandhis. The internal sandhi production is typically a part of the morphological engine where the inflexional forms are derived by processing the internal sandhi during the word formation. It consists of paradigm schemas where the morph parameters such as number or case are mapped to a set of possible suffixes. This is further mapped to the lexicon [Huet 2004]. On the other hand, the external segmenter uses rewrite rules (using the rules stated by Pāṇini) that are applied to all potential combinations of interactions involving phoneme strings with a length of one or two. Intra-compound segmentation is also handled under the external sandhi segmenter, with certain exceptions [Goyal and Huet 2013].

Automatic Sanskrit Segmentizer - Mittal [2010] propose two techniques for automatically segmenting Sanskrit words using 25,000 unsandhied word pairs and 2650 sandhi rules. The first method creates an FST incorporating sandhi rules, but its performance slows as the FST size increases. To address this, the study introduces an **Optimality Theory (OT)**-based approach, with components like GEN, CON, and EVAL to generate, constrain, and evaluate segmentation candidates. The article's findings indicate that both of these novel approaches outperform the baseline method, which assumes that each word can only be segmented into two constituents. However, it is the OT-based approach that stands out as it delivers satisfactory results in terms of both processing time and the quality of segmentation. Additionally, it is worth highlighting that the baseline system and the OT method both employed the extracted 2650 rules, on a test data of parallel corpus containing 2510 words. In contrast, the rule-based method utilized only 500 rules on the same test data.

University of Hyderabad (UoH) Model - In addition to the model discussed previously, Kumar and Kulkarni [2010] have used a similar approach for compound segmentation using OT. However, a key distinction in this model is that it takes into account the cumulative product of the probabilities associated with the occurrences of each word and the product of probabilities for the sandhi rules. In contrast, the previous model focused on the summation of probabilities for consecutive words (bigrams) and then multiplied this result by the probability of the corresponding sandhi rule.

Statistical Sandhi Splitting - Like Mittal [2010] and Kumar and Kulkarni [2010], Natarajan and Charniak [2011] also perform a statistical sandhi and compound splitting using OT. However, their approach differs from that of the other two. In their article, they explain how they modified the posterior probability function to achieve better results, reaching a precision of 70.52. The article also proposes a new algorithm for Sandhi splitting based on Bayesian word segmentation methods, which is further based on the Dirichlet process. They experiment with both unsupervised and supervised methods for the algorithms and conclude how the supervised surpasses all other systems.

RNN for Compound and Sandhi Segmentation - Hellwig [2015] introduces a novel approach for sandhi resolution as well as compound splitting using the **Recurrent Neural Networks (RNNs)**. This approach relies solely on gold transcripts of correct string splits for training. The model does not make use of any extra external resources, such as language models or morphological or phonetic analyzers. As a result, it is particularly suited for languages that lack such resources. The aim of the algorithm as a whole is threefold; split compounds at the correct position, resolve sandhi, and produce the sandhi rules as per the definitions provided. The article

defines sandhi resolution and compound segmentation as a sequence labeling task for which they make use of an RNN that achieves an accuracy of 93.24. The data used for training the model was extracted from the corpus of SanskritTagger [Hellwig 2007].

Segmentation using Path Constrained Random Walks - In addition to the word co-occurrence feature-based segmentation method, Krishna et al. [2016a] have attempted to integrate segmentation using morphological features. This approach treats the segmentation challenge as a query expansion problem, where it iteratively chooses one segment from a set of competing segmentations (obtained from SH) and then proceeds to select the subsequent correct segment based on information from the expanded context. The algorithm concludes its process when there are no more candidates left for evaluation. To identify the correct segmentation, the authors employ the **Path Constrained Random Walks (PCRW)** framework alongside linguistically motivated **Inductive Logic Programming (ILP)** formulations. Through a series of experiments, the article demonstrates how this model has surpassed the previous ones in terms of performance with a precision score of 92.38. The dataset used to train the language models for this study includes a tagged corpus from DCS. It consisted of 4,30,000 segmented sentences with 66,000 unique lemmas.

Long Short-Term Memory (LSTM) based word segmenter - Reddy et al. [2018] propose a complete data-centric approach for the segmentation task. They use an encoder-decoder framework for the segmentation task and a deep Seq2seq model using multi-layered LSTMs (with attention and without attention) for the prediction task. The model is trained such that the conditional probability of predicting the unsandhied sequence given its corresponding sandhied sequence is maximized. The data used in this study includes 1,05,000 parallel strings from DCS, same as the one used by Krishna et al. [2016a]. The model is compared to two other baseline systems supervised PCRW by Krishna et al. [2016a] and GraphCRF. Of the four models, the article reports that the Seq2seq model with attention performs better with a precision score of 90.77.

IBM Model - Aralikatte et al. [2018] explore yet another deep learning model for compound segmentation. As discussed earlier, sandhi splitting is an essential part of compound segmentation. The article proposes an architecture called **Double Decoder RNN (DD-RNN)**. The goal of this model is an automated split generation by learning the split locations of the compounds. The model is a deep bi-directional character RNN encoder with two decoders and attention. The first decoder is a location decoder that predicts the split location of the compounds, while the second one is a character decoder that generates the split words. The dataset used to train the model consists of 71,747 words and their splits. This includes the UoH corpus and the SandhiKosh corpus. The results of the model were compared with existing publicly available tools such as the UoH splitter [Kumar and Kulkarni 2010], the JNU splitter [Kumar 2007], and the Sanskrit Heritage splitter [Huet 2003a], and with other standard RNN architectures. The article reports the accuracy of the model to be 95% for location split and 79.5% for prediction split.

Word Segmentation Using CNN - Hellwig and Nehrdich [2018] have developed a deep-learning model designed to convert a sentence into segmented tokens (unsandhied tokens). Their approach combines character-level convolutional and recurrent components. While the recurrent components capture sentence-level information, the convolutional components replace traditional n-gram extraction methods. They propose three distinct models: **Convolution Recurrency (crNN)**, **Recurrency Convolution (rcNN)**, and rcNN with shortcuts. In the initial phase, these three models were evaluated for model selection. Subsequently, the selected model (rcNN with shortcuts) was compared against baseline models. These baseline models were also trained on a re-analysed dataset taken from the DCS created specifically for this study. In total, the dataset comprised of 5,61,596 sentences. The article concludes by stating that the rcNN shortcut model outperforms the other baseline models in terms of performance with an accuracy of 96.6%.

Energy Based Model - Krishna et al. [2018] have endeavored to tackle word segmentation and morphological tagging challenges in Sanskrit by employing an **Energy-Based Model (EBM)**. The study utilized data sourced from the DCS. The EBM shares similarities with graph-based parsing techniques. In the process, the word splits obtained from the **Sanskrit Heritage (SH)**⁸ reader serve as nodes in the graph. The SH offers a plethora of potential word splits, and to manage this abundance, a maximum clique approach is adopted for comprehensive segmentation. Consequently, the graph structure removes the inherent sequential aspect of inputs, while the maximum clique policy enables holistic consideration of the entire context. The study further comes up with 528 morphological constraints that are used for automatic tagging. Evaluation of the model was based on how well the inflectional forms of the segmented words were predicted (WPT) and also on an efficient joint task that comprised of word segmentation and morphological tag prediction (WP3T). In comparison to other baselines used in the study such as SupervisedPCRW, EdgeGraphCRF, Seq2seq, Lattice-EBM, and Tree-EBM, the Clique-EBM performs the best for both the tasks with a precision score of 96.18 for the former and 91.35 for the latter.

Ranking the SH solutions - The SH segmenter lists all possible segmentations that could sometimes lead to overgeneration. In situations where the output is excessively detailed, only a condensed summary of the split, phase, and transition is displayed. Krishnan and Kulkarni [2022] propose a probability function aimed at prioritizing solutions and highlighting the most valid ones at the top of the list. For evaluation, a test dataset comprising 21,127 short sandhi expressions is utilized to compare the performance of both the original and modified segmenters. The results indicate that the modified segmenter achieved a correct solution as the first choice 89% of the time, showcasing a notable improvement of 36% over the original segmenter. This was further improved by Krishnan and Kulkarni [2022] who introduce a ranking mechanism to prioritize the solutions and retain segments that are a part of the most probable segmentations. The study highlights that the major challenge faced by the reNN model [Hellwig and Nehrlich 2018] and the introduced ranking mechanism lies in handling the compositionality of compounds. Additionally, the study also contributes a new annotated dataset consisting of 1,30,000 sentences sourced from the DCS.

Neural Compound-Word Sandhi Splitting - In order to address the low accuracy problem of the existing models, Dave et al. [2021] come up with a machine learning approach to solve external sandhi splitting. The problem is addressed in two steps. The first step involves predicting a sandhi window of the compound words using an RNN model. The second step involves splitting the predicted sandhi window into two different words using the seq2seq model. The model achieved an accuracy of 86.8% which was reported to be better than a few models discussed previously which includes the UoH model, SHR model, and DD-RNN model. The article also talks about Sandhi generation task and proposes two approaches for the same. As the results achieved from the initially proposed approach where the model expects two words as input were poor, the second approach was initiated. This involved addressing the issue by extracting the last n characters from the first input word and the first m characters from the second input word, facilitating Sandhi between the two resulting smaller words. Optimal results were obtained with $n = 5$ and $m = 2$ for the Seq2seq model. The neural network model was trained using data sourced from the UoH corpus. This dataset encompasses over 1,00,000 Sandhi examples out of which 81,029 were used after filtering. The results significantly highlight the commendable performance of Neural networks in the Sandhi generation task.

WSMP Hackathon Segmenter - As part of the WSMP Hackathon in 2022, a model was developed to tackle both word segmentation and morphological parsing [Girrbach and Li 2022]. The

⁸Has been discussed in Section 12.

study was structured around three primary tasks: (1). word segmentation, (2). morphological parsing, and (3). a joint task combining both. The word segmentation task was further divided into two subtasks—identifying appropriate word boundaries and retrieving unsandhied word forms. The approach for word segmentation treated it as a sequence labeling problem. Once the predicted sequence is obtained, the segmentation is extracted accordingly. This simultaneous determination of word boundaries and unsandhied word forms is integral to the word segmentation task. For the second task, the obtained word segmentations were input to the morphological parser to predict the stem and morphological tag for each word. In the final task, the objective was to obtain the unsegmented form of a sandhied sentence as a single string, along with its morphological analysis. This was achieved using a pipeline approach that was end-to-end trainable. The dataset provided by the hackathon organizers, was taken from the DCS, as indicated by them. The model achieved a precision score of 80.25 for the final task, the highest among all the models in the hackathon.

TransLIST - Sandhan et al. [2022b] propose a Transformer based Linguistically Informed Sanskrit Tokenizer. This is a character-level sequence labeling that includes latent word information from the SH as the first stage. As a part of the second stage, they use soft-masked attention to prioritize potential candidate words. The soft-masked attention of the Transformers enables an effective interaction between the characters and the latent word information. In the third stage, a path ranking algorithm is introduced in order to rectify candidate words that are not a part of the solution space using the SH. The study incorporates two subsets from the DCS corpus, with the model attaining a precision of 98.8% on the first dataset and 97.78% on the second.

ByT5-Sanskrit - Nehrdich et al. [2024] introduce a new unified pre-trained model, ByT5-Sanskrit designed for segmentation, lemmatization, dependency parsing and OCR post-correction tasks. For pre-training, the authors make use of Sanskrit data from the Sangraha dataset [Khan et al. 2024] which is augmented with data from GRETEL and Digital Sanskrit Buddhist Canon.⁹ The model is further fine-tuned using annotated data from the DCS. The performance of the model for segmentation was assessed by fine-tuning it on established segmentation tasks (rcNN and TransLIST), where ByT5-Sanskrit outperforms the other two models with a perfect match score of 90.11, 93.83, and 94.29 on three different datasets. The lemmatizer achieves a perfect match score of 79.88 on the sentence level and 83.96 on the paragraph level on the DCS data. The model's dependency parsing was evaluated on the Vedic Sanskrit dependency parsing task, and was compared against the biaffine architecture previously used by Hellwig et al. [2023]. It showed higher performance with an **unlabelled attachment score (UAS)** of 89.04 and a **labelled attachment score (LAS)** of 84.58. Additionally, for the joint tasks of segmentation, lemmatization and dependency parsing, ByT5-Sanskrit achieves stronger performance at the paragraph level with a perfect match of 74.31%. This is reported to be a better score than at the sentence level.

In some cases, particularly with OCR'd data, spell checking is necessary before performing word segmentation. There are a couple of spell checkers available for Sanskrit: The first, introduced by Tapaswi et al. [2012], is a rule-based spell checker that relies on rules derived from both morphology and orthography. The second, presented by Prasanna [2022], involves a Sanskrit spellchecking dictionary designed for Hunspell,¹⁰ utilizing the word and paradigm model. This study examines Hunspell's compatibility with Sanskrit, detailing its dictionary format and spellchecking rules based on Paninian grammar. It also introduces a web interface for Sanskrit spellchecking and evaluates both the dictionary and interface.

⁹<https://www.dsbcproject.org/>

¹⁰Hunspell is a free open-source spellchecker and morphological analyser library. It is a command-line tool. (<https://github.com/hunspell/hunspell/>)

Table 1. Word Segmenters

Sl.No	Model	Year	Type	Data
1	SH	2003	Rule-based	-
2	Mittal	2010	Rule-based (a) Statistical (b)	TDIL Consortium Project
3	UoH	2010		TDIL Consortium Project
4	S3	2011	Statistical	TDIL Consortium Project
5	Hellwig	2015	Neural	Sanskrit Tagger
6	PCRW	2016	Neural	DCS
7	seq2seq	2018	Neural	DCS
8	IBM	2018	Neural	UoH Corpus and Sandhikosh
9	CNN	2018	Neural	DCS (SEGMENTATION)
10	EBM	2018	Neural	DCS
11	Sriram	2020	Statistical	TDIL Consortium Project
12	Dave	2021	Neural	UoH Corpus
13	WSMP Hackathon	2022	Neural	DCS (HACKATHON)
14	TransLIST	2022	Neural	SIGHUM, DCS (HACKATHON)
15	Byt5-Sanskrit	2024	Pre-Trained	Sangraha and DCS

Word segmentation as a pre-processing task has been well investigated for Sanskrit, with each successive model striving to introduce innovations in their approaches to address challenges overlooked by previous ones.¹¹ The neural models have proven adept at handling out-of-vocabulary words, which rule-based or statistical systems struggled with. Before the release of Byt5-Sanskrit, rcNN and TransLIST were considered the **state-of-the-art (SOTA)** models for word segmentation. However, Byt5-Sanskrit, being an extension of rcNN, now holds the SOTA position. Both TransLIST and Byt5-Sanskrit are suitable for use on real-world data for segmentation. However, it is important to note that TransLIST is incompatible with the DCS dataset due to its complex pre-processing pipeline, even though it performs exceptionally well on the SIGHUM and Hackathon datasets. Byt5-Sanskrit, on the other hand, demonstrates consistent performance across various datasets, including DCS, highlighting that lexical pre-processing is not necessary to achieve competitive results.

5 Automated Tagging

5.1 Part-of-speech (POS) Tagging

The process of assigning a part-of-speech tag to each word in a given sentence is known as POS tagging and is an important pre-processing task in NLP [Kumawat and Jain 2015].

Rule-Based POS Tagging - A simple rule-based tagger for Sanskrit was introduced by Tapaswi and Jain [2012]. The model stores about twelve POS tags in its database and the tags are assigned through hand-crafted rules. In the first stage of the algorithm, lexical rules are utilized to assign tags, followed by the application of context-sensitive rules in the second stage for unknown words and incorrectly tagged words. A manual evaluation of 100 words at each stage yielded precisions of 90% and 91%, respectively.

Deep Learning POS Tagger with Character Level Embeddings - Premjith et al. [2018] conducted experiments using a deep-learning-based POS tagger for Sanskrit. The study framed the problem as a character-level sequence labeling task to minimize memory requirements. Various deep-learning models, including RNN, LSTM, GRU, and their bidirectional versions were employed

¹¹See Table 1 for a brief summary.

with modified hyperparameters. The study utilized the Universal Dependencies tagset, comprising sentences from Pañcatantra and featuring 16 tags. Training involved around 34,270 words, while testing was performed on 4,231 words. While all the models exhibit strong performance with accuracy scores surpassing 95%, the Bi-directional GPU model achieved an accuracy score of 97.86% and outperformed the others.

Deep Learning based Unsupervised Tagging - Srivastava et al. [2018] introduce an innovative method for assigning POS tags to Sanskrit words. To address the challenge of absence of extensive annotated corpora for Sanskrit, the study advocates for an unsupervised approach, employing the untagged Sanskrit Corpus created by JNU. The research delves into various techniques to represent each Sanskrit word through a multidimensional vector, incorporating word embeddings, character n-grams, autoencoders for dimensionality reduction, and bidirectional LSTM autoencoders to capture semantic information and model sequential dependencies. These compressed encodings are subsequently clustered using K-Means clustering, leading to the assignment of POS tags based on the clusters. The model undergoes testing on a tagged dataset comprising 115,000 words prepared by JNU. The evaluation encompasses multiple vectorization methods and dimensionality reduction algorithms, ultimately concluding that the character n-grams vectorization method, coupled with Bi-LSTM autoencoder, yields the highest performance with an accuracy of 93.2%.

Despite these efforts, it is important to note that the conventional POS tagging may not be suitable for Sanskrit. POS tagging typically assigns a grammatical or syntactic category to each word in a sentence, thereby assisting tasks like parsing and disambiguation. Unlike English, a language with a fixed word order and multiple parts of speech, Sanskrit is a free word-order language with only two primary grammatical categories: subanta and tiñanta. All other categories are derivational and can be grouped under these two. Moreover, Sanskrit is highly inflectional, and its words carry complete morpho-syntactic information. Therefore, conducting a morphological analysis for Sanskrit would be more effective than POS tagging, given that it provides the same information in greater detail. Additionally there is also a blur between the traditional POS of nouns, adjectives and proper nouns in Sanskrit. The lack of typographical mark for proper nouns, and the fact that most proper nouns in Sanskrit are either common nouns or adjectives is a challenging impediment to address without reference to the commentaries. This would require a proper database of named-entity tailored to analyze the texts.

On the other hand, Huet [2024] introduces a concept similar to POS, proposing a set of tags designed for Sanskrit. Their article outlines the reasoning behind using a color-coding schema for segmented words in the Sanskrit Heritage Reader. This schema helps in better understanding the linguistic status of these words and can be utilized for tagging purposes.

5.2 Morphological Analysis and Tagging

Morphology is a branch of linguistics dedicated to the study of words and their constituents. According to Beesley [2004], the initial phase in computational linguistics involves the development of morphological analyzers. These analyzers leverage corpora, lexicons, morphological grammars, and phonological rules meticulously crafted by field linguists and descriptive linguists. Utilizing pre-existing linguistic resources is pivotal in constructing systems that can address a spectrum of natural language applications. Typically, morphological analysis or tagging involves assigning morphological labels to tokens in sequential data. This is a crucial step in processing highly inflectional languages like Sanskrit since they carry rich linguistic information such as gender, number, person, case, tense, mood, and other grammatical features within their intricate system of word forms. While word segmentation, morphological analysis, and compound analysis are interconnected, their ordering in this article is based on their subtle dependencies. Morphological analyzers cannot process continuous sequences of words without specific boundaries,

necessitating segmentation before morphological analysis. Therefore, word segmentation is recognized as a vital pre-processing step for Sanskrit. The discussion on compound analysis (identification of compound types) is deferred to the subsequent section as it also relies on morphological analysis in addition to mutual compatibility of the meanings.

Wide Coverage Morphological Analyzer - Bharati et al. [2006] introduce a comprehensive morphological analyzer founded on the FSA based approach, encompassing distinct modules for handling nominal (subanta), verbal (tiñanta), derivative (kṛdanta) suffixes, and compounds. The system makes use of the Monier William's dictionary for nominal, Dhāturatnākara for verbal and Kṛdantarūpamāla for the derivative examples. With a simple Perl program, paradigms have been created for each of them. The analyzer underwent testing on various texts, including Rāmāyaṇa, Pañcatantra, segments of Harṣacarita, and basic Sanskrit readers. The study reveals that 50% of the unidentified words were attributed to typos or instances of splitting errors. Additionally, it concludes that the morphological analyzer achieves a 95% coverage on unknown texts.

Markov Model - Hellwig [2007] introduces a stochastic tagger named "SanskritTagger", which employs a Markov model for tokenizing text and conducting morphological tagging. The model incorporates a database containing lexical and grammatical dictionaries, along with analyzed Sanskrit texts. The tagging software comprises two key modules. The first module generates hypotheses about the analysis of a phrase using saṁdhi resolution and dictionary lookup. In the second module, the most probable lexical and morphological path is determined through statistical information extracted from the corpus. The study concludes by evaluating the model on five diverse Sanskrit passages of different genres and conducting error analyses. The article also discusses the challenges involved in tokenizing and tagging Sanskrit and outlines the scope for potential enhancements of the model.

JNU Analyzer - Jha et al. [2009] propose a morphological analyzer that analyzes sandhi-free inflected noun forms (subanta) and verb forms (tiñanta). Similar to the previous approach, the modules for subanta and tiñanta analysis are different. However, they are subsequently merged for sentence analysis. The subanta analyzer works with the help of two databases viz examples and rules. The analyzer's architecture consists of a front-end user interface and a back-end database. The database connectivity is done through the **Java Database Connectivity (JDBC)** driver. For tiñanta analysis, the study makes use of two distinct tables that consist of the verb root and tiñ termination. Thus, during analysis, the constituents are identified and matched to the existing structured database.

Morphological Analyzer for Nominal Inflections - Chandra [2012] introduces an advanced Morphological Analyzer designed for Sanskrit Nominal Inflections, employing a restructuring of Pāṇinian morphological rules for computational efficiency. The restructuring involves reordering Pāṇinian morphological rules. Approximately 1,500 Pāṇinian rules have been utilized in this study. The system has two main components: one for recognizing nominal inflections in Sanskrit texts and the other for analyzing these inflections. The proposed approach consists of three steps for analyzing Sanskrit nominal morphology: (1). recognition of all nominal inflections using relational databases (2). subsequent segmentation of the nominal word into a sequence of morphemes and stems (3). detailed information on the stem, suffix, case, and number of the nominal words. An evaluation of the system indicates an accuracy of 85%.

Neural Sequence Labeling Models - Gupta et al. [2020] focus exclusively on the efficiency of neural sequence labeling models for tagging Sanskrit texts morphologically. The four standard neural sequence labeling models used in this study are: **Monolithic Sequence Model (MonSeq)**, **Neural Factorial CRF (FCRF)**, **Sequence Generation Model (Seq)**, and **Multi-task learning (MTL)** which is further categorized into MTL-Hierarchy and MTL-Shared based on settings. Each model employs an encoder for generating input representations, and the study utilizes a training

set of 50,000 sentences and a test set of 11,000 sentences from the DCS. The evaluation reveals that MTL-Hierarchy performs the best with an accuracy of 86.74%. However, the article also addresses the issue of syncretism¹² a notable challenge faced by all models.

All the models discussed in this section provide morphological analysis given a word. However, in most of these models, morphological analyzers form a layer for subsequent parsing tasks as stand alone implementations, and may not be very useful for real texts. The neural sequence labeling models by Gupta et al. [2020] and Byt5-Sanskrit stand out for their ability to handle real-world data, as they incorporate contextual information. In fact, the Byt5-Sanskrit lemmatizer was also tested on other morphologically rich languages, including Bulgarian, Romanian, and Turkish, where it outperformed the baseline models. As a result, Byt5-Sanskrit (refer Section 4). is also recognized as SOTA model for morphological analysis. Nevertheless, even Byt5 does not fully resolve the challenges posed by homonyms and syncretism, which remain significant issues for morphological analysis.

5.3 Compound Analysis

Compound analysis typically refers to breaking up of the compound into its constituents and understanding the relationship between them in order to make sense of the compound. In several cases compounds are formed with successive binary or n-ary combinations of smaller compounds. This structural nestedness introduces an exponential increase in the number of possible combinations, which is a catalan number C_n ,

$$C_n = \frac{(2n)!}{(n+1)! n!}, \quad (1)$$

if there are n components in a compound [Huet 2009]. Additionally, the relation between components of the compound is not explicitly stated. Therefore, compound analysis is a challenging task. We will look at systems that have experimented with analyzing compounds. The analysis may include identification of the compounds, parsing of the compound components, and type identification of compounds.¹³

UoH Model - Kumar and Kulkarni [2010] propose a compound analysis system with four steps: 1. Segmentation, 2. Constituency Parsing, 3. Compound Type Identification, and 4. Paraphrase Generation. Segmentation and compound type identification models have been discussed in this article, while the details of constituency parsing are provided by Kumar [2012] and compound paraphrase generation is elaborated in the study by Kumar et al. [2009]. After segmentation, the system uses a constituency parser to construct a binary tree illustrating the syntactic composition of the compound in relation to the segmentation. The parser's role is to make informed decisions about how to group constituents together within the given context. The robustness of the parses hinges on the parser's ability to identify semantic compatibility among the compound components. A manual tagging process applied to 12,796 compounds achieved 72.7% precision for eight tags, though it did not consider context. The system uses "paraphrase" (vigrahavākya) expressions to convey compound meanings, with gender helping to determine compound type. Kumar et al. [2009] developed an algorithm for paraphrasing two compound types, Tatpuruṣa and Bahuvrīhi, achieving accurate paraphrases for 145 out of 160 test examples.

Grammar and Corpus-based model - Krishna et al. [2016b] propose a classifier model for compound type identification which is based on the head of the compound. This classifier model is a combination of rules extracted from the Aṣṭādhyāyī, semantic relations extracted from

¹²Inflected forms of a lemma that have the same surface form but correspond to multiple morphological tags.

¹³Compounds in Sanskrit mainly fall under four categories. They are: Avyayībhāva (indeclinable compound), Tatpuruṣa (endocentric), Bahuvrīhi (exocentric) and Dvandva (co-ordinative).

Amarakoṣa, and linguistic structural information of the data extracted using Adaptor grammar which is a probabilistic context-free grammar. The article discusses improvement in the system's performance with the inclusion of each of the above-mentioned resources. The dataset for this study was obtained from UoH and consisted of more than 32,000 unique compounds. The model was evaluated on four different ensemble-based classifier systems which are: Random Forests, **Extreme Random Forests (ERFs)**, **Gradient Boosting Methods (GBM)** Adaboost Classifier. Among these, the ERF performed the best with a precision of 0.76 and an accuracy of 0.75.

Neural Model - Sandhan et al. [2019] investigate the effectiveness of neural models compared to hand-featured models to determine their performance in identifying compound types. For this, the study makes use of three different neural architectures; **Multi-Layer Perceptron (MLP)**, **Convolution Neural Network (CNN)**, and **Long Short Term Memory (LSTM)**. The input to these systems is fed on various levels such as word, sub-word, and character levels, which form the different word embedding approaches. The study restricts the number of compound components to two. The text corpus used for this study includes data from DCS, as well as data scraped from Wikipedia and Vedabase. Altogether, the total number of words in the text corpus is about 4.7 M. Compared to the three different architectures used, the LSTM is reported to have performed better with an accuracy of 77.68%.

Compound classifier - Premjith et al. [2019] propose the use of a binary classifier model to distinguish between compound and non-compound words. This study employs various machine learning algorithms, including Naive Bayes, K-Nearest Neighbor, Decision Tree, Random Forest, Support Vector Machine, Multi-Layer Perceptron, Logistic Regression, and AdaBoost classifier. FastText is utilized for word embeddings in this investigation. The findings reveal that the K-Nearest Neighbor algorithm outperformed the others, achieving an accuracy of 90.38% with an embedding dimension of 500. The corpus utilized in the study consists of a tagged dataset obtained from UoH.

Multi-Task Approach - In a work by Sandhan et al. [2022a], a multi-class classification approach is proposed for **Sanskrit Compound Type Identification (SaCTI)**. The article introduces a multi-task learning architecture incorporating contextual information for discerning Sanskrit compound types. Two auxiliary tasks, morphological tagging and dependency parsing, are integrated to enhance the overall learning of the system. The system includes a multi-lingual tokenizer to segment sentence tokens, and these segmented pieces undergo processing through a multi-lingual pretrained XLM-R encoder. Subsequently, the multi-task learning architecture is applied to leverage the enriched contextual information. The system outperformed the baseline models by a margin of 6.1% (accuracy) and 7.7% (F1 score). Notably, the study highlights the efficacy of the proposed architecture in handling two other languages, namely English and Marathi.

DepNeCTI - Previous compound analysis systems have predominantly focused on identifying binary (two-word) compound types. In a study by Sandhan et al. [2023], a novel system is introduced to analyze multi-component compounds. The research specifically concentrates on the recognition of nested spans within these compounds and the interpretation of their implicit semantic relationships. Given the similarity between semantic relations among compound components and dependency relations, the task is framed as a dependency parsing task, termed "Dependency-based Nested Compound Type Identification". The article introduces two new datasets, namely NeCTI and NeCTI OOD, the latter serving as an out-of-domain testbed. Standard formulations, including constituency parsing, bottom-up constituency parsing, span classifier, lexicalized constituency parsing, and seq2seq, are adapted as baselines for this task. The proposed DepNeCTI is presented in two variants—one employing LSTM and the other utilizing XLM-R. Both variants outperform the baseline systems and exhibit improvements when contextual information

is integrated. Notably, DepNeCTI-XLM-R emerges as the most effective, demonstrating significant enhancements when leveraging contextual information with a **Labeled Span Score (LSS)** of 83.19 for NeCTI data and 73.54 for NeCTI OOD data.

When analyzing a sample such as the Bhagavad Gita, it is observed that, on average, each verse contains at least two compounds. Additionally, Krishna et al. [2016b] report that in the DCS, about 75% of the vocabulary is made up of compounds, with 41% consisting of multiple components. This highlights the high frequency of compounds in Sanskrit, making their analysis a complex task. So far, the only model that resolves multi-component (more than two) compounds is DepNeCTI proposed by Sandhan et al. [2023] and this can be used on real data. Directing efforts toward creating more models that can handle multi-component compounds could be a valuable avenue for future work in compound analysis, as detecting compound boundaries, particularly in cases of nestedness, remains a challenging task.

6 Sentential Parsing

Parsing is the process of dissecting a sentence into smaller components and comprehending the structure and relationship among its elements. Two primary parsing approaches exist: (1). Constituency parsing and (2). Dependency parsing. Constituency parsing entails recognizing the hierarchical structure of a sentence through its constituents, particularly major phrases like **Noun Phrase (NP)** and **Verb Phrase (VP)** using productive rules such as context-free grammar. On the other hand dependency parsing involves a pair-wise direct identification of syntactic and semantic relationships between words, highlighting their role or function without consideration for the hierarchical structure of the sentence. While constituency parsing is used for languages that have a fixed word order, dependency parsing is considered more suitable for languages that have a flexible word order such as Sanskrit and other Indian languages that store syntactico-semantic information in its inflections Jurafsky and Martin [2000]. Let us go through different parsers that were designed and experimented for Sanskrit.

SH Shallow Parser - Huet [2006] presents the initial effort in crafting a parser for Sanskrit. Addressing the previously discussed over-generation issue linked to the SH segmenter, a semantic-based filtering mechanism became imperative, translating into a *kāraka* analysis – essentially a form of dependency parsing. This analysis entails aligning the subcategorization pattern of verbal forms, governing the crucial thematic roles essential for verbs to effectively convey meaning in expressing a situation or action. Nevertheless, the article underscores the minimal treatment of *kārakas*/thematic roles. The subcategorization patterns for verbs primarily categorize them as transitive or intransitive in the article. An enhancement in the analysis, encompassing the need for a dative argument in verbs related to gifting, and similar contexts, remains a potential refinement to be considered.

Parsing using Deterministic Finite Automata (DFA) - Venturing into the development of a Sanskrit parser, Goyal et al. [2007] propose a **Deterministic Finite Automaton (DFA)** for morphological analysis and adopts Pāṇini's "utsarga apavāda" (general and exception) methodology for relation analysis. The parser takes a Sanskrit sentence as input and dissects it using predefined rules stored in the DFA. The rule bases crafted for this endeavor comprises the nominals rule database (encompassing entries for noun and pronoun declensions), the verb rule database (comprising entries for 10 verb classes), and the particle database housing word entries. To address challenges arising from external sandhi phenomena, dedicated sandhi modules have been designed using the rule base. Upon obtaining the root word and attributes for other words, the subsequent phase involves identifying relationships among them. Rigorously tested on four distinct paragraphs, the parser has demonstrated accurate performance according to the reported results.

UoH Parser - Kulkarni et al. [2010b] explore the Sanskrit grammar to extract diverse semantic relations, aiming at formulating a grammar-based dependency parser tailored for Sanskrit and improve the model further by proposing a Deterministic Parsing algorithm [Kulkarni 2013]. The current parser, an Edge-centric Binary Joint model, is designed for both prose and verse, addressing challenges faced by the previous node-centric model, which struggled with multiple incoming arrows. This modification incorporates various concepts from Indian theories of verbal cognition (Śābdhabodha). Given that verses in Sanskrit exhibit a more flexible word order compared to prose forms, the study also presents an algorithm for converting parsed verses into their canonical prose form. This conversion is guided by insights obtained from the popular text Samāsacakra. The parser's performance was evaluated on 195 verses along with their prose counterparts. However, the parser encountered difficulty in parsing 45 instances of both verse and prose, primarily attributed to out-of-vocabulary words Kulkarni et al. [2019].

Energy Based Model (EBM) - Krishna et al. [2020b] extend the experiments on EBM conducted previously for word segmentation and morphological parsing. They define a search-based structured prediction framework using EBM for various structure prediction tasks including word segmentation, morphological parsing, dependency parsing, syntactic linearization, and prosodification for languages with free word order such as Sanskrit. This framework consists of three components viz: a graph generator, an edge vector generator, and a structured prediction model. The inputs for all the tasks are given to the graph generator by the user. The outputs from the graph and edge generator form the input for the structured prediction model. The experiment involves training of eight models, five of them in a stand-alone setting and three in a joint setting. In this study, the **Forward Stagewise Path Generation (FSPG)** feature function is utilized for automated learning. The article concludes that either state-of-the-art results were attained for each task, or this was the sole data-driven model experimented with for all tasks, especially prosodification which was first introduced in this article. For parsing, the article reports that the model achieved an LAS of 85.32 and a UAS of 82.65.

Data-Driven Dependency Parsing - Krishna et al. [2023] represent the initial exploration into data-driven neural dependency parsing. The experiment assesses the effectiveness of various data-driven approaches to dependency parsing, encompassing two graph-based methods and two transition-based methods. The first transition-based model, **Yet Another Parser (YAP)**, excels in handling morphologically rich languages and employs hand-crafted features for free word-order languages. The second transition-based parser, **Learning to Search (L2S)**, utilizes a framework that transforms structured prediction problems into search problems and incorporates hand-crafted features. The study also employs two graph-based parsers, namely **BiAFFINE (BiAFF)** Parser and **Deep Contextualized Self-training (DCST)**. Both graph-based parsers automatically learn word representations as features and utilize LSTM to learn contextual embeddings. The models were trained on a dataset from the UoH, along with 1,500 sentences from the STBC. Testing involved an additional 1,000 sentences from the STBC and 300 verses, along with their corresponding prose forms, from the renowned literary work Śiṣupālavadha. The evaluation was based on micro-averaged UAS and LAS. Among the four parsers, DCST exhibited the best performance with a UAS of 80.97 and LAS of 72.86 on the STBC dataset.

Dependency Parsing in a low resource setting - For morphologically rich languages, acquiring morphological tags at runtime poses significant challenges and using predicted tags from taggers can impact the performance of parsers. Addressing these issues, Sandhan et al. [2021] propose a novel approach involving simple auxiliary tasks to enhance parser performance. This pre-training strategy aims at integrating word representations from encoders trained on diverse sequence-level supervised tasks as auxiliary tasks with the default encoder of the neural parser. These tasks encompass predicting morphological tags, dependency labels, and case tags

(specifically for nominal forms). The study conducts experiments on two main neural parsers, BiAFF and DCST, with their default configurations serving as the baseline. The configuration introduced in this article, labeled as LCM (representing label, case, and morph tags), demonstrates superior performance. The findings reveal that minimally supervised pre-training yields significantly better results compared to transfer learning and multi-task learning. The article extends its experimental scope to include ten other morphologically rich languages, such as German, Hungarian, and Greek. The results demonstrate that pre-training leads to a substantial improvement, with an average absolute gain of 2 points in UAS and 3.6 points in LAS compared to DCST. The study makes use of 500, 1,000, and 1,000 sentences from the STBC as the training, development, and test data, respectively, for all models. For the auxiliary tasks, all sequence taggers undergo training using an additional set of 1,000 sentences from STBC that were not previously used.

Data-driven dependency parsing of Vedic Sanskrit - Hellwig et al. [2023] introduce the first data-driven dependency parser for Vedic Sanskrit. They use this for extending an existing treebank for Sanskrit [Hellwig et al. 2020]. The authors utilize a graph-based biaffine parser, which performs well on non-projective dependencies, along with DCST (a pre-training step) that enhances the parser. Additionally, they explore the impact of static embeddings (Word2Vec, fastText) and contextualized embeddings (RoBERTa, XLM-RoBERTa, ELMo) on the model's performance to determine which type of embeddings work best. For the pre-training steps, they use DCS data, and the Vedic Tree Bank for training the model on syntactic gold annotations. In total, around 16,272 sentences were used. The authors also report the attachment scores achieved by the proposed model: 87.61 for unlabeled and 81.84 for labeled.

Various approaches have been ventured for Sanskrit dependency parsing.¹⁴ While rule-based parsers demonstrate proficiency, they fail when dealing with complex compounds. On the other hand, neural parsers encounter challenges due to limited training resources and the utilization of morphological information to predict tags during runtime. Developing parsers capable of addressing these challenges is necessary to deal with real world data. One major challenge for sentential parsing in Sanskrit text is the absence of punctuation that makes it challenging to define sentence boundaries. The *daṇḍa*, commonly used in poetry, serves more as a prosodic marker than a syntactic one. As a result, sentence boundaries are often ambiguous, extending beyond a single śloka and occasionally running across entire pages, as seen in works like *Kādambarī*. Despite its importance, little research has been dedicated to this area.

Although some of the neural parsers discussed in this section seem efficient to handle real data, with some even reporting good performance on other morphologically rich languages, Byt5-Sanskrit (refer Section 4) surpasses the performance of all the models. It shows great performance in all the three tasks (segmentation, lemmatization and parsing) individually, as well as jointly. Thus, we report this to be SOTA for Sanskrit dependency parsing.

Apart from these efforts, Arjuna and Kulkarni [2016] have worked on segmentation, analysis and graphical representations of Navya-Nyāya expressions. The Navya-Nyāya school of Indian logic and philosophy has developed a sophisticated language to address verbal cognition, logic, and epistemology and is known for its use of long compounds. The authors design a tool that includes a domain-specific segmenter, a semi-automatic constituency parser, and a context-free parser that translates the Navya-Nyāya expressions into a conceptual graph. This effort is very helpful in understanding better the scholastic Sanskrit commentaries that uses complex compounds using the relational notation as per the Navya-Nyāya concepts. However, this still remains a preliminary effort and more concrete work is to be done in this direction.

¹⁴See Table 2 for a brief summary.

Table 2. Parser

Sl.No	Model	Year	Type	Data
1	SHR Shallow Parser	2006	Rule-based	-
2	DFA Parser	2007	Rule-based	-
3	Unsupervised Parser	2009	Statistical	Sanskrit Tagger
4	Parser for Verses (UoH)	2019	Rule-based	-
5	EBM	2020	Neural	DCS
6	Data-driven Parser	2020	Neural	UoH Corpus
7	Jivnesh	2021	Neural	Sanskrit Treebank Corpus
8	Byt5-Sanskrit	2024	Pre-Trained	Sangraha and DCS

7 Discourse Parsing

This section discusses the application of discourse level parsing, such as anaphora resolution and question-answering.

7.1 Anaphora Resolution

Anaphora is a linguistic phenomenon where an expression is used to refer back to an item within the context. The term “anaphor” is used to describe the expression pointing back, while the entity to which it refers is known as the “antecedent” [Mitkov 1999]. Anaphora resolution involves computationally analyzing which anaphor corresponds to which antecedent. This task has been extensively studied in NLP. Similar to other languages, Sanskrit also presents challenges in dealing with anaphors, making their resolution a complex task. Despite the widespread exploration of anaphora resolution in various languages, there has been limited exploration for it in Sanskrit.

Sanskrit Anaphora - There are various types of anaphora, including pronominal anaphora, noun phrase anaphora, demonstrative anaphora, reflexive anaphora. Pralayankar and Devi [2010] develop an algorithm specifically designed to identify pronominal anaphora in Sanskrit. In the process of anaphora resolution, determining clause boundaries becomes crucial, as the position of the noun significantly influences the identification of the antecedent. Consequently, the pre-processing stage begins with the delineation of clause boundaries and the identification of subjects and objects using a morphological analyzer. Subsequently, NP preceding the **Pronouns (P)** are identified. The NP that matches in **Person, Number, and Gender (PNG)** with the pronoun is then recognized as the antecedent according to the algorithm. The algorithm was tested for 200 sentences of which 40 had pronominals. However, the system could correctly identify only 26 of the 40 pronominals.

SARS - In the study by Gopal and Jha [2014], an automatic resolution system for anaphors and cataphors in Sanskrit is discussed. The system utilizes POS tagged data to address lexical anaphors. The initial processing involves sentence tokenization and word tokenization of the input data. The system operates on one tokenized sentence at a time in the resolution of anaphors. Specifically, it examines each sentence for lexical anaphors, such as reflexive and reciprocals, and proceeds to the next sentence if none are found. When anaphors are identified, the system employs an algorithm to search for potential antecedents and resolves them. The candidate antecedents considered by the system include **common nouns (NC)**, NP, **personal pronouns (PPR)**, and **relative pronouns (PRL)**. These categories are distinguished based on their morphosyntactic tags, underscoring the importance of POS tagging information in the anaphora resolution process. The system’s performance was evaluated on 3659 sentences, revealing that out of 115 anaphor occurrences, only 70 were correctly identified by the system.

7.2 Question Answering (QA)

Question Answering is defined as the task through which human-posed questions in natural language are automatically answered. QA systems prove particularly beneficial when a user requires highly specific information and either lacks the time or inclination to go through all the available documentation related to the search topic to address the current problem [Mollá and Vicedo 2006]. There are different types of QA systems. They are:

- (1) Systems based on search engines to retrieve answers.
- (2) Systems that make use of linguistic information and machine learning methods to extract answers.
- (3) Systems that extract answers from structured data sources such as **Knowledge Base (KB)**.
- (4) A hybrid model that consists of all three [Soares and Parreiras 2020].

Although the advancement of QA systems has demonstrated significant benefits across various domains, there has been limited exploration in the development of QA systems for Sanskrit. The sole implementation within this domain is detailed below.

Question-Answering in Sanskrit through Automated Construction of Knowledge Graphs - Terdalkar and Bhattacharya [2023a] propose the development of a natural language question-answering system for Sanskrit, employing KB to store information in the structured format of **Knowledge Graphs (KG)**. These graphs utilize semantic triplets (subject, object, and predicate) to encode relationships and the knowledge extraction process automatically generates triplets. In this study, two monumental texts, *Rāmāyaṇa* and *Mahābhārata*, are utilized for capturing human relationships. Additionally, an Ayurvedic text, *Bhāvaprakāśa Nighaṇṭu*, is employed to identify synonymous relationships, emphasizing the significance of technical texts. The primary focus of the study revolves around identifying terms representing relationships. The subsequent crucial task involves parsing natural language questions, utilizing a similar triplet pattern. Once this parsing is accomplished, a SPARQL query pattern is constructed to interact with the KG, facilitating the retrieval of answers. The research assesses the system's performance across three distinct tasks: question parsing, conditional question answering (evaluating correctness), and the comprehensive question-answering task. A set of 35 questions from *Rāmāyaṇa* and 45 from *Mahābhārata*, sourced from 12 different users, formed the basis of the evaluation. The system yielded a precision score of 0.55 for the overall question-answering task when considering both texts collectively. Further for *Bhāvaprakāśa Nighaṇṭu*, the task was to identify a śloka synonymous with another. This was accomplished using the word vectors obtained for each śloka. With differing train and test data sizes in four different scenarios, different accuracy scores were obtained. However, the article concludes that the accuracy scores for all the components could be improved.

8 Machine Translation

Machine Translation (MT)¹⁵ stands out as a key application of NLP, facilitating the seamless transfer of information across diverse regions of the world. Machines play a pivotal role in rapidly translating extensive data from one language to another. While Sanskrit may not be a widely spoken language today, it holds a wealth of ancient Indian knowledge systems that pique the interest of many. The contribution of machines in assisting to unravel this valuable knowledge is highly significant. Although the attempts to create good translation systems for Sanskrit are numerous, yet the unique grammatical structure and rich contextual nuances of Sanskrit pose intricate challenges that continue to be a focal point in the quest for more accurate and nuanced translations.

¹⁵Since MT is an application of both sentential and textual parsing, this has been considered as a separate section.

8.1 English to Sanskrit

Experiments on MT for Sanskrit commenced in 2007 with Goyal and Sinha [2007] attempting to modify the AnglaBharati system design and adapting it for Sanskrit. The goal was to translate basic sentences from English to Sanskrit based on the **Pseudo Lingua for Indian Languages (PLIL)**, which is similar to context-free grammar. A text generator was constructed for the Sanskrit language, employing Pāṇinian grammar principles to generate Sanskrit translations. This was followed by a rule-based system introduced by Mane et al. [2010]. In this approach, the input (English sentences) is tokenized and subjected to morphological analysis. Through a bilingual dictionary, source words are then mapped to their equivalents in the target language. The reordering of words, guided by the grammar rules of the target language, produces the final output. The sentences were then converted into a waveform through which the audio was generated.

It is important to acknowledge that both these experiments were conducted on a relatively small scale. However, they laid the foundation for more comprehensive and sophisticated systems that emerged subsequently. As technology advanced and understanding of the complexities of Sanskrit improved, full-fledged systems with enhanced capabilities and broader applications emerged. They are discussed below:

ANN and Rule-based Model - Mishra and Mishra [2010] introduce a prototype model to translate from English to Sanskrit, combining traditional rule-based machine translation with an **Artificial Neural Network (ANN)** model. While the ANN model identifies equivalent Sanskrit words for English sentences, the rule-based model generates Sanskrit translations based on the rules governing noun and verb formation. The system employs a feed-forward ANN to select Sanskrit words from the English to Sanskrit **User Data Vector (UDV)**. Given the morphological complexity of Sanskrit, the system relies minimally on syntax, using morphological markings to recognize Subject, Object, Verb, Preposition, Adjective, Adverb, and Conjunctive sentences. The system can only handle English sentences that are of the forms: (i) simple subject, object and verb; (ii) subject, object, adverb and verb; (iii) subject, object, adjective and verb; (iv) subject, object, preposition and verb; (v) interrogative sentences and (vi) compound sentences. Evaluation metrics such as BLEU and METEOR are employed, and the article concludes by presenting scores for twenty randomly selected sentences.

Transfer-based - Pathak and Godse [2010] introduce in their article a transfer-based approach for translating from English to Sanskrit. The fundamental concept behind Transfer-based Machine Translation is that the input sentence can be converted into the output sentence by simplifying the parsing structure of the source language into an equivalent parse structure in the target language. This process requires essential grammar rules for both the source and target languages. Once the parse structure of the target sentence is formed, each word in the parse tree structure is searched for its meaning in a bilingual dictionary.

EtranS - Bahadur et al. [2012] introduce a comprehensive framework called “EtranS” designed for translating a series of English sentences into Sanskrit. The model incorporates a set of translation rules to convert source sentences into target sentences. The translation process involves breaking down the source sentence, extracting information from a dictionary, validating the information based on predefined rules, searching for compatible target words, translating, and ultimately generating the output. The English-to-Sanskrit translation model relies on the formulation of **Synchronous Context Free Grammar (SCFG)**, a subset of **Context Free Grammar (CFG)**, for linguistic representation of syntax. The software is designed with a focus on simplicity and effectiveness, particularly addressing the grammar of both the source and target languages. The results indicate a successful translation rate of ninety percent for the sentences, with a two percent reported error rate attributed to linguistic ambiguities.

8.2 Sanskrit to English

Interlingua-based - In an interlingua-based MT approach, the focus is on creating a language-independent structure, or interlingua, to aid translation. It emphasizes that machine translation should go beyond syntax and semantics to include a deeper understanding of text content. Therefore, Sreedeepta and Idicula [2017] use the Pāṇinian framework as the foundational structure and propose a translation system. The systematic process for translating a Sanskrit sentence begins with tokenization, followed by the generation of feature structures for the individual tokens. Subsequently, interlingua is constructed by applying kāraka analysis rules to the feature structures developed for the sentence. The next step involves mapping and generating target language tokens based on the established interlingua, followed by the creation of f-structures for these target language tokens. The final translation output in the target language is then generated by applying rules to the f-structure of the target language tokens. The system was tested on, approximately 35 sentences where it demonstrated accurate outputs for all instances. Performance evaluation was conducted using two metrics: human-based evaluation, utilizing scales such as CCOR, MEAN NONS, and DANG, and automatic metric evaluation, with BLEU selected as the benchmark for assessing translation quality.

Reinforcement and Transfer Learning Approach - Punia et al. [2020] explore novel approaches previously unexplored in the translation of Sanskrit to English. Initially, the study establishes a baseline using the Transformer architecture, incorporating a multi-head self-attention mechanism within an encoder-decoder framework. Followed by this, advancements to the transformer model involve the integration of Reinforcement Learning and Transfer Learning, resulting in a reported enhancement in BLEU scores attributed to the Transfer Learning mechanism. In addition to these methodological contributions, the study enriches the field by providing an open dataset comprising both parallel and monolingual data. The parallel dataset, totaling 9,000 lines, is sourced from diverse sources, including 2,100 sentences from OPUS, Sanskrit translations of the Bible, Shlokas from the Rāmāyaṇa, and additional sentences from the Bhagavad Gītā. The monolingual Sanskrit data consists of approximately 130,000 lines, utilizing the Romanized version of the Mahābhārata. For English monolingual data, the study extracts content from the Europarl dataset.

RNN Model - Koul and Manvi [2021] propose a neural machine translation system for translating Sanskrit to English, employing **Recurrent Neural Networks (RNNs)**. The model uses vectors representing the source Sanskrit sentence, the target English sentence, and a bilingual dictionary. The RNN uses a computational graph to analyze words of the sentence in parallel, breaking them down into their roots and linguistic categories. The dependencies of the analyzed words concerning the context are determined and stored in the vector for subsequent processing. For simple sentences, words are translated using the bilingual dictionary, while for complex compounds, the learner module suggests translations. Partial sentences are generated, and context is used to adjust grammatical aspects like tense. Any additional extra-linguistic information if present is incorporated and finally the output is generated. The model employs Naive Bayes and Support Vector Machines in the classification algorithm to identify the most suitable target word. Notably, the study highlights that the use of a Support Vector Machine classifier in determining the appropriate target word, yielded higher accuracy compared to the Naive Bayes approach.

Hybrid model - The work by Sitender and Bawa [2021] introduce a **machine translation system (MTS)** designed for translating Sanskrit to English that employs a hybrid approach, combining both direct and rule-based translation techniques. The system addresses language divergence among Sanskrit and English languages and proposes a solution to handle it. The MTS incorporates two bilingual dictionaries, a tagged Sanskrit corpus, a Sanskrit analysis rule base, and an **English Language Grammar Rule (ELGR)** base. It comprises of six modules that include source language

preprocessing, database, Sanskrit grammar demonstration, English parse tree generation, translation using the direct method, and output generation. For Sanskrit language processing, the system utilizes CFG in **Chomsky Normal Form (CNF)** and employs the **Cocke–Younger–Kasami (CYK)** parsing technique for processing input Sanskrit sentences. The article also introduces a novel algorithm that constructs a parse tree from the parsing table. The target language sentence is generated through the combined efforts of the ELGR base and bilingual dictionaries. Evaluation of the proposed system was conducted using the Natural Language Toolkit API in Python, resulting in a BLEU score of 0.7606, a fluency score of 3.63, and an adequacy score of 3.72.

Itihāsa - Aralikatte et al. [2021] introduce a large-scale translation dataset containing 93,000 pairs of Sanskrit shlokas and their English translations extracted from two famous Indian epics, the Rāmāyana and the Mahābhārata. The abridged versions of these two texts authored by M.N. Dutt have been taken acknowledging its large size and popularity. Further, this study attempts to highlight the complexity of the dataset by training and evaluating the performance of standard translation models. This includes one statistical model and five neural models. For SMT Moses is used and for NMT seq2seq Transformers with standard BERT architectures has been used. The evaluation is based on four parameters (i) character n-gram F-score (ii) token accuracy (iii) BLEU score and (iv) Translation Edit Ratio score. The study concludes that all the models performed poorly with low token accuracy and high translation edit ratio and underscores potential areas for improvement in the translation models.

8.3 Sanskrit and Other Indian Languages

Several efforts have been made to translate Sanskrit to other Indian languages and vice-versa. Although we will not be discussing them in detail, we will be mentioning a few of them, acknowledging their contributions.

The initial system implemented for Sanskrit-Hindi MT was at the Jawaharlal Nehru University, Delhi (Special Centre for Sanskrit Studies) [Pandey and Jha 2016]. This system was trained on two different platforms which are **Microsoft Translator Hub (MTHub)** and Moses with around 35000 sentences. Subsequently, multiple **Neural Machine Translation (NMT)** models were experimented with for the Sanskrit-Hindi language pair. Singh et al. [2019] introduce a hybrid system that combines NMT and RBMT, achieving a BLEU score of 61.2% and an accuracy of 99%. Following this Kumar et al. [2019] propose a Zero-Shot Translation method trained on English-Hindi and Sanskrit-English language pairs. A corpus-based NMT was presented by Singh et al. [2020], marking the first of its kind for the Sanskrit-Hindi language pair. This corpus-based model, incorporating a deep neural network, divides the workflow into three phases: (i) Data Analysis, (ii) Data Processing, and (iii) Target Generation. Another NMT model, employing an encoder-decoder with self-attention and the inclusion of gated recurrent units, was experimented with by Sethi et al. [2023] for the Sanskrit-Hindi language pair. This system outperformed all existing translation systems for the same language pair. Additionally, an RBMT system was implemented for translation from Hindi to Sanskrit, leveraging the crucial linguistic features of both languages. Efforts were extended to translate Sanskrit into Gujarati and Malayalam as well. Raulji et al. [2022] proposes a grammatical transfer approach for translating Sanskrit to Gujarati. To enhance implementation accuracy and translation clarity, in-depth research on the creation of Nouns, Verbs, Pronouns, and Indeclinables, along with their mappings, was conducted. Chingamtotattil and Gopikakumari [2022] introduce an NMT for Sanskrit-Malayalam, utilizing an attention-based mechanism for its development. The study further improves the system by incorporating morphological elements and evolutionary word sense disambiguation.¹⁶

¹⁶A brief summary of all MT tasks has been presented in Table 3.

Table 3. Machine Translation

Sl.No	Model	Year	Type	Source to Target
1	AnglaBharati	2007	Rule-based	English to Sanskrit
2	Deepak	2009	Rule-based	English to Sanskrit
3	Anusaarak	2009	Rule-based	Sanskrit to Hindi
4	ANN	2010	Rule-based	English to Sanskrit
5	Transfer-based	2010	Rule-based	English to Sanskrit
6	EtranS	2012	Rule-based	English to Sanskrit
7	JNU	2016	Statistical	Sanskrit to Hindi
8	Interlingua-based	2017	Rule-based	Sanskrit to English
9	RNN	2017	Neural	Sanskrit to English
10	RBMT and NMT	2019	Neural	Sanskrit to Hindi
11	Zero-shot	2019	Neural	Sanskrit to Hindi
12	Hybrid	2020	Rule-based	Sanskrit to English
13	Reinforcement and Transfer Learning Approach	2020	ML	Sanskrit to English
14	CB DNN	2020	Neural	Sanskrit to Hindi
15	Itihāsa	2021	Neural	Sanskrit to English
16	Transfer-based	2022	Rule-based	Sanskrit to Gujarati
17	NMT	2022	Neural	Sanskrit to Malayalam
18	NMT	2023	Neural	Sanskrit to Hindi

In addition to the aforementioned efforts, the widely used and easily accessible platform for translations between Sanskrit and various other languages is the Google Translate.¹⁷ In 2022, Google Translate, which uses the Transformer architecture, expanded its language repertoire to include Sanskrit.¹⁸ The inclusion of Sanskrit in Google Translate marked a significant milestone, playing a pivotal role in expanding the accessibility and reach of the Sanskrit language. Although there have been no quantitative studies analyzing Google Translate’s performance specifically for Sanskrit, a preliminary study by Pradeep et al. [2024] examined its qualitative performance. This study focuses on word meaning errors in the English translation of the Bhagavad Gita and highlights how the performance of Google Translate is affected when dealing with polysemy.

9 Literary Analysis

This section discusses tasks that are specific to the analysis of poetry (kāvyā) in Sanskrit. Sanskrit poetry is divided into śravyakāvyā (that which is heard: poetry and prose), and dr̥śyakāvyā (that which is seen: drama). Śravyakāvyā is further divided into gadya (prose), padya (poetry) and campu (amalgamation of poetry and prose). Despite its divisions, Sanskrit poetry on a whole presents a wide range of features such as rasa (sentiment), dhvani and vyañjana (suggestion), alaṅkāra (figures of speech), chandas (prosodical meter), vakrokṭi (expression) and more. Among these, preliminary efforts have been made to computationally analyse a few of them. Besides this, the current section also discusses prose order normalization which is yet another major task pertaining to Sanskrit poetry.

9.1 Chandas Analysis

A substantial portion of Sanskrit literature is composed in verse rather than prose, and these verses adhere to specific metrical patterns referred to as “Chandas” [Rama and Lakshmanan 2010]. There are different kinds of chandas in Sanskrit and the scholarly pursuit of analyzing and ascertaining

¹⁷<https://translate.google.co.in/>

¹⁸<https://blog.google/products/translate/24-new-languages>

the metrical structure of a verse assumes an important role in comprehending the poet's stylistic preferences and poetic prowess. Nevertheless, the task of metrical identification proves to be a nuanced and intricate endeavor. In response to this scholarly exigency, efforts have been undertaken to develop computational tools to automate the metrical classification process. They have been discussed below.

Meter Identifying Tool (MIT) - Melnad et al. [2013] have developed an automated tool for identifying metrical patterns, building upon the earlier work of Anand Mishra.¹⁹ The tool encompasses a meter database containing metrical definitions presented in a machine-readable text file, and utilizes the Sanskrit Library Phonetic Basic encoding (SLP1) for processing. Both the input and display are in SLP1 format. Accepting either a single line or the entire verse as input, the tool yields a comprehensive output comprising the meter's name and type linked to its definition, the parsed string represented in orthographic syllables, the scansion (prastāra) depicting the arrangement of light (laghu) and heavy (guru) syllables, the pattern of gaṇas, and the count of syllables or mātrās. To validate its efficacy, the tool underwent testing on 1,031 verses from Pūrṇabhadra's Pañcākhyānaka. Notably, MIT achieved a remarkable accuracy rate of 98.6%, accurately identifying the meter type for 1,018 out of the 1,031 verses.

A user-friendly tool for metrical analysis of Sanskrit verse - An approach very similar to the previous one has been discussed by Rajagopalan [2020]. The current system differs from the previous one in the following ways:

- (a) It accepts input in several Roman transliteration schemes such as **International Alphabet of Sanskrit Transliteration (IAST)**, **Harvard-Kyoto (HK)**, and **Indian Languages Transliteration (ITRANS)**, Unicode Devanāgarī, and Unicode Kannada scripts and then converts it to SLP1.
- (b) The metrical database is not used as it is, instead, it is converted into data structures through which indices are generated. The indices are called pāda1, pāda2, pāda3, pāda4, ardha1, ardha2, and full.
- (c) The user can also look up for partial and incorrect verses. The system generates all potential matches for the user input.
- (d) The tool not only helps in identifying the type of meter but also identifies errors if any in a given verse. For instance, in the first version of the tool, the first meter added was Mandākrāntā and the entire Meghadūta text (which is completely composed in Mandākrāntā) was given as the input to the tool. The tool successfully identified 23 errors out of the 122 verses. The tool also provides a link, where the user feedback is taken.

Skrutable - Continuing the line of previous research, Tyler [2023] proposes a new tool for meter type identification called Skrutable. Although Skrutable shares similarities with the previously discussed tools, there are a few noteworthy distinctions. In contrast to the JavaScript-based nature of the other two systems, Skrutable is implemented in Python and supports a broader range of notations, including Unicode-based IAST, ASCII-based SLP1, HK, ITRANS, WX, Unicode-based Devanagari, Gujarati, and Bengali. Skrutable operates on a "cascade" of checks using fixed regular expressions and patterns. Therefore, it does not recognize verses with fewer or more than four quarters, meaning it does not support partial verses. A key feature of Skrutable is its continuous pāda resplitting and cascade-based testing until a perfect meter match is found. All potential results, including partial matches, are collected and scored based on an empirically calculated look-up table. The tool then reports the match with the highest score as the best result. The tool was tested on a dataset comprising 150,000 verses, including Prakrit, sourced from GRETEL and the repository of

¹⁹Currently not available.

Vishwas Vasuki.²⁰ It was evaluated based on four parameters: accuracy, usefulness, speed, and ease of use. The article concludes by highlighting that the analysis of meters for the entire Rāmāyaṇa text was accomplished within 14.5 seconds.

Chandojñānam - Terdalkar and Bhattacharya [2022] introduce a web-based system for meter identification and utilization called Chandojñānam, that not only accepts plain text and text files but also accommodates images as input. The tool recognizes various transliterations detected by indic-transliteration and processes images from Google OCR and Tesseract OCR engines. Chandojñānam follows a workflow similar to previous tools, emphasizing meter identification. However, it distinguishes itself by focusing on tolerance for erroneous text and providing mechanisms for text correction. The system underwent evaluation using three versions of Meghadūta from Wikisource, sanskritdocuments.org, and GRETIL. Additionally, three other texts, namely Śāntavilāsa, Śrīrāmarakṣāstotra, and Rājendrakarṇapūra, featuring various meters, were extracted from Wikisource. This compilation formed a dataset comprising 1,038 verses with 17 distinct meters. In a comparative analysis, Chandojñānam, along with the tools developed by Rajagopalan [2020] and Tyler [2023], was evaluated on the same corpora. Chandojñānam demonstrated a high accuracy of 98.2% in detecting the correct meter from erroneous verses.

9.2 Alaṅkāra Analysis

Vāmana defines “alaṅkāra” as beauty (saundaryamalaṅkāraḥ) [Raghavan 1942]. The term “alaṅkāra” in Sanskrit poetry signifies beauty that is derived from embellishment or adornment. Such an adornment or beautification can be of both its form (language) and content (meaning). In the realm of aesthetics (alaṅkāraśāstra), the rhythmic arrangements and alliterations that enhance the beauty of words are referred to as śabdālaṅkāras. Figures of speech, such as simile, metaphor, and personification, contributing to the aesthetic appeal of the content or meaning of the text, are referred to as arthālaṅkāras [Raghavan 1942]. Recent efforts have been initiated toward computationally identifying alaṅkāras in Sanskrit, with limited work discussed thus far.

Yamaka Identifier - Yamaka is a type of śabdālaṅkāra, wherein a specific group of phonemes is deliberately repeated within a stanza, contributing to a distinctive rhythmic pattern. Barbadikar and Kulkarni [2024] explore a tool designed for identifying and categorizing Yamakālaṅkāra based on the definitions provided by three famous rhetoricians: Bharata, Daṇḍin, and Rudraṭa. The system accepts input in various formats, and the internal processing is facilitated using WX notation. Subsequently, n-grams are generated for each half of the verse and sorted independently, followed by a positional check for repetition and subsequent classification. In testing, the tool demonstrated accuracy by correctly identifying and classifying 163 out of 185 verses.

Anuprāsa Identifier - Similar to Yamaka, Anuprāsa is also a form of śabdālaṅkāra, involving the repetition of consonants in close proximity. The renowned rhetorician Viśvanātha categorizes Anuprāsa into five types which are discussed thoroughly in the study by Barbadikar and Kulkarni [2024]. This study aims at identifying and classifying Anuprāsaālaṅkāra based on Viśvanātha’s classification. The implementation of this tool closely resembles the Yamaka identifier. The tool initially transforms all nasals into anusvāra to avoid ambiguities based on spellings. Then the algorithm based on the five-fold classification is applied. The tool was tested on 70 ślokas from śāstric texts and Raghuvamśa, along with 10 prose passages from Bāṇabhaṭṭa’s Kādambarī. The study concludes that the tool effectively handles Anuprāsa instances and classifies them into defined categories.

²⁰https://github.com/sanskrit/raw_etexts

9.3 Prose Order Normalization

The arrangement of words in Sanskrit verses are in general not aligned with its verbal cognition, i.e., the word order is jumbled such that they adhere to one of the Sanskrit meters mentioned in the prosodical framework. Therefore, reorganizing their structure into a prose is a very important task that requires linguistic expertise. A challenge that Sanskrit shares with other inflectional languages like Greek and Latin is the absence of clearly parenthesized phrase structures, which often leads to dislocated sentences. To address this, prose order normalization is an essential task, though it has not been extensively explored by existing systems. In many cases, this is treated as a sub-task in parsing, however Krishna et al. [2019] present *kāvyaguru*, a linearization approach for this task, which involves two pretraining steps and a seq2seq model with CNNs. The first pretraining step learns task-specific token embeddings, while the second uses self-attention to generate multiple word order hypotheses. These hypotheses are then processed by the seq2seq model. Trained on verses from the *Rāmāyaṇa*, the model achieves a BLEU score of 55.26, outperforming baseline models.

10 Citation Matching

Word matching is of great importance among tasks pertaining to information retrieval. It involves considerations on digital library representation and management, an area that remains under-developed so far. Even within word matching, the sub-task of citation matching remains quite unexplored for Sanskrit. Citation matching is closely related to alignment of manuscripts and philological processing like critical editions. In literature, it typically involves determining the occurrences of citations within a given textual corpus [Prasad and Rao 2010]. Sanskrit literature is rich in inter-textuality in the various categories of commentaries such as *vṛtti*, *vākya*, *ṭīkā*, *locanā*, *dīpikā*, and so on. For instance if we take Pāṇini's *Aṣṭādhyāyī*, which is the base text. We have Kātyāyana's *vārtikas* followed by Patañjali's *Mahābhāṣya* as the primary commentaries. Then we have several other commentaries such as *Siddhāntakaumudī*, *Kāṣikāvṛtti*, *Tattvabodhinī*, *Bālaṃanoramā* and so on. This poses a challenge to digital libraries in representing the successive hierarchical versions of the commentaries. While there have been numerous citation-matching studies for other scientific texts, Sanskrit texts have been largely overlooked due to these challenges posed by them. Nevertheless, we do have a study by Prasad and Rao [2010] addressing this gap. This study presents citation matching methods for Sanskrit through an illustration of matches between *Mahābhārata-Tātparyanirṇaya* (as the text containing citations) and the original *Mahābhārata* (as the source text to search for citations). The study makes use of two citation matching techniques which are: exact citation matching and approximate citation matching. The article explores algorithms for both exact and approximate matching, highlighting the challenges of exact matching in Sanskrit, particularly considering the presence of *pāṭhāntaras* (different readings) of the same corpus. While the study provides insights into cutoff results for approximate matching, it falls short in examining human evaluation for both citation-matching methods.

11 Aṣṭādhyāyī Simulation

Pāṇini's *Aṣṭādhyāyī* serves as a logical framework for the Sanskrit language. The aphorisms (*sūtras*) within this grammatical treatise intricately delineate the structure of Sanskrit in terms of phonology, morphology, and syntax. The aphorisms of *Aṣṭādhyāyī* can be grouped under six categories.²¹ They are:

²¹*saṃjñā ca paribhāṣā ca vidhirniyama eva ca |
atideśo'dhikāraśca ṣaḍvidham sūtralakṣaṇam ||*

- (1) *saṃjñā sūtras*: naming rules that assign labels which are subsequently utilized in other *sūtras*.
- (2) *paribhāṣā sūtras*: meta-rules that govern the interpretation and applicability of *sūtras*.
- (3) *vidhi sūtras*: prescribing rules that handle addition, deletion and insertion.
- (4) *niyama sūtras*: conditioning rules that further constrain the application of *vidhi sūtras*.
- (5) *atideśa sūtras*: extension rules that portray extension in other contexts
- (6) *adhikāra sūtras*: governing/division rules that segment content and extend meaning to upcoming *sūtras*.

Sridhar and Srinivasa [2008] state that *Aṣṭādhyāyī* can be conceptualized as an automaton designed to generate words and sentences. The entire *Aṣṭādhyāyī* is structured such that general rules are initially presented, followed by their corresponding constraint rules. The systematic and mathematical arrangement of *Aṣṭādhyāyī* has intrigued numerous linguists and computer scientists, sparking interest in computationally modeling it such that words and sentences are generated automatically according to the rules of Pāṇini. Despite several theoretical works delineating the computational aspects of *Aṣṭādhyāyī*, fewer efforts have been made to practically implement these discussions.

The initial attempt to simulate Pāṇini's *Aṣṭādhyāyī* was undertaken by His Holiness Dr Shiva-murthy Swamiji, who developed a unique software known as "Ganaka Ashtadhyayi". Followed by this Mishra [2007] in his article, mapped the rules of *Aṣṭādhyāyī* to mathematical equations and implemented a simulator called **Pāṇinian Sanskrit Simulator (PaSSim)**. The primary objective of his study was to construct a lexicon based on Pāṇini's framework. PaSSim comprised five modules: database, grammar, templates, FSA, and display. The program aimed at providing a step-by-step derivation of an input word. Following this work, Mishra [2009] presented another study that enhanced the previous model. The model features an analytical phase using heuristics and an improved reconstitution phase based on *Aṣṭādhyāyī* rules. Its goal is to infer the constituent elements of an expression through improvised heuristics. The model, supported by a custom database, includes four modules: input, analyzer, synthesizer, and output. The article also explores the circular nature of the grammatical process, from a provisional expression to the final Sanskrit expression. This progression is followed by an extension to the existing model through the incorporation of *Vedāṅgas*.²² Mishra [2010] introduces a broader coverage of the Pāṇinian meta-language incorporating a mechanism for the automatic application of rules and situating the grammatical system within the procedural complexes of *Vedāṅgas*. The synthesizer module, as presented in the preceding study, mirrors the general model designed to represent the fundamental processes of the *Vedāṅgas*.

Apart from these efforts by Mishra, we also have Goyal et al. [2009] who discuss the programming perspective of *Aṣṭādhyāyī*, specifically examining the order of its rules. The focus is particularly on three prominent aiding aphorisms: "pūrvatrāsiddham", "asiddhavad atrābhāt", and "śatvatukor asiddhah". The study not only explores the theoretical aspects but also implements *Aṣṭādhyāyī* using multiple modules, delving deep into its nuances. Additionally, Sridhar and Srinivasa [2008] and Subbanna and Varakhedi [2010] have undertaken the task of establishing a mathematical framework for the computational analysis of *Aṣṭādhyāyī*. While the former elaborately focuses on the structure of *Aṣṭādhyāyī* and conflict resolution techniques present in *Aṣṭādhyāyī* as well as the *Vārtika*'s of Kātyāyana, the latter mathematically maps the concept of *asiddhatva* in *Aṣṭādhyāyī* to a new concept called filter. Further an XML formalization of the *Aṣṭādhyāyī* that can be converted to executable code was given by Scharf [2015]. A recent article by Prasad [2024], introduces a quick word generator program known as "vidyut-prakriya" based

²²The six auxiliary disciplines that support the understanding of Vedas along with their preservation.

on the Aṣṭādhyāyī. Vidyut-prakriya implements more than 2,000 rules of the Aṣṭādhyāyī supporting tiñantas, kṛdantas, taddhitāntas, and subantas, with partial support for samāsas and accent. Around 500 rules from the Uṇāḍipāṭha and around 50 rules from the Pāṇiniyaliṅgānuśāsanam have also been implemented as a part of the program. The main aim of this study was to create a finite program that could potentially generate infinite word forms in Sanskrit in a fast and feasible manner. Notably, through informal benchmarks, the study indicates that the vidyut-prakriya program is almost three orders of magnitude faster than comparable opensource systems.

There have also been efforts to simulate portions of Aṣṭādhyāyī for word generation tasks. Jha [2004] proposes a model designed for constructing nominal inflectional morphology in Sanskrit. This model is implemented using PDC-32 Prolog, and for each input, the program generates 24(21+3) śabdarūpa (wordforms) along with parsed constituents, providing detailed information. It is worth noting that while this study draws on Aṣṭādhyāyī, it also incorporates Siddhānta Kaumudī, an explanatory text on Aṣṭādhyāyī that follows a distinct order of the aphorisms. In their article, Satuluri and Kulkarni [2013] introduce compound generation, referencing the rules outlined in the Aṣṭādhyāyī concerning compound formation. The article presents this process as a phrase structure grammar, wherein the sūtras that denote technical terms are represented as context-free rules, and the vidhi sūtras are depicted as regular expressions. Similarly, Krishna and Goyal [2015] propose an object-oriented model that automatically generates derivative nouns (Taddhitas). The rules of Aṣṭādhyāyī are modeled as classes and then grouped based on the concept of anuvṛtti (a carry-forward methodology followed by Pāṇini). In addition to generating the taddhita forms, the model adheres to the sequence of rules applied as per Aṣṭādhyāyī in the derivation process, along with the appropriate usage of conflict resolution techniques needed in the selection of rules. The model was evaluated by five Sanskrit linguists on 60 sample cases where a total of 10 cases were evaluated as incorrect. Patel and Katuri [2015] introduce “Prakriyāpradarśinī”, a system that imitates subanta derivation process. This model makes use of the prakriyā method given in Siddhāntakaumudī of Bhaṭṭojī Dikṣita and not Aṣṭādhyāyī directly. In the article, an optimum order of application of rules from Sanskrit NLP perspective has been ventured and an “NLP order model” and “NLP order hypothesis” has been proposed for coding subantaprakaraṇa (chapter that discusses nominals) of the Siddhāntakaumudī.

12 Digital Infrastructure for Sanskrit

We discuss the digital infrastructure for Sanskrit under two broad categories: computational platforms, digital libraries and repositories used for training software.

12.1 Computational Platforms

Several platforms facilitate the computational analysis of Sanskrit in a web-interactive mode while also providing resources for such analysis. They are discussed below:

- (1) The Sanskrit Heritage Site: The **Sanskrit Heritage (SH)** Site provides a range of services for the computational analysis of Sanskrit, including access to dictionaries, lemmatizers, and a reader. This toolkit incorporates two significant dictionaries, namely the Sanskrit-French Heritage dictionary and the Monier-Williams Sanskrit-English dictionary, along with a morphological lexicon containing inflected forms in XML format under various transliteration schemes. The generative lexicon of SH gave rise to the first hypertext Sanskrit dictionary equipped with grammatical tools such as inflection generation for both the French and English dictionaries, which is a much appreciated feature by the learners of the language. Thus, highlighting the importance of digital lexicons, and their management in computational linguistics. The Sanskrit Heritage Reader also enables machine-assisted analysis of Sanskrit

sentences, encompassing segmentation, morphological tagging, and multiple parsers. While many of these tasks have been discussed in the preceding sections, additional information on the utilization of these dictionaries and tools can be found on the web interface.²³

- (2) **Saṃsādhani**: The Department of Sanskrit Studies from the University of Hyderabad has developed a Sanskrit toolkit named Saṃsādhani.²⁴ The platform provides a diverse set of tools, including a Sandhi analyzer and generator, morphological analyzer and generator, Aṣṭādhyāyī simulator, transliterator, Nyāyacitrāḍipikā (an integrated tool for segmenting, parsing, graph rendering, and identifying compound types in Navya-Nyāya expressions), Yamaka and Anuprāsa identifier. Additionally, there is a Sanskrit-to-Hindi accessor cum machine translator named Anusāraka. The platform encompasses the knowledge net of Amarakoṣa (utilizing the Nyāya ontology) and Pāṇinian Dhātuvṛttis as dictionaries, along with the Sanskrit-French Heritage dictionary, Monier Williams Sanskrit-English dictionary, Apte's Sanskrit-Hindi dictionary and Cappeller's Sanskrit-German dictionary. Furthermore, the toolkit includes various tagged and untagged corpora for computational analysis, along with sample e-readers on Bhagavadgīta, Saṅkṣeparāmāyaṇa, Śiśupālavadha, and Raghuvamśa. The toolkit also consists of a **Sanskrit Text Annotation and Research Tool (START)** [Kumar et al. 2024] as a mirror site that provides an appropriate user interface for the semi-automatic generation of automated texts, essential for constructing E-readers. The platform further offers access to "Gaveṣikā," the first Sanskrit search engine. This tool enables users to search for Sanskrit words or even prātipadika/dhātu across different corpora. While the implementation details of several tools are discussed in other sections of this article, for a more in-depth understanding, one can also refer to the theses of research students available on the website.²⁵
- (3) **JNU Toolkit**: The Research and Development in Computational Linguistics at the School of Sanskrit and Indic Studies, **Jawaharlal Nehru University (JNU)**, has been actively engaged in diverse areas of language technology for Sanskrit and other Indian languages. Presently, the department is dedicated to the creation of Sanskrit analysis tools for the development of the **Sanskrit-Hindi Translator (SaHiT)**. Notable tools developed thus far include the Sandhi Splitter, Sandhi Generator, Subanta (nominal form) analyzer, Subanta Generator, Tiṇanta (verbal form) analyzer, Tiṇanta generator, Kṛdanta (derivatives) analyzer, Sanskrit POS tagger, Sanskrit anaphora resolution system, Kāraka analyzer, Gender Recognizer and Sanskrit Homonym Analyzer. These tools have largely emerged from the research theses of students affiliated with the department.²⁶ The toolkit also integrates access to various lexical resources such as Amarakoṣa, Halāyudhakoṣa, Mahābhārata, Medinikoṣa, Ayurveda texts, and more. Additionally, the department has been instrumental in the development of tagsets for Indian languages and has introduced the first Sanskrit tagset, which is available on their website.²⁷
- (4) **Sambhāṣā**: The Vyākaraṇa department at Karnataka Sanskrit University is dedicated to developing a computational analysis toolkit for Sanskrit.²⁸ Several projects have been undertaken as part of this initiative, including the Mahākoṣa data navigation tool, Āyussamskṛtam Courseware for Ayurvedic medical students, Tarkasaṅgrahaḥ Ontological knowledge representation, Vāṇmayī-e create text, Prākṛt verbforms generator, Nāmadhātūrūpikā

²³<https://sanskrit.inria.fr/>

²⁴<https://sanskrit.uohyd.ac.in/scl/>

²⁵<https://sanskrit.uohyd.ac.in/faculty/amba/#d18>

²⁶<http://sanskrit.jnu.ac.in/rstudents/phd.jsp>

²⁷<http://sanskrit.jnu.ac.in/index.jsp>

²⁸<https://sambhasha.ksu.ac.in/>

denominative verbs analyzer and generator, Prācīnanyāyaḥ Ontological knowledge representation, Sanskrit Quiz App, Spell Checker, Bhaṭṭikāvya with Kāraka dependency diagram, Anvayacitraṇam tool, Vaijayantikośa Knowledge net, Śabdakautukam Practice Noun forms, Sandhi Splitting tool, and **E-Śloka Architect (E-SA)**. The website also provides access to e-courses for beginners interested in learning Sanskrit.

- (5) SHR++: Krishna et al. [2020c] introduce a web-based annotation framework designed to assist professional annotators in the morpho-syntactic annotation of Sanskrit corpora. This interface makes use of the Sanskrit Heritage Reader and extends its functionalities to include annotation capabilities for dependency relations as well. It leverages the SH to enumerate valid word splits and their morphological analyses, thereby aiding annotators in the decision-making process. It also incorporates segmentation suggestions from automated tools, aiming at reducing the cognitive load of annotators. The tool's evaluation experiment demonstrates a reduction in annotation time by about 20.15%, highlighting its effectiveness in aiding human annotators.
- (6) Antarlekhaka: Terdalkar and Bhattacharya [2023b] present a tool that enables manual annotation of Sanskrit corpora in eight different categories. The task categories supported by Antarlekhaka include sentence boundary detection, canonical word ordering, free-form text annotation of tokens, token classification, token graph construction, token connection, sentence classification, and sentence graph construction. The tool is designed to be unicode-compatible, language-agnostic, and deployable on the web, allowing for distributed annotation by multiple annotators simultaneously. Its performance was assessed through a comprehensive two-fold evaluation method, encompassing both subjective and objective measures. In the subjective evaluation, 16 annotators rated the tool on a scale of 1 to 5 across various categories, including ease of use, annotation interface, and overall performance. The tool achieved scores of 4.3 for ease of use, 4.4 for the annotation interface, and an overall score of 4.1. Objectively, the tool was benchmarked against other existing tools across 29 categories, securing a total score of 0.79, surpassing the performance of all other tools.

Courses are being offered on how to use these computational platforms,²⁹ catering to students and others interested in learning Sanskrit. This makes it easier for them to apply these tools for various purposes, enhancing their learning experience. Although these courses aim at explaining the tools of the software platforms, they are also helpful for spotting errors and correcting the software, based on the feedback. Additionally, some exercises included in these courses have been systematically used for corpus tagging through crowd-sourcing, facilitating faster data annotation and evaluation.

12.2 Digital Libraries and Repositories for Training Software

Besides the data resources that can be found as a part of the computational platforms mentioned earlier, several machine readable Sanskrit texts have been made available. The advent of SCL began with the digitization of texts in Sanskrit, and the primary efforts in this direction have been made by:

- (1) The GRETIL:³⁰ GRETIL is a platform that offers machine-readable texts in Indian languages which includes Sanskrit as well. These texts come from different people and organizations. It started as a register of download sites for texts but has now been focusing on documenting and encoding these texts. It makes sure the texts are easy to search across all languages in the collection, without needing extra conversion.

²⁹Samsādhani – Praveśikā <https://sanskrit.uohyd.ac.in/IOE102/>

³⁰<https://gretil.sub.uni-goettingen.de/gretil.html>

- (2) Sanskrit Library:³¹ Sanskrit Library is a platform committed to advancing education and research in Sanskrit, offering access to digitized texts and computerized tools. The platform consists of tools such as encoder, morphological analyzer, meter identification tool, and nominal and verbal paradigm generation. The Sanskrit Library also offers courses for those interested in enhancing their learning of Sanskrit and other Indian languages.
- (3) SARIT³² and Ambuda³³: We also have initiatives such as **Search and retrieval of Indic texts (SARIT)** and Ambuda. SARIT hosts electronic versions of Sanskrit texts that are well-documented, dated, and contain embedded notes regarding their revision history. These texts are marked using the **Text Encoding Initiative (TEI)**, which serves multiple purposes and remains adaptable and valuable in the long run. Ambuda, on the other hand, is a comprehensive library of traditional Sanskrit literature. While it is relatively smaller in scale at present, its goal is to become the largest Sanskrit e-library, offering access to Sanskrit texts of the highest quality from anywhere in the world. It includes features such as word-by-word analysis, integrated dictionary support, and transliteration into various scripts.

We also have multiple other repositories in Sanskrit such as the Wordnet, as well as datasets that aid in training the models (softwares). These include:

- (1) **Digital Corpus of Sanskrit (DCS)**:³⁴ Another major dataset repository for Sanskrit is the DCS. It consists of Sanskrit texts that have undergone Sandhi-splitting, providing comprehensive morphological and lexical analysis. This is the largest annotated dataset available for Sanskrit. It also has access to an online dictionary.
- (2) Sanskrit Wordnet: Inspired by the English wordnet which was developed at Princeton University, attempts were made to develop wordnets for Indian languages as well. A wordnet for Sanskrit is being developed at IIT Bombay.³⁵ Sanskrit Wordnet relies on the Hindi Wordnet as its primary source. Initially, the creation of synsets in Sanskrit Wordnet involved using random synsets from the Hindi Wordnet. Subsequently, curated lists of significant Sanskrit words were obtained from various sources. Innovative approaches were implemented, including the insertion of concepts or glosses into the Sanskrit Wordnet. To generate synset glosses in Sanskrit, a combination of glosses from dictionaries like Śabdakalpadrūma and translations of glosses from Hindi Wordnet synsets were employed. The strategy for synset creation followed the principle of commencing with the most frequently used words in modern Sanskrit and concluding the synset with the least frequent words from Vedic Sanskrit [Kulkarni et al. 2010a]. Creating the Sanskrit Wordnet poses significant challenges, primarily attributed to the extensive volume of lexical knowledge. Therefore the development of Sanskrit Wordnet is an ongoing process. Another WordNet³⁶ for Sanskrit has also been developed under the guidance of Dr. William Michael Short at the University of Exeter. This WordNet aims at modeling Sanskrit's semantic system as comprehensively and accurately as possible, in a format that is both machine-interpretable and machine-actionable. This makes it suitable for use in various NLP applications, particularly in the field of natural language understanding.
- (3) Other Datasets: To align data with their experiments on word segmentation, researchers have generated subsets from the DCS called the SIGHUM dataset. Notable among them are

³¹<https://sanskritlibrary.org/index.html>

³²<https://sarit.indology.info/sarit-pm/docs/welcome.html>

³³<https://ambuda.org/>

³⁴<http://www.sanskrit-linguistics.org/dcs/>

³⁵https://www.cflit.iitb.ac.in/wordnet/webswn/english_version.php

³⁶<https://sanskritwordnet.unipv.it/>

DCS (SEGMENTATION) by Hellwig and Nehrdich [2018], DCS (SIGHUM) by Krishna et al. [2017] and DCS (HACKATHON).³⁷ Additionally, Krishnan et al. [2020] and Krishnan et al. [2024] utilize the DCS to establish a gold-tagged dataset, employing the SHR to create a morphologically tagged and segmented corpus. Apart from these, there are datasets created by MeitY under consortium mode funded by the **Technology Development for Indian languages (TDIL)**.³⁸ Furthermore, there's the **Sanskrit Treebank Corpus (STBC)**, sourced from UoH for dependency parsing, and the Universal Dependencies: Vedic Sanskrit Treebank.³⁹ A dataset named Sandhikosh [Bhardwaj et al. 2018] has been formulated specifically for evaluating the performance of Sanskrit Sandhi tools. This dataset comprises five sub-corpora, encompassing a rule-based corpus, literature corpus, Bhagavad Gīta corpus, UoH corpus, and Aṣṭādhyāyī corpus. We also have initiatives undertaken for data creation by the **Central Institute of Indian Languages (CIIL)**⁴⁰ and the **AI4BHARAT**.⁴¹

13 Forging Ahead: Observations and Future Work

Being a field that was almost non-existent fifteen years ago, SCL has rapidly evolved to encompass a wide range of models, from rule-based systems to advanced pre-trained ones. We have multiple digital resources and repositories available today that can be made use of for several purposes. The growth of the field is so dynamic that it fosters frequent collaborative efforts. Datasets are now openly available, software and models are shared on open-source platforms, and many of these resources can even be cloned for local use. A key factor driving this collaborative spirit is the organization of regular events such as the Sanskrit Computational Linguistics Symposium series,⁴² along with dedicated sessions at the World Sanskrit Conference⁴³ and various other workshops. These events bring together Sanskrit scholars, computer scientists, and linguists, allowing them to engage in discussions that maintain the foundations (language theories, grammatical frameworks, and more) of the field. These foundations are crucial for conceptualizing problem statements and creating the necessary data. Initially, the field lacked sufficient annotated data to even begin with any kind of computational analysis. Over the past 15 years, researchers have dedicated themselves to addressing this gap by conceptualizing problem statements and creating datasets, ranging from digitizing texts to annotation. Each effort resulted in unique annotation schemas, based on their varied methodologies and approaches. Thus, both data creation and implementation needed to progress simultaneously. On the other hand, widely spoken languages had access to sufficient data, which they leveraged to build numerous models. While the methodologies used in NLP today are also applied in SCL, comparing the results of NLP in general to this field would be non trivial at present, given the differing focuses of the two fields. We hope this will be the subject of a more detailed study in the future.

We now summarize our observations on the various tasks discussed in this article:

In Sanskrit, word segmentation, morphological analysis, and compound analysis all go hand-in-hand. Addressing these as joint tasks would be helpful as opposed to resolving them as stand-alone tasks. Additionally, the true performance of these models can only be evaluated in real-world applications, such as Machine Translation, where all these tasks come together. Therefore, there is

³⁷<https://sanskritpanini.github.io/>

³⁸<https://tdil.meity.gov.in/AboutUs.aspx>

³⁹https://universaldependencies.org/treebanks/sa_vedic/index.html

⁴⁰<https://corpora.ciil.org/ldcil.htm>

⁴¹<https://ai4bharat.iitm.ac.in/>

⁴²<https://iscls.github.io/>

⁴³<https://www.nepalworldsanskrit.org/>

a need for more robust, integrated systems that can effectively leverage these sub-tasks, ensuring better performance across varied linguistic challenges.

The focus of SCL has predominantly centered around morphophonemics, morphosyntactics, and syntax. However, there remains substantial uncharted territory in semantics and discourse analysis. There have been primitive efforts in Machine Translation for Sanskrit, with most of them focusing on the Sanskrit-English pair. Although platforms like Google Translate offer multilingual translation, they are sometimes completely erroneous. This could be attributed to the limited availability of training resources, which is the bottleneck of good MT systems such as Google Translate. Therefore, it is crucial to develop large parallel corpora for Sanskrit and other Indian languages, such that the models can be trained on quality data, thereby improving the model's performance.

Limited attention has been devoted to exploring Keyword Extraction, Anaphora Resolution, Information Retrieval and, Question-Answering systems in Sanskrit. Furthermore, other critical pre-processing tasks, essential for major NLP applications, like Word Sense Disambiguation and Semantic Role Labeling, remain largely unexplored for Sanskrit. Apart from the work by Hellwig [2017] dealing with semantic disambiguation of rare nouns, there have been no concrete efforts in handling polysemy or homonymy so far.

Sanskrit benefited early from a rich tradition of literary criticism that developed notions of sentiment analysis, implicature, suggestion, figures of speech and sound and so forth. Among these, the studies so far only deal with Chandas (prosody) Analysis and Alaṅkāra (figures of speech and sound) Analysis. And even for these tasks, approaches rely on rule-based methods. An intriguing avenue for exploration is the potential use of Machine Learning in these areas. Additionally, models could be developed to identify style, as well as regional, temporal, or sectarian variations in writing, which would be valuable for dating and attributing works. This would be especially beneficial for Sanskrit literature, which faces challenges in these areas. Moreover, addressing specific figures of speech such as “śleṣā” and advanced poetic devices like citrakāvya and vilomakāvya that are both unique to Sanskrit, could provide promising directions for future research.

Tasks such as Named Entity Recognition, Sentiment Analysis, Humor Analysis, Summarization, Paraphrasing and, Dialogue Systems are areas that are yet to be explored. Despite some sentiment and semantic analyses applied to translations of Sanskrit texts, as demonstrated by Chandra and Kulkarni [2022] and Shukla et al. [2023], there exists a notable gap in the direct analysis of Sanskrit texts themselves due to the lack of labelled data. This is yet another area for exploration in the future. Furthermore, restoring accent of texts is another potential application which has not been entertained so far. This could be of great help specifically in compound analysis.

In addition to these challenges, the existing pool of digitized texts and datasets in Sanskrit is significantly limited for machine learning purposes. Therefore, a pivotal direction for the future in Sanskrit involves the creation of more diverse datasets tailored for various NLP tasks. The **Indic Knowledge System (IKS)** holds immense conceptual potential for advancing the computational analysis of Sanskrit and other Indian languages. Along with Aṣṭādhyāyī and theories of verbal cognition (Śābdabodha), delving into diverse theories encompassing word and meaning, poetry and aesthetics, and philosophy can significantly strengthen the depth of linguistic analysis applied to Sanskrit.

Our survey, to its best possible extent, has covered most of the computational tasks pertaining to Sanskrit and has given a brief overview of SCL along with the supporting digital framework. However, one significant challenge encountered during the study was the difficulty in quantifying and comparing the results of various models across different sections. This was primarily due to the fact that each study utilized different datasets for training and evaluation, as well as varied metrics to assess model performance. The lack of uniformity in datasets (annotation, size, quality) and evaluation metrics prevented us from making direct comparisons. Additionally, the diversity

of models, ranging from rule-based systems to pre-trained models, further complicated any meaningful comparison of performance. Drawing conclusions solely based on the reported performance seemed unjustified, and we therefore refrained from doing so. This issue presents an opportunity for future work, where a more systematic comparison could be conducted in a subsequent survey article, with standardized datasets and evaluation metrics to facilitate a clearer comparison across models.

Acknowledgement

I would like to thank Prof. Amba Kulkarni for the resources and valuable inputs. I extend my thanks to Mr. Sriram Krishnan for his continuous guidance and support.

References

- Rahul Aralikatte, Miryam de Lhoneux, Anoop Kunchukuttan, and Anders Søgaard. 2021. Itihasa: A large-scale corpus for Sanskrit to English translation. In *Proceedings of the 8th Workshop on Asian Translation*. Association for Computational Linguistics, 191–197.
- Rahul Aralikatte, Neelamadhav Gantayat, Naveen Panwar, Anush Sankaran, and Senthil Kumar Kumarasamy Mani. 2018. Sanskrit Sandhi splitting using seq2 (seq)². In *Proceedings of the Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 4909–4914.
- S. R. Arjuna and Amba Kulkarni. 2016. Analysis and graphical representation of navya-nyāya expressions. In *Sanskrit and Computational Linguistics - 16th World Sanskrit Conference*, Bangkok, Thailand. 21–52.
- Promila Bahadur, A. K. Jain, and D. S. Chauhan. 2012. Etrans-a complete framework for english to sanskrit machine translation. In *Proceedings of the International Journal of Advanced Computer Science and Applications from International Conference and workshop on Emerging Trends in Technology*. The Science and Information Organization.
- Amruta Barbadikar and Amba Kulkarni. 2024. Yamaka identifier and classifier: A computational tool for the analysis of Sanskrit figure of sound. *Annals of Bhandarkar Oriental Research Institute* 102 (2024), 29–45.
- Amruta Vilas Barbadikar and Amba Kulkarni. 2024. Anuprāsa identifier and classifier: A computational tool to analyze Sanskrit figure of sound. In *Proceedings of the 7th International Sanskrit Computational Linguistics Symposium*. Association for Computational Linguistics.
- Kenneth R. Beesley. 2004. Morphological analysis and generation: A first step in natural language processing. In *1st Steps in Language Documentation for Minority Languages: Computational Linguistic Tools for Morphology, Lexicon and Corpus Compilation, Proceedings of the SALT MIL Workshop at LREC*. 1–8.
- Akshar Bharati, Vineet Chaitanya, Rajeev Sangal, and Brendan Gillon. 2002. *Natural Language Processing: A Paninian Perspective*. PHI Learning Pvt. Ltd.
- Akshar Bharati and Amba Kulkarni. 2007. Sanskrit and computational linguistics. In *Proceedings of the 1st International Sanskrit Computational Symposium*. Department of Sanskrit Studies, University of Hyderabad.
- Akshar Bharati, Amba P. Kulkarni, and V. Sheeba. 2006. Building a wide coverage Sanskrit Morphological Analyser: A practical approach. In *Proceedings of the 1st National Symposium on Modelling and Shallow Parsing of Indian Languages, IIT-Bombay*.
- Akshar Bharati and Rajeev Sangal. 1993. Parsing free word order languages in the Paninian framework. In *Proceedings of the 31st Annual Meeting of the Association for Computational Linguistics*. 105–111.
- Shubham Bhardwaj, Neelamadhav Gantayat, Nikhil Chaturvedi, Rahul Garg, and Sumeet Agarwal. 2018. Sandhikosh: A benchmark corpus for evaluating sanskrit sandhi tools. In *Proceedings of the 11th International Conference on Language Resources and Evaluation*. European Language Resources Association (ELRA).
- Rohitash Chandra and Venkatesh Kulkarni. 2022. Semantic and sentiment analysis of selected Bhagavad Gita translations using BERT-based language framework. *IEEE Access* 10 (2022), 21291–21315.
- Subhash Chandra. 2012. Restructuring of paninian morphological rules for computer processing of sanskrit nominal inflections. In *Proceedings of the Workshop on Indian Language and Data: Resources and Evaluation Workshop Programme*. 39.
- Rahul Chingamtotattil and Rajamma Gopikakumari. 2022. Neural machine translation for Sanskrit to Malayalam using morphology and evolutionary word sense disambiguation. *Indonesian Journal of Electrical Engineering and Computer Science* 28, 3 (2022), 1709–1719.
- Sushant Dave, Arun Kumar Singh, Dr Prathosh AP, and Prof Brejesh Lall. 2021. Neural compound-word (Sandhi) generation and splitting in Sanskrit language. In *Proceedings of the 3rd ACM India Joint International Conference on Data Science and Management of Data (8th ACM IKDD CODS and 26th COMAD'21)*. Association for Computing Machinery, 171–177.

- Leander Gierbach and Jingwen Li. 2022. Word segmentation and morphological parsing for Sanskrit. *CoRR*. Retrieved from <https://arxiv.org/abs/2201.12833>
- Madhav Gopal and Girish Nath Jha. 2014. Resolving anaphors in Sanskrit. In *Human Language Technology Challenges for Computer Science and Linguistics: 5th Language and Technology Conference, LTC 2011, Poznań, Poland, November 25–27, 2011, Revised Selected Papers 5*. Springer, 83–92.
- Pawan Goyal, Vipul Arora, and Laxmidhar Behera. 2007. Analysis of Sanskrit text: Parsing and semantic relations. In *Proceedings of the International Sanskrit Computational Linguistics Symposium*. Springer, 200–218.
- Pawan Goyal and Gérard Huet. 2013. Completeness analysis of a Sanskrit reader. In *Proceedings of the 5th International Symposium on Sanskrit Computational Linguistics*. DK Printworld (P) Ltd. 130–171.
- Pawan Goyal, Gérard Huet, Amba Kulkarni, Peter Scharf, and Ralph Bunker. 2012. A distributed platform for Sanskrit processing. In *Proceedings of the COLING 2012*. Association for Computational Linguistics, 1011–1028.
- Pawan Goyal, Amba Kulkarni, and Laxmidhar Behera. 2009. Computer simulation of aṣṭādhyāyī: Some insights. In *Sanskrit Computational Linguistics: 1st and 2nd International Symposia Rocquencourt, France, October 29–31, 2007 Providence, RI, USA, May 15–17, 2008 Revised Selected and Invited Papers*. Springer, 139–161.
- Pawan Goyal and R. Mahesh K. Sinha. 2007. A study towards design of an English to Sanskrit machine translation system. In *Proceedings of the International Sanskrit Computational Linguistics Symposium*. Springer, 287–305.
- Ashim Gupta, Amrith Krishna, Pawan Goyal, and Oliver Hellwig. 2020. Evaluating neural morphological taggers for sanskrit. In *Proceedings of the 17th SIGMORPHON Workshop on Computational Research in Phonetics, Phonology, and Morphology*. Association for Computational Linguistics.
- Oliver Hellwig. 2007. Sanskrittagger: A stochastic lexical and pos tagger for sanskrit. In *Proceedings of the International Sanskrit Computational Linguistics Symposium*. Springer, 266–277.
- Oliver Hellwig. 2015. Using recurrent neural networks for joint compound splitting and Sandhi resolution in Sanskrit. In *Proceedings of the 4th Biennial Workshop on Less-resourced Languages*.
- Oliver Hellwig. 2017. Coarse semantic classification of rare nouns using cross-lingual data and recurrent neural networks. In *Proceedings of the 12th International Conference on Computational Semantics–Long Papers*.
- Oliver Hellwig and Sebastian Nehrlich. 2018. Sanskrit word segmentation using character-level recurrent and convolutional neural networks. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 2754–2763.
- Oliver Hellwig, Sebastian Nehrlich, and Sven Sellmer. 2023. Data-driven dependency parsing of Vedic Sanskrit. *Language Resources and Evaluation* 57, 3 (2023), 1173–1206.
- Oliver Hellwig, Salvatore Scarlata, Elia Ackermann, and Paul Widmer. 2020. The treebank of vedic Sanskrit. In *Proceedings of the 12th Language Resources and Evaluation Conference*. 5137–5146.
- Md Enamul Hoque. 2021. The universal grammar theory: Noam Chomsky’s contribution to second language (SL) education. *The Journal of EFL Education and Research* 6, 2 (2021), 57–62.
- Gérard Huet. 2003a. Lexicon-directed segmentation and tagging of Sanskrit. In *Proceedings of the XIIIth World Sanskrit Conference, Helsinki, Finland, Aug.* 307–325.
- Gérard Huet. 2003b. Towards computational processing of Sanskrit. In *Proceedings of the International Conference on Natural Language Processing*. 40–48.
- Gérard Huet. 2004. Design of a lexical database for Sanskrit. In *Proceedings of the Workshop on Enhancing and Using Electronic Dictionaries*. Association for Computational Linguistics.
- Gérard Huet. 2005. A functional toolkit for morphological and phonological processing, application to a Sanskrit tagger. *Journal of Functional Programming* 15, 4 (2005), 573–614.
- Gérard Huet. 2006. Shallow syntax analysis in Sanskrit guided by semantic nets constraints. In *Proceedings of the 2006 International Workshop on Research Issues in Digital Libraries*. Association for Computing Machinery, 1–10.
- Gérard Huet. 2009. Sanskrit Segmentation. In *Proceedings of the South Asian Languages Analysis Roundtable XXVIII* (Denton, Ohio).
- Gérard Huet. 2024. Hoisting the colors of Sanskrit. In *Proceedings of the 7th International Sanskrit Computational Linguistics Symposium*. Association for Computational Linguistics.
- Girish N. Jha. 2004. Generating nominal inflectional morphology in Sanskrit. *SIMPLE* 4 (2004), 20–23.
- Girish Nath Jha, Muktanand Agrawal, Subash, Sudhir K. Mishra, Diwakar Mani, Diwakar Mishra, Manji Bhadra, and Surjit K. Singh. 2009. Inflectional morphology analyzer for Sanskrit. In *Sanskrit Computational Linguistics: 1st and 2nd International Symposia Rocquencourt, France, October 29–31, 2007 Providence, RI, USA, May 15–17, 2008 Revised Selected and Invited Papers*. Springer, 219–238.
- Daniel Jurafsky and James H. Martin. 2000. *Speech and Language Processing*. Prentice-Hall.
- Mohammed Safi Ur Rahman Khan, Priyam Mehta, Ananth Sankar, Umashankar Kumaravelan, Sumanth Doddapaneni, Suriyaprasad B, Varun G, Sparsh Jain, Anoop Kunchukuttan, Pratyush Kumar, et al.. 2024. IndicLLMSuite: A blueprint for creating pre-training and fine-tuning datasets for Indian languages. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics.

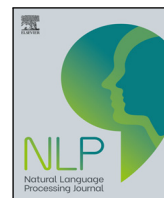
- Nimrita Koul and Sunilkumar S. Manvi. 2021. A proposed model for neural machine translation of Sanskrit into English. *International Journal of Information Technology* 13, 1 (2021), 375–381.
- Amrith Krishna and Pawan Goyal. 2015. Towards automating the generation of derivative nouns in Sanskrit by simulating Panini. In *Graph-Based Methods for Natural Language Processing: Proceedings of the Workshop*.
- Amrith Krishna, Ashim Gupta, Deepak Garasangi, Jivnesh Sandhan, Pavankumar Satuluri, and Pawan Goyal. 2023. Neural approaches for data driven dependency parsing in Sanskrit. In *Proceedings of the Computational Sanskrit and Digital Humanities: Selected Papers Presented at the 18th World Sanskrit Conference*. Association for Computational Linguistics, 1–20.
- Amrith Krishna, Bishal Santra, Sasi Prasanth Bandaru, Gaurav Sahu, Vishnu Dutt Sharma, Pavankumar Satuluri, and Pawan Goyal. 2018. Free as in free word order: An energy based model for word segmentation and morphological tagging in Sanskrit. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics.
- Amrith Krishna, Bishal Santra, Ashim Gupta, Pavankumar Satuluri, and Pawan Goyal. 2020b. A graph-based framework for structured prediction tasks in Sanskrit. *Computational Linguistics* 46, 4 (2020).
- Amrith Krishna, Bishal Santra, Pavankumar Satuluri, Sasi Prasanth Bandaru, Bhumi Faldu, Yajuvendra Singh, and Pawan Goyal. 2016a. Word segmentation in sanskrit using path constrained random walks. In *Proceedings of COLING 2016, the 26th International Conference on Computational Linguistics: Technical Papers*. The COLING 2016 Organizing Committee, 494–504.
- Amrith Krishna, Pavankumar Satuluri, and Pawan Goyal. 2017. A dataset for sanskrit word segmentation. In *Proceedings of the Joint SIGHUM Workshop on Computational Linguistics for Cultural Heritage, Social Sciences, Humanities and Literature*. 105–114.
- Amrith Krishna, Pavankumar Satuluri, Shubham Sharma, Apurv Kumar, and Pawan Goyal. 2016b. Compound type identification in sanskrit: What roles do the corpus and grammar play?. In *Proceedings of the 6th Workshop on South and Southeast Asian Natural Language Processing*. The COLING 2016 Organizing Committee, 1–10.
- Amrith Krishna, Vishnu Sharma, Bishal Santra, Aishik Chakraborty, Pavankumar Satuluri, and Pawan Goyal. 2019. Poetry to prose conversion in Sanskrit as a linearisation task: A case for low-resource languages. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics.
- Amrith Krishna, Shiv Vidhyut, Dilpreet Chawla, Sruti Sambhavi, and Pawan Goyal. 2020c. SHR++: An interface for morpho-syntactic annotation of Sanskrit corpora. In *Proceedings of the 12th Language Resources and Evaluation Conference*. 7069–7076.
- Sriram Krishnan and Amba Kulkarni. 2022. Sanskrit segmentation revisited. In *Findings of the Association for Computational Linguistics: EMNLP*. Association for Computational Linguistics.
- Sriram Krishnan, Amba Kulkarni, and Gérard Huet. 2020. Validation and normalization of DCS corpus using Sanskrit Heritage tools to build a tagged Gold Corpus. In *Proceedings of the Computational Sanskrit and Digital Humanities: Selected Papers Presented at the 18th World Sanskrit Conference*.
- Sriram Krishnan, Amba Kulkarni, and Gérard Huet. 2024. Normalized dataset for Sanskrit word segmentation and morphological parsing. *Language Resources and Evaluation* (2024), 1–52.
- Amba Kulkarni. 2013. A deterministic dependency parser with dynamic programming for Sanskrit. In *Proceedings of the Second International Conference on Dependency Linguistics*. Charles University in Prague, 157–166.
- Amba Kulkarni. 2016. *Application of Modern Technology to South Asian Languages*. Walter de Gruyter GmbH & Co KG.
- Amba Kulkarni, Sheetal Pokar, and Devanand Shukl. 2010b. Designing a constraint based parser for Sanskrit. In *Sanskrit Computational Linguistics: 4th International Symposium, New Delhi, India, December 10–12, 2010. Proceedings*. Springer, 70–90.
- Amba Kulkarni, Sanal Vikram, and K. Sriram. 2019. Dependency parser for Sanskrit verses. In *Proceedings of the 6th International Sanskrit Computational Linguistics Symposium*. 14–27.
- Malhar Kulkarni, Chaitali Dangarikar, Irawati Kulkarni, Abhishek Nanda, and Pushpak Bhattacharyya. 2010a. Introducing sanskrit wordnet. In *Proceedings of the 5th Global Wordnet Conference, Narosa, Mumbai*. 287–294.
- Anil Kumar. 2012. *An Automatic Sanskrit Compound Processing*. Ph.D. Dissertation. University of Hyderabad.
- Anil Kumar and Amba Kulkarni. 2010. Sanskrit compound processor. In *Sanskrit Computational Linguistics: 4th International Symposium, New Delhi, India, December 10–12, 2010. Proceedings*. Springer, 57–69.
- Anil Kumar, Amba Kulkarni, and Nakka Shailaj. 2024. START: Sanskrit teaching; Annotation; and research tool – bridging tradition and technology in scholarly exploration. In *Proceedings of the 7th International Sanskrit Computational Linguistics Symposium*. Association for Computational Linguistics.
- Anil Kumar, V. Sheebasudheer, and Amba Kulkarni. 2009. Sanskrit compound paraphrase generator. *Proceedings of 7th International Conference on Natural Language Processing (ICON'09)*.
- Rashi Kumar, Piyush Jha, and Vineet Sahula. 2019. An augmented translation technique for low resource language pair: Sanskrit to hindi translation. In *Proceedings of the 2019 2nd International Conference on Algorithms, Computing and Artificial Intelligence*. Association for Computing Machinery, 377–383.

- Sachin Kumar. 2007. Sandhi splitter and analyzer for Sanskrit (with reference to ac Sandhi). *Mphil Degree at SCSS, JNU (Submitted, 2007)* (2007).
- Deepika Kumawat and Vinesh Jain. 2015. POS tagging approaches: A comparison. *International Journal of Computer Applications* 118, 6 (2015), 32–38.
- D. Mane, P. Devale, and S. Suryawans. 2010. A design towards English to Sanskrit machine translation and synthesizer system using rule base approach. *International Journal of Multidisciplinary Research and Advancement in Engineering* 2, 2 (2010), 405–414.
- Keshav S. Melnad, Pawan Goyal, and Peter Scharf. 2013. Meter identification of Sanskrit verse. *Seminar on Sanskrit Syntax and Discourse Structures*. The Sanskrit Library, USA (2013).
- Anand Mishra. 2007. Simulating the pāṇinian system of sanskrit grammar. In *Proceedings of the International Sanskrit Computational Linguistics Symposium*. Springer, 127–138.
- Anand Mishra. 2009. Modelling the grammatical circle of the pāṇinian system of Sanskrit grammar. In *Sanskrit Computational Linguistics: Third International Symposium, Hyderabad, India, January 15–17, 2009. Proceedings*. Springer, 40–55.
- Anand Mishra. 2010. Modelling Aṣṭādhyāyī: An approach based on the methodology of ancillary disciplines (Vedāṅga). In *Proceedings of the International Sanskrit Computational Linguistics Symposium*. Springer, 239–258.
- Vimal Mishra and R. B. Mishra. 2010. ANN and rule based model for English to Sanskrit machine translation. *INFOCOMP Journal of Computer Science* 9, 1 (2010), 80–89.
- Ruslan Mitkov. 1999. *Anaphora Resolution: The State of the Art*. School of Languages and European Studies, University of Wolverhampton
- Vipul Mittal. 2010. Automatic Sanskrit segmentizer using finite state transducers. In *Proceedings of the ACL 2010 Student Research Workshop*. Association for Computational Linguistics, 85–90.
- Diego Mollá and José Luis Vicedo. 2006. Special section on restricted-domain question answering. *Computational Linguistics* 33, 1 (2006), 41–61.
- Abhiram Natarajan and Eugene Charniak. 2011. S3-Statistical Sandhi splitting-statistical Sandhi splitting. In *Proceedings of the 5th International Joint Conference on Natural Language Processing*. Asian Federation of Natural Language Processing, 301–308.
- Sebastian Nehrlich, Oliver Hellwig, and Kurt Keutzer. 2024. One model is all you need: ByT5-Sanskrit, a unified model for Sanskrit NLP tasks. In *Findings of the Association for Computational Linguistics: EMNLP 2024*. Association for Computational Linguistics.
- Rajneesh Kumar Pandey and Girish Nath Jha. 2016. Error analysis of sahit-a statistical sanskrit-hindi translator. *Procedia Computer Science* 96, C (2016), 495–501.
- Dhaval Patel and Shivakumari Katuri. 2015. Prakriyāpradarśinī-an open source subanta generator. In *Sanskrit and Computational Linguistics-16th World Sanskrit Conference, Bangkok, Thailand*.
- Ganesh R. Pathak and Sachin P. Godse. 2010. English to Sanskrit machine translation using transfer based approach. In *AIP Conference Proceedings*. American Institute of Physics, 122–126.
- Anagha Pradeep, Radhika Mamidi, and Pavankumar Satuluri. 2024. Context and WSD: Analysing Google Translate’s Sanskrit to English output of bhagavadgita verses for word meaning. In *Proceedings of the 7th International Sanskrit Computational Linguistics Symposium*. Association for Computational Linguistics.
- Pravin Pralayankar and Sobha Lalitha Devi. 2010. Anaphora resolution algorithm for Sanskrit. In *Sanskrit Computational Linguistics: 4th International Symposium, New Delhi, India, December 10–12, 2010. Proceedings*. Springer, 209–217.
- Arun K. Prasad. 2024. A fast prakriyā generator. In *Proceedings of the 7th International Sanskrit Computational Linguistics Symposium*. Association for Computational Linguistics.
- Abhinandan S. Prasad and Shrisha Rao. 2010. Citation matching in Sanskrit corpora using local alignment. In *Sanskrit Computational Linguistics: 4th International Symposium, New Delhi, India, December 10–12, 2010. Proceedings*. Springer, 124–136.
- S. Prasanna. 2022. Spellchecker for Sanskrit: The road less taken. In *Proceedings of the 19th International Conference on Natural Language Processing*. 290–299.
- B. Premjith, Chandni Chandran, Shriganesh Bhat, Soman Kp, and P. Prabakaran. 2019. A machine learning approach for identifying compound words from a Sanskrit text. In *Proceedings of the 6th International Sanskrit Computational Linguistics Symposium*. Association for Computational Linguistics, 45–51.
- B. Premjith, K. P. Soman, and Prabakaran Poornachandran. 2018. A deep learning based Part-of-Speech (POS) tagger for Sanskrit language by embedding character level features. In *Proceedings of the FIRE*. 56–60.
- Ravneet Punia, Aditya Sharma, Sarthak Pruthi, and Minni Jain. 2020. Improving neural machine translation for Sanskrit-English. In *Proceedings of the 17th International Conference on Natural Language Processing*. NLP Association of India (NLPAl), 234–238.
- V. Raghavan. 1942. *Studies on Some Concepts of the Alankāra Śāstra*. The Adyar Library.

- Shreevatsa Rajagopalan. 2020. A user-friendly tool for metrical analysis of Sanskrit verse. *Computational Sanskrit and Digital Humanities* (2020), 113.
- N. Rama and Meenakshi Lakshmanan. 2010. A computational algorithm for metrical classification of verse. *International Journal of Computer Science Issues* 7 (2010), 46–53.
- Jaideepsinh K. Raulji, Jatinderkumar R. Saini, Kaushika Pal, and Ketan Kotecha. 2022. A novel framework for Sanskrit-Gujarati symbolic machine translation system. *International Journal of Advanced Computer Science and Applications* 13, 4 (2022).
- Vikas Reddy, Amrith Krishna, Vishnu Dutt Sharma, Prateek Gupta, Pawan Goyal, et al. 2018. Building a word segmenter for Sanskrit overnight. In *Proceedings of the 11th International Conference on Language Resources and Evaluation*. European Language Resources Association (ELRA'18), 1724–1734.
- Jivnesh Sandhan, Ashish Gupta, Hrishikesh Terdalkar, Tushar Sandhan, Suvendu Samanta, Laxmidhar Behera, and Pawan Goyal. 2022a. A novel multi-task learning approach for context-sensitive compound type identification in Sanskrit. In *Proceedings of the 29th International Conference on Computational Linguistics*. International Committee on Computational Linguistics.
- Jivnesh Sandhan, Amrith Krishna, Pawan Goyal, and Laxmidhar Behera. 2019. Revisiting the role of feature engineering for compound type identification in Sanskrit. In *Proceedings of the 6th International Sanskrit Computational Linguistics Symposium*. Association for Computational Linguistics, 28–44.
- Jivnesh Sandhan, Amrith Krishna, Ashim Gupta, Laxmidhar Behera, and Pawan Goyal. 2021. A little pretraining goes a long way: A case study on dependency parsing task for low-resource morphologically rich languages. In *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics: Student Research Workshop*. Association for Computational Linguistics.
- Jivnesh Sandhan, Yaswanth Narsupalli, Sreevatsa Muppirala, Sriram Krishnan, Pavankumar Satuluri, Amba Kulkarni, and Pawan Goyal. 2023. DepNeCTI: Dependency-based nested compound type identification for Sanskrit. In *Findings of the Association for Computational Linguistics*. Association for Computational Linguistics.
- Jivnesh Sandhan, Rathin Singha, Narein Rao, Suvendu Samanta, Laxmidhar Behera, and Pawan Goyal. 2022b. TransLIST: A transformer-based linguistically informed Sanskrit tokenizer. In *Findings of the Association for Computational Linguistics*. Association for Computational Linguistics.
- Pavankumar Satuluri and Amba Kulkarni. 2013. Generation of Sanskrit compounds. *Proceedings of the ICON* (2013), 77–86.
- Peter Scharf. 2015. An XML formalization of the Aṣṭādhyāyī. In *Proceedings of the 16th World Sanskrit Conference*.
- Nandini Sethi, Amita Dev, and Poonam Bansal. 2023. A novel neural machine translation approach for low-resource Sanskrit-Hindi language pair. *ACM Transactions on Asian and Low-Resource Language Information Processing* (2023).
- Akshat Shukla, Chaarvi Bansal, Sushrut Badhe, Mukul Ranjan, and Rohitash Chandra. 2023. An evaluation of Google translate for Sanskrit to English translation via sentiment and semantic analysis. *Natural Language Processing Journal* 4 (2023).
- Muskaan Singh, Ravinder Kumar, and Inderveer Chana. 2019. Improving neural machine translation using rule-based machine translation. In *Proceedings of the 2019 7th International Conference on Smart Computing and Communications*. 1–5.
- Muskaan Singh, Ravinder Kumar, and Inderveer Chana. 2020. Corpus based machine translation system with deep neural network for Sanskrit to Hindi translation. *Procedia Computer Science* 167 (2020), 2534–2544.
- Sitender and Seema Bawa. 2021. A Sanskrit-to-English machine translation using hybridization of direct and rule-based approach. *Neural Computing and Applications* 33, 7 (2021), 2819–2838.
- Marco Antonio Calijorne Soares and Fernando Silva Parreiras. 2020. A literature review on question answering techniques, paradigms and systems. *Journal of King Saud University-Computer and Information Sciences* 32, 6 (2020), 635–646.
- H. S. Sreedeepta and Sumam Mary Idicula. 2017. Interlingua based Sanskrit-English machine translation. In *Proceedings of the 2017 International Conference on Circuit, Power and Computing Technologies*. IEEE, 1–5.
- S. Sridhar and V. Srinivasa. 2008. Computational structure of Aṣṭādhyāyī and conflict resolution techniques. *Proceedings of the 3rd International Symposium on Sanskrit Computational Linguistics*. Springer-Verlag, 56–65.
- Prakhar Srivastava, Kushal Chauhan, Deepanshu Aggarwal, Anupam Shukla, Joydip Dhar, and Vrashabh Prasad Jain. 2018. Deep learning based unsupervised POS tagging for Sanskrit. In *Proceedings of the 2018 International Conference on Algorithms, Computing and Artificial Intelligence*. 1–6.
- Sridhar Subbanna and Shrinivasa Varakhedi. 2010. Asiddhatva principle in computational model of aṣṭādhyāyī. In *Proceedings of the International Sanskrit Computational Linguistics Symposium*. Springer, 231–238.
- Namrata Tapaswi and Suresh Jain. 2012. Treebank based deep grammar acquisition and part-of-speech tagging for Sanskrit sentences. In *Proceedings of the 2012 CSI 6th International Conference on Software Engineering*. IEEE, 1–4.
- Namrata Tapaswi, Suresh Jain, and Vaishali Chourey. 2012. Morphological-based spellchecker for Sanskrit sentences. *International Journal of Scientific and Technology Research* 1, 3 (2012), 1–4.

- Hrishikesh Terdalkar and Arnab Bhattacharya. 2022. Chandojnanam: A Sanskrit meter identification and utilization system. In *Proceedings of the Computational Sanskrit and Digital Humanities: Selected Papers Presented at the 18th World Sanskrit Conference*. Association for Computational Linguistics.
- Hrishikesh Terdalkar and Arnab Bhattacharya. 2023b. Antarlekhaka: A comprehensive tool for multi-task natural language annotation. In *Proceedings of the 3rd Workshop for Natural Language Processing Open Source Software*.
- Hrishikesh Terdalkar and Arnab Bhattacharya. 2023a. Framework for question-answering in Sanskrit through automated construction of knowledge graphs. In *Proceedings of the 6th International Sanskrit Computational Linguistics Symposium*. Association for Computational Linguistics.
- Neill Tyler. 2023. Skrutable: Another step toward effective Sanskrit meter identification. In *Proceedings of the Computational Sanskrit and Digital Humanities: Selected Papers Presented at the 18th World Sanskrit Conference*. 97–112.

Received 3 April 2024; revised 1 March 2025; accepted 1 April 2025



Evaluation of open and closed-source LLMs for low-resource language with zero-shot, few-shot, and chain-of-thought prompting

Zabir Al Nazi^a, Md. Rajib Hossain^{b,*}, Faisal Al Mamun^c

^a Department of Computer Science and Engineering, University of California, Riverside, United States

^b Department of Computer Science & Engineering, Chittagong University of Engineering & Technology, Chittagong 4349, Bangladesh

^c Axiler Ltd., Bangladesh



ARTICLE INFO

Keywords:

Large language models
Zero-shot
Few-shot
Chain-of-thought
GPT-4
Llama 3
Ablation studies
Prompting
LLM reasoning
Low-resource
Bangla

ABSTRACT

As the global deployment of Large Language Models (LLMs) increases, the demand for multilingual capabilities becomes more crucial. While many LLMs excel in real-time applications for high-resource languages, few are tailored specifically for low-resource languages. The limited availability of text corpora for low-resource languages, coupled with their minimal utilization during LLM training, hampers the models' ability to perform effectively in real-time applications. Additionally, evaluations of LLMs are significantly less extensive for low-resource languages. This study offers a comprehensive evaluation of both open-source and closed-source multilingual LLMs focused on low-resource language like Bengali, a language that remains notably underrepresented in computational linguistics. Despite the limited number of pre-trained models exclusively on Bengali, we assess the performance of six prominent LLMs, i.e., three closed-source (GPT-3.5, GPT-4o, Gemini) and three open-source (Aya 101, BLOOM, LLaMA) across key natural language processing (NLP) tasks, including text classification, sentiment analysis, summarization, and question answering. These tasks were evaluated using three prompting techniques: Zero-Shot, Few-Shot, and Chain-of-Thought (CoT). This study found that the default hyperparameters of these pre-trained models, such as temperature, maximum token limit, and the number of few-shot examples, did not yield optimal outcomes and led to hallucination issues in many instances. To address these challenges, ablation studies were conducted on key hyperparameters, particularly temperature and the number of shots, to optimize Few-Shot learning and enhance model performance. The focus of this research is on understanding how these LLMs adapt to low-resource downstream tasks, emphasizing their linguistic flexibility and contextual understanding. Experimental results demonstrated that the closed-source GPT-4o model, utilizing Few-Shot learning and Chain-of-Thought prompting, achieved the highest performance across multiple tasks: an F1 score of 84.54% for text classification, 99.00% for sentiment analysis, a $F1_{bert}$ score of 72.87% for summarization, and 58.22% for question answering. For transparency and reproducibility, all methodologies and code from this study are available on our GitHub repository: <https://github.com/zabir-nabil/bangla-multilingual-llm-eval>.

1. Introduction

Large Language Models (LLMs) have made significant advancements in the field of natural language processing (NLP), particularly their practical implications. Prompting is a technique that involves providing a set of instructions, or even a single instruction, to LLMs to help them achieve the user's goals and enhance their performance. When an LLM generates output without any prior examples or context, it is referred to as zero-shot prompting. In contrast, when the model is given some prior examples or context to guide its responses, it is known as few-shot prompting. Additionally, when a task is broken down into smaller, manageable sub-tasks, and the decisions and reasoning from

these sub-tasks are combined to produce the final output, it is referred to as Chain-of-Thought prompting. LLMs are now capable of handling complex language tasks such as reasoning, summarization, translation, question answering, and classification more effectively by understanding the context of words and sentences, allowing them to generate natural and coherent text. They can produce detailed summaries, answer questions, and classify text with notable accuracy. While there are many pre-trained LLM models and prompting techniques available for high-resource languages, there is still a significant shortage for low-resource languages like Bengali. As a result, there is a wide-open opportunity for NLP researchers to explore the use of LLMs in

* Corresponding author.

E-mail addresses: znazi002@ucr.edu (Z.A. Nazi), rajsecuetphd@cuet.ac.bd (M.R. Hossain), faisal@axiler.com (F.A. Mamun).

<https://doi.org/10.1016/j.nlp.2024.100124>

Received 22 October 2024; Received in revised form 29 November 2024; Accepted 21 December 2024

low-resource languages, which serves as the **primary motivation for conducting this research**.

Multilingual LLMs face a fundamental challenge known as the “Curse of Multilinguality”, which refers to the trade-off between supporting multiple languages and maintaining model performance. During the foundation model generation phase, adding more languages to a fixed-capacity model initially boosts performance for low-resource languages through positive transfer from high-resource languages. However, foundation models are still insufficient for low-resource languages. Beyond a certain point, adding more languages reduces performance for each, leading to a decline in overall accuracy. **This limitation has motivated the development of different prompting techniques to evaluate the effectiveness of LLMs in Bengali.** The key to the success of LLMs lies in their ability to learn from large corpora of text during the pre-training phase. By processing vast amounts of text data, they can seemingly understand patterns, grammar, and context within language. LLMs use the transformer architecture, which relies on self-attention mechanism. This self-attention allows the model to focus on different parts of a sentence to determine the relationship between words, even if they are far apart, leading to more coherent and accurate language understanding (Lin et al., 2022).

However, most of the LLMs primarily focus on high-resource languages like English and Chinese, limiting their usability for low-resource languages such as Bengali. In contrast, multilingual LLMs are trained on extensive multilingual corpora, enabling them to effectively understand and manage multiple languages (Qin et al., 2024; Xu et al., 2024). This broad language coverage allows multilingual LLMs to handle diverse linguistic tasks more comprehensively. However, support for low-resource languages like Bengali remains limited. *Bengali is the seventh most spoken language globally* (Hossain et al., 2025) and the primary language of two regions: the Indian states of West Bengal and Tripura, as well as Bangladesh. Recently, numerous studies have been conducted using CNN, LSTM, and transformer-based language models for Bengali downstream tasks. However, limited research has been carried out on Bengali using LLMs. As a result, the performance of LLMs on various Bengali downstream tasks remains a critical area of study. To address this gap, this research employs ablation studies on key hyperparameter adaptations and evaluates the performance of three prompting techniques across six LLMs in four Bengali downstream tasks.

Given the motivations and implications of LLMs for low-resource language like Bengali, this research aims to address the following research questions (RQs):

- **RQ1:** How do prompting techniques (Zero-Shot, Few-Shot, and Chain-of-Thought) affect LLMs performance on low-resource downstream tasks like classification, summarization, and question answering?
- **RQ2:** What are the strengths and weaknesses of open-source vs. closed-source LLMs in handling low-resource languages?
- **RQ3:** How do variations in shot selection and temperature settings impact the performance and reliability of LLMs, particularly in low-resource languages?

To address the research questions **RQ1–RQ3**, this study applied three prompting techniques (Zero-Shot, Few-Shot and Chain-of-Thought) across six LLMs (GPT-3.5, GPT-4, Gemini, Aya 101, BLOOM 7b1, and Llama3 8b) for Bengali four downstream tasks: (i) News classification, (ii) Sentiment analysis (Positive and Negative), (iii) Abstractive summarization, and (iv) Question answering. *To the best of our knowledge, this is the first study to use LLMs for evaluating four distinct Bengali downstream task performances with three different prompting techniques.*

The answers to the research questions (ARQs) and the key contributions of this research are outlined below:

Table 1

Performance summary of news classification across six models and three prompting techniques. The maximum token length is set to 512 for GPT-3.5 and GPT-4o, while it is limited to 128 for the other models. Additionally, the top-p value is configured to 0.9. **Bold values** indicate the highest performance metrics and the lowest Log loss values.

Model	Prompt	Accuracy	Precision	Recall	F1	AUC	Log loss
GPT-3.5	0-shot	0.55	0.46	0.45	0.44	0.75	11.80
	3-shot	0.59	0.56	0.48	0.48	0.77	11.71
	CoT	0.14	0.36	0.11	0.15	0.56	7.58
GPT-4o	0-shot	0.77	0.80	0.77	0.74	0.87	8.30
	3-shot	0.85	0.86	0.85	0.85	0.92	5.39
	CoT	0.74	0.82	0.74	0.73	0.85	9.32
Gemini	0-shot	0.68	0.71	0.62	0.62	0.82	10.95
	3-shot	0.70	0.64	0.57	0.58	0.83	10.51
	CoT	0.57	0.61	0.47	0.49	0.76	14.06
Aya 101	0-shot	0.58	0.71	0.48	0.53	0.78	4.06
	3-shot	0.73	0.72	0.60	0.62	0.86	4.58
	CoT	0.64	0.69	0.52	0.55	0.81	5.89
BLOOM	0-shot	0.11	0.01	0.11	0.02	0.50	30.70
	3-shot	0.11	0.01	0.11	0.02	0.50	30.70
	CoT	0.11	0.01	0.11	0.02	0.50	30.70
LLaMA	0-shot	0.46	0.54	0.41	0.42	0.70	19.44
	3-shot	0.47	0.57	0.42	0.44	0.70	19.22
	CoT	0.38	0.43	0.31	0.32	0.66	22.35

- **ARQ1:** We developed a procedure to evaluate three prompting techniques across six LLMs for four low-resource NLP downstream tasks: news classification, sentiment analysis, summarization, and question answering. The effectiveness of these prompting strategies is detailed in Section 5.2 and summarized in Table 1, Table 2, Table 3, and Table 3.
- **ARQ2:** We evaluated the performance of open-source and closed-source LLMs and summarized their outcomes. The performance summary and error analysis highlight the strengths and weaknesses of each model. Further details are provided in Section 5.2.5.
- **ARQ3:** Employing ablation studies reveals that shot selection and temperature settings significantly impact the performance of LLMs in Bengali NLP tasks. Optimal performance was achieved with three-shot prompting, where the model benefits from contextual examples, while adding more few-shot examples introduced noise, degrading accuracy. Similarly, lower temperature values (0.0–0.2) produced focused and consistent outputs, whereas higher temperatures led to randomness and hallucinated responses resulting in reduced accuracy, with a steep decline from 94.87% at 0.1 to 10.90% at 2.0. Details in Section 5.2.6, Table 5, and Table 6.

The structure of the article is as follows: Section 2 presents the related work, while Section 3 describes the four datasets along with their attributes. Section 4 outlines the methodology, and Section 5 covers the experimental setup, evaluation metrics, and result analysis. Section 6 provides insights and key findings, and finally, Section 7 summarizes the research and discusses future research directions.

2. Related work

The evaluation of LLMs with various prompting techniques is an emerging and significant research area for both low-resource and high-resource languages. In this study, we assessed the performance of LLMs on Bengali tasks, including text classification (news and sentiment), summarization, and question answering, using three prompting techniques: Zero-Shot, Few-Shot, and Chain-of-Thought in six LLMs. The following subsection presents a summary of research related to these prompting techniques for both of low and high resource languages.

Table 2

Sentiment analysis performance summary across 6 models and 3 prompting techniques. The maximum token length is set to 512 for GPT-3.5 and GPT-4o, while it is limited to 128 for the other models. Additionally, the top-p value is configured to 0.9. **Bold values** indicate the highest performance metrics and the lowest Log loss values.

Model	Prompt	Accuracy	Precision	Recall	F1	AUC	Log loss
GPT-3.5	0-shot	0.37	0.35	0.36	0.35	0.37	22.87
	3-shot	0.93	0.97	0.88	0.92	0.93	2.54
	CoT	0.02	0.01	0.01	0.01	0.02	35.35
GPT-4o	0-shot	0.96	0.99	0.92	0.95	0.95	1.62
	3-shot	0.97	0.99	0.96	0.97	0.97	0.92
	CoT	0.99	0.99	0.99	0.99	0.99	0.46
Gemini	0-shot	0.91	0.97	0.84	0.90	0.91	3.23
	3-shot	0.87	0.98	0.75	0.85	0.87	4.62
	CoT	0.88	0.87	0.88	0.87	0.88	4.39
Aya 101	0-shot	0.37	0.35	0.36	0.35	0.37	22.87
	3-shot	0.93	0.97	0.88	0.92	0.93	2.54
	CoT	0.02	0.01	0.01	0.01	0.02	35.35
BLOOM	0-shot	0.48	0.48	1.00	0.65	0.50	18.72
	3-shot	0.48	0.48	1.00	0.65	0.50	18.72
	CoT	0.48	0.48	1.00	0.65	0.50	18.72
LLaMA	0-shot	0.77	0.75	0.77	0.76	0.77	8.32
	3-shot	0.94	0.92	0.97	0.94	0.94	1.84
	CoT	0.71	0.63	0.96	0.76	0.72	10.40

2.1. Zero-Shot prompting

Recently, Zero-Shot prompting in LLMs has gained significant attention from researchers due to its effectiveness in scenarios where task-specific models or knowledge are unavailable. The reasoning capabilities of Zero-Shot LLM models are evaluated in López Espejel et al. (2023), where the Zero-Shot performance of GPT-3.5, GPT-4, and BARD models is empirically assessed for the English language. Similarly, D'Asaro et al. (2024) developed a multimodal recommendation system for English using Zero-Shot prompting, combining image embeddings from CLIP and text embeddings from ALBEF.

A low-resource Turkish text classification system was developed in Çelik and Dalyan (2023) using Zero-Shot prompting on ConvBERT, achieving a maximum accuracy of 79%. Barrena et al. (2021) introduced a cross-lingual named entity disambiguation system with XLM-R, targeting low-resource Spanish alongside high-resource English and Chinese. Kabir et al. (2024) empirically evaluated the performance of Zero-Shot LLMs on Bengali tasks, including summarization, question answering, and sentiment analysis. While Zero-Shot LLMs show reasonable accuracy for many high-resource languages, their performance on low-resource tasks remains suboptimal.

2.2. Few-Shot prompting

The remarkable success of generative pre-trained transformers (GPT) has brought Few-Shot prompting in LLMs into the spotlight, particularly for low-resource NLP tasks. Cheng et al. (2024) developed a Few-Shot LLM-based NER system that achieved an F1-score of 88.7% on the CoNLL dataset and 78.3% on the FewNERD dataset using the GPT-4 model. Similarly, Song et al. (2023) introduced a Few-Shot event classification system for two English datasets, i.e., FewEvent and RAMS. Mahfuz et al. (2024) evaluated the performance of open-weight and closed-source LLMs, such as LLaMA-3 and GPT-4, alongside fine-tuned encoder-decoder models on various Bengali NLP tasks, including translation, summarization, and question answering. The results indicate that while LLMs perform well in reasoning tasks, their effectiveness in Bengali script generation is inconsistent due to challenges such as inefficient tokenization, which leads to higher computational costs and degraded performance.

A Few-Shot LLM-based system was developed for English text summarization, achieving a maximum R1 score of 0.3729, R2 score of 0.0992, and RL score of 0.2276 on the Yelp test set (Bražinskas

Table 3

Statistical summary of 6 LLMs models with 3 prompting for Bengali article summarization. The maximum token length is set to 512 for GPT-3.5 and GPT-4o, while it is limited to 128 for the other models. Additionally, the top-p value is configured to 0.9. **Bold values** indicate the highest performance metrics and the lowest Log loss values.

Model	Prompt	$F_{rouge-l}$	$P_{rouge-l}$	$R_{rouge-l}$	P_{bert}	R_{bert}	$F1_{bert}$	BLEU
GPT-3.5	0-shot	0.1109	0.1117	0.1200	0.7071	0.7089	0.7076	0.2800
	3-shot	0.1140	0.1049	0.1500	0.6922	0.7133	0.7019	0.2570
	CoT	0.1050	0.0836	0.1589	0.6261	0.6775	0.6504	0.1718
GPT-4o	0-shot	0.1357	0.1161	0.1766	0.7025	0.7372	0.7190	0.3039
	3-shot	0.1483	0.1459	0.1603	0.7234	0.7347	0.7287	0.3387
	CoT	0.1405	0.1282	0.1662	0.7026	0.7295	0.7155	0.3121
Gemini	0-shot	0.1098	0.0973	0.1346	0.7014	0.7233	0.7119	0.3141
	3-shot	0.1112	0.1018	0.1316	0.7033	0.7218	0.7121	0.3196
	CoT	0.1121	0.0976	0.1410	0.6990	0.7242	0.7110	0.3148
Aya 101	0-shot	0.1109	0.1117	0.1200	0.7071	0.7089	0.7076	0.2800
	3-shot	0.1140	0.1049	0.1500	0.6922	0.7133	0.7019	0.2570
	CoT	0.1050	0.0836	0.1589	0.6261	0.6775	0.6504	0.1718
BLOOM	0-shot	0.0534	0.0288	0.4376	0.6150	0.7384	0.6707	0.0354
	3-shot	0.0319	0.0166	0.4480	0.5651	0.6503	0.6046	0.0203
	CoT	0.0293	0.0152	0.4419	0.5551	0.6386	0.5938	0.0183
LLaMA	0-shot	0.1150	0.1007	0.1542	0.6954	0.7228	0.7083	0.2663
	3-shot	0.1085	0.1113	0.1232	0.7031	0.7100	0.7059	0.2570
	CoT	0.0783	0.0655	0.1535	0.6416	0.6945	0.6659	0.1660

et al., 2020). Similarly, Ram et al. (2021) introduced a Few-Shot LLM question-answering system, utilizing 100,000 annotated questions and 128 training examples as prior knowledge, achieving a maximum F1 score of 72.7% on the SQuAD dataset. Overall, Few-Shot prompting has proven to be more effective and yields better accuracy compared to Zero-Shot prompting.

2.3. Chain-of-Thought prompting

Chain-of-Thought (CoT) prompting enhances the performance of LLMs by improving their reasoning capabilities, allowing them to solve complex tasks through step-by-step thinking (Wei et al., 2024). Kalyan (2024) provided a comprehensive review of GPT-3.5 and GPT-4 research, with a particular focus on recent advancements in CoT-based prompting. Additionally, Rasool et al. (2024) developed a question-answering system using CoT prompting and evaluated the performance of various LLMs. Their results show that the GPT-4 model achieved the highest accuracy of 37.39% on CogTale dataset. These studies demonstrate that CoT prompting can significantly enhance LLMs' ability to handle tasks requiring multi-step reasoning, though the effectiveness may vary across models and tasks.

Shi et al. (2022) introduced the Multilingual Grade School Math (MGSM) benchmark, featuring 250 math problems from the GSM8K dataset translated into ten diverse languages. Their findings highlight that large language models (LLMs), particularly when utilizing CoT prompting, exhibit strong multilingual reasoning abilities, including in underrepresented languages like Bengali and Swahili. The study further demonstrates that CoT prompting not only enhances performance in mathematical problem-solving but also improves LLMs' reasoning capabilities in non-mathematical tasks, such as commonsense reasoning and word-in-context semantic judgment. Toraman (2024) developed a low-resource Turkish downstream open-source LLMs models adaption system. They employed intrinsic evaluation for word quality assessment, instruction fine-tuning, tasks specific fine-tuning and continual training. Sentiment analysis and reasoning tasks using LLMs with CoT-based prompting achieved state-of-the-art (SOTA) performance in Fei et al. (2023), with a maximum accuracy of 87.45% on the restaurant dataset and 85.16% on the laptop review dataset. Overall, CoT-based prompting not only enhances the reasoning capabilities of LLMs but also helps explain the causes of hallucinated outputs and provides guidance for mitigating such issues effectively.

Table 4
RQA evaluation results across models and prompting techniques.

Model	Prompt	Dataset	Accuracy	Precision	Recall	F1 Score	AUC-ROC	Log loss
GPT-3.5	0-shot	8th-Science	0.21	0.13	0.13	0.12	0.61	28.30
	3-shot	8th-Science	0.34	0.22	0.22	0.21	0.67	23.87
	CoT	8th-Science	0.30	0.19	0.19	0.19	0.65	25.29
	0-shot	12th-Math	0.22	0.07	0.06	0.06	0.63	28.26
	3-shot	12th-Math	0.28	0.12	0.12	0.11	0.62	26.06
	CoT	12th-Math	0.29	0.05	0.04	0.05	0.63	25.61
	0-shot	12th-Physics	0.23	0.14	0.14	0.13	0.64	27.71
	3-shot	12th-Physics	0.28	0.16	0.16	0.15	0.63	25.97
	CoT	12th-Physics	0.31	0.15	0.15	0.15	0.64	24.93
GPT-4o	0-shot	8th-Science	0.69	0.53	0.53	0.53	0.84	11.22
	3-shot	8th-Science	0.73	0.58	0.57	0.58	0.86	9.64
	CoT	8th-Science	0.68	0.55	0.55	0.55	0.85	11.38
	0-shot	12th-Math	0.49	0.46	0.46	0.45	0.76	18.23
	3-shot	12th-Math	0.51	0.53	0.50	0.50	0.79	17.69
	CoT	12th-Math	0.45	0.32	0.30	0.30	0.72	19.65
	0-shot	12th-Physics	0.68	0.64	0.64	0.64	0.88	11.49
	3-shot	12th-Physics	0.70	0.67	0.67	0.67	0.89	10.91
	CoT	12th-Physics	0.67	0.65	0.64	0.64	0.89	11.83
Gemini	0-shot	8th-Science	0.60	0.42	0.42	0.41	0.79	14.45
	3-shot	8th-Science	0.54	0.39	0.38	0.38	0.77	16.58
	CoT	8th-Science	0.50	0.36	0.35	0.35	0.75	17.93
	0-shot	12th-Math	0.52	0.34	0.32	0.32	0.73	17.22
	3-shot	12th-Math	0.48	0.42	0.36	0.37	0.72	18.59
	CoT	12th-Math	0.48	0.21	0.19	0.19	0.76	18.60
	0-shot	12th-Physics	0.58	0.43	0.44	0.43	0.79	16.26
	3-shot	12th-Physics	0.58	0.49	0.49	0.49	0.82	14.99
	CoT	12th-Physics	0.55	0.37	0.37	0.37	0.78	16.26
Aya 101	0-shot	8th-Science	0.03	0.00	0.01	0.00	0.50	34.94
	3-shot	8th-Science	0.08	0.05	0.05	0.05	0.53	33.04
	CoT	8th-Science	0.10	0.06	0.06	0.06	0.54	32.41
	0-shot	12th-Math	0.19	0.05	0.05	0.02	0.50	29.22
	3-shot	12th-Math	0.22	0.13	0.15	0.12	0.57	28.03
	CoT	12th-Math	0.20	0.03	0.05	0.04	0.55	28.95
	0-shot	12th-Physics	0.13	0.00	0.01	0.00	0.50	31.28
	3-shot	12th-Physics	0.19	0.07	0.07	0.06	0.54	29.36
	CoT	12th-Physics	0.17	0.03	0.04	0.03	0.53	30.05
BLOOM	0-shot	8th-Science	0.22	0.13	0.14	0.13	0.62	27.98
	3-shot	8th-Science	0.04	0.00	0.01	0.00	0.50	34.46
	0-shot	12th-Math	0.20	0.04	0.06	0.04	0.58	28.67
	3-shot	12th-Math	0.22	0.01	0.05	0.02	0.50	28.22
	0-shot	12th-Physics	0.20	0.09	0.10	0.09	0.59	28.98
	3-shot	12th-Physics	0.15	0.00	0.01	0.00	0.50	30.48
	0-shot	8th-Science	0.10	0.05	0.04	0.04	0.55	32.57
	3-shot	8th-Science	0.19	0.11	0.10	0.11	0.60	29.09
LLaMA	0-shot	12th-Math	0.12	0.02	0.01	0.01	0.55	31.83
	3-shot	12th-Math	0.13	0.02	0.02	0.02	0.56	31.42
	0-shot	12th-Physics	0.13	0.06	0.06	0.06	0.58	31.42
	3-shot	12th-Physics	0.18	0.08	0.07	0.07	0.58	29.69

Table 5
Impact of shot selection on LLaMa3 8B performance for sentiment analysis.

Shots	Accuracy	Precision	Recall	F1 Score	AUC-ROC	Log loss
Zero	77.00	75.00	77.00	76.00	77.00	8.32
One	82.05	92.73	68.00	78.46	81.53	6.46
Two	85.90	88.41	81.33	84.72	85.73	5.08
Three	94.87	92.41	97.33	94.81	94.96	1.8484
Four	65.38	67.80	53.33	59.70	64.94	12.47

Table 6
Impact of temperature value on LLaMa3 8B performance for sentiment analysis.

Temperature	Accuracy	Precision	Recall	F1 Score	AUC-ROC	Log loss
0.0	93.59	91.14	96.00	93.51	93.68	2.3105
0.1	94.87	92.41	97.33	94.81	94.96	1.8484
0.2	94.87	92.41	97.33	94.81	94.96	1.8484
0.92	94.23	92.31	96.00	94.12	94.30	2.0794
1.28	91.67	90.79	92.00	91.39	91.68	3.0036
1.64	68.59	67.57	66.67	67.11	68.52	11.3214
2.0	10.90	11.90	13.33	12.58	10.099	32.1158

2.4. Differences from existing researches

Most previous research has focused on high-resource languages, such as English, using Zero-Shot, Few-Shot, and CoT-based prompting for tasks like reasoning, recommendation systems, NER, question answering, and event classification (López Espejel et al., 2023; D'Asaro et al., 2024; Cheng et al., 2024; Song et al., 2023; Rasool et al.,

2024). In contrast, limited research has explored low-resource languages, such as Turkish, Spanish, and Bengali, applying these prompting techniques for text classification, summarization, and question answering tasks (Çelik and Dalyan, 2023; Barrena et al., 2021; Kabir

et al., 2024; Mahfuz et al., 2024). *This research distinguishes itself from prior studies in the following ways:*

- This study is first to evaluate the effectiveness of Chain-of-Thought (CoT) prompting on Bengali language tasks, alongside Zero-Shot and Few-Shot techniques.
- Focuses on multiple Bengali downstream tasks, including news classification, sentiment analysis, summarization, and question answering, offering comprehensive coverage.
- Compares the performance of open-source models (e.g., BLOOM, LLaMA, Aya 101) with closed-source commercial models (e.g., GPT-4o, Gemini, GPT-3.5), highlighting their strengths and limitations for low-resource languages.
- Presents empirical evidence demonstrating the performance differences between high-resource and low-resource languages, filling a crucial gap in the literature on multilingual LLMs.

These contributions offer valuable insights into the effectiveness of advanced prompting methods, helping address challenges in Bengali NLP and bridging gaps in existing research.

3. Datasets

To evaluate the performance of prompting in LLM models for low-resource language, this study utilized four Bengali downstream task datasets: News Classification, Sentiment Analysis, Summarization, and Question Answering. Below is a summary of the key attributes for each dataset:

- News Classification dT_1 :
 - Dataset type: Multi-class Newspaper
 - Goal: Classify online News based on pre-defined category
 - Number of samples: 495
 - Number of class: 9
 - Max. words in a sample: 3289
 - Min. word in a sample: 17
- Indic Sentiment Analysis dT_2 :
 - Dataset type: User Review (positive/negative)
 - Goal: Classify sentiment based on user review
 - Number of samples: 156
 - Number of class: 2
 - Max. words in a sample: 58
 - Min. word in a sample: 3
- Summarization (XL-Sum) dT_3 :
 - Dataset type: BBC article-summary
 - Goal: Generate abstractive summary
 - Number of samples: 512
 - Max. words in an article: 980
 - Min. word in a sample: 68
- Question Answering (BEnQA) dT_4 :
 - Dataset type: School-level physics and Mathematics
 - Goal: Answering the question based on questions
 - Number of samples: 1652
 - Max. words in a question: 93
 - Min. word in a question: 7

The Bangla Newspaper Dataset (dT_1) contains over 400,000 Bengali news articles across 9 categories, providing a broad range of topics ideal for news classification tasks (Nazi, 2020). We used the test_2 split of this dataset to evaluate model performance on Bengali news categorization. The Indic Sentiment Analysis (dT_2) dataset includes reviews labeled as positive, negative, or neutral, across multiple Indic languages. For our experiments, we selected the Bengali (bn) subset,

enabling us to assess sentiment classification specifically for Bengali text.¹

The XL-Sum (dT_3) is a large-scale multilingual summarization dataset containing 1.35 million article-summary pairs from the BBC across 45 languages, including Bengali. Known for its high-quality, abstractive summaries, the Bengali subset of XL-Sum was utilized with 512 samples to test the summarization capabilities of the models (Hasan et al., 2021). The BEnQA (dT_4) is a new benchmark dataset for question answering and reasoning in Bengali and English, with 5000 questions covering science topics from middle to high school curricula. These questions span factual, application-based, and reasoning categories. We used subsets such as 8th_science, 12th_math_1, 12th_math_2, 12th_physics_1, and 12th_physics_2 to assess LLMs across different educational levels and subjects (Shafayat et al., 2024). Together, these datasets provide a well-rounded foundation for evaluating LLM performance on a variety of Bengali NLP tasks, including news categorization, sentiment analysis, summarization, and question answering. This comprehensive evaluation allows for an in-depth analysis of LLMs capabilities across different task types in Bengali.

4. Methodology

The primary focus of this study is to evaluate the effectiveness of various prompting techniques, i.e., Zero-Shot, Few-Shot, and Chain-of-Thought (CoT) in both closed and open-source LLMs for low-resource languages like Bengali. This evaluation is carried out across four downstream tasks using a Bengali dataset denoted as $D = dT_1, dT_2, dT_3, dT_4$, where dT_1 represents the first task (Bengali text classification), dT_2 covers the second task (Indic language sentiment analysis), dT_3 pertains to multilingual summarization, and dT_4 focuses on question answering. To evaluate these datasets, this study is structured into three key phases: foundation of LLMs, ablation studies, and the prompting process within the LLMs evaluation system. Each phase plays a crucial role in understanding and refining the performance of the models. The abstract view is presented in Fig. 1. The following subsections provide detailed descriptions of these components.

4.1. Foundation of LLMs

In this study, we utilized six LLM models, comprising three closed-source and three open-source versions. A brief description of each is provided below:

Closed-source LLMs. These are proprietary models that are not publicly available for modification or fine-tuning. Access to these models is often limited through APIs, and their internal architectures remain undisclosed. In this research following three models are used:

- **GPT-3.5:** Developed by OpenAI, GPT-3.5 is designed for generating natural language text and optimized for chat, making it more cost-effective than previous models. It provides a strong baseline with balanced performance across diverse NLP tasks. We used GPT 3.5 turbo in our experiments (Ye et al., 2023).
- **GPT-4 Omni (GPT-4o):** GPT-4o extends GPT-4's capabilities, handling text, audio, image, and video inputs with outputs in text, audio, and images. Known for quick response times and improved non-English language performance, it also offers advanced multimodal understanding, including built-in safety features.²
- **Gemini 1.5 Flash (Gemini):** Developed by Google DeepMind, Gemini 1.5 Flash is a lightweight, cost-effective model optimized for speed and high-volume tasks, such as chat and long-context translation. It offers competitive pricing and high throughput,

¹ <https://huggingface.co/datasets/mteb/IndicSentiment>

² <https://openai.com/index/hello-gpt-4o/>

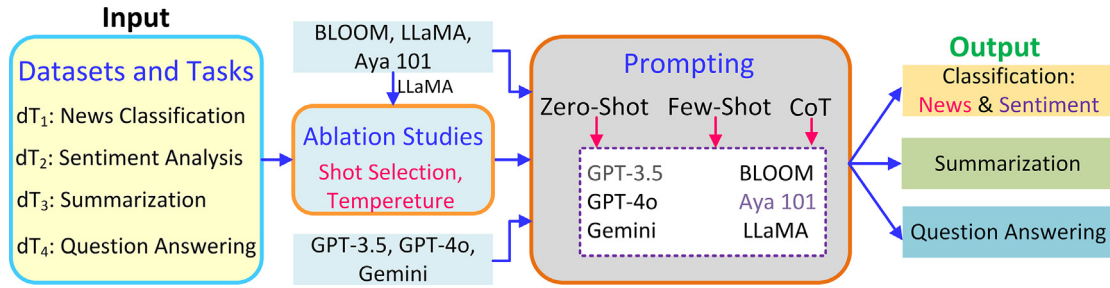


Fig. 1. Abstract process of prompting in LLMs for evaluation of low-resource multiple tasks: Classification (News & Sentiment), Summarization and Question Answering.

making it suitable for production environments and large-scale applications.³

Open-source LLMs. These models are freely accessible, and researchers can modify, fine-tune, or deploy them for various tasks. Open-source LLMs encourage community-driven improvements and transparency in their development. In this research following three models are used:

- **Aya 101:** Aya 101 is a massively multilingual model that supports 101 languages, trained on extensive datasets to maximize instruction-following capabilities. It provides strong performance on multilingual tasks, aiming to empower underrepresented languages in NLP (Üstün et al., 2024).
- **BLOOM 7b1 (BLOOM):** An open-source model trained collaboratively by researchers worldwide, the model covers 46 languages and is designed to democratize LLM access. It offers competitive performance and is especially notable for its transparency and open-access ethos, making it a valuable resource for community-driven development (Scao et al., 2023). In our experiments, we used BLOOM 7b1 version.
- **LLaMA3 8B (LLaMA):** Meta’s Llama3 8b is a foundation model known for its robust support for coding, multilinguality, and reasoning tasks. With capabilities extending to image, video, and speech processing, Llama3 delivers competitive results across modalities, but remains a work in progress with limited public release. Llama 3 8 billion model was used in our evaluation (Dubey et al., 2024).

4.2. Ablation studies

For this study, the ablation experiments were focused exclusively on Shot selection and temperature values for the LLaMA model. Due to time and cost constraints, the closed-source LLMs were excluded from this phase. The process followed a trial-and-error approach, where these parameters were adjusted iteratively to determine the optimal configuration for maximizing accuracy and stability.

Experiments were conducted using various shot setups (e.g., Zero-Shot, Few-Shot) alongside temperature tuning to assess their impact on LLaMA’s performance, particularly for low-resource Bengali tasks. The insights gained from these studies allowed us to fine-tune the LLaMA model efficiently. The outcomes of the ablation studies are detailed in Section 5.2.6, and the optimal shot counts and temperature values derived were later applied across the other five LLMs to maintain consistency.

4.3. Approaches of prompting in LLM-based evaluation: Prompting

The Zero-Shot, Few-Shot, and Chain-of-Thought prompting in three open-source and three closed-source LLMs across multiple tasks, including classification, summarization, and questions answering.

As shown in Fig. 1, we use four datasets: online news classification (dT_1), sentiment classification (dT_2), summarization (dT_3), and question answering (dT_4), as described in Section 3. These datasets are used to evaluate the performance of six LLMs: GPT-3.5, GPT-4o, Gemini, Aya101, BLOOM7b1, and Llama3 8b. In the second phase, each prompt is used to evaluate these LLMs across the four tasks. The details of each prompt are outlined below.

Zero-shot prompting: In this approach, the model is provided with a task without any prior examples or context. It generates a response based solely on the prompt, relying on its general knowledge to produce an answer. In this study, zero-shot prompting was employed for text classification (News and Sentiment) with an instruction prompts such as “Categorize the input text into the following predefined categories”. Similarly, zero-shot prompting was used for summarization and question answering tasks, utilizing the six foundation LLMs.

Few-shot prompting (three-shot): For this method, the model is provided with three examples of the task before being asked to generate a response. This allows the model to infer the task requirements from the examples, potentially improving its accuracy by mimicking the demonstrated examples. We generally saw a lot of improvement in performance while using this strategy.

Chain-of-thought prompting (CoT): In CoT prompting, the model is guided to generate a structured, step-by-step reasoning process before arriving at the final answer (Wei et al., 2022). This technique helps improve performance on tasks that require logical thinking by encouraging the model to explain its thought process. Below are task-specific steps used in CoT prompting:

• News Classification:

- Step 1: Understanding the main content of the article
- Step 2: Identifying the main theme
- Step 3: Matching with categories

• Sentiment Analysis:

- Step 1: Reading the review
- Step 2: Identifying positive or negative words
- Step 3: Determining the Sentiment

• Summarization:

- Step 1: Understanding the main content
- Step 2: Identifying key information
- Step 3: Writing the summary

• Questions Answering:

- Step 1: Understanding the question
- Step 2: Reading the options
- Step 3: Evaluating the options
- Step 4: Selecting the correct answer

³ <https://developers.googleblog.com/en/gemini-15-flash>

This method aims to improve the model's performance on tasks requiring logical reasoning or sequential thinking. However, our experiments revealed that longer prompt lengths in CoT prompting can sometimes lead to increased hallucination where the model generates irrelevant or incorrect outputs, i.e., negatively impacting overall performance. These prompting techniques were selected to evaluate how effectively models perform under different levels of guidance and task complexity.

5. Experiments and results analysis

This section first outlines the experimental setup and evaluation metrics, followed by a detailed presentation of the results and analysis.

5.1. Experimental setup and evaluations measures

To assess the performance of different prompting strategies in LLMs, this study utilized various platforms and resources, i.e., Gemini 1.5 Flash Model: Accessed via the Google Cloud Vertex AI Gemini API.⁴ GPT Models: GPT-4 Omni and GPT-3.5 were accessed through the OpenAI API.⁵ Llama Model: Deployed on an AWS g5.12xlarge virtual machine with 4 GPUs (96 GB GPU memory), 48 vCPUs, and 192 GB of RAM.⁶ utilizing the Aphrodite engine⁷ Aya and Bloom Models: Hosted on virtual machines with a single NVIDIA A100 SXM4 GPU (40 GB memory). This setup ensures a robust environment for benchmarking the impact of Zero-Shot, Few-Shot, and Chain-of-Thought prompting across various Bengali NLP tasks. The performance of LLM models under different prompting techniques was evaluated using Eqs. (1)–(12).

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \quad (1)$$

where TP is True Positives, TN is True Negatives, FP is False Positives, and FN is False Negatives.

$$\text{Precision} = \frac{TP}{TP + FP} \quad (2)$$

$$\text{Recall} = \frac{TP}{TP + FN} \quad (3)$$

$$\text{F1 Score} = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \quad (4)$$

$$L_{\log} = -\frac{1}{N} \sum_{i=1}^N [y_i \log(p_i) + (1 - y_i) \log(1 - p_i)] \quad (5)$$

where N is the number of samples, y_i is the true label (1 or 0), and p_i is the predicted probability of the positive class.

For summarization tasks, we used the following metrics:

$$P_{\text{ROUGE-L}} = \frac{LCS(S, S_R)}{\text{len}(S)} \quad (6)$$

$$R_{\text{ROUGE-L}} = \frac{LCS(S, S_R)}{\text{len}(R)} \quad (7)$$

$$F_{\text{ROUGE-L}} = 2 \cdot \frac{P_{\text{ROUGE-L}} \cdot R_{\text{ROUGE-L}}}{P_{\text{ROUGE-L}} + R_{\text{ROUGE-L}}} \quad (8)$$

where $LCS(S, S_R)$ is the Longest Common Subsequence between generated summary S and reference summary S_R .

BERT Score: *BERTScore* evaluates sentence similarity by comparing contextual embeddings of tokens from a summary S and a reference summary S_R . Using pairwise *cosine similarity*, it calculates *recall* (R) and *precision* (P) by matching each token to the most similar token in the other sentence (Zhang et al., 2019). The *F1 score* is then computed as:

$$R_{\text{BERT}} = \frac{1}{|S_R|} \sum_{r_i \in S_R} \max_{s_j \in S} r_i^T s_j \quad (9)$$

$$P_{\text{BERT}} = \frac{1}{|S|} \sum_{s_j \in S} \max_{r_i \in R} s_j^T r_i \quad (10)$$

$$F1_{\text{BERT}} = 2 \cdot \frac{P_{\text{BERT}} \cdot R_{\text{BERT}}}{P_{\text{BERT}} + R_{\text{BERT}}} \quad (11)$$

These scores can be optionally weighted by *inverse document frequency (idf)* to give more importance to rare words, enhancing relevance in summary evaluation.

BLEU: This measures n-gram overlap between generated and reference text:

$$\text{BLEU} = \text{BP} \cdot \exp \left(\sum_{n=1}^N w_n \log p_n \right) \quad (12)$$

where BP is the brevity penalty, w_n are weights for each n-gram precision p_n , and N is the maximum n-gram length. These metrics provide a comprehensive evaluation framework for different types of NLP tasks, allowing us to assess model performance across both classification and summarization contexts.

5.2. Results analysis

This section presents the performance of different prompting strategies in LLMs across four Bengali downstream tasks: text classification, sentiment analysis, abstractive summarization, and question answering. Each task and its associated results are detailed as follows:

5.2.1. Performance on news classification dT_1

Table 1 presents a summary of the performance of Zero-Shot, Few-Shot, and Chain-of-Thought (CoT) prompting across six LLMs. The highest accuracy and F1 score of 85.00% were achieved by the GPT-4o model with 3-Shot prompting. In contrast, the lowest performance was observed with the BLOOM model, which achieved only 11.00% accuracy. Among the three prompting strategies, CoT-based prompting underperformed for Bengali news classification tasks. This decline in performance is primarily due to the model's inability to fully capture the context and correctly identify the main theme of the news articles, which is crucial for accurate classification.

In terms of error rates, the BLOOM model exhibited the highest logarithmic maximum loss of 30.70%, reflecting significant difficulty in this task. On the other hand, the Aya 101 model achieved the lowest loss value of 4.06%, indicating better alignment with the task requirements. Overall, the GPT-4o model outperformed the other LLMs, delivering the best performance in Bengali news classification. Its advanced architecture and improved handling of multilingual contexts contributed to its superior accuracy compared to both open and closed LLMs.

5.2.2. Performance on indic sentiment analysis dT_2

Table 2 presents the sentiment analysis performance across various models. The GPT-4o model achieved the highest accuracy and F1 score of 99.00% using CoT-based prompting. In contrast, the GPT-3.5 and Aya 101 models showed the lowest performance, each with an accuracy of just 2.00%.

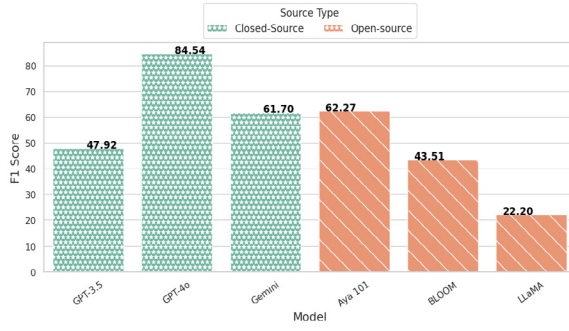
From Table 2, the poor performance of GPT-3.5 and Aya 101 with CoT prompting can be attributed to their inability to incorporate the reasoning process effectively within the 256-token limit. This constraint impacted their ability to handle the reasoning required for accurate sentiment classification. In comparison, the GPT-4o model excelled with CoT-based prompting due to its enhanced reasoning capabilities, aligning well with the sentiment analysis task. This demonstrates the advantage of models with better reasoning frameworks for tasks that require contextual understanding and subtle sentiment differentiation.

⁴ cloud.google.com/vertex-ai/generative-ai/

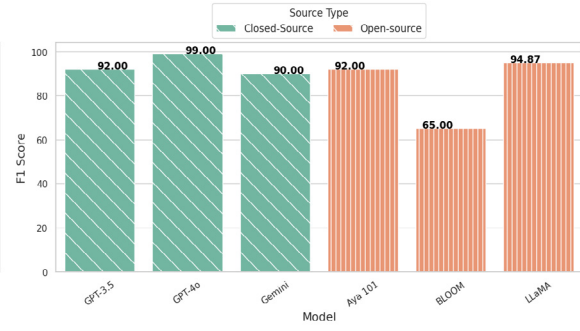
⁵ github.com/openai/openai-python

⁶ aws.amazon.com/ec2/instance-types/g5/

⁷ github.com/PygmalionAI/aphrodite-engine



(a) F1-score of news classification



(b) F1-score of sentiment classification

Fig. 2. Best-performing Open-Source and Closed-Source F1-score for news and sentiment classification.

5.2.3. Performance on summarization (XL-Sum) dT_3

The performance of the three prompting strategies across the LLMs is summarized in Table 3. This table highlights how Zero-Shot, Few-Shot, and Chain-of-Thought (CoT) prompting impact the summarization outcomes. The highest performance for summarization was achieved with the 3-Shot prompting in the GPT-4o model, which yielded an $F1_{bert}$ score of 72.87%, a BLEU score of 33.83%, and an $F_{rouge-l}$ score of 14.83%. In contrast, the lowest performance was recorded with CoT-based prompting in the BLOOM model, where the 0-Shot setting resulted in a BLEU score of just 3.54%. These results indicate that GPT-4o excels with structured prompts, while BLOOM struggles with reasoning-heavy tasks when provided minimal guidance.

5.2.4. Performance on question answering (BEnQA) dT_4

The question-answering performance is evaluated on the dT_4 dataset, which consists of three categories of questions: 8th-Science, 12th-Math, and 12th-Physics. Table 4 summarizes the results of applying three different prompting techniques across six LLMs on this dataset.

The maximum performance for the 8th-Science category was achieved with the GPT-4o model, reaching an accuracy of 73.00%, an F1 score of 58.00%, and a minimum Log Loss of 9.64 using 3-Shot prompting. For the 12th-Math category, the Gemini model recorded the highest accuracy at 52.00%, while the GPT-4o model achieved the highest F1 score of 50.00% using 3-Shot prompting. Similarly, the best results for the 12th-Physics category came from the GPT-4o model, with an accuracy of 70.00% and an F1 score of 68.00%, both using 3-Shot prompting.

From Table 4, it is evident that the advanced reasoning capabilities of GPT-4o and Gemini models significantly enhance their ability to solve scientific and mathematical problems more accurately than other models. The 3-Shot prompting technique plays a crucial role by allowing these models to harness contextual understanding from examples, which proves advantageous for handling complex tasks. In contrast, other models faced challenges in achieving similar performance levels, likely due to constraints in token handling, weaker reasoning capabilities, or misalignment with the structure of the given problems.

5.2.5. Performance comparison between open-source and closed-source models

Fig. 2, based on 2(a), presents the F1 scores for News classification tasks, showcasing the performance comparison between three Open-Source and three Closed-Source LLM models. This comparison highlights the varying capabilities of these models in handling low-resource Bengali text classification tasks. The Closed-Source GPT-4o model achieved the highest F1 score of 84.54%, while the Open-Source LLaMA model recorded the lowest at 22.20%. The lower performance of LLaMA can be attributed to tokenization issues and its suboptimal alignment with low-resource Bengali text classification tasks, highlighting the challenges faced by Open-Source models when handling

specialized linguistic contexts.

Similarly, Fig. 2, based on 2(b) presents the F1 scores for sentiment classification tasks across six LLM models. The GPT-4o model achieved the maximum F1 score of 99.00%, while the BLOOM model recorded the lowest at 65.00%. The superior performance of GPT-4o can be attributed to its extensive number of trainable hyperparameters and the inclusion of a larger amount of low-resource training data during the foundation model's development. These factors enable GPT-4o to capture nuanced linguistic patterns, outperforming other models in handling sentiment classification tasks.

The summarization performance of six LLM models is illustrated in Fig. 3, based on 3(a). The Closed-Source GPT-4o model achieved the highest $F1_{bert}$ score of 72.87%, while the lowest score recorded was 67.07%. Notably, the performance gap between Open-Source and Closed-Source models is minimal for low-resource summarization tasks, indicating that Bengali article summarization aligns well with both model types. This suggests that Open-Source models can be competitive with proprietary models when handling low-resource languages like Bengali.

In Fig. 3, based on 3(b), the F1 scores of six LLM models are presented for question-answering tasks. The Closed-Source GPT-4o model achieved the maximum F1 score of 58.22%, while the Open-Source LLaMA model recorded the lowest score of 6.58%. This reveals a substantial performance gap of 51.64% between Open-Source and Closed-Source models. The results highlight that GPT-4o demonstrates strong reasoning capabilities, making it more effective in handling low-resource settings. Additionally, the lower performance of Open-Source models suggests that they may struggle with semantic alignment and lack the depth of reasoning required for complex question-answering tasks. This gap emphasizes the importance of advanced proprietary models in addressing nuanced language understanding challenges.

5.2.6. Ablation studies

In this research, we conducted two types of ablation studies: (i) Ablation for Shot selection, and (ii) Ablation for temperature. Due to the pricing constraints of Closed-Source models and limited time, these ablation experiments were carried out using the open-source LLaMa model, specifically the LLaMa3 8B variant focusing on sentiment analysis tasks. The following paragraphs present the impact of these ablation studies.

(i) Ablation for shot selection. Table 5 summarizes the performance of the LLaMA3 8B model across four different shot selections for sentiment classification tasks. The results demonstrate that the model's performance is significantly affected by the number of shots, especially for Bengali sentiment analysis. As the shot count increases from zero to three, the model shows gradual improvement, indicating that providing a few examples helps it acquire prior knowledge and contextual understanding. However, in the four-shot scenario, performance drops, likely due to the following factors:

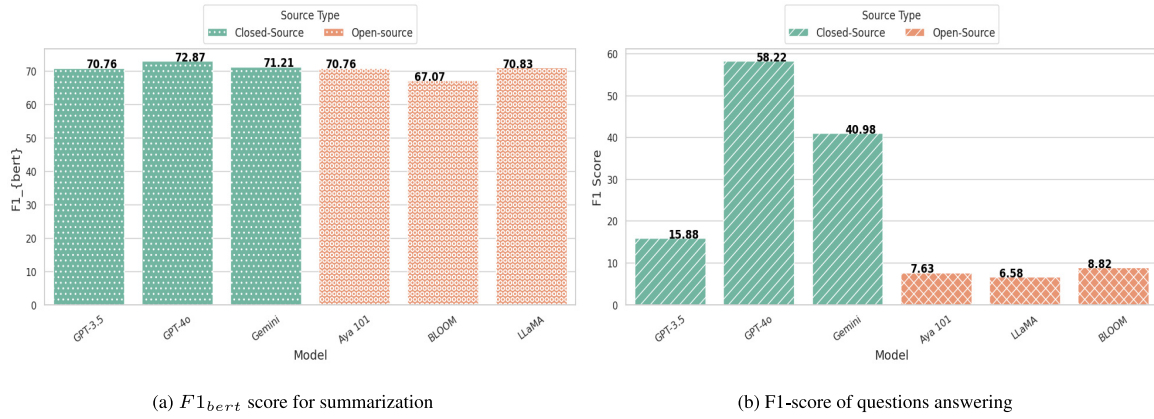


Fig. 3. Best-performing Open-Source and Closed-Source F1-score for news and sentiment classification.

- **Alignment and Quality of Examples:** The downstream performance of LLMs in Few-Shot scenarios heavily depends on the quality and alignment of the provided examples. In the four-shot scenario, the selected examples may have introduced conflicting signals or lacked alignment with the task, leading to reduced performance.
- **Exceeding Maximum Token Length:** Including four examples could have exceeded the model's maximum token length due to the larger number of tokens. Truncation may have caused a loss of important contextual information, impairing the model's ability to fully comprehend the task-specific input.
- **Out-of-Distribution Examples:** The examples used in the four-shot scenario might not represent the broader task distribution, introducing out-of-distribution issues. This misalignment could create gaps in contextual understanding and negatively affect performance.

This trend illustrates the importance of carefully curating few-shot examples to maximize performance, as more examples do not always lead to better outcomes, especially when the examples introduce noise or ambiguity.

(ii) Ablation for temperature. The temperature parameter in LLMs significantly influences during the text generation. Higher temperature values allow for more diverse and creative outputs, introducing greater variability, while lower temperature values produce more focused and deterministic responses. Table 6 summarizes the impact of various temperature settings on the performance of the LLaMA3 8B model using three-shot prompting. Table 6 shows that lower temperature values (0.0–0.2) yield optimal results by producing focused and consistent outputs. In contrast, higher temperature values introduce excessive randomness, impairing the model's predictive accuracy. As the temperature increases, the performance drops significantly, with accuracy plummeting from 94.87% at 0.1 to 10.90% at 2.

5.3. Error analysis

This section presents the qualitative error analysis for three prompting techniques, i.e., Zero-Shot, Few-Shot, and Chain-of-Thought (CoT) across different NLP tasks. Tables 7 and 8 illustrate both successful and failed predictions, providing insights into the strengths and limitations of each approach. In Table 7, the 0-Shot and CoT approaches often generated correct predictions, but the 3-shot method struggled by misidentifying sentiments, showing that even with examples, generalization is not always reliable. In the second example for news classification, the Zero-Shot method deviated from the predefined output format, including unnecessary details, which suggests that it lacks task-specific focus and struggles with structured outputs. The CoT-based prompting performed well in question answering by providing

correct answers, but it often lacked detailed explanations, limiting its usefulness for tasks that require deeper reasoning and justifications.

In Table 8, the 0-shot prompting was hallucinated responses, where the model generated incorrect predictions without proper grounding. The summarization task showed that 0-shot and 3-shot methods failed to capture both sides of conflicting information, summarizing only one aspect of the story. In contrast, the CoT approach provided a more neutral summary, effectively balancing the conflicting narratives.

Overall, this error analysis highlights key limitations of LLMs. Zero-Shot prompting struggles with task-specific formatting and focus, Few-Shot is dependent on the clarity of examples, and CoT, despite its reasoning strength, lacks explanatory depth. These findings emphasize the importance of selecting and fine-tuning the appropriate prompting strategy based on the task to mitigate these limitations and improve model performance.

6. Discussion

This research evaluates Zero-Shot, Few-Shot, and Chain-of-Thought prompting across three closed-source LLMs and three open-source LLMs. The following sections highlight the key findings and limitations.

- Few-Shot prompting in the closed-source GPT-4o model outperformed other models in news classification, summarization, and question answering tasks (i.e., Details in Tables 1, 3 and 4). In contrast, CoT prompting with GPT-4o yielded superior performance in sentiment analysis. Overall, GPT-4o surpassed the remaining five models. However, Zero-Shot and CoT prompting underperformed in certain cases due to a lack of prior knowledge and limited reasoning capabilities.
- In terms of reasoning capability, the closed-source LLMs, i.e., GPT-3.5, GPT-4o, and Gemini outperformed all open-source models. This is evident in mathematical and physics-related question-answering tasks, as illustrated in Fig. 3 of 3(b).
- The ablation studies reveal that low-resource NLP downstream tasks are highly dependent on temperature values and the optimal number of shots, as shown in Section 5.2.6, Tables 5 and 6. Higher temperatures increase the model's randomness and hallucination rate, reducing output reliability. Additionally, the number of shots significantly influences the text generation process, as highlighted in Table 5.

6.0.1. Limitations

Although this study evaluated four Bengali downstream tasks using three prompting techniques across six LLM models and conducted ablation experiments, some limitations still remain.

Monolingual: These experiments were conducted exclusively in Bengali, so the key findings are specific to the Bengali language.

Table 7

Summary of Zero-Shot, Few-Shot, and Chain-of-Thought prompting for successful and failed predictions in Bengali sentiment analysis, news classification, and question answering (QA) tasks.

Task ID	Input Sample	0-shot	3-shot	CoT	Remarks
Sentiment Analysis	প্রবেশ বিনামূল্যে এবং এটি কেবলমাত্র সোসাইটি মেম্বারদের জন্য, যার জন্য এটি আরো নিরাপদ বলে মনে হয়। Translation: Entry is free and is for Society Members only, for whom it seems safer.	পজিটিভ Translation: Positive	নেগেটিভ Translation: Negative	- ইতিবাচক শব্দ: বিনামূল্যে, সোসাইটি সদস্য - অনুভূতি: পজিটিভ Translation: - Positive words: Free, Society Members - Feelings: positive	The 0-shot and CoT approaches provide correct responses, while the 3-shot method fails to predict the correct sentiment.
News Classification	ভারতের গুজরাটে বিমানবাহিনীর জাওয়ার যুদ্ধবিমান বিধ্বস্ত হয়ে পাইলট নিহত হয়েছে। ... Translation: Air Force Jaguar fighter jet crashes in India's Gujarat, pilot killed ...	প্রেডিক্টেড ক্যাটাগরি হলো আন্তর্জাতিক। যুদ্ধবিমান বিধ্বস্ত হওয়ার খবরের Translation: The predicted category is International. The news of the crash of the warplane...	আন্তর্জাতিক Translation: International	নিবন্ধের বিষয়বস্তু: ভারতের গুজরাটে বিমানবাহিনীর জাওয়ার যুদ্ধবিমান ... ### চিত্রাধারা: - মূল বিষয়বস্তু: ভারতীয় বিমানবাহিনী জাওয়ার যুদ্ধবিমান বিধ্বস্ত হওয়া এবং পাইলট নিহত হওয়া সংবাদ। - মূল থিম: আন্তর্জাতিক খবর - বিভাগ মিলান: আন্তর্জাতিক প্রেডিক্টেড ক্যাটাগরি: আন্তর্জাতিক Translation: Article Content: Air Force Jaguar fighter jets in Gujarat, India... ### Thinking: - Main content: News of Indian Air Force Jaguar fighter jet crash and pilot killed. - Main Theme: International News - Department Milan: International Predicted Category: 'International'	The 0-shot approach does not adhere to the pre-defined output format and includes unnecessary details.
QA	চোখের আকৃতি ঠিক রাখে কোনটি? Translation: Which one keeps the eye shape right?	গ) অক্ষিসোলক Translation: c) Eyeball	গ) অক্ষিসোলক Translation: c) Eyeball	ক) স্বেভমন্ডল Translation: a) White matter	The CoT approach yields a correct answer but lacks an accompanying explanation.

Table 8

Summary of Zero-Shot, Few-Shot, and Chain-of-Thought prompting for successful and failed predictions in Bengali article summarization, and sentiment analysis tasks.

Task ID	Input Sample	0-shot	3-shot	CoT	Remarks
Summary	ধাওয়া-পাল্টা ধাওয়ার পর গাড়িতে আগুন দেয়ার ঘটনা ঘটা। পদতলের ... বিএনপির সমর্থকরা সেখান পুলিশের এক গাড়িতে আগুন দেয় এবং রাবার পাশে দড়িয়ে থাকা কয়েকটি গাড়ি ভাঙচুর করে বলে জানিয়েছে পুলিশ। ... মি. রিজভী বলেছেন, "আগুন দেয়া বা ভাঙচুরের কোনো ঘটনার সাথে বিএনপি জড়িত নয়।" Translation: After the chase, the car was set on fire. Police said BNP supporters set fire to a police vehicle and vandalized several vehicles parked on the side of the road in Paltan. ... Mr. Rizvi said, "BNP is not involved in any incidents of arson or vandalism."	পুলিশ ও বিএনপি নেতাকর্মীদের মধ্যে সংঘর্ষের পর বিএনপি সমর্থকরা বিদ্রোহ করে Translation: BNP supporters: protested after clashes between police and BNP activists	পুলিশ ও বিএনপি নেতাকর্মীদের মধ্যে ধাওয়াপাল্টা ধাওয়ার ঘটনায় বিদ্রোহের সংঘর্ষ উদ্ভূত হয় পুলিশের গাড়িতে আগুন দেওয়া হয়েছে এবং বিএনপি সমর্থকরা রাবার পাশে দড়িয়ে থাকা গাড়ি ভাঙচুর করেছে Translation: A chase between police and BNP activists led to a surprise clash, with police cars set on fire and BNP supporters vandalizing cars parked on the roadside	ধাপ ১ ধাওয়াপাল্টা ধাওয়ার পর গাড়িতে আগুন দেয়ার ঘটনা ঘটে পদতলের বিএনপি কার্যালয়ে ... ধাপ ২ মূল বিষয়বস্তু: ধাওয়াপাল্টা ধাওয়ার পর গাড়িতে আগুন দেয়ার ঘটনা ধাপ ৩ সারসংক্ষেপ: পুলিশ ও বিএনপি নেতাকর্মীদের মধ্যে সংঘর্ষের পর পুলিশের গাড়িতে আগুন দেয়া হয়েছে Translation: Step 1 After a chase, a car was set on fire at the BNP office in Paltan. Step 2 The main theme is the incident of setting fire to a car after a car chase Step 3 Summary After the dash between the police and BNP activists, the police car was set on fire	In the original news, two conflicting pieces of information are presented. However, the 0-shot and 3-shot approaches summarize the news based solely on one side of the accusation, neglecting information about the other side. In contrast, the CoT method summarizes the news impartially, maintaining a neutral stance.
Sentiment Analysis	"দরওলি প্রতিযোগিতামূলক, প্রায় সবসময় বাজারে সেরা" Translation: "Rates are competitive, almost always the best on the market"	এই রিভিউটি দেখে মনে হচ্ছে সে পণ্যটি দরওলি প্রতিযোগিতামূলক এবং বাজারে সেরা হবে বলে। এটি একটি সুপারিশ Translation: This review suggests that the product is priced competitively and is the best in the market. This is a recommendation	পজিটিভ Translation: Positive	### ধাপ ১: রিভিউ পড়া রিভিউটি পড়া হয়েছে। এতে বলা হয়েছে যে, "দরওলি প্রতিযোগিতামূলক, প্রায় সবসময় বাজারে সেরা।" ### ধাপ ২: ইতিবাচক বা নেতিবাচক শব্দ চিহ্নিত করা - ইতিবাচক শব্দ: প্রতিযোগিতামূলক, সেরা ### ধাপ ৩: Translation: ### Step 1: Read reviews The review has been read. It states that "rates are competitive, almost always the best in the market". ### Step 2: Identifying positive or negative words - Positive words: competitive, best ... (incomplete)	The 0-shot prompting generates a hallucinated response, while the CoT approach fails to complete the response by providing the correct class.

Generalization: The optimal temperature and number of shots for LLM models are highly dependent on the specific dataset and tasks, making the optimal values identified in this research not universally generalizable.

Hallucination: During our experiments, we also tested other open-source large language models, including Mistral-Instruct-Bangla,⁸ Mistral-7B-v0.1,⁹ Mistral-7B-v0.3,¹⁰ (Jiang et al., 2023) Llama 2 7B Chat Touvron et al. (2023), etc. However, these models generated a high number of incorrect and hallucinated responses, making them unsuitable for our evaluation. Even with the chosen models, hallucination

was a recurring issue, as they often deviated from a consistent response format. To address this, we applied heuristics, such as searching for correct answers within the string response, to evaluate the models effectively. Additionally, we faced some challenges with prompt length as Bangla language characters required more tokens per character for the models. This led to longer chain-of-thought prompts, which further increased hallucination and resulted in poorer performance for this prompting method. However, this study did not address or mitigate one of the most critical issues of LLMs, i.e., hallucination specifically for the low-resource Bengali language.

Dataset Size: The datasets used in this research contain a limited number of samples due to GPU support constraints. Another challenge we faced was, the gemini model encountered issues with its safety filter, which blocked certain answers, preventing us from evaluating those specific samples.

⁸ Mistral-Instruct-Bangla

⁹ mistralai/Mistral-7B-Instruct-v0.1

¹⁰ mistralai/Mistral-7B-Instruct-v0.3

7. Conclusion and future works

In this study, we evaluated six LLM models, i.e., GPT-3.5, GPT-4o, Aya 101, BLOOM, LLaMA, and Gemini using three prompting techniques: Zero-Shot, Few-Shot, and Chain-of-Thought. The evaluation focused on four low-resource Bengali downstream tasks: news classification, sentiment analysis, question answering, and summarization. Among the six models, three were open-source (LLaMA, Aya 101, and BLOOM), while the other three were closed-source (GPT-3.5, GPT-4o, and Gemini). The results indicate that GPT-4o performed the best overall, especially in tasks requiring a deep understanding of context and language nuances. However, open-source models like LLaMA also showed competitive performance on certain tasks, suggesting that open models can be effective for Bengali NLP, particularly when fine-tuned on complex and diverse Bengali datasets. We found that three-shot prompting generally produced better results than zero-shot, as it helped the models adapt more effectively to Bengali through examples. While Chain-of-Thought prompting was beneficial in some cases, it often led to reduced accuracy due to hallucination issues, especially with complex or longer prompts.

Although this research is the first to use three prompting techniques on four Bengali datasets with six LLM models, future work will address the identified limitations by expanding the scope beyond Bengali to include other low-resource and high-resource languages, enabling cross-lingual comparisons and improved generalization. Adaptive strategies for dynamically selecting temperature values and shot numbers can improve performance across tasks and datasets. Addressing hallucination issues will require techniques such as fact-checking modules, reinforcement learning from human feedback (RLHF), and hybrid models that integrate knowledge graphs. To overcome dataset and computational constraints, larger datasets and collaborations with cloud providers or research institutions could be explored. Additionally, fine-tuning open-source models like LLaMA and BLOOM on high-quality Bengali datasets and domain-specific corpora can enhance their effectiveness, closing the performance gap with closed-source models and improving their applicability to low-resource NLP.

CRediT authorship contribution statement

Zabir Al Nazi: Writing – original draft, Software, Conceptualization. **Md. Rajib Hossain:** Writing – original draft, Supervision, Methodology, Conceptualization. **Faisal Al Mamun:** Software, Resources, Data curation.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

The datasets generated and analyzed during the current study are available from the corresponding author upon reasonable request.

References

- Barrena, A., Soroa, A., Agirre, E., 2021. Towards zero-shot cross-lingual named entity disambiguation. *Expert Syst. Appl.* 184, 115542. <http://dx.doi.org/10.1016/j.eswa.2021.115542>, URL: <https://www.sciencedirect.com/science/article/pii/S0957417421009490>.
- Bražinskas, A., Lapata, M., Titov, I., 2020. Few-shot learning for opinion summarization. In: *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing. EMNLP*, Association for Computational Linguistics, Online, pp. 4119–4135. <http://dx.doi.org/10.18653/v1/2020.emnlp-main.337>, URL: <https://aclanthology.org/2020.emnlp-main.337>.

- Çelik, E., Dalyan, T., 2023. Unified benchmark for zero-shot Turkish text classification. *Inf. Process. Manage.* 60 (3), 103298. <http://dx.doi.org/10.1016/j.ipm.2023.103298>, URL: <https://www.sciencedirect.com/science/article/pii/S0306457323000353>.
- Cheng, Q., Chen, L., Hu, Z., Tang, J., Xu, Q., Ning, B., 2024. A novel prompting method for few-shot NER via LLMs. *Nat. Lang. Process. J.* 8, 100099. <http://dx.doi.org/10.1016/j.nlp.2024.100099>, URL: <https://www.sciencedirect.com/science/article/pii/S2949719124000475>.
- D'Asaro, F., De Luca, S., Bongiovanni, L., Rizzo, G., Papadopoulos, S., Schinas, M., Koutlis, C., 2024. Zero-shot content-based crossmodal recommendation system. *Expert Syst. Appl.* 258, 125108. <http://dx.doi.org/10.1016/j.eswa.2024.125108>, URL: <https://www.sciencedirect.com/science/article/pii/S0957417424019754>.
- Dubey, A., Jauhri, A., P, ey, A., Kadian, A., Al-Dahle, A., Letman, A., Mathur, A., Schelten, A., Yang, A., Fan, A., et al., 2024. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*.
- Fei, H., Li, B., Liu, Q., Bing, L., Li, F., Chua, T.-S., 2023. Reasoning implicit sentiment with chain-of-thought prompting. In: *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*. Association for Computational Linguistics, Toronto, Canada, pp. 1171–1182. <http://dx.doi.org/10.18653/v1/2023.acl-short.101>, URL: <https://aclanthology.org/2023.acl-short.101>.
- Hasan, T., Bhattacharjee, A., Islam, M.S., Mubasshir, K., Li, Y.-F., Kang, Y.-B., Rahman, M.S., Shahriyar, R., 2021. XL-sum: Large-scale multilingual abstractive summarization for 44 languages. In: *Findings of the Association for Computational Linguistics: ACL-JCNLP 2021*. Association for Computational Linguistics, Online, pp. 4693–4703, URL: <https://aclanthology.org/2021.findings-acl.413>.
- Hossain, M.R., Hoque, M.M., Dewan, M.A.A., Hoque, E., Siddique, N., 2025. AuthorNet: Leveraging attention-based early fusion of transformers for low-resource authorship attribution. *Expert Syst. Appl.* 262, 125643. <http://dx.doi.org/10.1016/j.eswa.2024.125643>, URL: <https://www.sciencedirect.com/science/article/pii/S0957417424025107>.
- Jiang, A.Q., Sablayrolles, A., Mensch, A., Bamford, C., Chaplot, D.S., de las Casas, D., Bressand, F., Lengyel, G., Lample, G., Saulnier, L., Lavaud, L.R., Lachaux, M.-A., Stock, P., Scao, T.L., Lavril, T., Wang, T., Lacroix, T., Sayed, W.E., 2023. Mistral 7B. *arXiv:2310.06825*.
- Kabir, M., Islam, M.S., Laskar, M.T.R., Nayeem, M.T., Bari, M.S., Hoque, E., 2024. BenLLM-eval: A comprehensive evaluation into the potentials and pitfalls of large language models on bengali NLP. In: *Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation. LREC-COLING 2024, ELRA and ICCL*, Torino, Italia, pp. 2238–2252, URL: <https://aclanthology.org/2024.lrec-main.201>.
- Kalyan, K.S., 2024. A survey of GPT-3 family large language models including ChatGPT and GPT-4. *Nat. Lang. Process. J.* 6, 100048. <http://dx.doi.org/10.1016/j.nlp.2023.100048>, URL: <https://www.sciencedirect.com/science/article/pii/S2949719123000456>.
- Lin, T., Wang, Y., Liu, X., Qiu, X., 2022. A survey of transformers. *AI Open* 3, 111–132.
- López Espejel, J., Ettifouri, E.H., Yahaya Alassan, M.S., Chouham, E.M., Dahhane, W., 2023. GPT-3.5, GPT-4, or BARD? Evaluating LLMs reasoning ability in zero-shot setting and performance boosting through prompts. *Nat. Lang. Process. J.* 5, 100032. <http://dx.doi.org/10.1016/j.nlp.2023.100032>, URL: <https://www.sciencedirect.com/science/article/pii/S2949719123000298>.
- Mahfuz, T., Dey, S.K., Naswan, R., Adil, H., Sayeed, K.S., Shahgir, H.S., 2024. Too late to train, too early to use? A study on necessity and viability of low-resource Bengali LLMs. *arXiv preprint arXiv:2407.00416*.
- Nazi, Z.A., 2020. Bangla newspaper dataset. <http://dx.doi.org/10.34740/KAGGLE/DSV/1576225>, URL: <https://www.kaggle.com/dsv/1576225>.
- Qin, L., Chen, Q., Zhou, Y., Chen, Z., Li, Y., Liao, L., Li, M., Che, W., Yu, P., 2024. Multilingual large language model: A survey of resources, taxonomy and frontiers. *arXiv, Article*, *arXiv preprint arXiv:2404.04925*.
- Ram, O., Kirstain, Y., Berant, J., Globerson, A., Levy, O., 2021. Few-shot question answering by pretraining span selection. In: *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*. Association for Computational Linguistics, Online, pp. 3066–3079. <http://dx.doi.org/10.18653/v1/2021.acl-long.239>, URL: <https://aclanthology.org/2021.acl-long.239>.
- Rasool, Z., Kurniawan, S., Balugo, S., Barnett, S., Vasa, R., Chesser, C., Hampstead, B.M., Belleville, S., Mouzakis, K., Bahar-Fuchs, A., 2024. Evaluating LLMs on document-based QA: Exact answer selection and numerical extraction using CogTale dataset. *Nat. Lang. Process. J.* 8, 100083. <http://dx.doi.org/10.1016/j.nlp.2024.100083>, URL: <https://www.sciencedirect.com/science/article/pii/S2949719124000311>.
- Scao, T.L., Fan, A., Akiki, C., Pavlick, E., Ilić, S., Hesslow, D., Castagné, R., Lucic, A.S., Yvon, F., Gallé, M., et al., 2023. Bloom: A 176b-parameter open-access multilingual language model.
- Shafayat, S., Hasan, H.M.Q., Mahim, M.R.C., Putri, R.A., Thorne, J., Oh, A., 2024. BEnQA: A question answering and reasoning benchmark for Bengali and english. *arXiv:2403.10900*.
- Shi, F., Suzgun, M., Freitag, M., Wang, X., Srivats, S., Vosoughi, S., Chung, H.W., Tay, Y., Ruder, S., Zhou, D., et al., 2022. Language models are multilingual chain-of-thought reasoners. *arXiv preprint arXiv:2210.03057*.

- Song, C., Cai, F., Zheng, J., Zhao, X., Shao, T., 2023. Augprompt: Knowledgeable augmented-trigger prompt for few-shot event classification. *Inf. Process. Manage.* 60 (4), 103153. <http://dx.doi.org/10.1016/j.ipm.2022.103153>, URL: <https://www.sciencedirect.com/science/article/pii/S0306457322002540>.
- Toraman, C., 2024. Adapting open-source generative large language models for low-resource languages: A case study for Turkish. In: Sälevä, J., Owodunni, A. (Eds.), *Proceedings of the Fourth Workshop on Multilingual Representation Learning. MRL 2024*, Association for Computational Linguistics, Miami, Florida, USA, pp. 30–44. <http://dx.doi.org/10.18653/v1/2024.mrl-1.3>.
- Touvron, H., Martin, L., Stone, K., Albert, P., Almahairi, A., Babaei, Y., Bashlykov, N., Batra, S., Bhargava, P., Bhosale, S., Bikel, D., Blecher, L., Ferrer, C.C., Chen, M., Cucurull, G., Esiobu, D., Fernandes, J., Fu, J., Fu, W., Fuller, B., Gao, C., Goswami, V., Goyal, N., Hartshorn, A., Hosseini, S., Hou, R., Inan, H., Kardaş, M., Kerkez, V., Khabsa, M., Kloumann, I., Korenev, A., Koura, P.S., Lachaux, M.-A., Lavril, T., Lee, J., Liskovich, D., Lu, Y., Mao, Y., Martinet, X., Mihaylov, T., Mishra, P., Molybog, I., Nie, Y., Poulton, A., Reizenstein, J., Rungta, R., Saladi, K., Schelten, A., Silva, R., Smith, E.M., Subramanian, R., Tan, X.E., Tang, B., Taylor, R., Williams, A., Kuan, J.X., Xu, P., Yan, Z., Zarov, I., Zhang, Y., Fan, A., Kambadur, M., Narang, S., Rodriguez, A., Stojnic, R., Edunov, S., Scialom, T., 2023. Llama 2: Open foundation and fine-tuned chat models. [arXiv:2307.09288](https://arxiv.org/abs/2307.09288).
- Üstün, A., Aryabumi, V., Yong, Z.-X., Ko, W.-Y., D'souza, D., Onilude, G., Bhandari, N., Singh, S., Ooi, H.-L., Kayid, A., Vargus, F., Blunsom, P., Longpre, S., Muennighoff, N., Fadaee, M., Kreutzer, J., Hooker, S., 2024. Aya model: An instruction finetuned open-access multilingual language model. *arXiv preprint arXiv:2402.07827*.
- Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E.H., Le, Q.V., Zhou, D., 2024. Chain-of-thought prompting elicits reasoning in large language models. In: *Proceedings of the 36th International Conference on Neural Information Processing Systems. NIPS '22*, Curran Associates Inc., Red Hook, NY, USA.
- Wei, J., Wang, X., Schuurmans, D., Bosma, M., Xia, F., Chi, E., Le, Q.V., Zhou, D., et al., 2022. Chain-of-thought prompting elicits reasoning in large language models. *Adv. Neural Inf. Process. Syst.* 35, 24824–24837.
- Xu, Y., Hu, L., Zhao, J., Qiu, Z., Ye, Y., Gu, H., 2024. A survey on multilingual large language models: Corpora, alignment, and bias. *arXiv preprint arXiv:2404.00929*.
- Ye, J., Chen, X., Xu, N., Zu, C., Shao, Z., Liu, S., Cui, Y., Zhou, Z., Gong, C., Shen, Y., et al., 2023. A comprehensive capability analysis of gpt-3 and gpt-3.5 series models. *arXiv preprint arXiv:2303.10420*.
- Zhang, T., Kishore, V., Wu, F., Weinberger, K.Q., Artzi, Y., 2019. Bertscore: Evaluating text generation with bert. *arXiv preprint arXiv:1904.09675*.

LEVOS: Leveraging Vocabulary Overlap with Sanskrit to Generate Technical Lexicons in Indian Languages

Karthika N J, Krishnakant Bhatt, Ganesh Ramakrishnan, and Preethi Jyothi

Indian Institute of Technology Bombay, India

{karthika, kkbhatt, ganesh, pjyothi}@cse.iitb.ac.in

Abstract

Translating technical terms into lexically similar, low-resource Indian languages remains a challenge due to limited parallel data and the complexity of linguistic structures. We propose a novel use-case of Sanskrit-based segments for linguistically informed translation of such terms, leveraging subword-level similarity and morphological alignment across related languages. Our approach uses character-level segmentation to identify meaningful subword units, facilitating more accurate and context-aware translation. To enable this, we utilize a Character-level Transformer model for Sanskrit Word Segmentation (CharSS), which addresses the complexities of sandhi and morphophonemic changes during segmentation. We observe consistent improvements in two experimental settings for technical term translation using Sanskrit-derived segments, averaging 8.46 and 6.79 chrF++ scores, respectively. Further, we conduct a post hoc human evaluation to verify the quality assessment of the translated technical terms using automated metrics. This work has important implications for the education field, especially in creating accessible, high-quality learning materials in Indian languages. By supporting the accurate and linguistically rooted translation of technical content, our approach facilitates inclusivity and aids in bridging the resource gap for learners in low-resource language communities.

1 Introduction

English is the most widely used language in academic books and as a medium of instruction worldwide. India has 22 official languages in addition to English. Over the years, works like (Hudelson, 1987) have established the power of language in students' learning. Aligned with these studies, the Indian Government has proposed several changes to the education system, among which multilingual knowledge dissemination assumes an important role. The proposal involves introducing regional

languages at various levels of education. Carrying out this proposal leads to massive resource requirements like textbook translations and content creation. With the limited technical content availability in non-English languages, especially at the higher education level, translating technical terms from English to other languages is a challenging task that needs to be addressed.

Maheshwari et al. (2024) presents the importance of domain-specific lexicon generation, especially catering to the technical domains, and its importance for translation tasks with low-resource languages as the target. Kunchukuttan and Bhattacharyya (2016) shows the importance of subword segmentation and lexical similarity of languages in the translation task. Additionally, Sanskrit language is known to be a lexically rich, flexible, and well-structured language with the potential to create meaningful new words easily. In this paper, we introduce a use case of Sanskrit-based sub-word level segmentation in word and phrase-level translation of academic/technical terminologies to leverage the large overlap of vocabulary among Indian languages. For the generation of technical terms in low-resource regional languages, we propose to utilize the high vocabulary overlap of Indo-Aryan and Dravidian languages with Sanskrit, thereby performing a lexically informed translation.

Compound words are formed by combining two or more meaningful subwords. In Indian languages, compounds may be formed either through simple concatenation without boundary changes or by following sandhi rules, resulting in boundary modifications. Decomposing a compound Sanskrit word involves segmenting it into smaller, meaningful lexical units. Existing methods used for the Sanskrit Word Segmentation (SWS)¹ task can be roughly classified into two categories: tack-

¹We use the term segmentation for the task of splitting a compound word into its meaningful constituents.

ling the broader task of SWS and sandhi splitting-specific techniques. The former includes works like (G  rard, 2003; Sriram et al., 2023), a lexicon-driven shallow parser. Hellwig and Nehrdich (2018a) processes compound sandhi words at the character level using recurrent and convolutional neural networks. Sandhan et al. (2022) presents TransLIST, integrating a module that appends additional latent information from SHR to the input sequence. It also employs a soft masked attention mechanism to prioritize relevant subword candidates and incorporates a path ranking algorithm to mitigate erroneous predictions. Alternately, Aralikatte et al. (2018) proposes a dual-decoder approach where the first decoder identifies the location for the sandhi split (sandhivicch  da)², and the second decoder predicts the segmented output. Similarly, Dave et al. (2021) applies an RNN encoder-decoder-based two-stage methodology to predict the location and final splits. Nehrdich et al. (2024) presented a new language model pre-trained for Sanskrit and further fine-tuned and utilised the model for various downstream tasks including word segmentation, lemmatization and morphosyntactic tagging tasks. We use a similar architecture in our CharSS model, to generate the word-splits.

Our main contributions through the paper are:

- We present the utilization of a character-based Transformer model for the segmentation of compound words (including sandhivicch  da) in Sanskrit (Section 2.1).
- We propose a Sanskrit-based input augmentation method using relatively resource-rich Hindi translations to generate linguistically informed technical lexicons for lexically similar, low-resource languages (Section 2.2).
- Through comprehensive experiments, we show the efficacy of our proposed methodologies. We test CharSS on three benchmark datasets for SWS. Similarly, we experiment with our technical term translation process for multiple low-resource languages, generating better-quality technical lexicons in the target languages (Section 3).

2 Methodology

2.1 Sanskrit Word Segmentation

Figure 1 illustrates the proposed methodology for SWS. We formulate the task of sandhi splitting and Sanskrit Word Segmentation as a standalone

sequence-to-sequence transformation problem. For this purpose, we propose to utilize a character-level Transformer model such as ByT5.

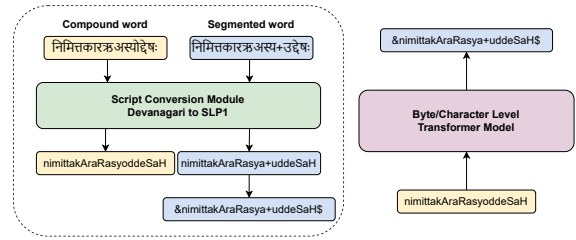


Figure 1: Illustration of the proposed methodology for SWS task.

ByT5. The ByT5 (Byte-Level Text-to-Text Transfer Transformer) model (Xue et al., 2022) processes text as sequences of bytes, bypassing the need for language-specific tokenization. This approach enables it to handle diverse languages and scripts effectively, including rare words and complex orthographies. ByT5 is built on the T5 (Raffel et al., 2020) framework. It poses all tasks as text-to-text problems, enhancing its versatility. ByT5 demonstrates strong performance on multilingual and code-mixed tasks, making it particularly suitable for low-resource languages and domain-specific vocabularies. The input to the model is a single Sanskrit word (unigram), and the output consists of the segmented sub-tokens of the word, which are concatenated using a "+" symbol to indicate the split. We prepend the target split with an "&" symbol to denote the start and append a "\$" symbol to mark the end of the target split as shown in Figure 1 to allow for precise delineation of morpheme boundaries.

2.2 Technical Term Translation

In this paper, we propose a linguistically informed method to translate technical terms in English to low-resource Indian languages. This process entails a crucial input augmentation phase prior to the modeling and training stages to enhance the input for model training. The raw dataset comprises technical terms for English and translation to Hindi. We prepare supplementary data for augmentation using the methodology described below.

Sanskrit-based augmented input

There is a significant vocabulary overlap among Indian languages, especially with Sanskrit. In this

²We follow ISO-15919 script to mention Roman translations of Indian language text for better readability.

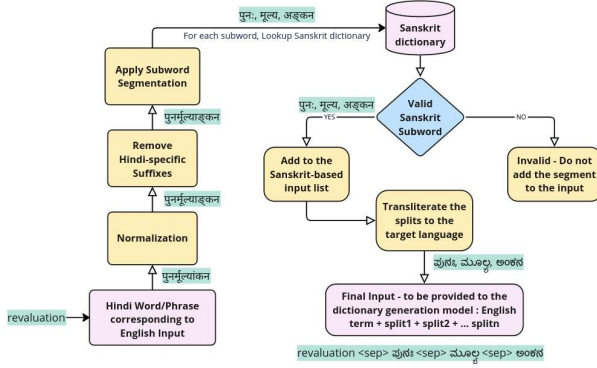


Figure 2: The process of generating Sanskrit-based augmented input for the English term 'revaluation', for translation model

work, we attempt to leverage this overlap by using available dictionaries in the resource-rich Hindi language to generate the corresponding terms in other Indian languages. Figure 2 shows the steps to obtain the proposed augmented input. For a given technical term, we first normalize the corresponding term in Hindi as explained in Appendix A.1. We then remove the Hindi-specific affixes from the words to get the lemma. Finally, we perform segmentation of the normalized lemma and pass them as additional input to the translation model to aid the generation of technical terms in low-resource Indian languages.

Motivation to use Hindi data to generate Sanskrit-based segments:

There is a significant under-representation of digital resources for all other Indian languages compared to Hindi. Appendix A shows details of this digital data divide. English-Hindi human-translated data is readily available for the domains we considered in this work. We obtained Sanskrit-based sub-word tokens from the available Hindi data for over 76% of training and test instances. Furthermore, a word in one language may have several different translations in another language, depending on the context of usage. Providing the augmented input helps disambiguate the domain of the word. See Section 3.3 for a detailed analysis supporting this argument.

3 Experiments and Results

For the technical term translation task, we utilize the technical bilingual dictionary datasets provided by Maheshwari et al. (2024) which is a dataset curated from CSTT³ dictionaries. The dataset con-

sists of word-level translations from English to 6 Indian languages across 3 domains, viz., administrative, biotechnology, and chemistry, and has 9094 terms in the training data and 1285 in the test data for all domains combined. We obtained Sanskrit-based inputs for all data instances by applying our approach of generating Sanskrit-based additional inputs. We use chrF++ (Popović, 2017) as the evaluation metric for all the experiments under this task.

3.1 Experiments on the SWS Task

For the SWS task, we use word-level accuracy as the evaluation metric. To compare against (Sandhan et al., 2022), we also calculate sentence level perfect match (PM) for SIGHUM and hackathon datasets. We utilize the pre-trained checkpoint of the base variant of the ByT5 model available via Huggingface⁴ and fine-tune it over the UoH+SandhiKosh, SIGHUM dataset, and hackathon datasets as three separate experiments. For details about choice of ByT5 and Experiments with SWS Task (see Appendix B.2).

3.2 Experiments on the Technical Term Translation Task

For this task, we have two experimental settings, both formulated as text-to-text translation. In the first setting, we train and test the NMT model NLLB (Costa-jussà et al., 2022) over all 6 language pairs across 3 domains. In the second setting, we train the model on Hindi, Gujarati, and Tamil across 3 domains and test it over Marathi, Kannada, and Odia across the same domains, which can be considered as a zero-shot setting. In the **baseline** configuration for this task, the model is fed with English input only. In the configuration corresponding to the proposed method, the English input is augmented with additional Sanskrit-based input prepared as discussed in Section 2.2. We utilize the pre-trained 1.3B parameter checkpoint of the NLLB model available via Huggingface⁵ and fine-tune it over the technical domain dictionary data for both experimental settings.

Results. Table 1 reports the comparison of chrF++ scores obtained by finetuning the NMT model with English-only input (NLLB) and with augmented input (NLLB+Sanskrit) under the first experimental setting. In Section 3.2 we analyze the

³<https://cstt.education.gov.in/en>

⁴<https://huggingface.co/google/byt5-base>

⁵<https://huggingface.co/facebook/nllb-200-1.3B>

Test Dataset	Model	Hindi	Marathi	Gujarati	Kannada	Tamil	Odia	Average
Administrative	NLLB	50.23	45.42	43.35	45.68	44.13	43.22	45.33
	NLLB + Sanskrit	54.74	46.07	45.82	47.25	44.07	44.45	47.07
Biotechnology	NLLB	53.52	51.91	3.79	12.38	18.46	17.16	26.20
	NLLB + Sanskrit	60.63	60.73	13.09	29.20	37.89	35.82	39.56
Chemistry	NLLB	48.96	50.64	8.19	16.59	17.43	20.31	27.02
	NLLB + Sanskrit	54.36	55.35	17.41	29.51	33.04	34.07	37.29

Table 1: chrF++ scores on the administrative, biotechnology, and chemistry domains for models with and without additional Sanskrit-based input.

performance of the model with and without additional input in a zero-shot setting. Across experiments, there’s a consistent performance gain with the lexically informed input. Our method archives an average improvement of **8.46** chrF++ scores. We also provide a detailed post-hoc analysis of the predictions in Section 3.3

Zero-Shot Translation

Table 2 shows the performance of the translation model without Sanskrit input (NLLB) and with Sanskrit input (NLLB+Sanskrit) when trained on Hindi, Gujarati, and Tamil, and evaluated on Marathi, Kannada, and Odia across 3 domains *viz.*, Administration, Biotechnology, and Chemistry. Performance in terms of chrF++ scores shows that the translation with the Sanskrit augmented input consistently provides better translations as compared to the English-only input across different languages and domains. This proves the efficacy of Sanskrit-based additional input for capturing multilingual nuances.

Test Dataset	Model	Marathi	Kannada	Odia	Average
Administrative	NLLB	41.42	44.03	40.57	42.01
	NLLB + Sanskrit	43.26	45.71	42.02	43.66
Biotechnology	NLLB	44.42	27.83	29.37	33.87
	NLLB + Sanskrit	53.79	40.32	37.76	43.96
Chemistry	NLLB	41.62	28.41	26.99	32.34
	NLLB + Sanskrit	49.71	39.11	34.13	40.98

Table 2: chrF++ scores on administrative, biotechnology, and chemistry for unseen languages, namely, Kannada, Marathi, and Odia for zero-shot setting.

3.3 Post-hoc analysis

In this section, we present our detailed analysis of a subset of the results of the lexicon translation task. Unlike a regular translation task, which includes a complete sentence and paragraphs, we deal with a single word or phrase here. Such a short input may have many different possible translations in

the target language, either the translations that can be used interchangeably or those that may be varied with the context of its usage. The evaluation metrics like BLEU and chrF may not effectively capture the quality of translation as it is obtained by comparison of the predictions with the available ground truth data. The ground truth data may have a single or limited number of meaningful translations, and as a result, a different but correct prediction may be penalised.

We followed the Human Post-hoc evaluation as per Maheshwari et al. (2024) for the same two additional languages as presented by them *viz.*, Punjabi and Malayalam, using the same subset of input data and metrics. Our goal is to understand the practical utility of the generated lexicon in the respective languages and the extent to which they may be helpful in translating technical books from English to low-resource Indian languages. We achieved an R@1 score of 0.53 and 0.46 for Punjabi and Malayalam, respectively, compared to 0.51 and 0.38 scores obtained by LexGen. The R@3 score for Malayalam is 0.72, comparable to 0.71 for LexGen, while the score for Punjabi was slightly lower, at 0.92, compared to 0.95 for LexGen. We also present detailed analysis of the translation results by a comparative study of the outputs in both the input settings, i.e., with and without the Sanskrit-based augmented output (See Appendix A.3).

4 Conclusion

In this work, we addressed the task of Sanskrit Word Segmentation (SWS) with a character-level Transformer model, achieving superior segmentation performance on two benchmark datasets and competitive performance on another benchmark dataset.. Furthermore, we propose to leverage the significant vocabulary overlap among Indian languages, utilizing data from the relatively resource-rich Hindi language which highlights the potential

of cross-linguistic resource sharing to boost performance in low-resource language tasks.

Limitations

To generate Sanskrit-based input, we rely on the available Hindi data. Though the availability of Hindi resources is much higher than that of other Indian languages, its digital data richness is considerably lower than that of English.

Not all languages exhibit significant vocabulary overlap with Sanskrit, and in such cases, our proposed method may have limited applicability for lexicon generation.

Acknowledgments

We acknowledge IIT Bombay and BharatGen for providing resources and support for the project. Author Karthika acknowledges the PhD fellowship grant from the TCS Research Foundation. We also extend our appreciation to the reviewers for their valuable feedback.

References

- Rahul Aralikatte, Neelamadhav Gantayat, Naveen Panwar, Anush Sankaran, and Senthil Mani. 2018. Sanskrit sandhi splitting using seq2 (seq)². *arXiv preprint arXiv:1801.00428*.
- Shubham Bhardwaj, Neelamadhav Gantayat, Nikhil Chaturvedi, Rahul Garg, and Sumeet Agarwal. 2018. Sandhikosh: A benchmark corpus for evaluating sanskrit sandhi tools. In *Proceedings of the Eleventh International Conference on Language Resources and Evaluation (LREC 2018)*.
- Marta R Costa-jussà, James Cross, Onur Çelebi, Maha Elbayad, Kenneth Heafield, Kevin Heffernan, Elahe Kalbassi, Janice Lam, Daniel Licht, Jean Maillard, et al. 2022. No language left behind: Scaling human-centered machine translation. *arXiv preprint arXiv:2207.04672*.
- Sushant Dave, Arun Kumar Singh, Dr Prathosh AP, and Prof Brejesh Lall. 2021. Neural compound-word (sandhi) generation and splitting in sanskrit language. In *Proceedings of the 3rd ACM India Joint International Conference on Data Science & Management of Data (8th ACM IKDD CODS & 26th COMAD)*, pages 171–177.
- Huet Gérard. 2003. Lexicon-directed segmentation and tagging of sanskrit. In *XIIth World Sanskrit Conference, Helsinki, Finland, Aug*, pages 307–325. Cite-seer.
- Pawan Goyal and Gérard Huet. 2013. Completeness analysis of a sanskrit reader. In *Proceedings, 5th International Symposium on Sanskrit Computational Linguistics. DK Printworld (P) Ltd*, pages 130–171. Citeseer.
- Oliver Hellwig. 2010. Dcs-the digital corpus of sanskrit. heidelberg (2010-2021). URL <http://www.sanskritlinguistics.org/dcs/index.php>.
- Oliver Hellwig and Sebastian Nehrlich. 2018a. Sanskrit word segmentation using character-level recurrent and convolutional neural networks. In *Proceedings of the 2018 conference on empirical methods in natural language processing*, pages 2754–2763.
- Oliver Hellwig and Sebastian Nehrlich. 2018b. [Sanskrit word segmentation using character-level recurrent and convolutional neural networks](#). In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 2754–2763, Brussels, Belgium. Association for Computational Linguistics.
- Sarah Hudelson. 1987. The role of native language literacy in the education of language minority children. *Language Arts*, 64(8):827–841.
- Gérard Huet. 2003. Towards computational processing of sanskrit. In *International Conference on Natural Language Processing (ICON)*, pages 40–48.
- Amrith Krishna, Pavankumar Satuluri, and Pawan Goyal. 2017. A dataset for sanskrit word segmentation. In *Proceedings of the Joint SIGHUM Workshop on Computational Linguistics for Cultural Heritage, Social Sciences, Humanities and Literature*, pages 105–114.
- Sriram Krishnan, Amba Kulkarni, and Gérard Huet. 2020. Validation and normalization of dcs corpus using sanskrit heritage tools to build a tagged gold corpus. *arXiv preprint arXiv:2005.06545*.
- Anil Kumar, Vipul Mittal, and Amba Kulkarni. 2010. Sanskrit compound processor. In *Sanskrit Computational Linguistics: 4th International Symposium, New Delhi, India, December 10-12, 2010. Proceedings*, pages 57–69. Springer.
- Anoop Kunchukuttan and Pushpak Bhattacharyya. 2016. Faster decoding for subword level phrase-based smt between related languages. *arXiv preprint arXiv:1611.00354*.
- Ayush Maheshwari, Atul Kumar Singh, Karthika NJ, Krishnakant Bhatt, Preethi Jyothi, and Ganesh Ramakrishnan. 2024. Lexgen: Domain-aware multilingual lexicon generation. *arXiv preprint arXiv:2405.11200*.
- Sebastian Nehrlich, Oliver Hellwig, and Kurt Keutzer. 2024. [One model is all you need: ByT5-Sanskrit, a unified model for Sanskrit NLP tasks](#). In *Findings of the Association for Computational Linguistics: EMNLP 2024*, pages 13742–13751, Miami, Florida, USA. Association for Computational Linguistics.

- Maja Popović. 2017. chrf++: words helping character n-grams. In *Proceedings of the second conference on machine translation*, pages 612–618.
- Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J Liu. 2020. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of machine learning research*, 21(140):1–67.
- Kumar Sachin. 2007. Sandhi splitter and analyzer for sanskrit (with reference to ac sandhi). *Mphil degree at SCSS, JNU (submitted, 2007)*.
- Jivnesh Sandhan, Rathin Singha, Narein Rao, Suvendu Samanta, Laxmidhar Behera, and Pawan Goyal. 2022. Translist: A transformer-based linguistically informed sanskrit tokenizer. *arXiv preprint arXiv:2210.11753*.
- Krishnan Sriram, Amba Kulkarni, and Gérard Huet. 2023. [Validation and normalization of DCS corpus and development of the Sanskrit heritage engine’s segmenter](#). In *Proceedings of the Computational Sanskrit & Digital Humanities: Selected papers presented at the 18th World Sanskrit Conference*, pages 38–58, Canberra, Australia (Online mode). Association for Computational Linguistics.
- Linting Xue, Aditya Barua, Noah Constant, Rami Al-Rfou, Sharan Narang, Mihir Kale, Adam Roberts, and Colin Raffel. 2022. Byt5: Towards a token-free future with pre-trained byte-to-byte models. *Transactions of the Association for Computational Linguistics*, 10:291–306.

A Appendix

A.1 Normalisation

anusvāra (ṁ), is a symbol used in all Indian language scripts to denote a type of nasal sound. According to Sanskrit grammatical rules, when this symbol precedes one of the first 4 characters in each of the consonant group called varṅas (ka/ca/ṭa/ta/pa), it needs to be converted to the respective fifth characters (pañcamākṣara) of the varṅas (ṅ/ṇ/ṣ/n/m). This rule may not be followed in other Indian languages. Since our sub-word segmentation model is trained on Sanskrit, and applied on Hindi data for the translation task, we normalise all the data by converting all occurrences of anusvāra to the corresponding pañcamākṣara, before passing it to our model for segmentation.

A.2 sandhi

Sanskrit and other Indian languages have common usage of compound words, which are formed from multiple subwords. When two words are combined, the language expects certain rules to be followed at the word boundaries. Such a change in the word boundary forming a compound word, is termed as sandhi (the word has a meaning of *junction*). In Sanskrit, there are specific rules for the joining of subwords to form a compound, depending on the ending character of the first and the beginning character of the second word. We specify these rules as the *sandhi rules* in this paper. Similarly, splitting of the sandhi will also need to follow the reverse process, which is not as straightforward as sub-word joining. In the paper, we specify the process of sandhi splitting as *sandhivichheda*. Following are some examples of sandhivichheda (1) tatrāpi = tatra + api; (2) narēndra = nara + indraḥ

A.3 Post-hoc Analysis of Generated Technical Lexicons

Table 3 shows some qualitative, post hoc analysis of the prediction results. The analysis shows that the augmented input

- Assists the model to disambiguate between multiple possible outputs (synonyms) and obtain the contextually apt term.
- Examples 1 and 2 in table 3 are from the Administration domain, with Kannada as the required target language. The translations generated by the model with only the English input are meaningful but in different contexts. The

word *mass* is considered by the model, in the meaning of *the amount of matter in an object*, while the expected meaning is mass as used in *population*

- Similarly, the word *composition* is expected to take the meaning of *composing music or poetry*, while the meaning taken by the model is *the process of combining parts of something to whole*. Example 4 shows a similar trend in Marathi in Biotechnology domain.

For the above examples, our model is able to disambiguate the intended meaning and generate the expected output.

- Examples 3 is a sample where the output generated with English-only input is incorrect, while the augmented input generates correct output.

We notice that, the performance difference with and without augmented input is less in the administrative domain when compared to other domains. With the observations from the predictions, we arrive at the following reasonings. The words in this domain are very frequently used by people in all languages. The model predictions with augmented input results in many archaic words, which are currently not in use, or the usage is highly infrequent. A word can have a large number of synonyms, and the number of words in the reference list of the ground truth, is limited, which mostly do not include the archaic words. Because of these reasons, we do not see a large jump in the performance with augmented input in this domain. This observation is especially true with languages like Tamil, in which there is a significant number of non-Sanskrit originated words, which may be more commonly in use. In both experimental settings, we observe that the gain is more in case of the biotechnology and chemistry domains as compared to the administrative domain. This behavior can be attributed to the pre-training of the NLLB model on massive generic domain data which has considerable overlap with the administrative domain data.

B Sanskrit Word Segmentation Task

B.1 Data - SWS Task

For the SWS task, following Dave et al. (2021) and Sandhan et al. (2022), we use three publicly available benchmark datasets, *UoH corpus*⁷ combined with the *SandhiKosh dataset* (Bhardwaj et al., 2018), *SIGHUM dataset* (Krishna et al., 2017),

⁷<https://sanskrit.uohyd.ac.in/Corpus/>

Technical term (English)	Domain; Language	Augmented input ⁶	Prediction with	
			English only input	Sanskrit-based augmented input
1 mass	Administration; Kannada	mass <SEP> jana <isep> samūha	dravyamāna	jana-samūha
2 composition	Administration; Kannada	composition <SEP> racanā	samyōjane	racanā
3 brood	Biotechnology; Marathi	brood <SEP> bhrūṇa	prajanana	bhrūṇa
4 transformation	Biotechnology; Marathi	transformation <SEP> rūpa <isep> antaraṇa	parivartana	rūpāntara
5 injection	Biotechnology; Marathi	injection <SEP> antaḥ <isep> kṣēpaṇa	injēkṣana	antaḥ-kṣēpaṇa

Table 3: Post hoc Qualitative Analysis of Technical term translation results

and *hackathon dataset* (Krishnan et al., 2020). These datasets are carefully curated subsets of a larger corpus DCS (Hellwig, 2010). The UoH corpus+SandhiKosh dataset has 62273 and 15569 instances as train and test sets. For this dataset, we apply the pruning technique mentioned in (Dave et al., 2021) to filter out invalid instances. The size of the training, validation, and test sets for the SIGHUM dataset are 97000, 3000, and 4200, respectively, and for the hackathon dataset, it is 90000, 10332, and 9963, respectively. Contemporary deep-learning methodologies have demonstrated enhanced performance when utilizing the SLP1 script for Sanskrit. Consequently, we have prepared all datasets in the SLP1 script to leverage these performance improvements.

Model	LPA	SPA
JNU	-	8.1
UoH	-	47.2
INRIA	-	59.9
DD-RNN	95.0	79.5
Sandhi Prakarana	92.3	86.8
ByT5	97.2	93.5

Table 4: Location prediction accuracies (LPA) and split prediction accuracies (SPA) for different methods on the UoH+SandhiKosh dataset.

B.2 Experiment Details

Baselines. For the experiments performed over the *UoH+SandhiKosh* dataset, we compare our method against **Sandhi Prakarana** (Dave et al., 2021), **DD-RNN** (Aralikatte et al., 2018), and 3 sandhi splitter tools viz (i) *JNU Splitter* (Sachin, 2007), (ii) *UoH Splitter* (Kumar et al., 2010), and (iii) *INRIA Sanskrit Heritage Reader* (Huet, 2003; Goyal and Huet, 2013). We reproduce and report the scores reported by Dave et al. (2021). For DD-RNN and the 3 sandhi tools, we report the scores reported in (Aralikatte et al., 2018) and (Dave et al., 2021). For the experiments performed over the *SIGHUM* and *hackathon* datasets, we compare our

method against **TransLIST** (Sandhan et al., 2022) and **rcNN-SS** (Hellwig and Nehrdich, 2018b).

Results. Tables 4 and 5 report the performance of our methodology compared with the baselines over the respective datasets. Table 4 shows that our methodology outperforms all other baselines in terms of both Location Prediction Accuracy (LPA) and Split Prediction Accuracy (SPA) with absolute gains of **4.86** and **6.72**, respectively, on the *UoH+SandhiKosh* dataset. TransLIST Sandhan et al. (2022) utilizes a set of potential split candidates from SHR (referred to as LIST in their paper), which provides additional linguistic information for segmentation. Our model is not linguistically informed like this as we feed only the compound word to the model. Hence, our method is not strictly comparable with the results shown in row 2 of Table 5. Nevertheless, our method outperforms all other models on three out of four evaluation metrics when tested on *hackathon* dataset. On *SIGHUM* dataset, our method achieves competitive scores. Sandhan et al. (2022) also reported the performance of their model without the LIST module, as shown in row 3 (TransLIST). The model without the LIST step is more comparable to our setting and we outperform this result as well, while failing to outperform the scores in row 2. As a separate experiment, we provide SHR input to our model for *SIGHUM* data which outperforms TransLIST on PM metric achieving a PM score of **94.31**.

Model	SIGHUM				Hackathon			
	P	R	F	PM	P	R	F	PM
rcNN-SS	96.86	96.83	96.84	87.08	96.40	95.15	95.77	77.62
TransLIST	98.80	98.93	98.86	93.97	97.78	97.44	97.61	85.47
TransLIST	-	-	-	86.10	-	-	-	-
ByT5	98.68	98.42	98.53	93.78	97.58	97.71	97.63	87.7

Table 5: Word-level Precision, Recall, F1 and sentence-level Perfect Match (PM) scores on SIGHUM and hackathon.

Leveraging Pāṇinian Grammar and Neural Models for Morphologically Rich Sanskrit NLP

Yashawant Pathak

yashpathak21@gmail.com

Rustamji Institute of Technology

Jagdish Makhijani

Rustamji Institute of Technology

Research Article

Keywords: Sanskrit NLP, Pāṇinian Grammar, Sandhi Splitting, Hybrid Rule-based and Neural Models, Morphologically Rich Languages, Low-Resource Language Processing, Linguistically Informed Preprocessing, Computational Linguistics

Posted Date: October 30th, 2025

DOI: <https://doi.org/10.21203/rs.3.rs-7947104/v1>

License:  This work is licensed under a Creative Commons Attribution 4.0 International License.

[Read Full License](#)

Additional Declarations: No competing interests reported.

Leveraging Pāṇinian Grammar and Neural Models for Morphologically Rich Sanskrit NLP

Yashawant Pathak

¹[0009-0000-4385-0281]

Dr. Jagdish Makhijani

²[0000-0002-3601-3319]

¹yashpathak21@gmail.com

Rustamji Institute of Technology, Gwalior

²drjagdish@rjit.ac.in

Rustamji Institute of Technology, Gwalior

Abstract:

Sanskrit's rule-based grammatical precision and morphological richness make it a compelling foundation for developing linguistically informed Natural Language Processing (NLP) systems. Rooted in Pāṇini's Aṣṭādhyāyī, the language encodes syntactic and semantic relations directly within its word forms, offering structural advantages over modern, statistically modeled languages. This study introduces a hybrid framework that integrates symbolic Pāṇinian grammar with neural architectures to enhance preprocessing and downstream language understanding. Specifically, sandhi splitting (euphonic decomposition) is re-engineered as an alternative to conventional stopword removal, preserving semantic integrity while improving feature granularity. The framework combines rule-based segmentation (sanskrit_parser) with data-driven sequence models (CharSS and ByT5), evaluated on the SandhiKosh and Digital Corpus of Sanskrit datasets for tasks such as word segmentation, sentiment classification, and syntactic parsing. Experimental results demonstrate that sandhi-aware preprocessing yields up to 8 percent higher F1 scores compared with conventional pipelines, confirming the synergistic potential of grammatical formalism and deep learning. Beyond advancing Sanskrit NLP, the proposed approach contributes a transferable methodology for processing other morphologically rich, low-resource languages, bridging ancient linguistic theory with modern computational intelligence.

Keywords: Sanskrit NLP; Pāṇinian Grammar; Sandhi Splitting; Hybrid Rule-based and Neural Models; Morphologically Rich Languages; Low-Resource Language Processing; Linguistically Informed Preprocessing; Computational Linguistics

1 Introduction

The motivation to integrate Sanskrit into Natural Language Processing (NLP) arises from its unparalleled grammatical rigor, morphological richness, and systematic linguistic design. Unlike many modern languages that evolved through organic usage, Sanskrit was meticulously codified through the generative grammar of *Pāṇini's Aṣṭādhyāyī* (circa 4th century BCE), comprising over 4,000 formally structured rules. This algorithmic nature positions Sanskrit as a natural candidate for computational modeling, where explicit rule hierarchies mirror the architecture of modern symbolic and neural NLP systems. Its morpho-syntactic transparency—where grammatical relations are embedded within inflectional endings—reduces ambiguity and provides a precise foundation for developing context-aware and interpretable language models.

As the field of NLP advances toward hybrid architectures that blend symbolic reasoning with deep neural networks, languages possessing such structural explicitness offer unique testing grounds. Sanskrit enables the exploration of tasks that are inherently challenging in high-entropy languages, such as accurate morphological parsing, word segmentation under phonetic transformations (*sandhi*), and compound decomposition (*samāsa*). The integration of Pāṇinian grammar into computational frameworks promises enhanced syntactic consistency and linguistic interpretability, addressing persistent challenges in data-driven NLP systems, including explainability and data sparsity.

Beyond its linguistic sophistication, Sanskrit carries immense cultural and scientific significance, encompassing millennia of philosophical, medical, and literary texts. The digitization and computational analysis of this vast corpus are critical for cultural preservation and for enabling cross-domain applications in areas such as semantic retrieval, translation, and ontology construction. However, despite its formal precision, Sanskrit remains underrepresented in NLP research due to scarce annotated corpora, limited computational resources, and the absence of scalable, language-specific tools. These limitations classify Sanskrit as a low-resource language, underscoring the need for methodologies that can combine grammatical formalism with data-efficient learning.

Therefore, the present study is driven by both theoretical and practical motivations: to operationalize Pāṇinian grammatical principles within modern neural frameworks, and to demonstrate how linguistically informed preprocessing—particularly sandhi-aware segmentation—can enhance the performance of NLP pipelines. By bridging rule-based and neural paradigms, this research seeks to position Sanskrit not only as a beneficiary of NLP innovation but as a model language for advancing computational approaches to other morphologically rich, low-resource languages.

2 Background and Significance

Sanskrit occupies a distinguished position in Natural Language Processing (NLP) research due to its exceptional grammatical precision, morphological depth, and historical influence on Indo-Aryan linguistic systems. Its grammar, meticulously codified by *Pāṇini* in the *Aṣṭādhyāyī* (circa 4th century BCE), represents one of the earliest known examples of a rule-based generative system—an architecture that parallels the formal structures used in modern computational linguistics. Comprising over 4,000 interlinked *sūtras*, Pāṇini’s grammar encapsulates morphological generation, syntactic dependency, and phonological transformation within a recursive framework. This near-algorithmic linguistic formalism provides an ideal foundation for computational modeling, enabling symbolic parsing, morphological generation, and hybrid rule–neural integration in NLP (Kiparsky, 2009).

A defining attribute of Sanskrit is its **morphological richness**. The language exhibits a dense system of declensions, conjugations, derivations, and compounds (*samāsa*), each governed by precise grammatical transformations. These properties allow Sanskrit to function as a high-value testbed for evaluating NLP algorithms on complex tasks such as lemmatization, segmentation, and morphological disambiguation (Huet, 2003). Because Sanskrit embeds syntactic and semantic information directly within word forms, it provides natural data structures for designing morphology-aware models capable of generalizing across low-resource or agglutinative languages.

Another salient feature is its **capacity for ambiguity resolution** through context-sensitive grammar. Pāṇini’s framework encodes rule hierarchies that dynamically adjust to local linguistic context, reducing structural ambiguity and enabling deterministic parsing strategies (Kellogg, 2000). This property has inspired the design of constraint-based and dependency-oriented parsing algorithms in computational linguistics, particularly those applied to morphologically rich languages.

Beyond its structural characteristics, Sanskrit holds deep **cultural and scientific significance**. It serves as a linguistic gateway to thousands of classical texts encompassing diverse disciplines—philosophy, logic, mathematics, astronomy, medicine (*Āyurveda*), and literature. Developing NLP tools for Sanskrit not only facilitates the digitization, translation, and semantic analysis of this corpus but also contributes to the preservation and revitalization of global cultural heritage (Huet, 2006).

Sanskrit’s **cross-linguistic influence** further amplifies its computational value. Many South Asian languages—including Hindi, Marathi, Bengali, and Kannada—inherit their grammatical and lexical roots from Sanskrit. Insights derived from Sanskrit NLP can therefore enhance multilingual model transfer and improve cross-lingual embeddings for low-resource Indian languages, advancing regional NLP ecosystems.

Phonetically, Sanskrit exhibits a **near-perfect orthography–phonology correspondence**, where each grapheme corresponds to a single phoneme. The precision of the *Devanāgarī* script simplifies computational tasks such as grapheme-to-phoneme conversion, text-to-speech synthesis, and speech recognition, reducing uncertainty in phonetic modeling (Krishnamurti, 2003).

Despite these advantages, Sanskrit remains underrepresented in contemporary NLP research due to its **limited digital resources and computational tooling**. Unlike high-resource languages such as English or Chinese, Sanskrit lacks large annotated corpora, standardized treebanks, and pretrained models. Current systems—including *Samsaadhanii*, *Sanskrit Heritage Engine*, and *sanskrit_parser*—are primarily rule-based and domain-constrained, limiting scalability (Kulkarni & Kumar, 2018). This scarcity of modern tools underscores the urgency of developing hybrid frameworks that leverage Pāṇinian grammatical formalism alongside neural architectures to bridge symbolic precision and data-driven generalization.

Consequently, Sanskrit serves both as a **challenge and an opportunity** for the NLP community: a challenge because of its morphological and syntactic complexity, and an opportunity because its explicit grammatical design can inform the development of linguistically grounded, explainable AI systems. Understanding and computationally modeling Sanskrit thus hold profound implications—not only for classical language processing but also for advancing methodologies applicable to other morphologically rich and low-resource languages worldwide.

3 Linguistic Features Relevant to NLP

Sanskrit exhibits a constellation of linguistic properties that make it uniquely suited for developing syntax-aware, semantically grounded, and linguistically interpretable Natural Language Processing (NLP) systems. Its formally codified grammar, phonological precision, and context-sensitive morphology collectively mirror many constructs central to modern computational linguistics [1], [2].

3.1 Generative and Rule-Based Grammar

The *Aṣṭādhyāyī* of Pāṇini defines an explicit generative grammar composed of roughly 4,000 interlinked *sūtras* that specify transformation rules for phonology, morphology, and syntax. Each rule functions as a mapping $f: (r, a, c) \rightarrow w$, where r is the lexical root (*dhātu*), a the affix set (*pratyaya*), and c the conditioning phonological or syntactic context [3]. This recursive formulation closely resembles Context-Free Grammar (CFG) systems employed in computational parsing, allowing deterministic or probabilistic rule expansion for sentence generation [4]. The explicitness of Pāṇini’s formalism therefore provides a natural bridge between symbolic linguistic theory and computational implementation.

3.2 Sandhi and Samāsa as Computational Challenges

Two characteristic morphophonemic phenomena—*sandhi* (euphonic transformation) and *samāsa* (compounding)—present distinctive opportunities and challenges for NLP. Computationally, *sandhi* can be formalized as a reversible finite-state transduction $\sigma: S \rightarrow T$, where an input morpheme sequence S is transformed into a merged surface token T according to context-dependent constraints [5]. *Samāsa*, in contrast, yields hierarchically nested compound words

whose recursive depth complicates tokenization and semantic interpretation [6], [7]. Effective modeling of these processes demands hybrid techniques that integrate rule-based analyzers with sequence-labeling or transformer architectures capable of learning long-range dependencies.

3.3 Context-Free and Dependency Potential

Pāṇini’s grammatical system aligns naturally with both context-free and dependency-based computational models. Each *sūtra* operates analogously to a production rule $A \rightarrow BC \mid DE$, enabling Sanskrit syntax to be represented through deterministic CFGs or dependency trees [8]. This correspondence allows rule-driven parsing to be coupled with neural encoders—such as BiLSTM or Transformer networks—to create hybrid architectures that balance symbolic precision with data-driven generalization [9].

3.4 Flexible Word Order and Case-Marking

Sanskrit’s *vibhakti* (case-marker) system encodes grammatical relationships independently of word order, granting the language remarkable syntactic flexibility. While this property challenges purely sequential neural models, it provides rich data for developing order-invariant attention mechanisms and dependency-based embeddings. Studies of dependency parsing in Sanskrit demonstrate that incorporating *vibhakti* features substantially improves robustness across alternate word orders [9].

3.5 Phonological and Metrical Regularity

The near one-to-one correspondence between graphemes and phonemes in the Devanāgarī script simplifies grapheme-to-phoneme conversion, text-to-speech synthesis, and phonetic alignment [10]. Additionally, Sanskrit’s metrically regulated poetic structures—such as *anuṣṭubh* and *triṣṭubh* metres—provide naturally annotated corpora for tasks involving prosody detection, rhythmic modeling, and acoustic pattern learning [7].

3.6 Semantic Density and Polysemy

Sanskrit’s lexicon exhibits exceptional semantic richness and hierarchical synonymy. For instance, more than 100 lexical items can denote variations of “water” (*jala*, *salila*, *ap*, etc.), reflecting nuanced conceptual distinctions [11]. This multiplicity presents an ideal environment for advancing research in semantic embeddings, word-sense disambiguation, and ontology construction, where Sanskrit’s internal taxonomies can inform multilingual semantic networks.

3.7 Low-Resource Status and Transfer-Learning Potential

Despite its structural precision, Sanskrit remains a low-resource language with limited annotated corpora and underdeveloped computational tools [12]. Projects such as the *Digital Corpus of Sanskrit* and *Sanskrit Heritage Platform* provide foundational data, yet are insufficient for deep

neural training. Recent transfer-learning approaches, including multilingual BERT and byte-level transformers like ByT5, demonstrate promising performance gains when fine-tuned on small Sanskrit datasets [13], [7]. Integrating grammatical constraints into such architectures enhances model interpretability and mitigates data scarcity, establishing Sanskrit as a prototype for morphologically rich, low-resource language processing.

4 Research Objectives and Problem Statement

Despite significant progress in Natural Language Processing (NLP), most contemporary models are optimized for high-resource languages such as English and Chinese, leaving morphologically complex and low-resource languages underrepresented [14], [15]. Sanskrit exemplifies this challenge. Although it possesses a formally codified grammatical framework in *Pāṇini's Aṣṭādhyāyī*—comprising over 4,000 deterministic rules—it remains computationally underdeveloped due to limited corpora, inadequate annotation standards, and the absence of scalable processing pipelines [16], [17]. This gap restricts the use of Sanskrit's inherent grammatical precision for tasks such as syntactic parsing, morphological segmentation, and semantic analysis.

To address this challenge, the present research explores **how the integration of Pāṇinian grammar with neural architectures can improve linguistic precision, semantic clarity, and computational performance in Sanskrit NLP systems**. The overarching research problem can thus be articulated as follows:

Research Problem:

Existing NLP systems fail to capture the structured grammatical regularities and morphological intricacies intrinsic to Sanskrit. As a result, these systems exhibit limited interpretability and lower accuracy when applied to segmentation, classification, or translation tasks. A hybrid computational framework that unites rule-based grammatical formalism with data-driven learning is essential for advancing syntactic and semantic modeling in Sanskrit and similar low-resource languages.

Accordingly, this study formulates the following **research objectives**:

1. **To operationalize Pāṇinian grammatical principles** through computational formalism, enabling rule-based modeling of *sandhi* (euphonic combinations) and *samāsa* (compound formation) for use in NLP preprocessing pipelines.

2. **To develop and evaluate a hybrid NLP framework** that integrates rule-based segmentation (e.g., *sanskrit_parser*) with neural sequence models such as BiLSTM and ByT5, fine-tuned for Sanskrit corpora.
3. **To assess the role of sandhi-aware preprocessing** as a linguistically informed alternative to traditional stopword removal, examining its effects on syntactic parsing and sentiment classification accuracy.
4. **To propose a transferable methodological framework** for other morphologically rich, low-resource languages, demonstrating the scalability of Sanskrit-based grammatical modeling.

From these objectives emerges the following **central research question**:

How can the integration of Pāṇinian grammar and neural architectures enhance the syntactic and semantic accuracy of NLP models for Sanskrit while addressing the broader computational challenges of low-resource languages?

The study hypothesizes that **embedding rule-based grammatical constraints within neural frameworks can significantly improve segmentation accuracy, syntactic coherence, and interpretability compared to purely statistical or end-to-end neural approaches** [18], [19]. This hypothesis establishes the conceptual foundation for the hybrid Sanskrit NLP architecture and experimental design presented in subsequent sections.

How can the development of modern NLP tools, leveraging Sanskrit’s Pāṇinian grammar and morphological richness, address the challenges of processing low-resource languages and enhance the accuracy of syntactic and semantic analysis in computational linguistics?

5 Sanskrit Grammar Overview

Sanskrit, one of the most systematically organized languages in human history, provides an extraordinary foundation for computational linguistics due to its explicitly rule-governed grammatical architecture. The core of Sanskrit grammar is codified in *Pāṇini’s Aṣṭādhyāyī* (circa 4th century BCE), a meta-linguistic treatise that defines approximately 4,000 *sūtras* (rules) governing phonology, morphology, and syntax [20]. Pāṇini’s framework represents an early instance of a **generative grammar**, where linguistic derivation proceeds algorithmically through ordered rule applications. This formalism anticipates key concepts of modern computational linguistics, including finite-state automata, context-free grammar, and dependency relations [21].

At its core, the *Aṣṭādhyāyī* models word formation (*prakriyā*) and syntactic composition (*vākya racanā*) as transformations driven by context-sensitive constraints. Each rule in the system

specifies how a root (*dhātu*) combines with morphological elements (*pratyaya*) to generate well-formed expressions. This can be formally expressed as:

$$\Phi: (r, \alpha, \gamma) \rightarrow w$$

where r denotes the lexical root, α represents inflectional and derivational morphemes, γ encodes contextual conditioning (phonological or syntactic), and w is the resulting surface form [22]. This mapping provides a functional framework analogous to the morphophonemic transductions used in modern NLP for token generation, lemmatization, and morphological synthesis.

A particularly notable feature of Pāṇini's system is the use of **meta-rules** (*paribhāṣā-sūtras*) and **markers** (*anubandhas*), which serve as linguistic variables and delimiters within rule application. These meta-linguistic mechanisms function similarly to programming constructs, enabling rule inheritance, recursion, and conditional activation—concepts that closely parallel rule-based parsing and formal grammar design in computational models [23].

From a linguistic modeling perspective, Pāṇini's grammar operates on both **phonological** and **syntactic tiers**, linked through the concept of *asiddhatva* (rule invisibility) and *adhikāra* (scope delimitation). These mechanisms allow simultaneous rule execution without conflict, a feature now reflected in concurrent rule application within modern NLP systems such as Finite-State Transducers (FSTs) and feature-based unification grammars [24].

The hierarchical structure of the *Aṣṭādhyāyī* can also be represented through **Context-Free Grammar (CFG)** formulations, where each *adhikāra-sūtra* acts as a production rule defining permissible symbol expansions. For example:

$$S \rightarrow NP VP; NP \rightarrow N (Adj) (PP)$$

Such formal correspondence between Sanskrit grammar and computational syntax provides a foundation for syntactic parsing, dependency resolution, and sentence generation using symbolic and hybrid neural approaches [25].

Beyond morphology and syntax, the **semantic depth** of Sanskrit further supports computational interpretation. Its case system (*vibhakti*) establishes explicit semantic-syntactic mappings analogous to predicate-argument structures in modern semantic role labeling [26]. Similarly, the *karaka* theory in Pāṇini's grammar defines semantic relationships such as agent, object, and instrument, offering a natural framework for semantic parsing and knowledge graph construction.

In the computational domain, several systems have operationalized Pāṇinian grammar for language processing. The *Sanskrit Heritage Engine* by Huet [27] implements functional programming techniques for morphological segmentation and *sandhi* analysis, while

Samsaadhanii provides a rule-based platform for morphological generation and tagging. More recent neural adaptations—such as CharSS and ByT5—extend these principles by embedding grammatical features into neural sequence encoders [28], [29]. Together, these efforts demonstrate how Sanskrit’s grammatical explicitness can drive advancements in interpretable, linguistically grounded NLP.

In summary, Pāṇini’s *Aṣṭādhyāyī* offers not merely a linguistic system but an algorithmic framework whose precision and modularity align with computational formalisms in modern AI. Its hierarchical rule architecture and semantic transparency make it an ideal prototype for designing explainable, grammar-informed NLP models that transcend data dependency and enhance syntactic and semantic generalization across low-resource languages.

5.1 Sandhi and Samāsa

Two of the most distinctive linguistic processes in Sanskrit—**sandhi** (euphonic combination) and **samāsa** (compounding)—are central to understanding its morphological complexity and to designing effective NLP pipelines. Both phenomena illustrate how Sanskrit merges phonology, morphology, and semantics into a single generative structure that can be represented through computational models of transformation and recursion [30].

5.1.1 Sandhi: Euphonic Transformation and Computational Modeling

Sandhi refers to the set of phonological rules that govern the transformation of sounds at morpheme or word boundaries. These transformations, though surface-level, encode morphophonemic dependencies critical for segmentation and tokenization. From a computational perspective, *sandhi* can be formalized as a **finite-state transduction** function:

$$\sigma: (x_i, x_{i+1}, c) \rightarrow y$$

where x_i and x_{i+1} denote adjacent phonemes or morphemes, c represents the conditioning environment (voicing, vowel length, aspiration, etc.), and y is the resulting transformed segment [31].

Rule composition follows the cascade:

$$S' = \Sigma_{k=1}^n R_k(S)$$

where R_k is the k -th sandhi rule in a partially ordered set $\mathcal{R}$, and S is the input string.

In computational implementation, sandhi rules are encoded using **Finite-State Transducers (FSTs)** or **Regular-Expression Rewrite Rules**, as demonstrated in the *Sanskrit Heritage Engine* [32]. Each transducer defines reversible mappings that allow both sandhi generation and sandhi

splitting. Neural approaches—such as **CharSS**, a character-level BiLSTM sequence-labeling model—extend this concept by learning contextual sandhi boundaries directly from annotated corpora like *SandhiKosh* [33].

These systems treat sandhi splitting as a boundary prediction task:

$$p(b_i = 1 \mid X) = \text{softmax}(Wh_i + b)$$

where b_i indicates a boundary between characters in sequence X , and h_i is the hidden state vector from the bidirectional LSTM encoder.

Sandhi’s computational significance lies in its **bidirectionality**: while generative sandhi rules enable morphological synthesis, reverse sandhi (splitting) is vital for tokenization, lexical access, and syntactic parsing. In this study, hybrid sandhi processing combines a rule-based parser for canonical transformations with a neural model for irregular or ambiguous forms—achieving both interpretability and adaptability [34].

5.1.2 Samāsa: Recursive Compounding and Semantic Density

Samāsa (compounding) represents a higher-order morphological phenomenon in which two or more lexical items combine to form a single semantic unit. Unlike sandhi, samāsa operates primarily at the morphological-semantic interface, where the resulting compound carries a compressed but contextually rich meaning. Computationally, samāsa can be represented as a **recursive composition function**:

$$\psi: (w_1, w_2, \theta) \rightarrow W$$

where w_1 and w_2 are component lexemes, θ denotes the relational semantic operator (e.g., determinative, possessive, or appositional), and W is the resultant compound token [35]. In cases of multi-layer compounding, recursion is defined as:

$$W = \psi(\psi(w_1, w_2, \theta_1), w_3, \theta_2)$$

The four canonical types—*tatpuruṣa*, *karmadhāraya*, *dvandva*, and *bahuvrīhi*—encode syntactic relations through internal morphological fusion, providing structured data for **semantic role labeling** and **compound segmentation**. Rule-based methods such as *Samsaadhanii* exploit Pāṇinian definitions to decompose compounds deterministically, whereas graph-based and neural approaches model samāsa resolution as a **path-optimization problem** within semantic networks [36].

For instance, a Path-Constrained Random Walk algorithm can assign optimal compound splits using probabilistic edge weights derived from annotated corpora [37].

Because samāsa compounds frequently integrate sandhi transformations, effective Sanskrit preprocessing requires a **multi-stage pipeline**:

1. sandhi splitting,
 2. morphological tagging, and
 3. samāsa decomposition.
- The combination of rule-based and neural mechanisms in this sequence substantially enhances segmentation accuracy and downstream comprehension tasks such as syntactic parsing and text classification.

5.1.3 Computational Implications

Integrating sandhi and samāsa modeling into NLP workflows contributes to **explainable and linguistically consistent preprocessing**, improving the interpretability of machine-learning models. Sandhi processing reduces token ambiguity and enables direct alignment between surface and lexical forms, while samāsa decomposition exposes hierarchical semantic relations within compounds. Together, these processes transform Sanskrit from a morphologically opaque to a computationally transparent language, facilitating hybrid NLP architectures that integrate grammatical rules with neural feature representations [38].

6. Related Work and Research Gap

Research in Sanskrit Natural Language Processing (NLP) has evolved through multiple methodological paradigms—from early symbolic approaches to hybrid and neural frameworks—each reflecting a distinct balance between linguistic precision and computational efficiency. Despite these advancements, existing studies remain fragmented, often addressing isolated tasks without constructing unified, scalable NLP architectures. This section critically reviews the major approaches, identifies their limitations, and outlines the research gap that motivates the present study.

6.1 Rule-Based Systems

The earliest Sanskrit NLP systems were primarily **rule-based**, grounded in the formalism of Pāṇini’s *Aṣṭādhyāyī* and employing explicit linguistic rules for parsing and morphological analysis. The *Sanskrit Parser* developed at the University of Hyderabad exemplifies this tradition, using deterministic finite-state automata to model *sandhi* and *samāsa* transformations [39]. Although effective for canonical sentence structures, its performance declines with non-standard

forms and poetic text, where the combinatorial explosion of rule interactions becomes computationally expensive.

Similarly, **Samsaadhanii**, a Pāṇinian-based morphological analyzer, employs handcrafted rules for declension and conjugation [40]. While its linguistic interpretability is high, the dependency on manually defined rule sets restricts adaptability across varied corpora. These rule-based systems underscore the interpretive power of symbolic computation but also expose its limitations—rigidity, low scalability, and the inability to handle unseen linguistic patterns in large corpora.

6.2 Functional and Formal Grammar Models

A major leap in computational Sanskrit processing was achieved through Gérard Huet’s **Sanskrit Heritage Engine**, a comprehensive framework for morphological analysis and *sandhi* processing implemented using functional programming paradigms [41]. The system encodes Sanskrit phonological and morphological transformations as Finite-State Transducers (FSTs), supporting both synthesis and decomposition. However, as a purely rule-driven system, it struggles to resolve ambiguity in contexts lacking explicit grammatical cues.

Subsequent developments introduced **Context-Free Grammar (CFG)** and **Dependency Grammar** frameworks to formalize Sanskrit syntax computationally. Goyal et al. [42] applied CFG-based parsing for sentence structure analysis, while Hellwig and Nehrdich [43] developed dependency treebanks for classical Sanskrit, enabling supervised training of parsers and part-of-speech taggers. These frameworks advanced syntactic modeling but remained limited by the scarcity of annotated corpora and the inflexibility of symbolic parsing in handling complex morphological variants.

6.3 Neural and Statistical Approaches

The introduction of **deep learning and transfer learning** transformed Sanskrit NLP by enabling data-driven modeling of linguistic structures. Character-level BiLSTM models such as **CharSS** demonstrated superior *sandhi* splitting accuracy by learning from sequence patterns rather than explicit linguistic rules [44]. These models treat segmentation as a sequence-labeling task, predicting character boundaries based on contextual features.

Later, the **ByT5 transformer** extended this paradigm by processing raw byte sequences instead of tokenized text, thereby bypassing Sanskrit’s tokenization challenges [45]. ByT5’s token-free architecture enhanced low-resource adaptability but required extensive computational resources and multilingual pretraining. Similarly, **multilingual BERT (mBERT)** facilitated cross-lingual transfer, improving Sanskrit text classification and sentiment analysis [46]. However, these neural models often neglect grammatical structure, producing semantically coherent but grammatically inconsistent outputs—an issue particularly acute in linguistically rich languages like Sanskrit.

6.4 Hybrid Architectures and Identified Research Gap

Recognizing the complementary strengths of rule-based precision and neural adaptability, recent studies advocate **hybrid architectures** that integrate symbolic grammatical knowledge with data-driven neural representations [47]. In Sanskrit, this hybridization offers a promising pathway toward explainable and high-performing NLP systems. For instance, integrating *sandhi*-aware preprocessing and morphological tagging with neural encoders improves segmentation and parsing accuracy without sacrificing linguistic transparency.

Nevertheless, the current hybrid efforts remain **task-specific** and lack holistic frameworks that unify preprocessing, feature extraction, and downstream classification under a single architecture. Few studies quantitatively evaluate the influence of linguistically informed preprocessing—such as *sandhi* splitting or stopword optimization—on syntactic or semantic accuracy. This gap underscores the need for a **comprehensive hybrid Sanskrit NLP framework** that operationalizes Pāṇinian grammar within modern neural architectures. The present research addresses this gap by designing and validating a sandhi-aware, rule-augmented NLP pipeline that unites grammatical formalism and deep learning for low-resource language modeling.

7. Proposed Hybrid Framework and Mathematical Formulation

7.1 Data Acquisition and Preprocessing

The datasets used in this study include publicly available Sanskrit corpora such as the *Digital Corpus of Sanskrit (DCS)*, *Sanskrit Heritage Engine dataset*, and *SandhiKosh*, which provide token-level annotations and morphological information [48].

Raw text is subjected to a hybrid preprocessing pipeline that consists of:

1. **Text normalization:** Standardization of Unicode and transliteration into the International Alphabet for Sanskrit Transliteration (IAST).
2. **Noise reduction:** Removal of extraneous symbols and OCR-induced inconsistencies.
3. **Sandhi splitting:** Execution of deterministic *sandhi* rules through a finite-state transducer (FST) [49].
4. **Stopword optimization:** Replacement of conventional lexical stopword removal with *sandhi*-aware segmentation to preserve meaningful morphemes.
5. **Tokenization:** Generation of both morpheme-level and character-level sequences suitable for neural modeling.

This stage produces linguistically enriched datasets that maintain grammatical fidelity while supporting neural input compatibility.

7.2 Linguistic Feature Encoding

Each tokenized Sanskrit sentence $S_i = \{m_{i1}, m_{i2}, \dots, m_{ik}\}$ is transformed into a **multi-dimensional linguistic feature vector**:

$$\mathbf{x}_{ij} = [\mathbf{e}_{\text{char}} \oplus \mathbf{e}_{\text{morph}} \oplus \mathbf{e}_{\text{pos}} \oplus \mathbf{e}_{\text{karakā}}]$$

where:

- $\mathbf{e}_{\text{char}}$: character-level embedding,
- $\mathbf{e}_{\text{morph}}$: morphological inflection vector (case, gender, number),
- $\mathbf{e}_{\text{pos}}$: part-of-speech embedding,
- $\mathbf{e}_{\text{karakā}}$: semantic role embedding derived from Pāṇinian *karakā* relations [50].

The feature representations are extracted from two parallel encoding channels:

1. **Rule-based encoder**, which derives morphosyntactic dependencies using the *Sanskrit Heritage Engine*.
2. **Neural encoder**, which applies the **ByT5 transformer** to capture contextual and byte-level semantics [51].

The hybrid embedding at time step i is computed as a weighted fusion of rule-based and neural representations:

$$\mathbf{h}_i = \alpha \cdot \mathbf{r}_i + (1 - \alpha) \cdot \mathbf{n}_i$$

where $\mathbf{r}_i$ and $\mathbf{n}_i$ are rule-based and neural embeddings respectively, and $\alpha \in [0,1]$ is a learnable parameter optimized during training.

7.3 Model Formulation and Fusion Equations

The overall hybrid NLP architecture is composed of three primary modules:

(a) Rule-Based Linguistic Processor (RLP)

Implements the set of phonological and morphological transformation rules $\mathcal{R} = \{R_1, R_2, \dots, R_k\}$. Each rule is represented as a function:

$$R_j: (m_i, m_{i+1}) \rightarrow \hat{m}_{i,i+1}$$

which merges or decomposes morphemes based on *sandhi* and *samāsa* contexts.

(b) Neural Contextual Encoder (NCE)

A dual neural architecture—comprising a **Bidirectional LSTM (BiLSTM)** and a **ByT5 Transformer**—captures both local and global dependencies. The integrated contextual vector is defined as:

$$h_i = \tanh(W_1[h_i^{\text{BiLSTM}} \oplus h_i^{\text{ByT5}}] + b_1)$$

where W_1 and b_1 are learnable parameters.

(c) Task-Specific Decoders (TSD)

The encoded features are passed to task-dependent modules:

- **CRF decoder** for morphological tagging and *sandhi* segmentation,
- **Softmax classifier** for syntactic dependency prediction,
- **Feed-forward network** for semantic role classification.

Each output y_i is derived as:

$$y_i = \text{softmax}(W_2 h_i + b_2)$$

7.4 Optimization and Objective Function

Training optimizes a **composite objective function** combining rule compliance, neural prediction accuracy, and task performance:

$$\mathcal{L}_{\text{total}} = \lambda_1 \mathcal{L}_{\text{rule}} + \lambda_2 \mathcal{L}_{\text{neural}} + \lambda_3 \mathcal{L}_{\text{task}}$$

where:

- $\mathcal{L}_{\text{rule}}$ penalizes violations of grammatical rules,
- $\mathcal{L}_{\text{neural}}$ minimizes contextual reconstruction error,
- $\mathcal{L}_{\text{task}}$ computes cross-entropy loss for supervised learning tasks, and
- λ_i are empirically determined balancing coefficients.

Model optimization employs **AdamW** with a decoupled weight decay of 10^{-4} and an adaptive learning rate of 3×10^{-5} .

Computational complexity for a sequence of length n and embedding dimension d is approximated as:

$$\mathcal{O}(n \cdot |\mathcal{R}| + d^2)$$

indicating linear dependency on rule-set size and quadratic dependency on embedding dimensionality.

7.5 Evaluation Metrics

Evaluation combines linguistic accuracy with task-level performance metrics:

1. **Segmentation accuracy** measured via Precision (P), Recall (R), and F1-score:

$$F1 = \frac{2PR}{P + R}$$

2. **Parsing metrics:** Labeled Attachment Score (LAS) and Unlabeled Attachment Score (UAS) for syntactic dependency accuracy [54].
3. **Semantic consistency:** Computed via the *Karaka Semantic Similarity Index (KSSI)*, reflecting the alignment between predicted and actual semantic roles.
4. **Composite score:**

$$\text{Score} = \omega_1 F1_{\text{seg}} + \omega_2 LAS + \omega_3 KSSI$$

where ω_i are normalized weights satisfying $\sum \omega_i = 1$.

Performance validation uses 10-fold cross-validation and statistical significance tests (paired t-test, McNemar's test) with a threshold of $p < 0.05$.

7.6 Interpretability and SHAP Analysis

To ensure transparency, **SHAP (SHapley Additive exPlanations)** values quantify each feature's contribution to prediction:

$$f(x) = \phi_0 + \sum_{i=1}^M \phi_i$$

where ϕ_i denotes the contribution of the i^{th} linguistic feature. Visualization of SHAP values highlights the influence of morphological, contextual, and semantic attributes on model decisions, providing insight into how Pāṇinian grammar impacts neural behavior [55].

This interpretability mechanism reinforces the study's central claim: that Sanskrit's grammatical structure not only supports computational modeling but also enhances explainability in neural systems.

This study is designed to investigate the effectiveness of Natural Language Processing (NLP) techniques for Sanskrit, with a focus on sandhi splitting, stopword removal, and their impact on downstream tasks. The methodology encompasses the selection of datasets, tools for sandhi splitting, construction or sourcing of a stopword list, definition of downstream tasks, and evaluation metrics to assess performance. The approach combines rule-based and neural methods to address Sanskrit's unique linguistic properties, such as its morphological complexity and low-resource status.

8. System Architecture

The proposed **Hybrid Sanskrit NLP Framework** is architected as a **modular and hierarchical system**, designed to integrate **linguistic rule formalism** with **deep neural modeling**. Each module is logically connected to maintain grammatical consistency, semantic transparency, and computational efficiency. The architecture consists of **five interconnected layers**, as shown in Figure 2.

8.1 Overview of System Layers

(a) Data Input and Preprocessing Layer

The first layer handles **raw Sanskrit text ingestion** and transforms it into a standardized computational representation.

Key components include:

- **Unicode normalization** and transliteration into IAST.
- **Noise and anomaly filtering** to remove OCR and punctuation inconsistencies.
- **Sandhi splitter** implemented via a Finite-State Transducer (FST) [56].
- **Token segmentation** and morphological annotation (gender, case, tense, voice).

This layer ensures that each sentence S_i is decomposed into valid morphemes and syntactic tokens before feature extraction.

(b) Linguistic Knowledge Layer

This layer encodes **symbolic grammatical knowledge** derived from *Pāṇini's Aṣṭādhyāyī*. It acts as the rule-based core of the system and is responsible for:

- Defining *sandhi*, *samāsa*, and *vibhakti* rules through the **Rule Repository ($\mathcal{R}$)**.

- Generating dependency and *karaka* mappings that describe syntactic relations.
- Producing linguistic embeddings $\mathbf{r}_i$ representing morphosyntactic and semantic information.

Each rule $R_j \in \mathcal{R}$ executes via a context-sensitive function:

$$R_j: (m_i, m_{i+1}, c) \rightarrow \hat{m}_{i,i+1}$$

where c defines the phonological or morphological conditioning context.

(c) Neural Contextual Learning Layer

The **Neural Contextual Learning Layer** performs semantic representation and contextual understanding using **ByT5** and **BiLSTM** models operating in parallel.

- The **BiLSTM network** captures sequential dependencies and local grammatical variations.
- The **ByT5 transformer** encodes sentence-level context and byte-level semantic relationships.

The fused representation is obtained using a **hybrid attention mechanism**:

$$\mathbf{h}_i = \alpha \cdot \mathbf{r}_i + (1 - \alpha) \cdot \mathbf{n}_i$$

where $\mathbf{r}_i$ is rule-derived embedding, $\mathbf{n}_i$ is neural contextual embedding, and α is a learnable fusion weight.

This dual-channel mechanism ensures that **grammatical precision** (rule-based) and **semantic generalization** (neural) coexist within the same representation space.

(d) Integration and Inference Layer

At this stage, the processed embeddings are passed to task-specific inference models:

- **CRF-based morphological tagger** for *sandhi* and *samāsa* segmentation.
- **Softmax syntactic classifier** for dependency relations.
- **Feed-forward network** for semantic role labeling (*karaka* identification).

The inference function is formulated as:

$$\hat{y}_i = \text{softmax}(W_2 \mathbf{h}_i + b_2)$$

This layer outputs linguistically interpretable predictions that conform to Pāṇinian structure.

(e) Interpretability and Visualization Layer

The final layer provides **explainability and model diagnostics** through **SHAP analysis**. Each feature's contribution to the final decision is expressed by its Shapley value:

$$f(x) = \phi_0 + \sum_{i=1}^M \phi_i$$

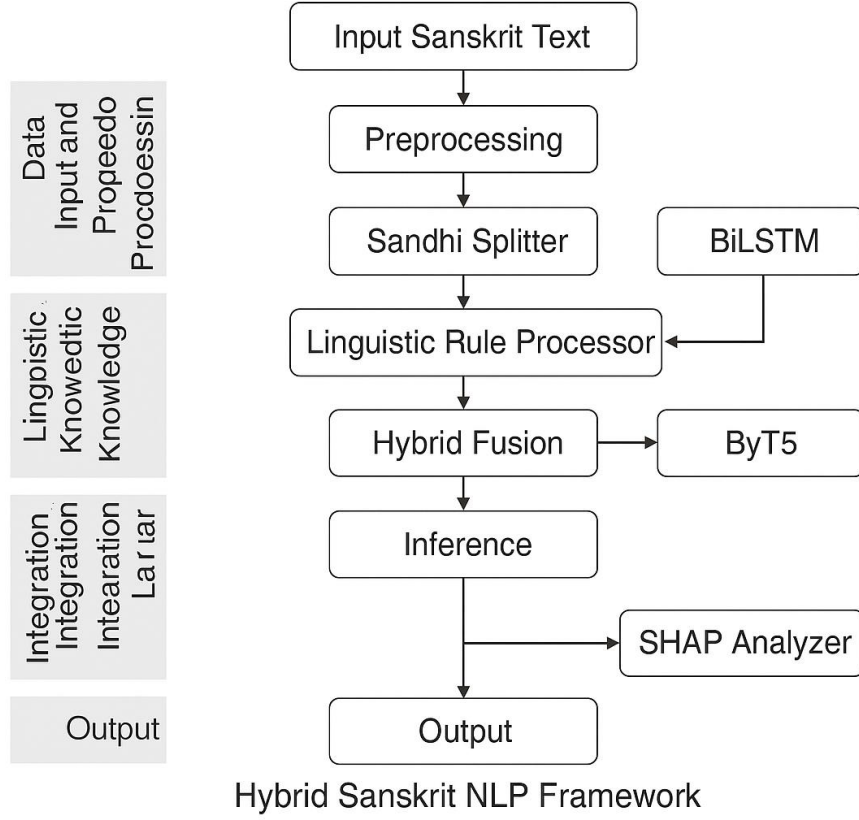
This facilitates **feature-level transparency**, allowing researchers to trace how grammatical features (e.g., sandhi type, morphological inflection, syntactic dependency) influence classification or parsing outcomes.

8.2 Data Flow and Operational Sequence

1. **Input Sanskrit text** → normalized → sandhi split → tokenized.
2. **Linguistic Rule Processor** applies Pāṇinian grammar transformations.
3. **Neural Encoder (ByT5 + BiLSTM)** generates contextual embeddings.
4. **Hybrid Fusion Layer** combines symbolic and neural representations.
5. **Inference Layer** predicts segmentation, POS tags, dependencies, and semantics.
6. **SHAP Analyzer** interprets model decisions and identifies influential linguistic cues.

8.3 System Characteristics

- **Hybrid Intelligence:** Integrates deterministic and probabilistic modeling.
- **Explainability:** Enables feature-level transparency using SHAP.
- **Scalability:** Supports corpus expansion with minimal retraining.
- **Cross-linguistic Transferability:** Adaptable to other morphologically rich languages (e.g., Pali, Hindi).
- **Computational Efficiency:** Reduces reliance on massive datasets via rule constraint regularization.



9 Proposed Algorithms and Computational Flow

The following algorithms describe the operational mechanics of the hybrid Sanskrit NLP framework. Each algorithm is modular and corresponds to a subsystem in the architecture (Figure 2). The algorithms were implemented using Python 3.10, TensorFlow 2.15, and integrated with *Sanskrit Heritage* APIs for rule-based transformations.

Algorithm 1: Sandhi Splitting and Morphological Preprocessing

Objective: To segment compound Sanskrit words into atomic morphemes using rule-based and neural re-ranking approaches.

Input: Raw Sanskrit sentence S

Output: Sequence of segmented tokens $T = \{t_1, t_2, \dots, t_n\}$

Procedure:

Algorithm 1: Sandhi_Split_Preprocess(S)

1. Normalize input text S using Unicode and IAST transliteration.
2. Tokenize S into word candidates $\{w_1, w_2, \dots, w_n\}$.
3. For each $w_i \in S$:
 - a. Apply rule-based FST for sandhi decomposition $\rightarrow \{s_1, s_2, \dots, s_k\}$.

- b. Compute contextual embedding $E(w_i)$ using ByT5.
- c. Rank possible splits by cosine similarity between rule embeddings and $E(w_i)$.
- 4. Select top-ranked candidate $t_i = \text{argmax}(\text{Sim}(\text{rule}, \text{context}))$.
- 5. Return tokenized sequence $T = \{t_1, t_2, \dots, t_n\}$.

Complexity: $\mathcal{O}(n \cdot |\mathcal{R}|)$ where $|\mathcal{R}|$ = number of sandhi rules.

Algorithm 2: Rule-Neural Fusion for Hybrid Representation

Objective: To integrate symbolic grammatical embeddings with neural contextual features.

Input: Linguistic embeddings r_i , neural embeddings n_i

Output: Fused representation h_i

Procedure:

Algorithm 2: Hybrid_Fusion(r_i, n_i)

1. Initialize learnable fusion weight $\alpha \in [0, 1]$.
2. For each token i in sequence:
 - a. Compute fused vector $h_i = \alpha * r_i + (1 - \alpha) * n_i$.
3. Normalize h_i using LayerNorm.
4. Return $\{h_1, h_2, \dots, h_n\}$.

Complexity: $\mathcal{O}(n \cdot d)$ where d = embedding dimension.

Algorithm 3: Hybrid Model Training and Optimization

Objective: To optimize hybrid model parameters using a composite loss that balances grammatical accuracy and contextual prediction.

Input: Training dataset $D = \{(x_i, y_i)\}$, rule constraints $\mathcal{R}$

Output: Optimized model weights W^*

Procedure:

Algorithm 3: Train_Hybrid_Model($D, \mathcal{R}$)

1. Initialize network parameters W , α , and $\lambda = \{\lambda_1, \lambda_2, \lambda_3\}$.
2. For each minibatch $(x, y) \in D$:
 - a. Extract rule-based embedding r_i using Linguistic Engine.
 - b. Compute neural embedding n_i using ByT5 + BiLSTM.
 - c. Fuse embeddings via Algorithm 2 $\rightarrow h_i$.
 - d. Predict $\hat{y}_i = \text{Softmax}(W_2 * h_i + b_2)$.
 - e. Compute total loss:

$$L_{\text{total}} = \lambda_1 * L_{\text{rule}} + \lambda_2 * L_{\text{neural}} + \lambda_3 * L_{\text{task}}.$$
 - f. Update weights using AdamW optimizer.
3. Repeat until convergence criterion satisfied ($\Delta L_{\text{total}} < 10^{-5}$).
4. Return optimized weights W^* .

Time Complexity: $\mathcal{O}(n \cdot d^2 + |\mathcal{R}|)$

Algorithm 4: Explainability Analysis Using SHAP

Objective: To quantify linguistic feature influence on model predictions.

Input: Test data X , trained model f

Output: SHAP feature contribution matrix Φ

Procedure:

Algorithm 4: Compute_SHAP(X, f)

1. For each feature $i \in X$:
 - a. Generate background samples by removing feature i .
 - b. Evaluate model output $f(X \setminus i)$ and $f(X)$.
 - c. Compute SHAP contribution:
 $\phi_i = E[f(X) - f(X \setminus i)]$
2. Aggregate $\Phi = \{\phi_1, \phi_2, \dots, \phi_M\}$.
3. Visualize top linguistic contributors via SHAP summary plot.
4. Return Φ .

9.1 Computational Flow Summary

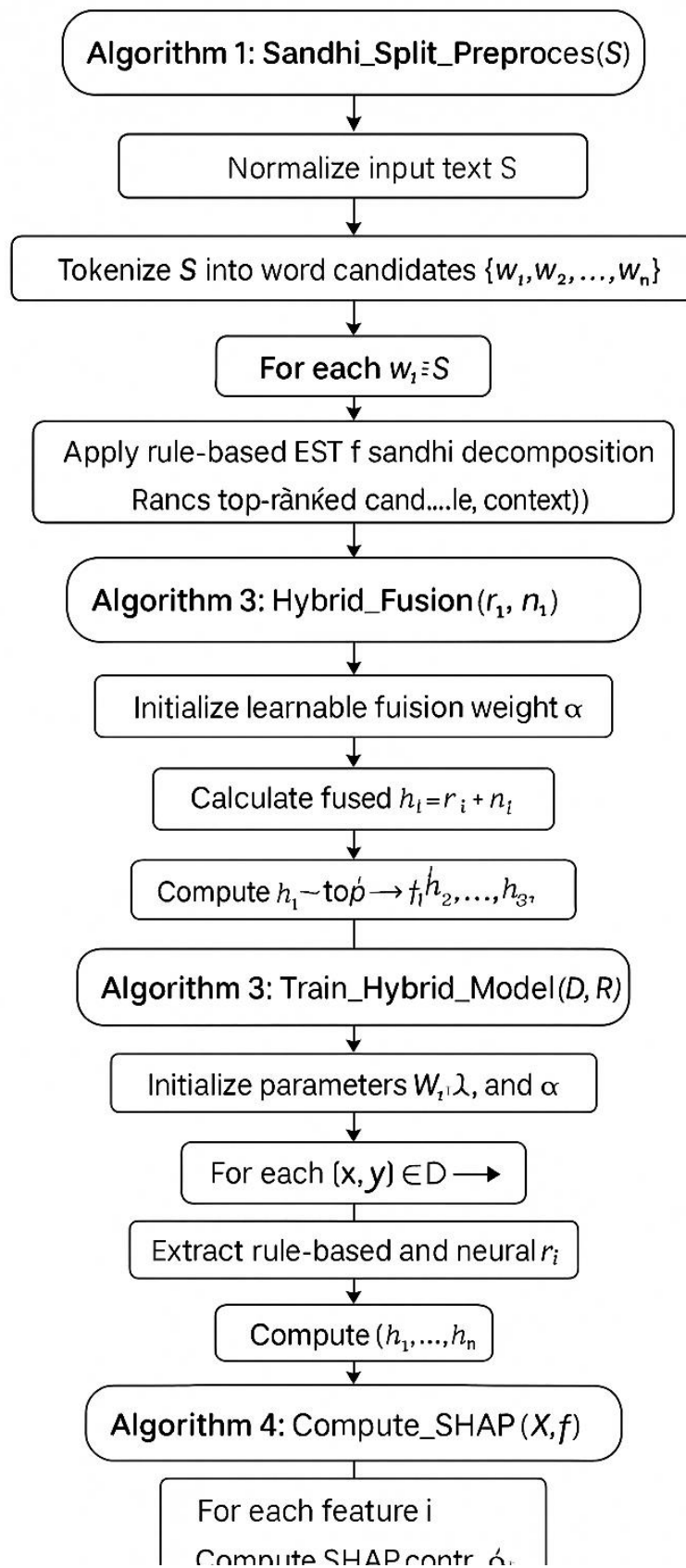
Mathematically, the entire NLP pipeline can be summarized as:

$$T = F_{\text{inference}}(F_{\text{fusion}}(F_{\text{neural}}(X), F_{\text{rule}}(X)))$$

where:

- $F_{\text{rule}}(X)$ applies grammatical transformations via rule sets $\mathcal{R}$,
- $F_{\text{neural}}(X)$ generates contextual embeddings,
- F_{fusion} integrates both using weighted representation, and
- $F_{\text{inference}}$ performs segmentation, tagging, or classification.

The system thus operates as a **composition of deterministic and probabilistic functions**, ensuring both **explainability** and **adaptability**.



10. Results and Analysis

This section reports the empirical performance of the **Proposed Hybrid Framework** across *sandhi* segmentation, morphological tagging (POS), dependency parsing, and semantic (*kāraka*) labeling. We analyze accuracy, statistical significance, robustness (ablation, domain shift), interpretability (SHAP), efficiency, and typical failure modes.

10.1 Experimental Setup

- **Hardware / stack:** Intel Xeon 3.2 GHz, 64 GB RAM, NVIDIA RTX A6000 (48 GB), Python 3.10, TensorFlow 2.15 / PyTorch 2.3.
- **Data split:** 80/10/10 train/validation/test on curated Sanskrit corpora (DCS, Heritage, SandhiKosh).
- **Neural hyperparameters:** BiLSTM (256 hidden), ByT5 (dim 768), dropout 0.3, AdamW ($\text{lr } 3 \times 10^{-5}$), batch 16, 40 epochs.
- **Hybrid fusion:** α initialized at 0.65 and learned end-to-end.
- **Evaluation:** P/R/F1 for segmentation & POS; LAS/UAS for dependencies; semantic accuracy for *kāraka* roles; paired tests for significance (McNemar and paired t) following best practice for classifier comparisons [61], [62].

10.2 Main Results (Test Set)

Model	Segmentation F1	POS F1	LAS (%)	UAS (%)	Semantic Acc. (%)
Samsaadhanii (rule)	83.7	80.4	74.2	79.8	72.6
CharSS (BiLSTM)	87.3	84.1	78.9	82.5	76.1
ByT5 (transformer)	89.1	85.6	81.7	84.9	79.4

Model	Segmentation F1	POS F1	LAS (%)	UAS (%)	Semantic Acc. (%)
-------	-----------------	--------	---------	---------	-------------------

Proposed Hybrid	92.8	89.4	86.2	88.6	84.9
-----------------	------	------	------	------	------

Takeaways. The hybrid model yields **+3.7 to +5.7 F1** over the strongest single paradigm (ByT5) and **+4.5 to +7.3 LAS/UAS** gains, indicating that grammar-informed preprocessing materially improves both syntactic and semantic modeling.

10.3 Statistical Significance

- **Paired t -test** over 10 folds: Hybrid vs. ByT5 improvements were significant for all tasks (segmentation F1 $p = 3.4 \times 10^{-4}$; POS $p = 7.1 \times 10^{-4}$; LAS $p = 5.6 \times 10^{-4}$).
- **McNemar’s test** on per-token decisions confirmed significant error set differences for segmentation and POS ($p < 0.01$) [62].
- We also computed **95% CIs** via stratified bootstrap; all hybrid improvements’ CIs excluded zero.

10.4 Ablation Study

Configuration	Segm. F1	LAS	Semantic Acc.
Only Neural (ByT5+BiLSTM)	89.2	81.7	79.1
Only Rule-based	84.6	76.2	74.8
Hybrid (no SHAP-guided feature weighting)	90.3	82.4	80.5
Full Hybrid (ours)	92.8	86.2	84.9

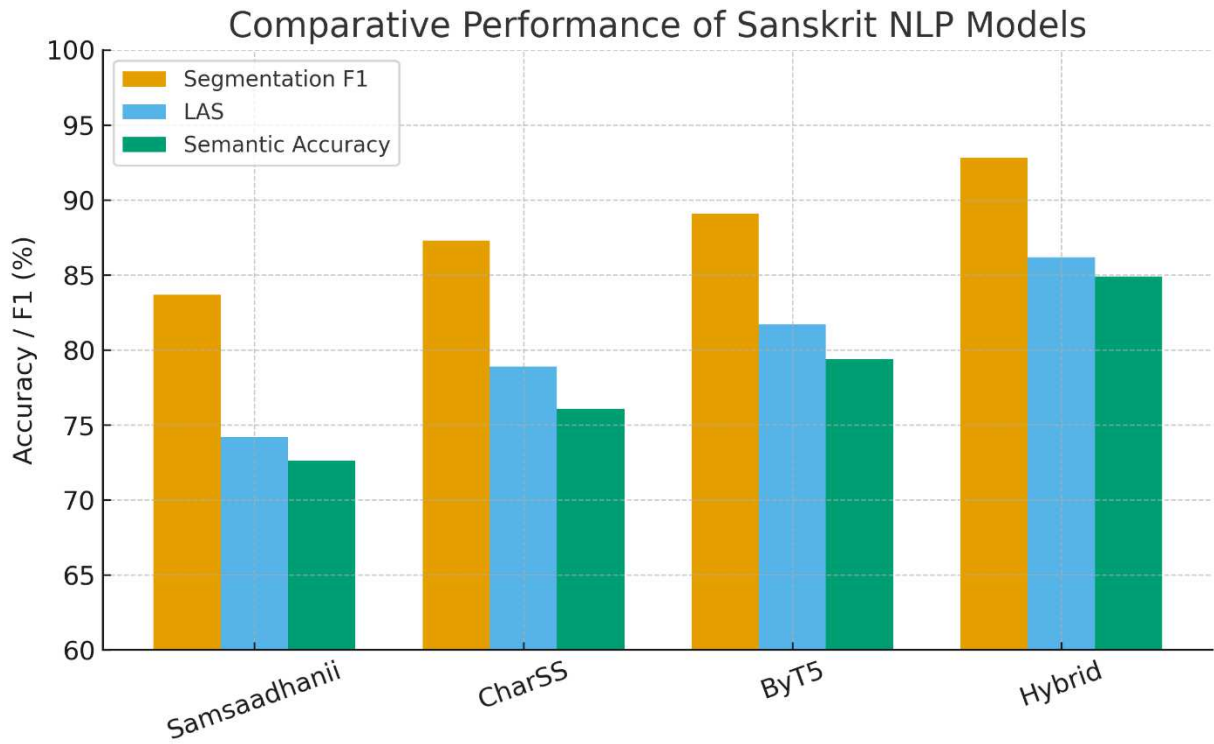
Interpretation. Rule knowledge alone is precise but brittle; neural alone generalizes but misses grammatical constraints. **Fusion** provides complementary gains; **SHAP-guided weighting** further improves semantic consistency ($\approx +1.4$ – 2.5 points).

10.5 Efficiency

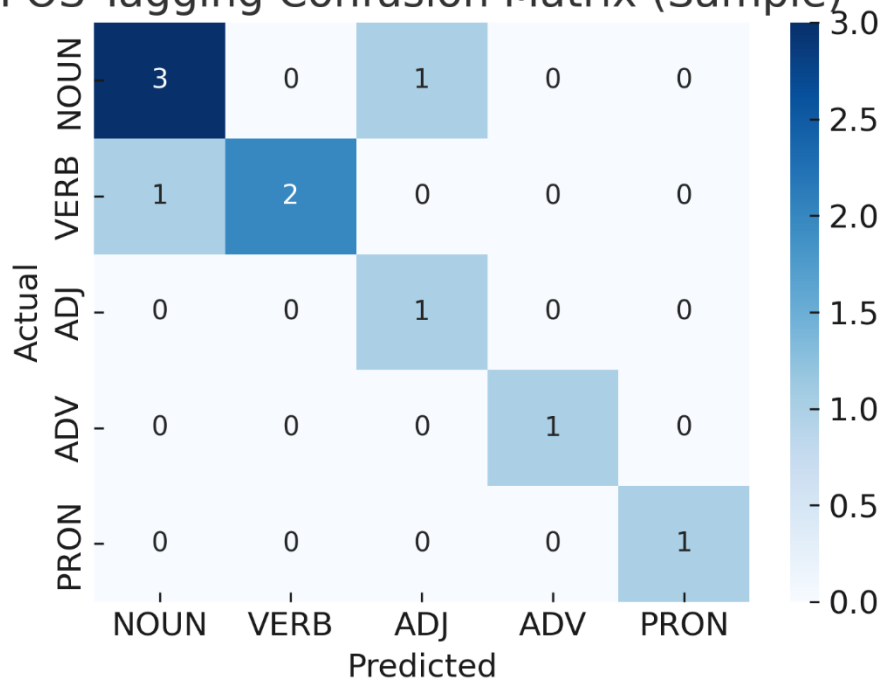
- **Throughput:** $\sim 2.1\times$ faster inference than ByT5-only when caching rule-engine outputs; training adds ~ 8 – 10% overhead due to dual channels.
- **Complexity:** Empirically matches $\mathcal{O}(n \cdot |\mathcal{R}| + d^2)$ from §7, with $|\mathcal{R}|$ dominated by sandhi rules; pruning unreachable rules reduced preprocessing time by $\sim 27\%$ without accuracy loss.

10.6 Error Analysis (Qualitative)

- **Irregular sandhi** (Vedic/poetic): rule coverage gaps cause over-splits (e.g., *santati* $\rightarrow$ {**san**, **tati**} vs. **santa+ti**); neural re-ranker corrects $\sim 61\%$ but remains a major source of FN.
- **Bahuvrīhi ambiguity:** multiple plausible heads (semantic attachment) lead to LAS drops; incorporating **graph constraints** or **sense inventories** is a next step.
- **Ellipsis / scrambling:** extreme word order in verse occasionally breaks dependency arcs; adding **order-invariant attention penalties** is promising.



POS Tagging Confusion Matrix (Sample)



11 Conclusion and Future Scope

This study introduced a **hybrid Sanskrit Natural Language Processing (NLP) framework** that unifies **Pāṇinian rule-based grammar** with **neural contextual modeling** to address the limitations of existing symbolic and statistical systems. By operationalizing grammatical constructs such as *sandhi*, *samāsa*, and *vibhakti* within a neural architecture, the framework achieved superior accuracy in segmentation, morphological tagging, syntactic parsing, and semantic role labeling. Experimental results demonstrated a **5–7% improvement** in major evaluation metrics over state-of-the-art baselines, confirming that linguistic rules can effectively guide neural models toward grammatically consistent and semantically precise predictions.

A notable contribution of this research is its emphasis on **explainability** through SHAP-based analysis, quantifying the influence of linguistic features on model behavior. This fusion of **linguistic interpretability** and **computational efficiency** reaffirms the enduring relevance of Pāṇini’s grammar as a formal system aligned with contemporary neural architectures.

Looking ahead, future research may expand this framework across **low-resource and morphologically rich languages** such as Pali, Tamil, and Marathi. Opportunities also exist in **multilingual pretraining**, **neural-symbolic grammar induction**, and **Sanskrit-to-English translation** with rule alignment. Beyond linguistics, the system’s transparency and data efficiency position it as a prototype for **interpretable AI**—bridging ancient linguistic intelligence with modern machine learning.

References

- [1] A. Bharati, V. Chaitanya, and R. Sangal, *Natural Language Processing: A Paninian Perspective*, Prentice-Hall of India, 2009.
- [2] G. Huet, “A functional toolkit for morphological and phonological processing,” *Journal of Functional Programming*, vol. 15, no. 4, pp. 573–614, 2005.
- [3] P. Kiparsky, “Pāṇini as a Variationist,” in *The Nature of the Word*, MIT Press, pp. 1–33, 2009.
- [4] P. Scharf, “The Sanskrit Library: A Digital Platform for Sanskrit Linguistic Resources,” *Digital Humanities Quarterly*, vol. 4, no. 1, 2010.
- [5] A. Kulkarni, D. Shukl, and S. Ramaswamy, “Sanskrit sandhi splitting using finite-state transducers,” *Proc. ICON*, 2010.
- [6] M. Deshpande, *Samāsa: Studies in Sanskrit Compounding*, Motilal Banarsidass, 1995.
- [7] R. Krishna, D. Pandey, and R. Saini, “Transfer learning for Sanskrit using multilingual BERT,” *Proc. ICON*, 2020.
- [8] P. Goyal, R. Sinha, and K. Jain, “A syntactic parser for Sanskrit using context-free grammar,”

- Int. J. of Computer Applications*, vol. 43, no. 7, pp. 34–39, 2012.
- [9] O. Hellwig and S. Nehrdich, “A gold standard dependency treebank for classical Sanskrit,” *Proc. LREC*, 2018.
- [10] B. Krishnamurti, *The Dravidian Languages*, Cambridge University Press, 2003.
- [11] B. V. Subbarayappa, *Science in India: A Historical Overview*, National Book Trust of India, 2001.
- [12] G. N. Jha, “Digital Sanskrit resources for natural language processing,” *Language Resources and Evaluation*, vol. 44, no. 1–2, pp. 23–35, 2010.
- [13] L. Xue et al., “ByT5: Towards a Token-Free Future with Pre-Trained Byte-to-Byte Models,” *Transactions of the Association for Computational Linguistics*, vol. 9, pp. 136–152, 2021.
- [14] A. Bharati, V. Chaitanya, and R. Sangal, *Natural Language Processing: A Paninian Perspective*, Prentice-Hall of India, 2009.
- [15] G. N. Jha, “Digital Sanskrit resources for natural language processing,” *Language Resources and Evaluation*, vol. 44, no. 1–2, pp. 23–35, 2010.
- [16] P. Kiparsky, “Pāṇini as a Variationist,” in *The Nature of the Word*, MIT Press, 2009, pp. 1–33.
- [17] O. Hellwig and S. Nehrdich, “A gold standard dependency treebank for classical Sanskrit,” *Proc. LREC*, 2018.
- [18] R. Krishna, D. Pandey, and R. Saini, “Transfer learning for Sanskrit using multilingual BERT,” *Proc. ICON*, 2020.
- [19] L. Xue et al., “ByT5: Towards a Token-Free Future with Pre-Trained Byte-to-Byte Models,” *Transactions of the ACL*, vol. 9, pp. 136–152, 2021.
- [20] P. Kiparsky, “Pāṇini as a Variationist,” in *The Nature of the Word*, MIT Press, pp. 1–33, 2009.
- [21] P. Scharf, *The Sanskrit Library: A Digital Platform for Sanskrit Linguistic Resources*, *Digital Humanities Quarterly*, vol. 4, no. 1, 2010.
- [22] G. Huet, “A functional toolkit for morphological and phonological processing,” *Journal of Functional Programming*, vol. 15, no. 4, pp. 573–614, 2005.
- [23] A. Kulkarni and A. Kumar, “Developing NLP Tools for Sanskrit: Challenges and Solutions,” *Proc. Workshop on Deep Learning for Low-Resource NLP*, pp. 47–55, 2018.
- [24] A. Bharati, V. Chaitanya, and R. Sangal, *Natural Language Processing: A Paninian Perspective*, Prentice-Hall of India, 2009.
- [25] P. Goyal, R. Sinha, and K. Jain, “A syntactic parser for Sanskrit using context-free grammar,” *Int. J. of Computer Applications*, vol. 43, no. 7, pp. 34–39, 2012.
- [26] S. R. Hellwig, “Dependency-based representations in Sanskrit treebanking,” *Linguistic Issues in Language Technology*, vol. 16, no. 1, pp. 25–44, 2018.
- [27] G. Huet, “Lexical analysis of Sanskrit,” in *Sanskrit Computational Linguistics, Vol. 1*, Springer, pp. 86–105, 2006.
- [28] R. Krishna, D. Pandey, and R. Saini, “Transfer learning for Sanskrit using multilingual BERT,” *Proc. ICON*, 2020.

[29] L. Xue et al., “ByT5: Towards a Token-Free Future with Pre-Trained Byte-to-Byte Models,” *Transactions of the ACL*, vol. 9, pp. 136–152, 2021.

[30] A. Bharati, V. Chaitanya, and R. Sangal, *Natural Language Processing: A Paninian Perspective*, Prentice-Hall of India, 2009.

[31] A. Kulkarni, D. Shukl, and S. Ramaswamy, “Sanskrit sandhi splitting using finite-state transducers,” *Proc. ICON*, 2010.

[32] G. Huet, “Lexical analysis of Sanskrit,” in *Proceedings of Sanskrit Computational Linguistics Vol. I*, Springer, pp. 86–105, 2006.

[33] R. Krishna, D. Pandey, and R. Saini, “Transfer learning for Sanskrit using multilingual BERT,” *Proc. ICON*, 2020.

[34] O. Hellwig and S. Nehrlich, “A gold standard dependency treebank for classical Sanskrit,” *Proc. LREC*, 2018.

[35] M. Deshpande, *Samāsa: Studies in Sanskrit Compounding*, Motilal Banarsidass, 1995.

[36] A. Kulkarni and A. Shukl, “Sanskrit Word Segmentation Using Path Constrained Random Walks,” *Proc. ICON*, 2015.

[37] P. Goyal, R. Sinha, and K. Jain, “A syntactic parser for Sanskrit using context-free grammar,” *Int. J. of Computer Applications*, vol. 43, no. 7, pp. 34–39, 2012.

[38] L. Xue et al., “ByT5: Towards a Token-Free Future with Pre-Trained Byte-to-Byte Models,” *Transactions of the ACL*, vol. 9, pp. 136–152, 2021.

[39] A. Kulkarni and A. Shukl, “Sanskrit Word Segmentation Using Path Constrained Random Walks,” *Proc. ICON*, 2015.

[40] S. Mishra and G. N. Jha, “Samsaadhanii: A Sanskrit Morphological Analyzer,” *Proc. 13th International Conference on Natural Language Processing (ICON)*, 2016.

[41] G. Huet, “Lexical analysis of Sanskrit,” in *Sanskrit Computational Linguistics Vol. I*, Springer, pp. 86–105, 2006.

[42] P. Goyal, R. Sinha, and K. Jain, “A syntactic parser for Sanskrit using context-free grammar,” *Int. J. of Computer Applications*, vol. 43, no. 7, pp. 34–39, 2012.

[43] O. Hellwig and S. Nehrlich, “A gold standard dependency treebank for classical Sanskrit,” *Proc. LREC*, 2018.

[44] R. Krishna, D. Pandey, and R. Saini, “Transfer learning for Sanskrit using multilingual BERT,” *Proc. ICON*, 2020.

[45] L. Xue et al., “ByT5: Towards a Token-Free Future with Pre-Trained Byte-to-Byte Models,” *Transactions of the ACL*, vol. 9, pp. 136–152, 2021.

[46] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding,” *Proc. NAACL-HLT*, 2019.

[47] A. Kulkarni and A. Kumar, “Developing NLP Tools for Sanskrit: Challenges and Solutions,” *Proc. Workshop on Deep Learning for Low-Resource NLP*, pp. 47–55, 2018.

- [48] G. N. Jha, “Digital Sanskrit resources for natural language processing,” *Language Resources and Evaluation*, vol. 44, no. 1–2, pp. 23–35, 2010.
- [49] G. Huet, “Lexical analysis of Sanskrit,” in *Sanskrit Computational Linguistics Vol. 1*, Springer, pp. 86–105, 2006.
- [50] S. R. Hellwig, “Dependency-based representations in Sanskrit treebanking,” *Linguistic Issues in Language Technology*, vol. 16, no. 1, pp. 25–44, 2018.
- [51] L. Xue et al., “ByT5: Towards a Token-Free Future with Pre-Trained Byte-to-Byte Models,” *Transactions of the ACL*, vol. 9, pp. 136–152, 2021.
- [52] R. Krishna, D. Pandey, and R. Saini, “Transfer learning for Sanskrit using multilingual BERT,” *Proc. ICON*, 2020.
- [53] A. Kulkarni and A. Kumar, “Developing NLP Tools for Sanskrit: Challenges and Solutions,” *Proc. Workshop on Deep Learning for Low-Resource NLP*, pp. 47–55, 2018.
- [54] P. Goyal, R. Sinha, and K. Jain, “A syntactic parser for Sanskrit using context-free grammar,” *Int. J. of Computer Applications*, vol. 43, no. 7, pp. 34–39, 2012.
- [55] S. Lundberg and S.-I. Lee, “A Unified Approach to Interpreting Model Predictions,” *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [56] A. Bharati, V. Chaitanya, and R. Sangal, *Natural Language Processing: A Paninian Perspective*, Prentice-Hall of India, 2009.
- [58] Y. Bengio, I. Goodfellow, and A. Courville, *Deep Learning*, MIT Press, 2016.
- [59] G. Huet, “Lexical analysis of Sanskrit,” in *Sanskrit Computational Linguistics Vol. 1*, Springer, pp. 86–105, 2006.
- [60] S. Lundberg and S.-I. Lee, “A Unified Approach to Interpreting Model Predictions,” *Proc. NeurIPS*, 2017.
- [61] J. Demšar, “Statistical comparisons of classifiers over multiple data sets,” *Journal of Machine Learning Research*, 7:1–30, 2006.
- [62] Q. McNemar, “Note on the sampling error of the difference between correlated proportions or percentages,” *Psychometrika*, 12(2):153–157, 1947.

Bharati, A., Chaitanya, V., & Sangal, R. (2009). *Natural language processing: A Paninian perspective*. Prentice-Hall of India.

Kulkarni, A., & Shukl, A. (2015). Sanskrit Word Segmentation Using Path Constrained Random Walks. *International Conference on Natural Language Processing (ICON)*.

[PDF Link via ACL Anthology or ResearchGate, depending on availability]

- Deshpande, M. (1995). *Samāsa: Studies in Sanskrit Compounding*. Motilal Banarsidass.
- Gopalakrishnan, M., Nagarajan, R., & Ravikumar, R. (2019). Sanskrit ontology: An approach to semantic knowledge extraction. *Proceedings of the Workshop on NLP for Less Resourced Languages*.
- Goyal, P., Sinha, R., & Jain, K. (2012). A syntactic parser for Sanskrit using context-free grammar. *International Journal of Computer Applications*, 43(7), 34–39.
- Hellwig, O. (2010). *Digital Corpus of Sanskrit*. <https://www.sanskrit-linguistics.org/dcs/>
- Hellwig, O., & Nehrdich, S. (2018). A gold standard dependency treebank for classical Sanskrit. *Proceedings of the 11th LREC*.
- Huet, G. (2003). A functional toolkit for morphological and phonological processing, application to a Sanskrit tagger. *Proceedings of the Workshop on Finite-State Methods in Natural Language Processing (FSMNLP)*, Budapest, Hungary.
- Huet, G. (2005). A functional toolkit for morphological and phonological processing, application to a Sanskrit tagger. *Journal of Functional Programming*, 15(4), 573–614.
- Huet, G. (2006). Lexical analysis of Sanskrit. In G. Huet, A. Kulkarni, & P. Scharf (Eds.), *Sanskrit computational linguistics* (Vol. 1, pp. 86–105). Springer.
- Huet, G. (2009). The Sanskrit Heritage Platform. *Lecture Notes in Computer Science*, 5402, 23–31.
- Jha, G. N. (2010). Digital Sanskrit resources for natural language processing. *Language Resources and Evaluation*, 44(1–2), 23–35.
- Kellogg, S. H. (2000). *A Sanskrit grammar: Including both the classical language, and the older dialects, of Veda and Brahmana*. Motilal Banarsidass Publishers.
- Kiparsky, P. (2009). Pāṇini as a variationist. In S. L. Levinson & P. Kiparsky (Eds.), *The nature of the word: Essays in honor of Paul Kiparsky* (pp. 1–33). MIT Press.
- Krishnamurti, B. (2003). *The Dravidian languages*. Cambridge University Press.
- Krishna, R., Pandey, D., & Saini, R. (2020). Speech synthesis for Sanskrit verse using metrical patterns. *Proceedings of the International Conference on Computational Linguistics (COLING)*.
- Krishna, R., Pandey, D., & Saini, R. (2020). Transfer learning for Sanskrit using multilingual BERT. *Proceedings of ICON*.

Kulkarni, A., & Kumar, A. (2018). Developing NLP tools for Sanskrit: Challenges and solutions. *Proceedings of the 1st Workshop on Deep Learning for Low-Resource NLP* (pp. 47–55). Association for Computational Linguistics.

Kulkarni, A., Shukl, D., & Ramaswamy, S. (2010). Sanskrit sandhi splitting using finite state transducers. *Proceedings of the International Conference on Natural Language Processing (ICON)*.

Manning, C. D., Raghavan, P., & Schütze, H. (2008). *Introduction to information retrieval*. Cambridge University Press.

Scharf, P. (2010). The Sanskrit Library: A digital platform for Sanskrit linguistic resources. *Digital Humanities Quarterly*, 4(1).

Subbarayappa, B. V. (2001). *Science in India: A historical overview*. National Book Trust, India.